where $D_m = \dim(\boldsymbol{\theta})$ and we have assumed $\mathbf{H}$ is full rank. We can drop the $\log|\hat{\mathbf{H}}|$ term, since it is independent of N, and thus will get overwhelmed by the likelihood. Putting all the pieces together, we get the **BIC score** that we want to maximize:

$$J_{\text{BIC}}(m) = \log p(\mathcal{D}|m) \approx \log p(\mathcal{D}|\hat{\boldsymbol{\theta}}, m) - \frac{D_m}{2}\log N \tag{5.59}$$

We can also define the **BIC loss**, that we want to minimize, by multiplying by -2:

$$\mathcal{L}_{\text{BIC}}(m) = -2\log p(\mathcal{D}|\hat{\boldsymbol{\theta}}, m) + D_m \log N \tag{5.60}$$

(The use of 2 as a scale factor is chosen to simplify the expression when using a model with a Gaussian likelihood.)

5.2.5.2 Akaike information criterion

The **Akaike information criterion** [Aka74] is closely related to the BIC. It has the form

$$\mathcal{L}_{\text{AIC}}(m) = -2\log p(\mathcal{D}|\hat{\boldsymbol{\theta}}, m) + 2D_m \tag{5.61}$$

This penalizes complex models less heavily than BIC, since the regularization term is independent of N. This estimator can be derived from a frequentist perspective.

5.2.5.3 Minimum description length (MDL)

We can think about the problem of scoring different models by using tools from information theory (Chapter 6). In particular, suppose we want to choose a model so that the sender can send some dataa to the receiver using the fewest number of bits. Choosing models this way is known as the **minimum description length** or **MDL** principle (see e.g., [HY01b; Gru07; GR19] for details, and see [Wal05] for the closely related **minimum message length** criterion).

We now derive an approximation to the MDL objective. First, the sender needs to specify which model to use. Let $\hat{\boldsymbol{\theta}} \in \mathbb{R}^{D_m}$ be the parameters estimated using N data samples. Since we can only reliably estimate each parameter to an accuracy of $O(1/\sqrt{N})$ (see Section 4.6.4.1), we only need to use $\log_2(1/\sqrt{N}) = \frac{1}{2}\log_2(N)$ bits to encode each parameter. Second, the sender needs to use this model to encode the data, which takes $-\log p(\mathcal{D}|\hat{\boldsymbol{\theta}}, m) = -\sum_n \log p(y_n|\hat{\boldsymbol{\theta}}, m)$ bits. The total cost is

$$\mathcal{L}_{\text{MDL}}(m) = -\log p(\mathcal{D}|\hat{\boldsymbol{\theta}}, m) + \frac{D_m}{2}\log N \tag{5.62}$$

We see that this **two-part code** has the same basic form as BIC.

5.2.5.4 Widely applicable information criterion (WAIC)

The main problem with BIC, AIC and MDL is that it can be hard to compute the degrees of a freedom of a model, needed to define the complexity term, since most parameters are highly correlated and not uniquely identifiable from the likelihood. In particular, if the mapping from parameters to the likelihood is not one-to-one, then the model known as a **singular statistical model**, since the corresponding Fisher information matrix (Section 4.7.2), and hence the Hessian $\mathbf{H}$ above, may be

singular. (Similar problems arise in over-parameterized models [Dwi+23].) An alternative criterion that works even in the singular case is known as the **widely applicable information criterion** (WAIC), also known as the **Watanabe–Akaike information criterion** [Wat10; Wat13].

The WAIC replaces the plug-in approximation to the marginal log likelihood, $\ell(m) = \sum_n \log p(y_n|\hat{\boldsymbol{\theta}}, m)$ with the the **expected log pointwise predictive density** or **ELPD**, defined as $\text{ELPD}(m) = \sum_{n=1}^N \log p(y_n|\mathcal{D}, m) = \sum_{n=1}^N \log \mathbb{E}_{\boldsymbol{\theta}|\mathcal{D},m}[p(y_n|\boldsymbol{\theta}, m)]$, which is usually approximated by Monte Carlo. In addition, the complexity term is defined by $C(m) = \sum_{n=1}^N \log \mathbb{V}_{\boldsymbol{\theta}|\mathcal{D},m}[p(y_n|\boldsymbol{\theta}, m)]$, which again is usually approximated by Monte Carlo. (The intuition for this is as follows: if, for a given datapoint y_n, the different posterior samples $\boldsymbol{\theta}_s$ make very different predictions, then the model is uncertain, and likely too flexible. The complexity term essentially counts how often this occurs.) The WAIC loss we want to minimize is defined as $\mathcal{L}_{\text{WAIC}}(m) = -2\text{LPPD}(m) + 2C(m)$.

Note that the WAIC evaluates the expected log likelihood using the posterior of the parameters. By contrast, the marginal likelihood averages the log likelihood wrt the prior. This makes the ML more sensitive to the prior. It is therefore generally better to use WAIC for model selection. Efficient Monte Carlo appproximations are discussed in [VGG17].

5.2.6 Posterior inference over effect sizes and Bayesian significance testing

The approach to hypothesis testing discussed in Section 5.2.1 relies on computing the Bayes factors for the null vs the alternative model, $p(\mathcal{D}|H_0)/p(\mathcal{D}|H_1)$. Unfortunately, computing the necessary marginal likelihoods can be computationally difficult, and the results can be sensitive to the choice of prior. Furthermore, we are often more interested in estimating an **effect size**, which is the difference in magnitude between two parameters, rather than in deciding if an effect size is 0 (null hypothesis) or not (alternative hypothesis) — the latter is called a **point null hypothesis**, and is often regarded as an irrelevant "straw man" (see e.g., [Mak+19] and references therein).

For example, suppose we have two classifiers, m_1 and m_2, and we want to know which one is better. That is, we want to perform a **comparison of classifiers**. Let μ_1 and μ_2 be their average accuracies, and let $\Delta = \mu_1 - \mu_2$ be the difference in their accuracies. The probability that model 1 is more accurate, on average, than model 2 is given by $p(\Delta > 0|\mathcal{D})$. However, even if this probability is large, the improvement may be not be practically significant. So it is better to compute a probability such as $p(\Delta > \epsilon|\mathcal{D})$ or $p(|\Delta| > \epsilon|\mathcal{D})$, where ϵ represents the minimal magnitude of effect size that is meaningful for the problem at hand. This is called a **one-sided test** or **two-sided test**.

More generally, let $R = [-\epsilon, \epsilon]$ represent a **region of practical equivalence** or **ROPE** [Kru15; KL17]. We can define 3 events of interest: the null hypothesis $H_0 : \Delta \in R$, which says both methods are practically the same (which is a more realistic assumption than $H_0 : \Delta = 0$); $H_A : \Delta > \epsilon$, which says m_1 is better than m_2; and $H_B : \Delta < -\epsilon$, which says m_2 is better than m_1. To choose amongst these 3 hypotheses, we just have to compute $p(\Delta|\mathcal{D})$, which avoids the need to compute Bayes factors. In the sections below, we discuss how to compute this quantity using two different kinds of model.

5.2.6.1 Bayesian t-test for difference in means

Suppose we have two classifiers, m_1 and m_2, which are evaluated on the same set of N test examples. Let e_i^m be the error of method m on test example i. (Or this could be the conditional log likelihood, $e_i^m = \log p^m(y_i|\boldsymbol{x}_i)$.) Since the classifiers are applied to the same data, we can use a **paired test** for comparing them, which is more sensitive than looking at average performance, since the factors that

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

make one example easy or hard to classify (e.g., due to label noise) will be shared by both methods. Thus we will work with the differences, $d_i = e_i^1 - e_i^2$. We assume $d_i \sim \mathcal{N}(\Delta, \sigma^2)$. We are interested in $p(\Delta|\boldsymbol{d})$, where $\boldsymbol{d} = (d_1, \ldots, d_N)$.

If we use an uninformative prior for the unknown parameters (Δ, σ), one can show that the posterior marginal for the mean is given by a Student distribution:

$$p(\Delta|\boldsymbol{d}) = \mathcal{T}_{N-1}(\Delta|\mu, s^2/N)$$

where $\mu = \frac{1}{N}\sum_{i=1}^N d_i$ is the sample mean, and $s^2 = \frac{1}{N-1}\sum_{i=1}^N (d_i - \mu)^2$ is an unbiased estimate of the variance. Hence we can easily compute $p(|\Delta| > \epsilon|\boldsymbol{d})$, with a ROPE of $\epsilon = 0.01$ (say). This is known as a **Bayesian t-test** [Ben+17]. (See also [Rou+09] for Bayesian t-test based on Bayes factors, and [Die98] for a non-Bayesian approach to comparing classifiers.)

An alternative to a formal test is to just plot the posterior $p(\Delta|\boldsymbol{d})$. If this distribution is tightly centered on 0, we can conclude that there is no significant difference between the methods. (In fact, an even simpler approach is to just make a boxplot of the data, $\{d_i\}$, which avoids the need for any formal statistical analysis.)

Note that this kind of problem arises in many applications, not just evaluating classifiers. For example, suppose we have a set of N people, each of whom is exposed two drugs; let e_i^m be the outcome (e.g., sickness level) when person i is exposed to drug m, and let $d_i^m = e_i^1 - e_i^2$ be the difference in response. We can then analyse the effect of the drug by computing $p(\Delta|\boldsymbol{d})$ as we discussed above.

5.2.6.2 Bayesian χ^2-test for difference in rates

Now suppose we have two classifiers which are evaluated on different test sets. Let y_m be the number of correct examples from method $m \in \{1, 2\}$ out of N_m trials, so the accuracy rate is y_m/N_m. We assume $y_m \sim \text{Bin}(N_m, \theta_m)$, so we are interested in $p(\Delta|\mathcal{D})$, where $\Delta = \theta_1 - \theta_2$, and $\mathcal{D} = (y_1, N_1, y_2, N_2)$ is all the data.

If we use a uniform prior for θ_1 and θ_2 (i.e., $p(\theta_j) = \text{Beta}(\theta_j|1, 1)$), the posterior is given by

$$p(\theta_1, \theta_2|\mathcal{D}) = \text{Beta}(\theta_1|y_1 + 1, N_1 - y_1 + 1)\text{Beta}(\theta_2|y_2 + 1, N_2 - y_2 + 1)$$

The posterior for Δ is given by

$$\begin{aligned} p(\Delta|\mathcal{D}) &= \int_0^1 \int_0^1 \mathbb{I}(\Delta = \theta_1 - \theta_2)\, p(\theta_1|\mathcal{D}_1)p(\theta_2|\mathcal{D}_2) \\ &= \int_0^1 \text{Beta}(\theta_1|y_1 + 1, N_1 - y_1 + 1)\text{Beta}(\theta_1 - \Delta|y_2 + 1, N_2 - y_2 + 1)d\theta_1 \end{aligned}$$

We can then evaluate this for any value of Δ that we choose. For example, we can compute

$$p(\Delta > \epsilon|\mathcal{D}) = \int_\epsilon^\infty p(\Delta|\mathcal{D})d\Delta \tag{5.63}$$

(We can compute this using 1 dimensional numerical integration or analytically [Coo05].) This is called a **Bayesian χ^2-test**.

	LH	RH	
Male	9	43	$N_1 = 52$
Female	4	44	$N_2 = 48$
Totals	13	87	100

Table 5.7: A 2×2 contingency table from http://en.wikipedia.org/wiki/Contingency_table. The MLEs for the left handedness rate in males and females are $\hat{\theta}_1 = 9/52 = 0.1731$ and $\hat{\theta}_2 = 4/48 = 0.0417$.

Note that this kind of problem arises in many applications, not just evaluating classifiers, For example, suppose the two groups are different companies selling the same product on Amazon, and y_m is the number of positive reviews for merchant m. Or suppose the two groups correspond to men and women, and y_m is the number of people in group m who are left handed, and $N_m - y_m$ to be the number who are right handed.[3] We can represent the data as a 2×2 **contingency table** of counts, as shown in Table 5.7.

The MLEs for the left handedness rate in males and females are $\hat{\theta}_1 = 9/52 = 0.1731$ and $\hat{\theta}_2 = 4/48 = 0.0417$. It seems that there is a difference, but the sample size is low, so we cannot be sure. Hence we will represent our uncertainty by computing $p(\Delta|\mathcal{D})$, where $\Delta = \theta_1 - \theta_2$ and $\mathcal{D}$ is the table of counts. We find $p(\theta_1 > \theta_2|\mathcal{D}) = \int_0^\infty p(\Delta|\mathcal{D}) = 0.901$, which suggests that left handedness is more common in males, consistent with other studies [PP+20].

5.3 Frequentist decision theory

In this section, we discuss **frequentist decision theory**. In this approach, we treat the unknown state of nature (often denoted by $\boldsymbol{\theta}$ instead of h) as a fixed but unknown quantity, and we treat the data $\boldsymbol{x}$ as random. Thus instead of conditioning on $\boldsymbol{x}$, we average over it, to compute the loss we expect to incur if we apply our decision procedure (estimator) to many different datasets. We give the details below.

5.3.1 Computing the risk of an estimator

We define the frequentist **risk** of an estimator δ given an unknown state of nature $\boldsymbol{\theta}$ to be the expected loss when applying that estimator to data $\boldsymbol{x}$, where the expectation is over the data, sampled from $p(\boldsymbol{x}|\boldsymbol{\theta})$:

$$R(\boldsymbol{\theta}, \delta) \triangleq \mathbb{E}_{p(\boldsymbol{x}|\boldsymbol{\theta})}\left[\ell(\boldsymbol{\theta}, \delta(\boldsymbol{x}))\right] \tag{5.64}$$

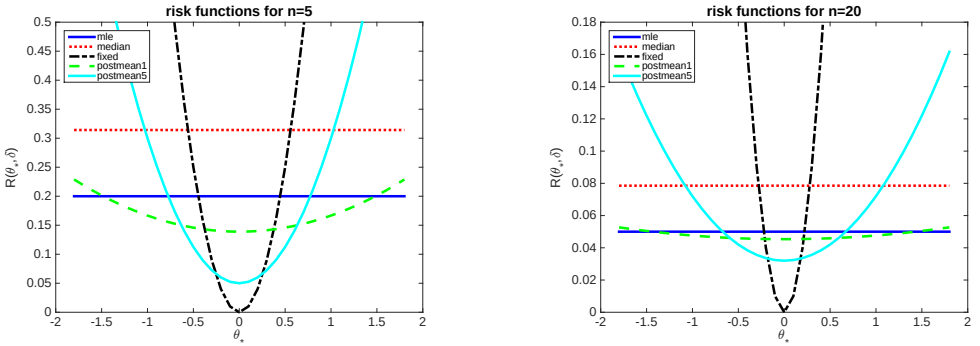
We give an example of this in Section 5.3.1.1.

5.3.1.1 Example

In this section, we consider the problem of estimating the mean of a Gaussian. We assume the data is sampled from $x_n \sim \mathcal{N}(\theta^*, \sigma^2 = 1)$, and we let $\boldsymbol{x} = (x_1, \ldots, x_N)$. If we use quadratic loss, $\ell_2(\theta, \hat{\theta}) = (\theta - \hat{\theta})^2$, the corresponding risk function is the MSE.

3. This example is based on the following blog post by Bob Carpenter: https://bit.ly/2FykD1C.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 5.8: Risk functions for estimating the mean of a Gaussian. Each curve represents $R(\hat{\theta}_i(\cdot), \theta^*)$ plotted vs θ^*, where i indexes the estimator. Each estimator is applied to N samples from $\mathcal{N}(\theta^*, \sigma^2 = 1)$. The dark blue horizontal line is the sample mean (MLE); the red line horizontal line is the sample median; the black curved line is the estimator $\hat{\theta} = \theta_0 = 0$; the green curved line is the posterior mean when $\kappa = 1$; the light blue curved line is the posterior mean when $\kappa = 5$. (a) $N = 5$ samples. (b) $N = 20$ samples. Adapted from Figure B.1 of [BS94]. Generated by riskFnGauss.ipynb.

We now consider 5 different estimators for computing θ:

- $\delta_1(\boldsymbol{x}) = \overline{x}$, the sample mean.
- $\delta_2(\boldsymbol{x}) = \text{median}(\boldsymbol{x})$, the sample median.
- $\delta_3(\boldsymbol{x}) = \theta_0$, a fixed value
- $\delta_\kappa(\boldsymbol{x})$, the posterior mean under a $\mathcal{N}(\theta|\theta_0, \sigma^2/\kappa)$ prior:

$$\delta_\kappa(\boldsymbol{x}) = \frac{N}{N+\kappa}\overline{x} + \frac{\kappa}{N+\kappa}\theta_0 = w\overline{x} + (1-w)\theta_0 \tag{5.65}$$

For δ_κ, we use $\theta_0 = 0$, and consider a weak prior, $\kappa = 1$, and a stronger prior, $\kappa = 5$.

Let $\hat{\theta} = \hat{\theta}(\boldsymbol{x}) = \delta(\boldsymbol{x})$ be the estimated parameter. The risk of this estimator is given by the MSE. In Section 4.7.6.3, we show that the MSE can be decomposed into squared bias plus variance:

$$\text{MSE}(\hat{\theta}|\theta^*) = \mathbb{V}\left[\hat{\theta}\right] + \text{bias}^2(\hat{\theta}) \tag{5.66}$$

where the bias is defined as $\text{bias}(\hat{\theta}) = \mathbb{E}\left[\hat{\theta} - \theta^*\right]$. We now use this expression to derive the risk for each estimator.

δ_1 is the sample mean. This is unbiased, so its risk is

$$\text{MSE}(\delta_1|\theta^*) = \mathbb{V}\left[\overline{x}\right] = \frac{\sigma^2}{N} \tag{5.67}$$

δ_2 is the sample median. This is also unbiased. Furthermore, one can show that its variance is approximately $\pi/(2N)$ (where $\pi = 3.14$) so the risk is

$$\text{MSE}(\delta_2|\theta^*) = \frac{\pi}{2N} \tag{5.68}$$

δ_3 returns the constant θ_0, so its bias is $(\theta^* - \theta_0)$ and its variance is zero. Hence the risk is

$$\text{MSE}(\delta_3|\theta^*) = (\theta^* - \theta_0)^2 \tag{5.69}$$

Finally, δ_4 is the posterior mean under a Gaussian prior. We can derive its MSE as follows:

$$\text{MSE}(\delta_\kappa|\theta^*) = \mathbb{E}\left[(w\overline{x} + (1-w)\theta_0 - \theta^*)^2\right] \tag{5.70}$$

$$= \mathbb{E}\left[(w(\overline{x} - \theta^*) + (1-w)(\theta_0 - \theta^*))^2\right] \tag{5.71}$$

$$= w^2\frac{\sigma^2}{N} + (1-w)^2(\theta_0 - \theta^*)^2 \tag{5.72}$$

$$= \frac{1}{(N+\kappa)^2}\left(N\sigma^2 + \kappa^2(\theta_0 - \theta^*)^2\right) \tag{5.73}$$

These functions are plotted in Figure 5.8 for $N \in \{5, 20\}$. We see that in general, the best estimator depends on the value of θ^*, which is unknown. If θ^* is very close to θ_0, then δ_3 (which just predicts θ_0) is best. If θ^* is within some reasonable range around θ_0, then the posterior mean, which combines the prior guess of θ_0 with the actual data, is best. If θ^* is far from θ_0, the MLE is best.

5.3.1.2 Bayes risk

In general, the true state of nature $\boldsymbol{\theta}$ that generates the data $\boldsymbol{x}$ is unknown, so we cannot compute the risk given in Equation (5.64). One solution to this is to assume a prior π_0 for $\boldsymbol{\theta}$, and then average it out. This gives us the **Bayes risk**, also called the **integrated risk**:

$$R_{\pi_0}(\delta) \triangleq \mathbb{E}_{\pi_0(\boldsymbol{\theta})}\left[R(\boldsymbol{\theta}, \delta)\right] = \int d\boldsymbol{\theta}\, d\boldsymbol{x}\, \pi_0(\boldsymbol{\theta}) p(\boldsymbol{x}|\boldsymbol{\theta}) \ell(\boldsymbol{\theta}, \delta(\boldsymbol{x})) \tag{5.74}$$

A decision rule that minimizes the Bayes risk is known as a **Bayes estimator**. This is equivalent to the optimal policy recommended by Bayesian decision theory in Equation (5.2) since

$$\delta(\boldsymbol{x}) = \operatorname*{argmin}_a \int d\boldsymbol{\theta}\, \pi_0(\boldsymbol{\theta}) p(\boldsymbol{x}|\boldsymbol{\theta}) \ell(\boldsymbol{\theta}, a) = \operatorname*{argmin}_a \int d\boldsymbol{\theta}\, p(\boldsymbol{\theta}|\boldsymbol{x}) \ell(\boldsymbol{\theta}, a) \tag{5.75}$$

Hence we see that picking the optimal action on a case-by-case basis (as in the Bayesian approach) is optimal on average (as in the frequentist approach). In other words, the Bayesian approach provides a good way of achieving frequentist goals. See [BS94, p448] for further discussion of this point.

5.3.1.3 Maximum risk

Of course the use of a prior might seem undesirable in the context of frequentist statistics. We can therefore define the **maximum risk** as follows:

$$R_{\max}(\delta) \triangleq \sup_{\boldsymbol{\theta}} R(\boldsymbol{\theta}, \delta) \tag{5.76}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

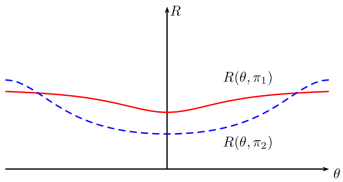


Figure 5.9: Risk functions for two decision procedures, δ_1 and δ_2. Since δ_1 has lower worst case risk, it is the minimax estimator, even though δ_2 has lower risk for most values of θ. Thus minimax estimators are overly conservative.

A decision rule that minimizes the maximum risk is called a **minimax estimator**, and is denoted δ_{MM}. For example, in Figure 5.9, we see that δ_1 has lower worst-case risk than δ_2, ranging over all possible values of $\boldsymbol{\theta}$, so it is the minimax estimator.

Minimax estimators have a certain appeal. However, computing them can be hard. And furthermore, they are very pessimistic. In fact, one can show that all minimax estimators are equivalent to Bayes estimators under a **least favorable prior**. In most statistical situations (excluding game theoretic ones), assuming nature is an adversary is not a reasonable assumption.

5.3.2 Consistent estimators

Suppose we have a dataset $\boldsymbol{x} = \{\boldsymbol{x}_n : n = 1 : N\}$ where the samples $\boldsymbol{x}_n \in \mathcal{X}$ are generated iid from a distribution $p(\mathbf{X}|\boldsymbol{\theta}^*)$, where $\boldsymbol{\theta}^* \in \Theta$ is the true parameter. Furthermore, suppose the parameters are **identifiable**, meaning that $p(\boldsymbol{x}|\boldsymbol{\theta}) = p(\boldsymbol{x}|\boldsymbol{\theta}')$ iff $\boldsymbol{\theta} = \boldsymbol{\theta}'$ for any dataset $\boldsymbol{x}$. Then we say that an estimator $\delta : \mathcal{X}^N \to \Theta$ is a **consistent estimator** if $\hat{\boldsymbol{\theta}}(\boldsymbol{x}) \to \boldsymbol{\theta}^*$ as $N \to \infty$ (where the arrow denotes convergence in probability). In other words, the procedure δ recovers the true parameter (or a subset of it) in the limit of infinite data. This is equivalent to minimizing the 0-1 loss, $\mathcal{L}(\boldsymbol{\theta}^*, \hat{\boldsymbol{\theta}}) = \mathbb{I}\left(\boldsymbol{\theta}^* \neq \hat{\boldsymbol{\theta}}\right)$. An example of a consistent estimator is the maximum likelihood estimator (MLE).

Note that an estimator can be unbiased but not consistent. For example, consider the estimator $\delta(\boldsymbol{x}) = \delta(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}) = \boldsymbol{x}_N$. This is an unbiased estimator of the true mean $\boldsymbol{\mu}$, since $\mathbb{E}[\delta(\boldsymbol{x})] = \mathbb{E}[\boldsymbol{x}_N] = \boldsymbol{\mu}$. But the sampling distribution of $\delta(\boldsymbol{x})$ does not converge to a fixed value, so it cannot converge to the point $\boldsymbol{\theta}^*$.

Although consistency is a desirable property, it is of somewhat limited usefulness in practice since most real datasets do not come from our chosen model family (i.e., there is no $\boldsymbol{\theta}^*$ such that $p(\cdot|\boldsymbol{\theta}^*)$ generates the observed data $\boldsymbol{x}$). In practice, it is more useful to find estimators that minimize some discrepancy measure between the empirical distribution and the estimated distribution. If we use KL divergence as our discrepancy measure, our estimate becomes the MLE.

5.3.3 Admissible estimators

We say that δ_1 **dominates** δ_2 if $R(\boldsymbol{\theta}, \delta_1) \leq R(\boldsymbol{\theta}, \delta_2)$ for all $\boldsymbol{\theta}$. The domination is said to be strict if the inequality is strict for some $\boldsymbol{\theta}^*$. An estimator is said to be **admissible** if it is not strictly dominated by any other estimator. Interestingly, [Wal47] proved that all admissible decision rules are equivalent to some kind of Bayesian decision rule, under some technical conditions. (See [DR21]

for a more general version of this result.)

For example, in Figure 5.8, we see that the sample median (dotted red line) always has higher risk than the sample mean (solid blue line). Therefore the sample median is not an admissible estimator for the mean. More surprisingly, one can show that the sample mean is not always an admissible estimator either, even under a Gaussian likelihood model with squared error loss (this is known as **Stein's paradox** [Ste56]).

However, the concept of admissibility is of somewhat limited value. For example, let $X \sim \mathcal{N}(\theta, 1)$, and consider estimating θ under squared loss. Consider the estimator $\delta_1(x) = \theta_0$, where θ_0 is a constant *independent of the data*. We now show that this is an admissible estimator.

To see this, suppose it were not true. Then there would be some other estimator δ_2 with smaller risk, so $R(\theta^*, \delta_2) \leq R(\theta^*, \delta_1)$, where the inequality must be strict for some θ^*. Consider the risk at $\theta^* = \theta_0$. We have $R(\theta_0, \delta_1) = 0$, and

$$R(\theta_0, \delta_2) = \int (\delta_2(x) - \theta_0)^2 p(x|\theta_0) dx \tag{5.77}$$

Since $0 \leq R(\theta^*, \delta_2) \leq R(\theta^*, \delta_1)$ for all θ^*, and $R(\theta_0, \delta_1) = 0$, we have $R(\theta_0, \delta_2) = 0$ and hence $\delta_2(x) = \theta_0 = \delta_1(x)$. Thus the only way δ_2 can avoid having higher risk than δ_1 at θ_0 is by being equal to δ_1. Hence there is no other estimator δ_2 with strictly lower risk, so δ_2 is admissible.

Thus we see that the estimator $\delta_1(x) = \theta_0$ is admissible, even though it ignores the data, so is useless as an estimator. Conversely, it is possible to construct useful estimators that are not admissable (see e.g., [Jay03, Sec 13.7]).

5.4 Empirical risk minimization

In this section, we consider how to apply frequentist decision theory in the context of supervised learning.

5.4.1 Empirical risk

In standard accounts of frequentist decision theory used in statistics textbooks, there is a single unknown "state of nature", corresponding to the unknown parameters $\boldsymbol{\theta}^*$ of some model, and we define the risk as in Equation (5.64), namely $R(\delta, \boldsymbol{\theta}^*) = \mathbb{E}_{p(\mathcal{D}|\boldsymbol{\theta}^*)}[\ell(\boldsymbol{\theta}^*, \delta(\mathcal{D}))]$.

In supervised learning, we have a different unknown state of nature (namely the output y) for each input $\boldsymbol{x}$, and our estimator δ is a prediction function $\hat{\boldsymbol{y}} = f(\boldsymbol{x})$, and the state of nature is the true distribution $p^*(\boldsymbol{x}, \boldsymbol{y})$. Thus the risk of an estimator is as follows:

$$R(f, p^*) = R(f) \triangleq \mathbb{E}_{p^*(\boldsymbol{x})p^*(\boldsymbol{y}|\boldsymbol{x})}[\ell(\boldsymbol{y}, f(\boldsymbol{x}))] \tag{5.78}$$

This is called the **population risk**, since the expectations are taken wrt the true joint distribution $p^*(\boldsymbol{x}, \boldsymbol{y})$. Of course, p^* is unknown, but we can approximate it using the empirical distribution with N samples:

$$p_{\mathcal{D}}(\boldsymbol{x}, \boldsymbol{y}|\mathcal{D}) \triangleq \frac{1}{|\mathcal{D}|} \sum_{(\boldsymbol{x}_n, \boldsymbol{y}_n) \in \mathcal{D}} \delta(\boldsymbol{x} - \boldsymbol{x}_n)\delta(\boldsymbol{y} - \boldsymbol{y}_n) \tag{5.79}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $p_{\mathcal{D}}(\boldsymbol{x}, \boldsymbol{y}) = p_{\text{tr}}(\boldsymbol{x}, \boldsymbol{y})$. Plugging this in gives us the **empirical risk**:

$$R(f, \mathcal{D}) \triangleq \mathbb{E}_{p_{\mathcal{D}}(\boldsymbol{x},\boldsymbol{y})}\left[\ell(\boldsymbol{y}, f(\boldsymbol{x})\right] = \frac{1}{N}\sum_{n=1}^{N} \ell(\boldsymbol{y}_n, f(\boldsymbol{x}_n)) \tag{5.80}$$

Note that $R(f, \mathcal{D})$ is a random variable, since it depends on the training set.

A natural way to choose the predictor is to use

$$\hat{f}_{\text{ERM}} = \operatorname*{argmin}_{f \in \mathcal{H}} R(f, \mathcal{D}) = \operatorname*{argmin}_{f \in \mathcal{H}} \frac{1}{N}\sum_{n=1}^{N} \ell(\boldsymbol{y}_n, f(\boldsymbol{x}_n)) \tag{5.81}$$

where we optimize over a specific **hypothesis space** $\mathcal{H}$ of functions. This is called **empirical risk minimization** (**ERM**).

5.4.1.1 Approximation error vs estimation error

In this section, we analyze the theoretical performance of functions that are fit using the ERM principle. Let $f^{**} = \operatorname{argmin}_f R(f)$ be the function that achieves the minimal possible population risk, where we optimize over all possible functions. Of course, we cannot consider all possible functions, so let us also define $f^*_{\mathcal{H}} = \operatorname{argmin}_{f \in \mathcal{H}} R(f)$ to be the best function in our hypothesis space, $\mathcal{H}$. Unfortunately we cannot compute $f^*_{\mathcal{H}}$, since we cannot compute the population risk, so let us finally define the prediction function that minimizes the empirical risk in our hypothesis space, given a fixed training set $\mathcal{D}$:

$$f^*_{\mathcal{D}} = \operatorname*{argmin}_{f \in \mathcal{H}} R(f, \mathcal{D}) \tag{5.82}$$

This is a random function, since $\mathcal{D} \sim p^*$ is random.

By adding and substracting terms, we can show that the expected risk of our chosen predictor compared to the best possible predictor can be decomposed into two terms, as follows:

$$\mathbb{E}_{\mathcal{D}\sim p^*}\left[R(f^*_{\mathcal{D}}) - R(f^{**})\right] = \underbrace{R(f^*_{\mathcal{H}}) - R(f^{**})}_{\mathcal{E}_{\text{app}}(\mathcal{H})} + \underbrace{\mathbb{E}_{\mathcal{D}\sim p^*}\left[R(f^*_{\mathcal{D}}) - R(f^*_{\mathcal{H}})\right]}_{\mathcal{E}_{\text{est}}(\mathcal{H},N)} \tag{5.83}$$

The first term, $\mathcal{E}_{\text{app}}(\mathcal{H})$, is the **approximation error**, which measures how closely $f^*_{\mathcal{H}}$ can model the true optimal function f^{**}. The second term, $\mathcal{E}_{\text{est}}(\mathcal{H}, N)$, is the **estimation error**, which measures the error in finding the best function in the class, due to having a finite training set of size N to evaluate performance. (One can also study the extra error introduced by the training process [BB08].)

We can decrease the approximation error by using a more expressive family of functions $\mathcal{H}$. However, this usually increases overfitting, which increases the estimation error. We can quantify the degree of overfitting of any model f by computing the **generalization gap**, defined as follows:

$$\text{GenGap}(f) = R(f) - R(f, \mathcal{D}_{\text{train}}) \approx R(f, \mathcal{D}_{\text{test}}) - R(f, \mathcal{D}_{\text{train}}) \tag{5.84}$$

Thus we need to find models that tradeoff approximation error and estimation error. We discuss solutions to this tradeoff below.

5.4.1.2 Regularized risk

To avoid the chance of overfitting, it is common to add a complexity penalty to the objective function, giving us the **regularized empirical risk**:

$$R_\lambda(f, \mathcal{D}) = R(f, \mathcal{D}) + \lambda C(f) \tag{5.85}$$

where $C(f)$ measures the complexity of the prediction function $f(\boldsymbol{x}; \boldsymbol{\theta})$, and $\lambda \geq 0$, which is known as a **hyperparameter**, controls the strength of the complexity penalty. (We discuss how to pick λ in Section 5.4.2.)

In practice, we usually work with parametric functions, and apply the regularizer to the parameters themselves. This yields the following form of the objective:

$$R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = R(\boldsymbol{\theta}, \mathcal{D}) + \lambda C(\boldsymbol{\theta}) \tag{5.86}$$

Note that, if the loss function is log loss, and the regularizer is a negative log prior, the regularized risk is given by

$$R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = -\frac{1}{N} \sum_{n=1}^{N} \log p(\boldsymbol{y}_n | \boldsymbol{x}_n, \boldsymbol{\theta}) - \lambda \log p(\boldsymbol{\theta}) \tag{5.87}$$

Minimizing this is equivalent to MAP estimation.

5.4.2 Structural risk

A natural way to estimate the hyperparameters is to minimize for the lowest achievable empirical risk:

$$\hat{\lambda} = \operatorname*{argmin}_{\lambda} \min_{\boldsymbol{\theta}} R_\lambda(\boldsymbol{\theta}, \mathcal{D}) \tag{5.88}$$

(This is an example of **bilevel optimization**, also called **nested optimization**.) Unfortunately, this technique will not work, since it will always pick the least amount of regularization, i.e., $\hat{\lambda} = 0$. To see this, note that

$$\operatorname*{argmin}_{\lambda} \min_{\boldsymbol{\theta}} R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = \operatorname*{argmin}_{\lambda} \min_{\boldsymbol{\theta}} R(\boldsymbol{\theta}, \mathcal{D}) + \lambda C(\boldsymbol{\theta}) \tag{5.89}$$

which is minimized by setting $\lambda = 0$. The problem is that the empirical risk underestimates the population risk, resulting in overfitting when we choose λ. This is called **optimism of the training error**.

If we knew the regularized population risk $R_\lambda(\boldsymbol{\theta})$, instead of the regularized empirical risk $R_\lambda(\boldsymbol{\theta}, \mathcal{D})$, we could use it to pick a model of the right complexity (e.g., value of λ). This is known as **structural risk minimization** [Vap98]. There are two main ways to estimate the population risk for a given model (value of λ), namely cross-validation (Section 5.4.3), and statistical learning theory (Section 5.4.4), which we discuss below.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

5.4.3 Cross-validation

In this section, we discuss a simple way to estimate the population risk for a supervised learning setup. We simply partition the dataset into two, the part used for training the model, and a second part, called the **validation set** or **holdout set**, used for assessing the risk. We can fit the model on the training set, and use its performance on the validation set as an approximation to the population risk.

To explain the method in more detail, we need some notation. First we make the dependence of the empirical risk on the dataset more explicit as follows:

$$R_\lambda(\boldsymbol{\theta}, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(\boldsymbol{x},\boldsymbol{y})\in\mathcal{D}} \ell(\boldsymbol{y}, f(\boldsymbol{x};\boldsymbol{\theta})) + \lambda C(\boldsymbol{\theta}) \tag{5.90}$$

Let us also define $\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}) = \text{argmin}_{\boldsymbol{\theta}} R_\lambda(\mathcal{D}, \boldsymbol{\theta})$. Finally, let $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{valid}}$ be a partition of $\mathcal{D}$. (Often we use about 80% of the data for the training set, and 20% for the validation set.)

For each model λ, we fit it to the training set to get $\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{\text{train}})$. We then use the unregularized empirical risk on the validation set as an estimate of the population risk. This is known as the **validation risk**:

$$R_\lambda^{\text{val}} \triangleq R_0(\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{\text{train}}), \mathcal{D}_{\text{valid}}) \tag{5.91}$$

Note that we use different data to train and evaluate the model.

The above technique can work very well. However, if the number of training cases is small, this technique runs into problems, because the model won't have enough data to train on, and we won't have enough data to make a reliable estimate of the future performance.

A simple but popular solution to this is to use **cross validation** (**CV**). The idea is as follows: we split the training data into K **folds**; then, for each fold $k \in \{1, \ldots, K\}$, we train on all the folds but the k'th, and test on the k'th, in a round-robin fashion, as sketched in Figure 4.6. Formally, we have

$$R_\lambda^{\text{cv}} \triangleq \frac{1}{K} \sum_{k=1}^{K} R_0(\hat{\boldsymbol{\theta}}_\lambda(\mathcal{D}_{-k}), \mathcal{D}_k) \tag{5.92}$$

where $\mathcal{D}_k$ is the data in the k'th fold, and $\mathcal{D}_{-k}$ is all the other data. This is called the **cross-validated risk**. Figure 4.6 illustrates this procedure for $K = 5$. If we set $K = N$, we get a method known as **leave-one-out cross-validation**, since we always train on $N - 1$ items and test on the remaining one.

We can use the CV estimate as an objective inside of an optimization routine to pick the optimal hyperparameter, $\hat{\lambda} = \text{argmin}_\lambda R_\lambda^{\text{cv}}$. Finally we combine all the available data (training and validation), and re-estimate the model parameters using $\hat{\boldsymbol{\theta}} = \text{argmin}_{\boldsymbol{\theta}} R_{\hat{\lambda}}(\boldsymbol{\theta}, \mathcal{D})$.

5.4.4 Statistical learning theory *

The principal problem with cross validation is that it is slow, since we have to fit the model multiple times. This motivates the desire to compute analytic approximations or bounds on the population risk. This is studied in the field of **statistical learning theory** (SLT) (see e.g., [Vap98]).

More precisely, the goal of SLT is to upper bound the generalization error with a certain probability. If the bound is satisfied, then we can be confident that a hypothesis that is chosen by minimizing the empirical risk will have low population risk. In the case of binary classifiers, this means the hypothesis will make the correct predictions; in this case we say it is **probably approximately correct**, and that the hypothesis class is **PAC learnable** (see e.g., [KV94] for details).

5.4.4.1 Bounding the generalization error

In this section, we establish conditions under which we can prove that a hypothesis class is PAC learnable. Let us initially consider the case where the hypothesis space is finite, with size $\dim(\mathcal{H}) = |\mathcal{H}|$. In other words, we are selecting a hypothesis from a finite list, rather than optimizing real-valued parameters. In this case, we can prove the following.

Theorem 5.4.1. *For any data distribution p^*, and any dataset $\mathcal{D}$ of size N drawn from p^*, the probability that the generalization error of a binary classifier will be more than ϵ, in the worst case, is upper bounded as follows:*

$$P\left(\max_{h\in\mathcal{H}} |R(h) - R(h,\mathcal{D})| > \epsilon\right) \leq 2\dim(\mathcal{H})e^{-2N\epsilon^2} \tag{5.93}$$

where $R(h,\mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\left(f(\boldsymbol{x}_i) \neq y_i^*\right)$ is the empirical risk, and $R(h) = \mathbb{E}\left[\mathbb{I}\left(f(\boldsymbol{x}) \neq y^*\right)\right]$ is the population risk.

Proof. Before we prove this, we introduce two useful results. First, **Hoeffding's inequality**, which states that if $E_1, \ldots, E_N \sim \text{Ber}(\theta)$, then, for any $\epsilon > 0$,

$$P(|\overline{E} - \theta| > \epsilon) \leq 2e^{-2N\epsilon^2} \tag{5.94}$$

where $\overline{E} = \frac{1}{N}\sum_{i=1}^{N} E_i$ is the empirical error rate, and θ is the true error rate. Second, the **union bound**, which says that if $A_1, \ldots, A_d$ are a set of events, then $P(\cup_{i=1}^{d} A_i) \leq \sum_{i=1}^{d} P(A_i)$. Using these results, we have

$$P\left(\max_{h\in\mathcal{H}} |R(h) - R(h,\mathcal{D})| > \epsilon\right) = P\left(\bigcup_{h\in\mathcal{H}} |R(h) - R(h,\mathcal{D})| > \epsilon\right) \tag{5.95}$$

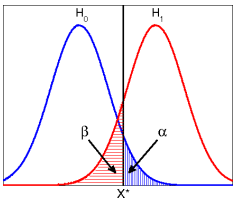
$$\leq \sum_{h\in\mathcal{H}} P\left(|R(h) - R(h,\mathcal{D})| > \epsilon\right) \tag{5.96}$$

$$\leq \sum_{h\in\mathcal{H}} 2e^{-2N\epsilon^2} = 2\dim(\mathcal{H})e^{-2N\epsilon^2} \tag{5.97}$$

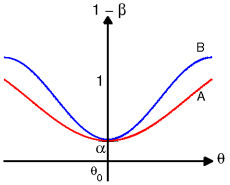
□

This bound tells us that the optimism of the training error increases with $\dim(\mathcal{H})$ but decreases with $N = |\mathcal{D}|$, as is to be expected.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a)



(b)

Figure 5.10: (a) Illustration of the Neyman-Pearson hypothesis testing paradigm. Generated by neyman-Pearson2.ipynb. (b) Two hypothetical two-sided power curves. B dominates A. Adapted from Figure 6.3.5 of [LM86]. Generated by twoPowerCurves.ipynb.

5.4.4.2 VC dimension

If the hypothesis space $\mathcal{H}$ is infinite (e.g., we have real-valued parameters), we cannot use $\dim(\mathcal{H}) = |\mathcal{H}|$. Instead, we can use a quantity called the **VC dimension** of the hypothesis class, named after Vapnik and Chervonenkis; this measures the degrees of freedom (effective number of parameters) of the hypothesis class. See e.g., [Vap98] for the details.

Unfortunately, it is hard to compute the VC dimension for many interesting models, and the upper bounds are usually very loose, making this approach of limited practical value. However, various other, more practical, estimates of generalization error have recently been devised, especially for DNNs, such as [Jia+20].

5.5 Frequentist hypothesis testing *

In this section, we discuss ways to determining if a hypothesis (model) is plausible or not, in the light of data $\mathcal{D}$.

5.5.1 Likelihood ratio test

When deciding if a model is a good description of some data or not, it is always useful to ask "relative to what". To make this concrete, suppose we have two hypotheses, known as the **null hypothesis** H_0 and an **alternative hypothesis** H_1, and we want to choose the one we think is more likely. We can think of this as a binary classification problem, where $H \in \{0, 1\}$ represents the identity of the "true" model. A natural approach is to use Bayesian model selection, as we discussed in Section 5.2.1, to compute $p(H|\mathcal{D})$, and then to pick the most probable model. Here we discuss a frequentist approach.

Suppose we have a uniform prior, so $p(H = 0) = p(H = 1) = 0.5$, and that we use 0-1 loss. Then the optimal decision rule is to accept H_0 iff $\frac{p(\mathcal{D}|H_0)}{p(\mathcal{D}|H_1)} > 1$. This is called the **likelihood ratio test**. We give some examples of this below.

5.5.1.1 Example: comparing Gaussian means

Suppose we are interested in testing whether some data comes from a Gaussian with mean μ_0 or from a Gaussian with mean μ_1. (We assume a known shared variance σ^2.) This is illustrated in

Figure 5.10a, where we plot $p(x|H_0)$ and $p(x|H_1)$. We can derive the likelihood ratio as follows:

$$\frac{p(\mathcal{D}|H_0)}{p(\mathcal{D}|H_1)} = \frac{\exp\left(-\frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n-\mu_0)^2\right)}{\exp\left(-\frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n-\mu_1)^2\right)} \tag{5.98}$$

$$= \exp\left(\frac{1}{2\sigma^2}(2N\overline{x}(\mu_0-\mu_1)+N\mu_1^2-N\mu_0^2)\right) \tag{5.99}$$

We see that this ratio only depends on the observed data via its mean, $\overline{x}$. From Figure 5.10a, we can see that $\frac{p(\mathcal{D}|H_0)}{p(\mathcal{D}|H_1)} > 1$ iff $\overline{x} < x^*$, where x^* is the point where the two pdf's intersect (we are assuming this point is unique).

5.5.1.2 Simple vs compound hypotheses

In Section 5.5.1.1, the parameters for the null and alternative hypotheses were either fully specified (μ_0 and μ_1) or shared (σ^2). This is called a **simple hypothesis** test. In general, a hypothesis might not fully specify all the parameters; this is called a **compound hypothesis**. In this case, we could integrate out these unknown parameters, as in the Bayesian approach, since a hypothesis with more parameters will always have higher likelihood. However, this can be computationally difficult, and is prone to problems caused by prior misspecification. As an alternative approach, we can "maximize out" the parameters, which gives us the maximum likelihood ratio test:

$$\frac{p(H_0|\mathcal{D})}{p(H_1|\mathcal{D})} = \frac{\int_{\boldsymbol{\theta}\in H_0} p(\boldsymbol{\theta})p_{\boldsymbol{\theta}}(\mathcal{D})}{\int_{\boldsymbol{\theta}\in H_1} p(\boldsymbol{\theta})p_{\boldsymbol{\theta}}(\mathcal{D})} \approx \frac{\max_{\boldsymbol{\theta}\in H_0} p_{\boldsymbol{\theta}}(\mathcal{D})}{\max_{\boldsymbol{\theta}\in H_1} p_{\boldsymbol{\theta}}(\mathcal{D})} \tag{5.100}$$

5.5.2 Type I vs type II errors and the Neyman-Pearson lemma

Hypothesis testing is a kind of binary classification problem. As we discussed in Section 5.1.3, there are two kinds of error we can make, known as a false positive or **type I error**, which corresponds to accidentally accepting the alterative when the null is true (i.e, $p(\hat{H}=1|H=0)$), and a false negative or **type II error**, which corresponds to accidentally accepting the null when the alternative is true (i.e, $p(\hat{H}=0|H=1)$). The type I error rate α is called the **significance** of the test. In our Gaussian mean example, we see from Figure 5.10a that the type I error rate is the vertical shaded blue area:

$$\alpha(\mu_0) = p(\text{type I error}) = p(\text{reject } H_0|H_0 \text{ is true}) \tag{5.101}$$

$$= p(\overline{X}(\tilde{\mathcal{D}}) > x^*|\tilde{\mathcal{D}} \sim H_0) \tag{5.102}$$

$$= p\left(\frac{\overline{X}-\mu_0}{\sigma/\sqrt{N}} > \frac{x^*-\mu_0}{\sigma/\sqrt{N}}\right) \tag{5.103}$$

Hence $x^* = z_\alpha\sigma/\sqrt{N}+\mu_0$, where z_α is the upper α quantile of the standard Normal. The type II error rate is denoted by β, and is given by

$$\beta(\mu_1) = p(\text{type II error}) = p(\text{accept } H_0|H_1 \text{ is true}) = p(\overline{X}(\tilde{\mathcal{D}}) < x^*|\tilde{\mathcal{D}} \sim H_1) \tag{5.104}$$

This is shown by the horizontal shaded red area in Figure 5.10a.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We define the **power** of a test as $1 - \beta(\mu_1)$; this is the probability that we reject H_0 given that H_1 is true (i.e, $p(\hat{H} = 1|H = 1)$, which is the true positive rate). In other words, it is the ability to correctly recognize that the null hypothesis is wrong. Clearly the least power occurs if $\mu_1 = \mu_0$ (so the curves overlap); in this case, we have $1 - \beta(\mu_1) = \alpha(\mu_0)$. As μ_1 and μ_0 become further apart, the power approaches 1 (because the shaded red area gets smaller, $\beta \to 0$). If we have two tests, A and B, where $\text{power}(B) \geq \text{power}(A)$ for the same type I error rate, we say B **dominates** A. See Figure 5.10b. A test with highest power under H_1 amongst all tests with significance level α is called a **most powerful test**. It turns out that the likelihood ratio test is a most powerful test, a result known as the **Neyman-Pearson lemma**.

5.5.3 Null hypothesis significance testing (NHST) and p-values

In the above decision-theoretic (or Neyman-Pearson) approach to hypothesis testing, we had to specify a null hypothesis H_0 as well as an alternative hypothesis H_1 so that we can compute $p(\mathcal{D}|H_0)$ and $p(\mathcal{D}|H_1)$. In some cases, it is difficult to define an alternative hypothesis, and we just want to test if a simple null hypothesis is "plausible" given some data. To do this, we can define a **test statistic** $\text{test}(\mathcal{D})$, and then we can compare its observed value to the value we would expect if the data came from the null hypothesis, $\text{test}(\tilde{\mathcal{D}})$ where $\tilde{\mathcal{D}} \sim H_0$. If the observed value is unexpected given H_0, we **reject the null hypothesis**. To quantify this, we compute the probability of seeing a test value that is as large or larger than the observed value (assuming that larger values make H_1 more likely). More precisely, we define the **p-value** to be the probability, under the null hypothesis, of observing a test statistic that is as large or larger than that actually observed:

$$\text{pval} \triangleq \Pr(\text{test}(\tilde{\mathcal{D}}) \geq \text{test}(\mathcal{D})|\tilde{\mathcal{D}} \sim H_0) \tag{5.105}$$

In other words, $\text{pval} \triangleq \Pr(\text{test}_{\text{null}} \geq \text{test}_{\text{obs}})$, where $\text{test}_{\text{obs}} = \text{test}(\mathcal{D})$ and $\text{test}_{\text{null}} = \text{test}(\tilde{\mathcal{D}})$, where $\tilde{\mathcal{D}} \sim H_0$ is hypothetical future data. Smaller values correspond to stronger evidence against H_0.

Traditionally we reject the null hypothesis if the p-value is less than $\alpha = 0.05$; this is called the **significance level** of the test, and the whole approach is called **null hypothesis significance testing** or **NHST**. By construction, such a test will have a type I error rate (accidently rejecting the null when it is true) of value α. Note that this decision rule corresponds to picking decision threshold t^* such that $\Pr(\text{test}(\tilde{\mathcal{D}}) \geq t^*|H_0) = \alpha$. If we set $t^* = \text{test}(\mathcal{D})$, then α will be equal to the observed p-value. Thus the p-value is the smallest value of α for which we can reject H_0.

We can compute the p-value using $\text{pval} = 1 - \Phi(\text{test}(\mathcal{D}))$, where Φ is the cdf of the sampling distribution of the test statistic. This is called a **one-sided p-value**. In some case it can be more appropriate to use a **two-sided p-value** of the form $\text{pval} = \Pr(\text{test}(\tilde{\mathcal{D}}) \geq \text{test}(\mathcal{D})|\tilde{\mathcal{D}} \sim H_0) + \Pr(\text{test}(\tilde{\mathcal{D}}) \leq -\text{test}(\mathcal{D})|\tilde{\mathcal{D}} \sim H_0)$, where we have assumed $\text{test}(\mathcal{D}) \geq 0$. For example, suppose we use $\text{test}(\tilde{\mathcal{D}}) = (\hat{\theta}(\tilde{\mathcal{D}}) - \theta_0)/\hat{\text{se}}(\tilde{\mathcal{D}})$, where θ_0 is the value for θ^* given H_0, and $\hat{\theta}$ is the MLE; ths is known as the **Wald statistic**. Based on the asymptotic normality of the MLE discussed in Section 4.7.2, we have that $\text{pval} = \Pr(|\text{test}(\tilde{\mathcal{D}})| > |\text{test}(\mathcal{D})| \mid H_0) \approx \Pr(|Z| > |\text{test}(\mathcal{D})|) = 2\Phi(-|\text{test}(\mathcal{D})|)$, where $Z \sim \mathcal{N}(0, 1)$.

We see that, to compute the p-value, we need to compute the sampling distribution of the test statistic under the null hypothesis. Suppose we want to compare an empirical distribution or outcome to an expected (theoretical) distribution or outcome. In some cases we can use a large sample (Gaussian approxmation) to the sampling distribution, as we illustrated above. If not, we can use a non-parametric bootstrap approximation. Another important case arises when we want to compare

two empirical distributions to test if they are the same; for this we can use the non-parametric **permutation test**, which makes no assumptions about the distribution. For example, suppose we have m samples X_i from P_X and n samples Y_i from P_y and the null hypothesis is $P_x = P_y$. Define the test statistic $\text{test}(X_1, \ldots, X_m, Y_1, \ldots, Y_n) = |\overline{X} - \overline{Y}|$. If we permute the order of the samples, then, under H_0, this statistic should not change. So we can sample random permutations to approximate $p(\text{test}(\tilde{\mathcal{D}})|\tilde{\mathcal{D}} \sim H_0)$, from which we can compute the tail probability of $\text{test}(\mathcal{D})$ computed using the unshuffled data. For more details, see e.g. [Was04, p162].

Note that a p-value of 0.05 does *not* mean that the alternative hypothesis H_1 is true with probability 0.95. Indeed, even most scientists misinterpret p-values.[4] The quantity that most people want to compute is the Bayesian posterior $p(H|\mathcal{D})$. For more on this important distinction, see Section 5.5.4.

5.5.4 p-values considered harmful

A p-value is often interpreted as the likelihood of the data under the null hypothesis, so small values are interpreted to mean that H_0 is unlikely, and therefore that H_1 is likely. The reasoning is roughly as follows:

> If H_0 is true, then this test statistic would probably not occur. This statistic did occur. Therefore H_0 is probably false.

However, this is invalid reasoning. To see why, consider the following example (from [Coh94]):

> If a person is an American, then he is probably not a member of Congress. This person is a member of Congress. Therefore he is probably not an American.

This is obviously fallacious reasoning. By contrast, the following logical argument is valid reasoning:

> If a person is a Martian, then he is not a member of Congress. This person is a member of Congress. Therefore he is not a Martian.

The difference between these two cases is that the Martian example is using **deduction**, that is, reasoning forward from logical definitions to their consequences. More precisely, this example uses a rule from logic called **modus tollens**, in which we start out with a definition of the form $P \Rightarrow Q$; when we observe $\neg Q$, we can conclude $\neg P$. By contrast, the American example concerns **induction**, that is, reasoning backwards from observed evidence to probable (but not necessarily true) causes using statistical regularities, not logical definitions.

To perform induction, we need to use probabilistic inference (as explained in detail in [Jay03]). In particular, to compute the probability of the null hypothesis, we should use Bayes rule, as follows:

$$p(H_0|\mathcal{D}) = \frac{p(\mathcal{D}|H_0)p(H_0)}{p(\mathcal{D}|H_0)p(H_0) + p(\mathcal{D}|H_1)p(H_1)} \tag{5.106}$$

If the prior is uniform, so $p(H_0) = p(H_1) = 0.5$, this can be rewritten in terms of the **likelihood ratio** $LR = p(\mathcal{D}|H_0)/p(\mathcal{D}|H_1)$ as follows:

$$p(H_0|\mathcal{D}) = \frac{LR}{LR + 1} \tag{5.107}$$

4. See e.g., https://fivethirtyeight.com/features/not-even-scientists-can-easily-explain-p-values/.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

	Ineffective	Effective	
"Not significant"	171	4	175
"Significant"	9	16	25
	180	20	200

Table 5.8: Some statistics of a hypothetical clinical trial. Source: [SAM04, p74].

In the American Congress example, $\mathcal{D}$ is the observation that the person is a member of Congress. The null hypothesis H_0 is that the person is American, and the alternative hypothesis H_1 is that the person is not American. We assume that $p(\mathcal{D}|H_0)$ is low, since most Americans are not members of Congress. However, $p(\mathcal{D}|H_1)$ is also low — in fact, in this example, it is 0, since only Americans can be members of Congress. Hence $LR = \infty$, so $p(H_0|\mathcal{D}) = 1.0$, as intuition suggests. Note, however, that NHST ignores $p(\mathcal{D}|H_1)$ as well as the prior $p(H_0)$, so it gives the wrong results — not just in this problem, but in many problems.

In general there can be huge differences between p-values and $p(H_0|\mathcal{D})$. In particular, [SBB01] show that even if the p-value is as low as 0.05, the posterior probability of H_0 can be as high as 30% or more, even with a uniform prior.

Consider this concrete example from [SAM04, p74]. Suppose 200 clinical trials are carried out for some drug, and we get the data in Table 5.8. Suppose we perform a statistical test of whether the drug has a significant effect or not. The test has a type I error rate of $\alpha = 9/180 = 0.05$ and a type II error rate of $\beta = 4/20 = 0.2$.

We can compute the probability that the drug is not effective, given that the result is supposedly "significant", as follows:

$$p(H_0|\text{'significant'}) = \frac{p(\text{'significant'}|H_0)p(H_0)}{p(\text{'significant'}|H_0)p(H_0) + p(\text{'significant'}|H_1)p(H_1)} \tag{5.108}$$

$$= \frac{p(\text{type I error})p(H_0)}{p(\text{type I error})p(H_0) + (1 - p(\text{type II error}))p(H_1)} \tag{5.109}$$

$$= \frac{\alpha p(H_0)}{\alpha p(H_0) + (1-\beta)p(H_1)} \tag{5.110}$$

If we have prior knowledge, based on past experience, that most (say 90%) drugs are ineffective, then we find $p(H_0|\text{'significant'}) = 0.36$, which is much more than the 5% probability people usually associate with a p-value of $\alpha = 0.05$.

Thus we should distrust claims of statistical significance if they violate our prior knowledge.

5.5.5 Why isn't everyone a Bayesian?

In Section 4.7.5 and Section 5.5.4, we have seen that inference based on frequentist principles can exhibit various forms of counter-intuitive behavior that can sometimes contradict common sense reason, as has been pointed out in multiple articles (see e.g., [Mat98; MS11; Kru13; Gel16; Hoe+14; Lyu+20; Cha+19b; Cla21]).

The fundamental reason for these problems is that frequentist inference violates the **likelihood principle** [BW88], which says that inference should be based on the likelihood of the observed data,

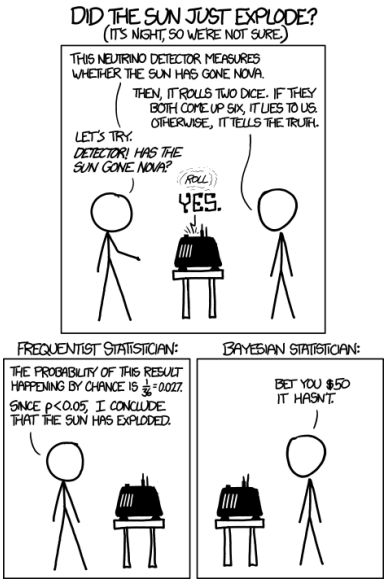


Figure 5.11: Cartoon illustrating the difference between frequentists and Bayesians. (The $p < 0.05$ comment is explained in Section 5.5.4. The betting comment is a reference to the Dutch book theorem, which essentially proves that the Bayesian approach to gambling (and other decision theory problems) is optimal, as explained in e.g., [Háj08].) From https://xkcd.com/1132/. Used with kind permission of Rundall Munroe (author of xkcd).

not on hypothetical future data that you have not observed. Bayes obviously satisfies the likelihood principle, and consequently does not suffer from these pathologies.

Given these fundamental flaws of frequentist statistics, and the fact that Bayesian methods do not have such flaws, an obvious question to ask is: "Why isn't everyone a Bayesian?" The (frequentist) statistician Bradley Efron wrote a paper with exactly this title [Efr86]. His short paper is well worth reading for anyone interested in this topic. Below we quote his opening section:

> The title is a reasonable question to ask on at least two counts. First of all, everyone used to be a Bayesian. Laplace wholeheartedly endorsed Bayes's formulation of the inference problem, and most 19th-century scientists followed suit. This included Gauss, whose statistical work is usually presented in frequentist terms.
>
> A second and more important point is the cogency of the Bayesian argument. Modern statisticians, following the lead of Savage and de Finetti, have advanced powerful theoretical arguments for preferring Bayesian inference. A byproduct of this work is a disturbing catalogue of inconsistencies in the frequentist point of view.
>
> Nevertheless, everyone is not a Bayesian. The current era (1986) is the first century in which statistics has been widely used for scientific reporting, and in fact, 20th-century statistics is mainly non-Bayesian. However, Lindley (1975) predicts a change for the 21st century.

Time will tell whether Lindley was right. However, the trends seem to be going in this direction.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

For example, some journals have banned p-values [TM15; AGM19], and the journal *The American Statistician* (produced by the American Statistical Association) published a whole special issue warning about the use of p-values and NHST [WSL19].

Traditionally, computation has been a barrier to using Bayesian methods, but this is less of an issue these days, due to faster computers and better algorithms (which we will discuss in the sequel to this book, [Mur23]). Another, more fundamental, concern is that the Bayesian approach is only as correct as its modeling assumptions. However, this criticism also applies to frequentist methods, since the sampling distribution of an estimator must be derived using assumptions about the data generating mechanism. (In fact [BT73] show that the sampling distributions for the MLE for common models are identical to the posterior distributions under a noninformative prior.) Fortunately, we can check modeling assumptions empirically using cross validation (Section 4.5.5), calibration, and Bayesian model checking. We discuss these topics in the sequel to this book, [Mur23].

To summarize, it is worth quoting Donald Rubin, who wrote a paper [Rub84] called "Bayesianly Justifiable and Relevant Frequency Calculations for the Applied Statistician". In it, he writes

> The applied statistician should be Bayesian in principle and calibrated to the real world in practice. [They] should attempt to use specifications that lead to approximately calibrated procedures under reasonable deviations from [their assumptions]. [They] should avoid models that are contradicted by observed data in relevant ways — frequency calculations for hypothetical replications can model a model's adequacy and help to suggest more appropriate models.

5.6 Exercises

Exercise 5.1 [Reject option in classifiers]

(Source: [DHS01, Q2.13].) In many classification problems one has the option either of assigning $\boldsymbol{x}$ to class j or, if you are too uncertain, of choosing the **reject option**. If the cost for rejects is less than the cost of falsely classifying the object, it may be the optimal action. Let α_i mean you choose action i, for $i = 1 : C+1$, where C is the number of classes and $C+1$ is the reject action. Let $Y = j$ be the true (but unknown) **state of nature**. Define the loss function as follows

$$\lambda(\alpha_i|Y=j) = \begin{cases} 0 & \text{if } i = j \text{ and } i, j \in \{1, \ldots, C\} \\ \lambda_r & \text{if } i = C+1 \\ \lambda_s & \text{otherwise} \end{cases} \tag{5.111}$$

In other words, you incur 0 loss if you correctly classify, you incur λ_r loss (cost) if you choose the reject option, and you incur λ_s loss (cost) if you make a substitution error (misclassification).

a. Show that the minimum risk is obtained if we decide $Y = j$ if $p(Y = j|\boldsymbol{x}) \geq p(Y = k|\boldsymbol{x})$ for all k (i.e., j is the most probable class) *and* if $p(Y = j|\boldsymbol{x}) \geq 1 - \frac{\lambda_r}{\lambda_s}$; otherwise we decide to reject.

b. Describe qualitatively what happens as λ_r/λ_s is increased from 0 to 1 (i.e., the relative cost of rejection increases).

Exercise 5.2 [Newsvendor problem †]

Consider the following classic problem in decision theory / economics. Suppose you are trying to decide how much quantity Q of some product (e.g., newspapers) to buy to maximize your profits. The optimal amount will depend on how much demand D you think there is for your product, as well as its cost to you C and its selling price P. Suppose D is unknown but has pdf $f(D)$ and cdf $F(D)$. We can evaluate the expected profit by considering two cases: if $D > Q$, then we sell all Q items, and make profit $\pi = (P - C)Q$; but if $D < Q$,

we only sell D items, at profit $(P-C)D$, but have wasted $C(Q-D)$ on the unsold items. So the expected profit if we buy quantity Q is

$$E\pi(Q) = \int_Q^\infty (P-C)Qf(D)dD + \int_0^Q (P-C)Df(D)dD - \int_0^Q C(Q-D)f(D)dD \tag{5.112}$$

Simplify this expression, and then take derivatives wrt Q to show that the optimal quantity Q^* (which maximizes the expected profit) satisfies

$$F(Q^*) = \frac{P-C}{P} \tag{5.113}$$

Exercise 5.3 [Bayes factors and ROC curves †]

Let $B = p(D|H_1)/p(D|H_0)$ be the Bayes factor in favor of model 1. Suppose we plot two ROC curves, one computed by thresholding B, and the other computed by thresholding $p(H_1|D)$. Will they be the same or different? Explain why.

Exercise 5.4 [Posterior median is optimal estimate under L1 loss]

Prove that the posterior median is the optimal estimate under L1 loss.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6 Information Theory

In this chapter, we introduce a few basic concepts from the field of **information theory**. More details can be found in other books such as [Mac03; CT06], as well as the sequel to this book, [Mur23].

6.1 Entropy

The **entropy** of a probability distribution can be interpreted as a measure of uncertainty, or lack of predictability, associated with a random variable drawn from a given distribution, as we explain below.

We can also use entropy to define the **information content** of a data source. For example, suppose we observe a sequence of symbols $X_n \sim p$ generated from distribution p. If p has high entropy, it will be hard to predict the value of each observation X_n. Hence we say that the dataset $\mathcal{D} = (X_1, \ldots, X_n)$ has high information content. By contrast, if p is a degenerate distribution with 0 entropy (the minimal value), then every X_n will be the same, so $\mathcal{D}$ does not contain much information. (All of this can be formalized in terms of data compression, as we discuss in the sequel to this book.)

6.1.1 Entropy for discrete random variables

The entropy of a discrete random variable X with distribution p over K states is defined by

$$\mathbb{H}(X) \triangleq -\sum_{k=1}^{K} p(X=k) \log_2 p(X=k) = -\mathbb{E}_X[\log p(X)] \tag{6.1}$$

(Note that we use the notation $\mathbb{H}(X)$ to denote the entropy of the rv with distribution p, just as people write $\mathbb{V}[X]$ to mean the variance of the distribution associated with X; we could alternatively write $\mathbb{H}(p)$.) Usually we use log base 2, in which case the units are called **bits** (short for binary digits). For example, if $X \in \{1, \ldots, 5\}$ with histogram distribution $p = [0.25, 0.25, 0.2, 0.15, 0.15]$, we find $H = 2.29$ bits. If we use log base e, the units are called **nats**.

The discrete distribution with **maximum entropy** is the uniform distribution. Hence for a K-ary random variable, the entropy is maximized if $p(x=k) = 1/K$; in this case, $\mathbb{H}(X) = \log_2 K$. To see this, note that

$$\mathbb{H}(X) = -\sum_{k=1}^{K} \frac{1}{K} \log(1/K) = -\log(1/K) = \log(K) \tag{6.2}$$

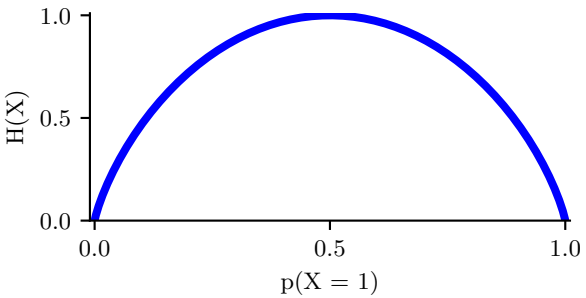
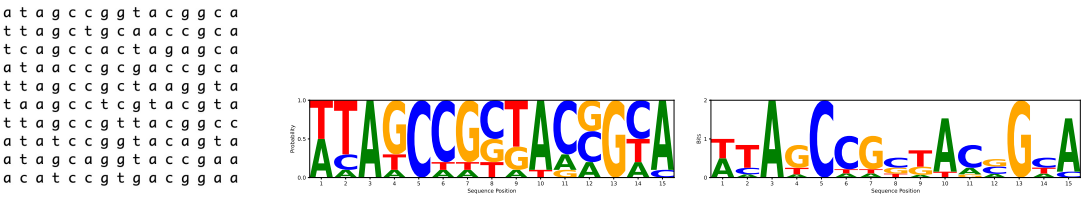


Figure 6.1: Entropy of a Bernoulli random variable as a function of θ. The maximum entropy is $\log_2 2 = 1$. Generated by bernoulli_entropy_fig.ipynb.



(a) *(b)* *(c)*

Figure 6.2: (a) Some aligned DNA sequences. Each row is a sequence, each column is a location within the sequence. (b) The corresponding position weight matrix, visualized as a sequence of histograms. Each column represents a probability distribution over the alphabet $\{A, C, G, T\}$ for the corresponding location in the sequence. The size of the letter is proportional to the probability. (c) A sequence logo. See text for details. Generated by seq_logo_demo.ipynb.

Conversely, the distribution with minimum entropy (which is zero) is any delta-function that puts all its mass on one state. Such a distribution has no uncertainty.

For the special case of binary random variables, $X \in \{0, 1\}$, we can write $p(X = 1) = \theta$ and $p(X = 0) = 1 - \theta$. Hence the entropy becomes

$$\mathbb{H}(X) = -[p(X = 1) \log_2 p(X = 1) + p(X = 0) \log_2 p(X = 0)] \tag{6.3}$$

$$= -[\theta \log_2 \theta + (1 - \theta) \log_2 (1 - \theta)] \tag{6.4}$$

This is called the **binary entropy function**, and is also written $\mathbb{H}(\theta)$. We plot this in Figure 6.1. We see that the maximum value of 1 bit occurs when the distribution is uniform, $\theta = 0.5$. A fair coin requires a single yes/no question to determine its state.

6.1.1.1 Application: DNA sequence logos

As an interesting application of entropy, consider the problem of representing **DNA sequence motifs**, which is a distribution over short DNA strings. We can estimate this distribution by aligning a set of DNA sequences (e.g., from different species), and then estimating the empirical distribution of each possible nucleotide from the 4 letter alphabet $X \sim \{A, C, G, T\}$ at each location t in the ith

sequence as follows:

$$\mathbf{N}_t = \left(\sum_{i=1}^{N} \mathbb{I}\left(X_{it} = A\right), \sum_{i=1}^{N} \mathbb{I}\left(X_{it} = C\right), \sum_{i=1}^{N} \mathbb{I}\left(X_{it} = G\right), \sum_{i=1}^{N} \mathbb{I}\left(X_{it} = T\right) \right) \tag{6.5}$$

$$\hat{\boldsymbol{\theta}}_t = \mathbf{N}_t / N, \tag{6.6}$$

This $\mathbf{N}_t$ is a length four vector counting the number of times each letter appears at each location amongst the set of sequences. This $\hat{\boldsymbol{\theta}}_t$ distribution is known as a **position weight matrix** or a **sequence motif**. We can visualize this as shown in Figure 6.2b. Here we plot the letters A, C, G and T, where the size of letter k at location t is proportional to the empirical frequency θ_{tk}.

An alternative visualization, known as a **sequence logo**, is shown in Figure 6.2c. Each column is scaled by $R_t = 2 - H_t$, where H_t is the entropy of $\hat{\boldsymbol{\theta}}_t$, and $2 = \log_2(4)$ is the maximum possible entropy for a distribution over 4 letters. Thus a deterministic distribution, which has entropy 0 and thus maximal information content, has height 2. Such informative locations are highly conserved by evolution, often because they are part of a gene coding region. We can also just compute the most probable letter in each location, regardless of the uncertainty; this is called the **consensus sequence**.

6.1.1.2 Estimating entropy

Estimating the entropy of a random variable with many possible states requires estimating its distribution, which can require a lot of data. For example, imagine if X represents the identity of a word in an English document. Since there is a long tail of rare words, and since new words are invented all the time, it can be difficult to reliably estimate $p(X)$ and hence $\mathbb{H}(X)$. For one possible solution to this problem, see [VV13].

6.1.2 Cross entropy

The **cross entropy** between distribution p and q is defined by

$$\mathbb{H}_{ce}(p, q) \triangleq -\sum_{k=1}^{K} p_k \log q_k \tag{6.7}$$

One can show that the cross entropy is the expected number of bits needed to compress some data samples drawn from distribution p using a code based on distribution q. This can be minimized by setting $q = p$, in which case the expected number of bits of the optimal code is $\mathbb{H}_{ce}(p, p) = \mathbb{H}(p)$ — this is known as **Shannon's source coding theorem** (see e.g., [CT06]).

6.1.3 Joint entropy

The joint entropy of two random variables X and Y is defined as

$$\mathbb{H}(X, Y) = -\sum_{x,y} p(x, y) \log_2 p(x, y) \tag{6.8}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

For example, consider choosing an integer from 1 to 8, $n \in \{1, \ldots, 8\}$. Let $X(n) = 1$ if n is even, and $Y(n) = 1$ if n is prime:

n	1	2	3	4	5	6	7	8
X	0	1	0	1	0	1	0	1
Y	0	1	1	0	1	0	1	0

The joint distribution is

$p(X,Y)$	$Y=0$	$Y=1$
$X=0$	$\frac{1}{8}$	$\frac{3}{8}$
$X=1$	$\frac{3}{8}$	$\frac{1}{8}$

so the joint entropy is given by

$$\mathbb{H}(X,Y) = -\left[\frac{1}{8}\log_2\frac{1}{8} + \frac{3}{8}\log_2\frac{3}{8} + \frac{3}{8}\log_2\frac{3}{8} + \frac{1}{8}\log_2\frac{1}{8}\right] = 1.81 \text{ bits} \tag{6.9}$$

Clearly the marginal probabilities are uniform: $p(X = 1) = p(X = 0) = p(Y = 0) = p(Y = 1) = 0.5$, so $\mathbb{H}(X) = \mathbb{H}(Y) = 1$. Hence $\mathbb{H}(X,Y) = 1.81$ bits $< \mathbb{H}(X) + \mathbb{H}(Y) = 2$ bits. In fact, this upper bound on the joint entropy holds in general. If X and Y are independent, then $\mathbb{H}(X,Y) = \mathbb{H}(X) + \mathbb{H}(Y)$, so the bound is tight. This makes intuitive sense: when the parts are correlated in some way, it reduces the "degrees of freedom" of the system, and hence reduces the overall entropy.

What is the lower bound on $\mathbb{H}(X,Y)$? If Y is a deterministic function of X, then $\mathbb{H}(X,Y) = \mathbb{H}(X)$. So

$$\mathbb{H}(X,Y) \geq \max\{\mathbb{H}(X), \mathbb{H}(Y)\} \geq 0 \tag{6.10}$$

Intuitively this says combining variables together does not make the entropy go down: you cannot reduce uncertainty merely by adding more unknowns to the problem, you need to observe some data, a topic we discuss in Section 6.1.4.

We can extend the definition of joint entropy from two variables to n in the obvious way.

6.1.4 Conditional entropy

The **conditional entropy** of Y given X is the uncertainty we have in Y after seeing X, averaged over possible values for X:

$$\mathbb{H}(Y|X) \triangleq \mathbb{E}_{p(X)}\left[\mathbb{H}(p(Y|X))\right] \tag{6.11}$$

$$= \sum_x p(x)\,\mathbb{H}(p(Y|X=x)) = -\sum_x p(x)\sum_y p(y|x)\log p(y|x) \tag{6.12}$$

$$= -\sum_{x,y} p(x,y)\log p(y|x) = -\sum_{x,y} p(x,y)\log\frac{p(x,y)}{p(x)} \tag{6.13}$$

$$= -\sum_{x,y} p(x,y)\log p(x,y) + \sum_x p(x)\log p(x) \tag{6.14}$$

$$= \mathbb{H}(X,Y) - \mathbb{H}(X) \tag{6.15}$$

If Y is a deterministic function of X, then knowing X completely determines Y, so $\mathbb{H}(Y|X) = 0$. If X and Y are independent, knowing X tells us nothing about Y and $\mathbb{H}(Y|X) = \mathbb{H}(Y)$. Since

$\mathbb{H}(X,Y) \leq \mathbb{H}(Y) + \mathbb{H}(X)$, we have

$$\mathbb{H}(Y|X) \leq \mathbb{H}(Y) \tag{6.16}$$

with equality iff X and Y are independent. This shows that, on average, conditioning on data never increases one's uncertainty. The caveat "on average" is necessary because for any *particular* observation (value of X), one may get more "confused" (i.e., $\mathbb{H}(Y|x) > \mathbb{H}(Y)$). However, in expectation, looking at the data is a good thing to do. (See also Section 6.3.8.)

We can rewrite Equation (6.15) as follows:

$$\mathbb{H}(X_1, X_2) = \mathbb{H}(X_1) + \mathbb{H}(X_2|X_1) \tag{6.17}$$

This can be generalized to get the **chain rule for entropy**:

$$\mathbb{H}(X_1, X_2, \ldots, X_n) = \sum_{i=1}^{n} \mathbb{H}(X_i|X_1, \ldots, X_{i-1}) \tag{6.18}$$

6.1.5 Perplexity

The **perplexity** of a discrete probability distribution p is defined as

$$\text{perplexity}(p) \triangleq 2^{\mathbb{H}(p)} \tag{6.19}$$

This is often interpreted as a measure of predictability. For example, suppose p is a uniform distribution over K states. In this case, the perplexity is K. Obviously the lower bound on perplexity is $2^0 = 1$, which will be achieved if the distribution can perfectly predict outcomes.

Now suppose we have an empirical distribution based on data $\mathcal{D}$:

$$p_{\mathcal{D}}(x|\mathcal{D}) = \frac{1}{N} \sum_{n=1}^{N} \delta(x - x_n) \tag{6.20}$$

We can measure how well p predicts $\mathcal{D}$ by computing

$$\text{perplexity}(p_{\mathcal{D}}, p) \triangleq 2^{\mathbb{H}_{ce}(p_{\mathcal{D}}, p)} \tag{6.21}$$

Perplexity is often used to evaluate the quality of statistical language models, which is a generative model for sequences of tokens. Suppose the data is a single long document x of length N, and suppose p is a simple unigram model. In this case, the cross entropy term is given by

$$H = -\frac{1}{N} \sum_{n=1}^{N} \log p(x_n) \tag{6.22}$$

and hence the perplexity is given by

$$\text{perplexity}(p_{\mathcal{D}}, p) = 2^H = 2^{-\frac{1}{N} \log(\prod_{n=1}^{N} p(x_n))} = \sqrt[N]{\prod_{n=1}^{N} \frac{1}{p(x_n)}} \tag{6.23}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is sometimes called the **exponentiated cross entropy**. We see that this is the geometric mean of the inverse predictive probabilities.

In the case of language models, we usually condition on previous words when predicting the next word. For example, in a bigram model, we use a first order Markov model of the form $p(x_i|x_{i-1})$. We define the **branching factor** of a language model as the number of possible words that can follow any given word. We can thus interpret the perplexity as the weighted average branching factor. For example, suppose the model predicts that each word is equally likely, regardless of context, so $p(x_i|x_{i-1}) = 1/K$. Then the perplexity is $((1/K)^N)^{-1/N} = K$. If some symbols are more likely than others, and the model correctly reflects this, its perplexity will be lower than K. However, as we show in Section 6.2, we have $\mathbb{H}(p^*) \leq \mathbb{H}_{ce}(p^*, p)$, so we can never reduce the perplexity below the entropy of the underlying stochastic process p^*.

See [JM08, p96] for further discussion of perplexity and its uses in language models.

6.1.6 Differential entropy for continuous random variables *

If X is a continuous random variable with pdf $p(x)$, we define the **differential entropy** as

$$h(X) \triangleq -\int_{\mathcal{X}} p(x) \log p(x)\, dx \tag{6.24}$$

assuming this integral exists. For example, suppose $X \sim U(0, a)$. Then

$$h(X) = -\int_0^a dx\, \frac{1}{a} \log \frac{1}{a} = \log a \tag{6.25}$$

Note that, unlike the discrete case, *differential entropy can be negative*. This is because pdf's can be bigger than 1. For example if $X \sim U(0, 1/8)$, we have $h(X) = \log_2(1/8) = -3$.

One way to understand differential entropy is to realize that all real-valued quantities can only be represented to finite precision. It can be shown [CT91, p228] that the entropy of an n-bit quantization of a continuous random variable X is approximately $h(X) + n$. For example, suppose $X \sim U(0, \frac{1}{8})$. Then in a binary representation of X, the first 3 bits to the right of the binary point must be 0 (since the number is $\leq 1/8$). So to describe X to n bits of accuracy only requires $n - 3$ bits, which agrees with $h(X) = -3$ calculated above.

6.1.6.1 Example: Entropy of a Gaussian

The entropy of a d-dimensional Gaussian is

$$h(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})) = \frac{1}{2} \ln |2\pi e \boldsymbol{\Sigma}| = \frac{1}{2} \ln[(2\pi e)^d |\boldsymbol{\Sigma}|] = \frac{d}{2} + \frac{d}{2} \ln(2\pi) + \frac{1}{2} \ln |\boldsymbol{\Sigma}| \tag{6.26}$$

In the 1d case, this becomes

$$h(\mathcal{N}(\mu, \sigma^2)) = \frac{1}{2} \ln \left[2\pi e \sigma^2\right] \tag{6.27}$$

6.1.6.2 Connection with variance

The entropy of a Gaussian increases monotonically as the variance increases. However, this is not always the case. For example, consider a mixture of two 1d Gaussians centered at -1 and +1. As we move the means further apart, say to -10 and +10, the variance increases (since the average distance from the overall mean gets larger). However, the entropy remains more or less the same, since we are still uncertain about where a sample might fall, even if we know that it will be near -10 or +10. (The exact entropy of a GMM is hard to compute, but a method to compute upper and lower bounds is presented in [Hub+08].)

6.1.6.3 Discretization

In general, computing the differential entropy for a continuous random variable can be difficult. A simple approximation is to **discretize** or **quantize** the variables. There are various methods for this (see e.g., [DKS95; KK06] for a summary), but a simple approach is to bin the distribution based on its empirical quantiles. The critical question is how many bins to use [LM04]. Scott [Sco79] suggested the following heuristic:

$$B = N^{1/3} \frac{\max(\mathcal{D}) - \min(\mathcal{D})}{3.5\sigma(\mathcal{D})} \tag{6.28}$$

where $\sigma(\mathcal{D})$ is the empirical standard deviation of the data, and $N = |\mathcal{D}|$ is the number of datapoints in the empirical distribution. However, the technique of discretization does not scale well if X is a multi-dimensional random vector, due to the curse of dimensionality.

6.2 Relative entropy (KL divergence) *

Given two distributions p and q, it is often useful to define a **distance metric** to measure how "close" or "similar" they are. In fact, we will be more general and consider a **divergence measure** $D(p, q)$ which quantifies how far q is from p, without requiring that D be a metric. More precisely, we say that D is a divergence if $D(p, q) \geq 0$ with equality iff $p = q$, whereas a metric also requires that D be symmetric and satisfy the **triangle inequality**, $D(p, r) \leq D(p, q) + D(q, r)$. There are many possible divergence measures we can use. In this section, we focus on the **Kullback-Leibler divergence** or **KL divergence**, also known as the **information gain** or **relative entropy**, between two distributions p and q.

6.2.1 Definition

For discrete distributions, the KL divergence is defined as follows:

$$D_{\mathbb{KL}}\left(p \parallel q\right) \triangleq \sum_{k=1}^{K} p_k \log \frac{p_k}{q_k} \tag{6.29}$$

This naturally extends to continuous distributions as well:

$$D_{\mathbb{KL}}\left(p \parallel q\right) \triangleq \int dx\, p(x) \log \frac{p(x)}{q(x)} \tag{6.30}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6.2.2 Interpretation

We can rewrite the KL as follows:

$$D_{\mathbb{KL}}\left(p \parallel q\right) = \underbrace{\sum_{k=1}^{K} p_k \log p_k}_{-\mathbb{H}(p)} \underbrace{- \sum_{k=1}^{K} p_k \log q_k}_{\mathbb{H}_{ce}(p,q)} \tag{6.31}$$

We recognize the first term as the negative entropy, and the second term as the cross entropy. It can be shown that the cross entropy $\mathbb{H}_{ce}(p, q)$ is a lower bound on the number of bits needed to compress data coming from distribution p if your code is designed based on distribution q; thus we can interpret the KL divergence as the "extra number of bits" you need to pay when compressing data samples if you use the incorrect distribution q as the basis of your coding scheme compared to the true distribution p.

There are various other interpretations of KL divergence. See the sequel to this book, [Mur23], for more information.

6.2.3 Example: KL divergence between two Gaussians

For example, one can show that the KL divergence between two multivariate Gaussian distributions is given by

$$\begin{aligned} &D_{\mathbb{KL}}\left(\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) \parallel \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)\right) \\ &= \frac{1}{2}\left[\mathrm{tr}(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^{\mathsf{T}}\boldsymbol{\Sigma}_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - D + \log\left(\frac{\det(\boldsymbol{\Sigma}_2)}{\det(\boldsymbol{\Sigma}_1)}\right)\right] \end{aligned} \tag{6.32}$$

In the scalar case, this becomes

$$D_{\mathbb{KL}}\left(\mathcal{N}(x|\mu_1, \sigma_1) \parallel \mathcal{N}(x|\mu_2, \sigma_2)\right) = \log\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} \tag{6.33}$$

6.2.4 Non-negativity of KL

In this section, we prove that the KL divergence is always non-negative.

To do this, we use **Jensen's inequality**. This states that, for any convex function f, we have that

$$f(\sum_{i=1}^{n} \lambda_i \boldsymbol{x}_i) \leq \sum_{i=1}^{n} \lambda_i f(\boldsymbol{x}_i) \tag{6.34}$$

where $\lambda_i \geq 0$ and $\sum_{i=1}^{n} \lambda_i = 1$. In words, this result says that f of the average is less than the average of the f's. This is clearly true for $n = 2$, since a convex function curves up above a straight line connecting the two end points (see Section 8.1.3). To prove for general n, we can use induction.

For example, if $f(x) = \log(x)$, which is a concave function, we have

$$\log(\mathbb{E}_x g(x)) \geq \mathbb{E}_x \log(g(x)) \tag{6.35}$$

We use this result below.

Theorem 6.2.1. *(Information inequality)* $D_{\mathbb{KL}}(p \parallel q) \geq 0$ *with equality iff* $p = q$.

Proof. We now prove the theorem following [CT06, p28]. Let $A = \{x : p(x) > 0\}$ be the support of $p(x)$. Using the concavity of the log function and Jensen's inequality (Section 6.2.4), we have that

$$-D_{\mathbb{KL}}(p \parallel q) = -\sum_{x \in A} p(x) \log \frac{p(x)}{q(x)} = \sum_{x \in A} p(x) \log \frac{q(x)}{p(x)} \tag{6.36}$$

$$\leq \log \sum_{x \in A} p(x) \frac{q(x)}{p(x)} = \log \sum_{x \in A} q(x) \tag{6.37}$$

$$\leq \log \sum_{x \in \mathcal{X}} q(x) = \log 1 = 0 \tag{6.38}$$

Since $\log(x)$ is a strictly concave function ($-\log(x)$ is convex), we have equality in Equation (6.37) iff $p(x) = cq(x)$ for some c that tracks the fraction of the whole space $\mathcal{X}$ contained in A. We have equality in Equation (6.38) iff $\sum_{x \in A} q(x) = \sum_{x \in \mathcal{X}} q(x) = 1$, which implies $c = 1$. Hence $D_{\mathbb{KL}}(p \parallel q) = 0$ iff $p(x) = q(x)$ for all x. □

This theorem has many important implications, as we will see throughout the book. For example, we can show that the uniform distribution is the one that maximizes the entropy:

Corollary 6.2.1. *(Uniform distribution maximizes the entropy)* $\mathbb{H}(X) \leq \log |\mathcal{X}|$, *where* $|\mathcal{X}|$ *is the number of states for* X, *with equality iff* $p(x)$ *is uniform.*

Proof. Let $u(x) = 1/|\mathcal{X}|$. Then

$$0 \leq D_{\mathbb{KL}}(p \parallel u) = \sum_x p(x) \log \frac{p(x)}{u(x)} = \log |\mathcal{X}| - \mathbb{H}(X) \tag{6.39}$$

□

6.2.5 KL divergence and MLE

Suppose we want to find the distribution q that is as close as possible to p, as measured by KL divergence:

$$q^* = \arg\min_q D_{\mathbb{KL}}(p \parallel q) = \arg\min_q \int p(x) \log p(x) dx - \int p(x) \log q(x) dx \tag{6.40}$$

Now suppose p is the empirical distribution, which puts a probability atom on the observed training data and zero mass everywhere else:

$$p_{\mathcal{D}}(x) = \frac{1}{N} \sum_{n=1}^{N} \delta(x - x_n) \tag{6.41}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Using the sifting property of delta functions we get

$$D_{\mathbb{KL}}\left(p_{\mathcal{D}} \parallel q\right) = -\int p_{\mathcal{D}}(x) \log q(x) dx + C \tag{6.42}$$

$$= -\int \left[\frac{1}{N}\sum_n \delta(x - x_n)\right] \log q(x) dx + C \tag{6.43}$$

$$= -\frac{1}{N}\sum_n \log q(x_n) + C \tag{6.44}$$

where $C = \int p(x) \log p(x) dx$ is a constant independent of q. This is called the **cross entropy** objective, and is equal to the average negative log likelihood of q on the training set. Thus we see that minimizing KL divergence to the empirical distribution is equivalent to maximizing likelihood.

This perspective points out the flaw with likelihood-based training, namely that it puts too much weight on the training set. In most applications, we do not really believe that the empirical distribution is a good representation of the true distribution, since it just puts "spikes" on a finite set of points, and zero density everywhere else. Even if the dataset is large (say 1M images), the universe from which the data is sampled is usually even larger (e.g., the set of "all natural images" is much larger than 1M). We could smooth the empirical distribution using kernel density estimation (Section 16.3), but that would require a similar kernel on the space of images. An alternative, algorithmic approach is to use **data augmentation**, which is a way of perturbing the observed data samples in way that we believe reflects plausible "natural variation". Applying MLE on this augmented dataset often yields superior results, especially when fitting models with many parameters (see Section 19.1).

6.2.6 Forward vs reverse KL

Suppose we want to approximate a distribution p using a simpler distribution q. We can do this by minimizing $D_{\mathbb{KL}}\left(q \parallel p\right)$ or $D_{\mathbb{KL}}\left(p \parallel q\right)$. This gives rise to different behavior, as we discuss below.

First we consider the **forwards KL**, also called the **inclusive KL**, defined by

$$D_{\mathbb{KL}}\left(p \parallel q\right) = \int p(x) \log \frac{p(x)}{q(x)} dx \tag{6.45}$$

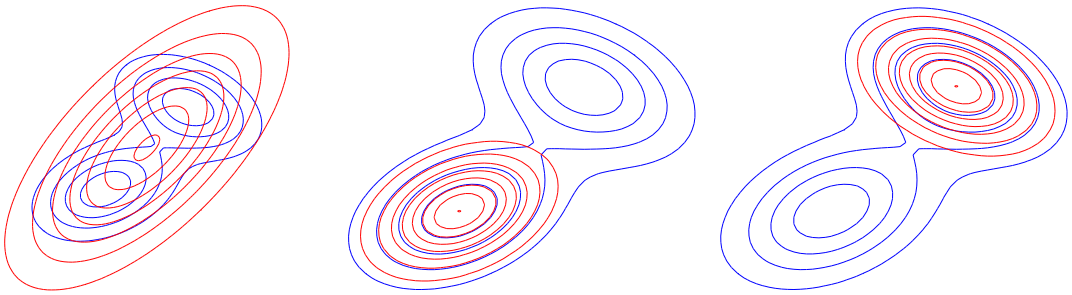
Minimizing this wrt q is known as an **M-projection** or **moment projection**.

We can gain an understanding of the optimal q by considering inputs x for which $p(x) > 0$ but $q(x) = 0$. In this case, the term $\log p(x)/q(x)$ will be infinite. Thus minimizing the KL will force q to include all the areas of space for which p has non-zero probability. Put another way, q will be **zero-avoiding** or **mode-covering**, and will typically over-estimate the support of p. Figure 6.3(a) illustrates mode covering where p is a bimodal distribution but q is unimodal.

Now consider the **reverse KL**, also called the **exclusive KL**:

$$D_{\mathbb{KL}}\left(q \parallel p\right) = \int q(x) \log \frac{q(x)}{p(x)} dx \tag{6.46}$$

Minimizing this wrt q is known as an **I-projection** or **information projection**.



(a) (b) (c)

Figure 6.3: Illustrating forwards vs reverse KL on a bimodal distribution. The blue curves are the contours of the true distribution p. The red curves are the contours of the unimodal approximation q. (a) Minimizing forwards KL, $D_{\mathbb{KL}}(p \parallel q)$, wrt q causes q to "cover" p. (b-c) Minimizing reverse KL, $D_{\mathbb{KL}}(q \parallel p)$ wrt q causes q to "lock onto" one of the two modes of p. Adapted from Figure 10.3 of [Bis06]. Generated by KLfwdReverseMixGauss.ipynb.

We can gain an understanding of the optimal q by consider inputs x for which $p(x) = 0$ but $q(x) > 0$. In this case, the term $\log q(x)/p(x)$ will be infinite. Thus minimizing the exclusive KL will force q to exclude all the areas of space for which p has zero probability. One way to do this is for q to put probability mass in very few parts of space; this is called **zero-forcing** or **mode-seeking** behavior. In this case, q will typically under-estimate the support of p. We illustrate mode seeking when p is bimodal but q is unimodal in Figure 6.3(b-c).

6.3 Mutual information *

The KL divergence gave us a way to measure how similar two distributions were. How should we measure how dependant two random variables are? One thing we could do is turn the question of measuring the dependence of two random variables into a question about the similarity of their distributions. This gives rise to the notion of **mutual information** (MI) between two random variables, which we define below.

6.3.1 Definition

The mutual information between rv's X and Y is defined as follows:

$$\mathbb{I}(X;Y) \triangleq D_{\mathbb{KL}}(p(x,y) \parallel p(x)p(y)) = \sum_{y \in Y} \sum_{x \in X} p(x,y) \log \frac{p(x,y)}{p(x)\,p(y)} \tag{6.47}$$

(We write $\mathbb{I}(X;Y)$ instead of $\mathbb{I}(X,Y)$, in case X and/or Y represent sets of variables; for example, we can write $\mathbb{I}(X;Y,Z)$ to represent the MI between X and (Y,Z).) For continuous random variables, we just replace sums with integrals.

It is easy to see that MI is always non-negative, even for continuous random variables, since

$$\mathbb{I}(X;Y) = D_{\mathbb{KL}}(p(x,y) \parallel p(x)p(y)) \geq 0 \tag{6.48}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We achieve the bound of 0 iff $p(x, y) = p(x)p(y)$.

6.3.2 Interpretation

Knowing that the mutual information is a KL divergence between the joint and factored marginal distributions tells us that the MI measures the information gain if we update from a model that treats the two variables as independent $p(x)p(y)$ to one that models their true joint density $p(x, y)$.

To gain further insight into the meaning of MI, it helps to re-express it in terms of joint and conditional entropies, as follows:

$$\mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y) = \mathbb{H}(Y) - \mathbb{H}(Y|X) \tag{6.49}$$

Thus we can interpret the MI between X and Y as the reduction in uncertainty about X after observing Y, or, by symmetry, the reduction in uncertainty about Y after observing X. Incidentally, this result gives an alternative proof that conditioning, on average, reduces entropy. In particular, we have $0 \leq \mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y)$, and hence $\mathbb{H}(X|Y) \leq \mathbb{H}(X)$.

We can also obtain a different interpretation. One can show that

$$\mathbb{I}(X;Y) = \mathbb{H}(X,Y) - \mathbb{H}(X|Y) - \mathbb{H}(Y|X) \tag{6.50}$$

Finally, one can show that

$$\mathbb{I}(X;Y) = \mathbb{H}(X) + \mathbb{H}(Y) - \mathbb{H}(X,Y) \tag{6.51}$$

See Figure 6.4 for a summary of these equations in terms of an **information diagram**. (Formally, this is a signed measure mapping set expressions to their information-theoretic counterparts [Yeu91].)

6.3.3 Example

As an example, let us reconsider the example concerning prime and even numbers from Section 6.1.3. Recall that $\mathbb{H}(X) = \mathbb{H}(Y) = 1$. The conditional distribution $p(Y|X)$ is given by normalizing each row:

	Y=0	Y=1
X=0	$\frac{1}{4}$	$\frac{3}{4}$
X=1	$\frac{3}{4}$	$\frac{1}{4}$

Hence the conditional entropy is

$$\mathbb{H}(Y|X) = -\left[\frac{1}{8}\log_2\frac{1}{4} + \frac{3}{8}\log_2\frac{3}{4} + \frac{3}{8}\log_2\frac{3}{4} + \frac{1}{8}\log_2\frac{1}{4}\right] = 0.81 \text{ bits} \tag{6.52}$$

and the mutual information is

$$\mathbb{I}(X;Y) = \mathbb{H}(Y) - \mathbb{H}(Y|X) = (1 - 0.81) \text{ bits} = 0.19 \text{ bits} \tag{6.53}$$

You can easily verify that

$$\mathbb{H}(X,Y) = \mathbb{H}(X|Y) + \mathbb{I}(X;Y) + \mathbb{H}(Y|X) \tag{6.54}$$

$$= (0.81 + 0.19 + 0.81) \text{ bits} = 1.81 \text{ bits} \tag{6.55}$$

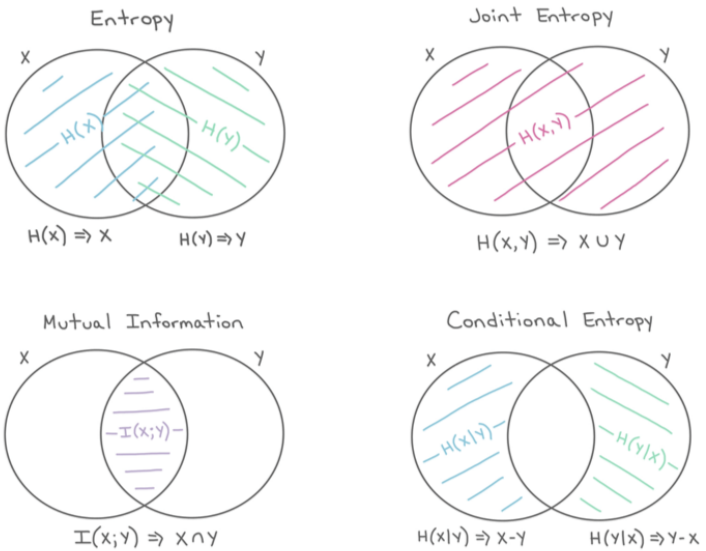


Figure 6.4: The marginal entropy, joint entropy, conditional entropy and mutual information represented as information diagrams. Used with kind permission of Katie Everett.

6.3.4 Conditional mutual information

We can define the **conditional mutual information** in the obvious way

$$\begin{aligned}
\mathbb{I}(X;Y|Z) &\triangleq \mathbb{E}_{p(Z)}\left[\mathbb{I}(X;Y)|Z\right] && (6.56)\\
&= \mathbb{E}_{p(x,y,z)}\left[\log \frac{p(x,y|z)}{p(x|z)p(y|z)}\right] && (6.57)\\
&= \mathbb{H}(X|Z) + \mathbb{H}(Y|Z) - \mathbb{H}(X,Y|Z) && (6.58)\\
&= \mathbb{H}(X|Z) - \mathbb{H}(X|Y,Z) = \mathbb{H}(Y|Z) - \mathbb{H}(Y|X,Z) && (6.59)\\
&= \mathbb{H}(X,Z) + \mathbb{H}(Y,Z) - \mathbb{H}(Z) - \mathbb{H}(X,Y,Z) && (6.60)\\
&= \mathbb{I}(Y;X,Z) - \mathbb{I}(Y;Z) && (6.61)
\end{aligned}$$

The last equation tells us that the conditional MI is the extra (residual) information that X tells us about Y, excluding what we already knew about Y given Z alone.

We can rewrite Equation (6.61) as follows:

$$\mathbb{I}(Z,Y;X) = \mathbb{I}(Z;X) + \mathbb{I}(Y;X|Z) \tag{6.62}$$

Generalizing to N variables, we get the **chain rule for mutual information**:

$$\mathbb{I}(Z_1,\ldots,Z_N;X) = \sum_{n=1}^{N} \mathbb{I}(Z_n;X|Z_1,\ldots,Z_{n-1}) \tag{6.63}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6.3.5 MI as a "generalized correlation coefficient"

Suppose that (x, y) are jointly Gaussian:

$$\begin{pmatrix} x \\ y \end{pmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{pmatrix} \sigma^2 & \rho\sigma^2 \\ \rho\sigma^2 & \sigma^2 \end{pmatrix}\right) \tag{6.64}$$

We now show how to compute the mutual information between X and Y.

Using Equation (6.26), we find that the entropy is

$$h(X, Y) = \frac{1}{2} \log\left[(2\pi e)^2 \det \Sigma\right] = \frac{1}{2} \log\left[(2\pi e)^2 \sigma^4 (1 - \rho^2)\right] \tag{6.65}$$

Since X and Y are individually normal with variance σ^2, we have

$$h(X) = h(Y) = \frac{1}{2} \log\left[2\pi e \sigma^2\right] \tag{6.66}$$

Hence

$$\begin{aligned} I(X, Y) &= h(X) + h(Y) - h(X, Y) && (6.67)\\ &= \log[2\pi e\sigma^2] - \frac{1}{2}\log[(2\pi e)^2\sigma^4(1-\rho^2)] && (6.68)\\ &= \frac{1}{2}\log[(2\pi e\sigma^2)^2] - \frac{1}{2}\log[(2\pi e\sigma^2)^2(1-\rho^2)] && (6.69)\\ &= \frac{1}{2}\log\frac{1}{1-\rho^2} = -\frac{1}{2}\log[1-\rho^2] && (6.70) \end{aligned}$$

We now discuss some interesting special cases.

1. $\rho = 1$. In this case, $X = Y$, and $I(X, Y) = \infty$, which makes sense. Observing Y tells us an infinite amount of information about X (as we know its real value exactly).
2. $\rho = 0$. In this case, X and Y are independent, and $I(X, Y) = 0$, which makes sense. Observing Y tells us nothing about X.
3. $\rho = -1$. In this case, $X = -Y$, and $I(X, Y) = \infty$, which again makes sense. Observing Y allows us to predict X to infinite precision.

Now consider the case where X and Y are scalar, but not jointly Gaussian. In general it can be difficult to compute the mutual information between continuous random variables, because we have to estimate the joint density $p(X, Y)$. For scalar variables, a simple approximation is to **discretize** or **quantize** them, by dividing the ranges of each variable into bins, and computing how many values fall in each histogram bin [Sco79]. We can then easily compute the MI using the empirical pmf.

Unfortunately, the number of bins used, and the location of the bin boundaries, can have a significant effect on the results. One way to avoid this is to use K-nearest neighbor distances to estimate densities in a non-parametric, adaptive way. This is the basis of the **KSG estimator** for MI proposed in [KSG04]. This is implemented in the sklearn.feature_selection.mutual_info_regression function. For papers related to this estimator, see [GOV18; HN19].

6.3.6 Normalized mutual information

For some applications, it is useful to have a normalized measure of dependence, between 0 and 1. We now discuss one way to construct such a measure.

First, note that

$$\mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y) \leq \mathbb{H}(X) \tag{6.71}$$

$$= \mathbb{H}(Y) - \mathbb{H}(Y|X) \leq \mathbb{H}(Y) \tag{6.72}$$

so

$$0 \leq \mathbb{I}(X;Y) \leq \min(\mathbb{H}(X), \mathbb{H}(Y)) \tag{6.73}$$

Therefore we can define the **normalized mutual information** as follows:

$$NMI(X,Y) = \frac{\mathbb{I}(X;Y)}{\min(\mathbb{H}(X), \mathbb{H}(Y))} \leq 1 \tag{6.74}$$

This normalized mutual information ranges from 0 to 1. When $NMI(X,Y) = 0$, we have $\mathbb{I}(X;Y) = 0$, so X and Y are independent. When $NMI(X,Y) = 1$, and $\mathbb{H}(X) < \mathbb{H}(Y)$, we have

$$\mathbb{I}(X;Y) = \mathbb{H}(X) - \mathbb{H}(X|Y) = \mathbb{H}(X) \implies \mathbb{H}(X|Y) = 0 \tag{6.75}$$

and so X is a deterministic function of Y. For example, suppose X is a discrete random variable with pmf $[0.5, 0.25, 0.25]$. We have $MI(X,X) = 1.5$ (using log base 2), and $H(X) = 1.5$, so the normalized MI is 1, as is to be expected.

For continuous random variables, it is harder to normalize the mutual information, because of the need to estimate the differential entropy, which is sensitive to the level of quantization. See Section 6.3.7 for further discussion.

6.3.7 Maximal information coefficient

As we discussed in Section 6.3.6, it is useful to have a normalized estimate of the mutual information, but this can be tricky to compute for real-valued data. One approach, known as the maximal information coefficient (MIC) [Res+11], is to define the following quantity:

$$\text{MIC}(X,Y) = \max_G \frac{\mathbb{I}((X,Y)|_G)}{\log ||G||} \tag{6.76}$$

where G is the set of 2d grids, and $(X,Y)|_G$ represents a discretization of the variables onto this grid, and $||G||$ is $\min(G_x, G_y)$, where G_x is the number of grid cells in the x direction, and G_y is the number of grid cells in the y direction. (The maximum grid resolution depends on the sample size n; they suggest restricting grids so that $G_x G_y \leq B(n)$, where $B(n) = n^\alpha$, where $\alpha = 0.6$.) The denominator is the entropy of a uniform joint distribution; dividing by this ensures $0 \leq \text{MIC} \leq 1$.

The intuition behind this statistic is the following: if there is a relationship between X and Y, then there should be some discrete gridding of the 2d input space that captures this. Since we don't know the correct grid to use, MIC searches over different grid resolutions (e.g., 2x2, 2x3, etc), as well as over locations of the grid boundaries. Given a grid, it is easy to quantize the data and compute

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

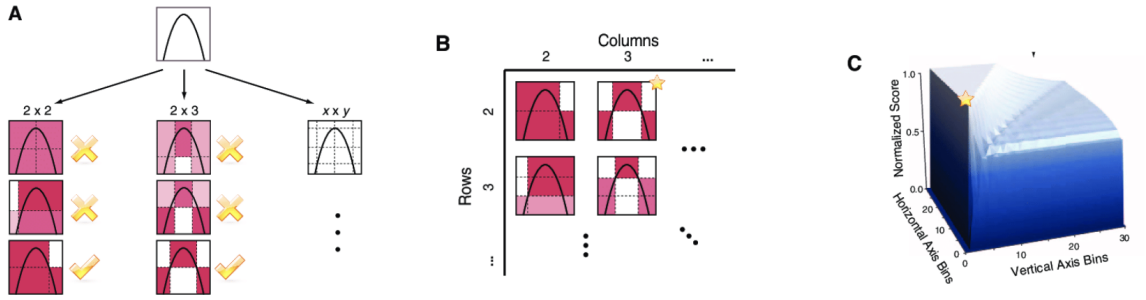


Figure 6.5: Illustration of how the maximal information coefficient (MIC) is computed. (a) We search over different grid resolutions, and grid cell locations, and compute the MI for each. (b) For each grid resolution (k, l), we define set $M(k, l)$ to be the maximum MI for any grid of that size, normalized by $\log(\min(k, l))$. *(c) We visualize the matrix* $\mathbf{M}$. *The maximum entry (denoted by a star) is defined to be the MIC. From Figure 1 of [Res+11]. Used with kind permission of David Reshef.*

MI. We define the **characteristic matrix** $M(k, l)$ to be the maximum MI achievable by any grid of size (k, l), normalized by $\log(\min(k, l))$. The MIC is then the maximum entry in this matrix, $\max_{kl \le B(n)} M(k, l)$. See Figure 6.5 for a visualization of this process.

In [Res+11], they show that this quantity exhibits a property known as **equitability**, which means that it gives similar scores to equally noisy relationships, regardless of the type of relationship (e.g., linear, non-linear, non-functional).

In [Res+16], they present an improved estimator, called **MICe**, which is more efficient to compute, and only requires optimizing over 1d grids, which can be done in $O(n)$ time using dynamic programming. They also present another quantity, called **TICe** (total information content), that has higher power to detect relationships from small sample sizes, but lower equitability. This is defined to be $\sum_{kl \le B(n)} M(k, l)$. They recommend using TICe to screen a large number of candidate relationships, and then using MICe to quantify the strength of the relationship. For an efficient implementation of both of these metrics, see [Alb+18].

We can interpret MIC of 0 to mean there is no relationship between the variables, and 1 to represent a noise-free relationship of any form. This is illustrated in Figure 6.6. Unlike correlation coefficients, MIC is not restricted to finding linear relationships. For this reason, the MIC has been called "a correlation for the 21st century" [Spe11].

In Figure 6.7, we give a more interesting example, from [Res+11]. The data consists of 357 variables measuring a variety of social, economic, health and political indicators, collected by the World Health Organization (WHO). On the left of the figure, we see the correlation coefficient (CC) plotted against the MIC for all 63,546 variable pairs. On the right of the figure, we see scatter plots for particular pairs of variables, which we now discuss:

- The point marked C (near 0,0 on the plot) has a low CC and a low MIC. The corresponding scatter plot makes it clear that there is no relationship between these two variables (percentage of lives lost to injury and density of dentists in the population).
- The points marked D and H have high CC (in absolute value) and high MIC, because they

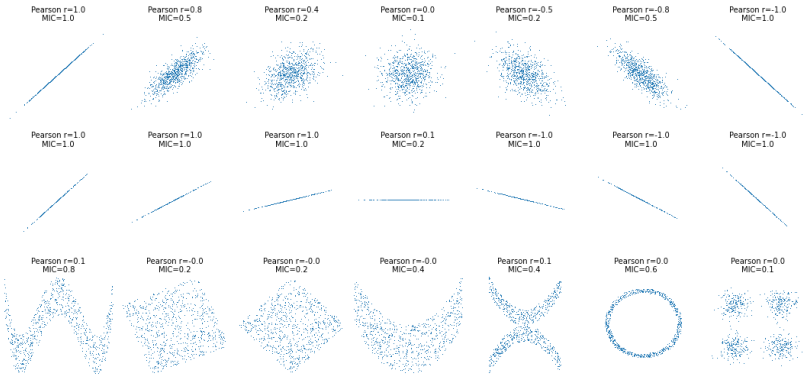


Figure 6.6: Plots of some 2d distributions and the corresponding estimate of correlation coefficient R^2 and the maximal information coefficient (MIC). Compare to Figure 3.1. Generated by MIC_correlation_2d.ipynb.

represent nearly linear relationships.

- The points marked E, F, and G have low CC but high MIC. This is because they correspond to non-linear (and sometimes, as in the case of E and F, non-functional, i.e., one-to-many) relationships between the variables.

6.3.8 Data processing inequality

Suppose we have an unknown variable X, and we observe a noisy function of it, call it Y. If we process the noisy observations in some way to create a new variable Z, it should be intuitively obvious that we cannot increase the amount of information we have about the unknown quantity, X. This is known as the **data processing inequality**. We now state this more formally, and then prove it.

Theorem 6.3.1. *Suppose $X \to Y \to Z$ forms a Markov chain, so that $X \perp Z|Y$. Then $\mathbb{I}(X;Y) \geq \mathbb{I}(X;Z)$.*

Proof. By the chain rule for mutual information (Equation (6.62)), we can expand the mutual information in two different ways:

$$\mathbb{I}(X;Y,Z) = \mathbb{I}(X;Z) + \mathbb{I}(X;Y|Z) \tag{6.77}$$
$$= \mathbb{I}(X;Y) + \mathbb{I}(X;Z|Y) \tag{6.78}$$

Since $X \perp Z|Y$, we have $\mathbb{I}(X;Z|Y) = 0$, so

$$\mathbb{I}(X;Z) + \mathbb{I}(X;Y|Z) = \mathbb{I}(X;Y) \tag{6.79}$$

Since $\mathbb{I}(X;Y|Z) \geq 0$, we have $\mathbb{I}(X;Y) \geq \mathbb{I}(X;Z)$. Similarly one can prove that $\mathbb{I}(Y;Z) \geq \mathbb{I}(X;Z)$. □

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

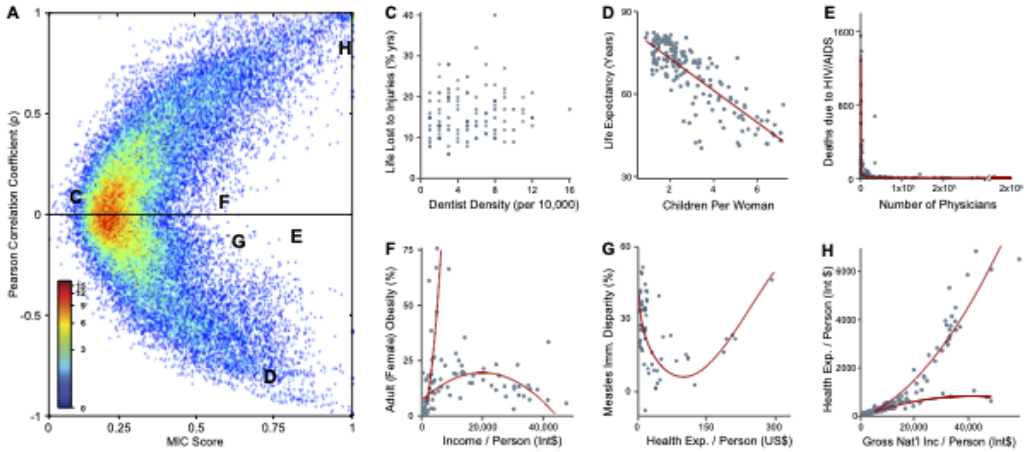


Figure 6.7: Left: Correlation coefficient vs maximal information criterion (MIC) for all pairwise relationships in the WHO data. Right: scatter plots of certain pairs of variables. The red lines are non-parametric smoothing regressions fit separately to each trend. From Figure 4 of [Res+11]. Used with kind permission of David Reshef.

6.3.9 Sufficient Statistics

An important consequence of the DPI is the following. Suppose we have the chain $\theta \to \mathcal{D} \to s(\mathcal{D})$. Then

$$\mathbb{I}(\theta; s(\mathcal{D})) \leq \mathbb{I}(\theta; \mathcal{D}) \tag{6.80}$$

If this holds with equality, then we say that $s(\mathcal{D})$ is a **sufficient statistic** of the data $\mathcal{D}$ for the purposes of inferring θ. In this case, we can equivalently write $\theta \to s(\mathcal{D}) \to \mathcal{D}$, since we can reconstruct the data from knowing $s(\mathcal{D})$ just as accurately as from knowing θ.

An example of a sufficient statistic is the data itself, $s(\mathcal{D}) = \mathcal{D}$, but this is not very useful, since it doesn't summarize the data at all. Hence we define a **minimal sufficient statistic** $s(\mathcal{D})$ as one which is sufficient, and which contains no extra information about θ; thus $s(\mathcal{D})$ maximally compresses the data $\mathcal{D}$ without losing information which is relevant to predicting θ. More formally, we say s is a *minimal* sufficient statistic for $\mathcal{D}$ if for all sufficient statistics $s'(\mathcal{D})$ there is some function f such that $s(\mathcal{D}) = f(s'(\mathcal{D}))$. We can summarize the situation as follows:

$$\theta \to s(\mathcal{D}) \to s'(\mathcal{D}) \to \mathcal{D} \tag{6.81}$$

Here $s'(\mathcal{D})$ takes $s(\mathcal{D})$ and adds redundant information to it, thus creating a one-to-many mapping.

For example, a minimal sufficient statistic for a set of N Bernoulli trials is simply N and $N_1 = \sum_n \mathbb{I}(X_n = 1)$, i.e., the number of successes. In other words, we don't need to keep track of the entire sequence of heads and tails and their ordering, we only need to keep track of the total number of heads and tails. Similarly, for inferring the mean of a Gaussian distribution with known variance we only need to know the empirical mean and number of samples.

6.3.10 Fano's inequality *

A common method for **feature selection** is to pick input features X_d which have high mutual information with the response variable Y. Below we justify why this is a reasonable thing to do. In particular, we state a result, known as **Fano's inequality**, which bounds the probability of misclassification (for any method) in terms of the mutual information between the features X and the class label Y.

Theorem 6.3.2. *(Fano's inequality) Consider an estimator $\hat{Y} = f(X)$ such that $Y \to X \to \hat{Y}$ forms a Markov chain. Let E be the event $\hat{Y} \neq Y$, indicating that an error occured, and let $P_e = P(Y \neq \hat{Y})$ be the probability of error. Then we have*

$$\mathbb{H}(Y|X) \leq \mathbb{H}\left(Y|\hat{Y}\right) \leq \mathbb{H}(E) + P_e \log |\mathcal{Y}| \tag{6.82}$$

Since $\mathbb{H}(E) \leq 1$, as we saw in Figure 6.1, we can weaken this result to get

$$1 + P_e \log |\mathcal{Y}| \geq \mathbb{H}(Y|X) \tag{6.83}$$

and hence

$$P_e \geq \frac{\mathbb{H}(Y|X) - 1}{\log |\mathcal{Y}|} \tag{6.84}$$

Thus minimizing $\mathbb{H}(Y|X)$ (which can be done by maximizing $\mathbb{I}(X;Y)$) will also minimize the lower bound on P_e.

Proof. (From [CT06, p38].) Using the chain rule for entropy, we have

$$\mathbb{H}\left(E, Y|\hat{Y}\right) = \mathbb{H}\left(Y|\hat{Y}\right) + \underbrace{\mathbb{H}\left(E|Y, \hat{Y}\right)}_{=0} \tag{6.85}$$

$$= \mathbb{H}\left(E|\hat{Y}\right) + \mathbb{H}\left(Y|E, \hat{Y}\right) \tag{6.86}$$

Since conditioning reduces entropy (see Section 6.2.4), we have $\mathbb{H}\left(E|\hat{Y}\right) \leq \mathbb{H}(E)$. The final term can be bounded as follows:

$$\mathbb{H}\left(Y|E, \hat{Y}\right) = P(E=0)\,\mathbb{H}\left(Y|\hat{Y}, E=0\right) + P(E=1)\,\mathbb{H}\left(Y|\hat{Y}, E=1\right) \tag{6.87}$$

$$\leq (1 - P_e)0 + P_e \log |\mathcal{Y}| \tag{6.88}$$

Hence

$$\mathbb{H}\left(Y|\hat{Y}\right) \leq \underbrace{\mathbb{H}\left(E|\hat{Y}\right)}_{\leq \mathbb{H}(E)} + \underbrace{\mathbb{H}\left(Y|E, \hat{Y}\right)}_{P_e \log |\mathcal{Y}|} \tag{6.89}$$

Finally, by the data processing inequality, we have $\mathbb{I}(Y;\hat{Y}) \leq \mathbb{I}(Y;X)$, so $\mathbb{H}(Y|X) \leq \mathbb{H}\left(Y|\hat{Y}\right)$, which establishes Equation (6.82). □

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

6.4 Exercises

Exercise 6.1 [Expressing mutual information in terms of entropies †]
Prove the following identities:

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) \tag{6.90}$$

and

$$H(X,Y) = H(X|Y) + H(Y|X) + I(X;Y) \tag{6.91}$$

Exercise 6.2 [Relationship between $D(p||q)$ and χ^2 statistic]
(Source: [CT91, Q12.2].)
Show that, if $p(x) \approx q(x)$, then

$$D_{\mathbb{KL}}\left(p \parallel q\right) \approx \frac{1}{2}\chi^2 \tag{6.92}$$

where

$$\chi^2 = \sum_x \frac{(p(x) - q(x))^2}{q(x)} \tag{6.93}$$

Hint: write

$$p(x) = \Delta(x) + q(x) \tag{6.94}$$

$$\frac{p(x)}{q(x)} = 1 + \frac{\Delta(x)}{q(x)} \tag{6.95}$$

and use the Taylor series expansion for $\log(1+x)$.

$$\log(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} \cdots \tag{6.96}$$

for $-1 < x \leq 1$.

Exercise 6.3 [Fun with entropies †]
(Source: Mackay.)
Consider the joint distribution $p(X, Y)$

		x			
		1	2	3	4
y	1	1/8	1/16	1/32	1/32
	2	1/16	1/8	1/32	1/32
	3	1/16	1/16	1/16	1/16
	4	1/4	0	0	0

a. What is the joint entropy $H(X, Y)$?
b. What are the marginal entropies $H(X)$ and $H(Y)$?
c. The entropy of X conditioned on a specific value of y is defined as

$$H(X|Y = y) = -\sum_x p(x|y) \log p(x|y) \tag{6.97}$$

Compute $H(X|y)$ for each value of y. Does the posterior entropy on X ever increase given an observation of Y?

d. The conditional entropy is defined as

$$H(X|Y) = \sum_y p(y) H(X|Y=y) \tag{6.98}$$

Compute this. Does the posterior entropy on X increase or decrease when averaged over the possible values of Y?

e. What is the mutual information between X and Y?

Exercise 6.4 [Forwards vs reverse KL divergence]

(Source: Exercise 33.7 of [Mac03].) Consider a factored approximation $q(x, y) = q(x)q(y)$ to a joint distribution $p(x, y)$. Show that to minimize the forwards KL $D_{\mathbb{KL}}(p \parallel q)$ we should set $q(x) = p(x)$ and $q(y) = p(y)$, i.e., the optimal approximation is a product of marginals.

Now consider the following joint distribution, where the rows represent y and the columns x.

	1	2	3	4
1	1/8	1/8	0	0
2	1/8	1/8	0	0
3	0	0	1/4	0
4	0	0	0	1/4

Show that the reverse KL $D_{\mathbb{KL}}(q \parallel p)$ for this p has three distinct minima. Identify those minima and evaluate $D_{\mathbb{KL}}(q \parallel p)$ at each of them. What is the value of $D_{\mathbb{KL}}(q \parallel p)$ if we set $q(x, y) = p(x)p(y)$?

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7 Linear Algebra

This chapter is co-authored with Zico Kolter.

7.1 Introduction

Linear algebra is the study of matrices and vectors. In this chapter, we summarize the key material that we will need throughout the book. Much more information can be found in other sources, such as [Str09; Ips09; Kle13; Mol04; TB97; Axl15; Tho17; Agg20; LLM14].

7.1.1 Notation

In this section, we define some notation.

7.1.1.1 Vectors

A **vector** $\boldsymbol{x} \in \mathbb{R}^n$ is a list of n numbers, usually written as a **column vector**

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \tag{7.1}$$

The vector of all ones is denoted $\mathbf{1}$. The vector of all zeros is denoted $\mathbf{0}$.
The **unit vector** $\boldsymbol{e}_i$ is a vector of all 0's, except entry i, which has value 1:

$$\boldsymbol{e}_i = (0, \ldots, 0, 1, 0, \ldots, 0) \tag{7.2}$$

This is also called a **one-hot vector**.

7.1.1.2 Matrices

A **matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ with m rows and n columns is a 2d array of numbers, arranged as follows:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}. \tag{7.3}$$

If $m = n$, the matrix is said to be **square**.

We use the notation A_{ij} or $A_{i,j}$ to denote the entry of $\mathbf{A}$ in the ith row and jth column. We use the notation $\mathbf{A}_{i,:}$ to denote the i'th row and $\mathbf{A}_{:,j}$ to denote the j'th column. We treat all vectors as column vectors by default (so $\mathbf{A}_{i,:}$ is viewed as a column vector with n entries). We use bold upper case letters to denote matrices, bold lower case letters to denote vectors, and non-bold letters to denote scalars.

We can view a matrix as a set of columns stacked along the horizontal axis:

$$\mathbf{A} = \begin{bmatrix} | & | & & | \\ \mathbf{A}_{:,1} & \mathbf{A}_{:,2} & \cdots & \mathbf{A}_{:,n} \\ | & | & & | \end{bmatrix}. \tag{7.4}$$

For brevity, we will denote this by

$$\mathbf{A} = [\mathbf{A}_{:,1}, \mathbf{A}_{:,2}, \ldots, \mathbf{A}_{:,n}] \tag{7.5}$$

We can also view a matrix as a set of rows stacked along the vertical axis:

$$\mathbf{A} = \begin{bmatrix} — & \mathbf{A}_{1,:}^\mathsf{T} & — \\ — & \mathbf{A}_{2,:}^\mathsf{T} & — \\ & \vdots & \\ — & \mathbf{A}_{m,:}^\mathsf{T} & — \end{bmatrix}. \tag{7.6}$$

For brevity, we will denote this by

$$\mathbf{A} = [\mathbf{A}_{1,:}; \mathbf{A}_{2,:}; \ldots; \mathbf{A}_{m,:}] \tag{7.7}$$

(Note the use of a semicolon.)

The **transpose** of a matrix results from "flipping" the rows and columns. Given a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, its transpose, written $\mathbf{A}^\mathsf{T} \in \mathbb{R}^{n \times m}$, is defined as

$$(\mathbf{A}^\mathsf{T})_{ij} = A_{ji} \tag{7.8}$$

The following properties of transposes are easily verified:

$$(\mathbf{A}^\mathsf{T})^\mathsf{T} = \mathbf{A} \tag{7.9}$$

$$(\mathbf{AB})^\mathsf{T} = \mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T} \tag{7.10}$$

$$(\mathbf{A} + \mathbf{B})^\mathsf{T} = \mathbf{A}^\mathsf{T} + \mathbf{B}^\mathsf{T} \tag{7.11}$$

If a square matrix satisfies $\mathbf{A} = \mathbf{A}^\mathsf{T}$, it is called **symmetric**. We denote the set of all symmetric matrices of size n as $\mathbb{S}^n$.

7.1.1.3 Tensors

A **tensor** (in machine learning terminology) is just a generalization of a 2d array to more than 2 dimensions, as illustrated in Figure 7.1. For example, the entries of a 3d tensor are denoted by A_{ijk}.

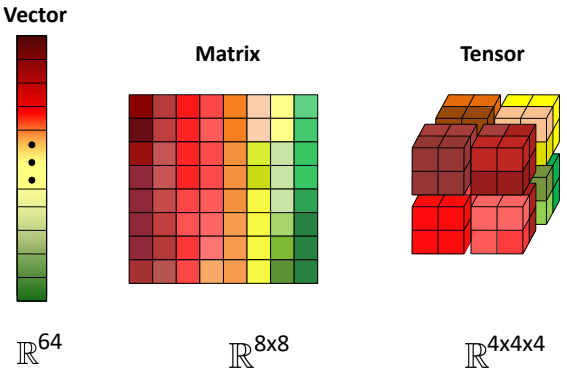


Figure 7.1: Illustration of a 1d vector, 2d matrix, and 3d tensor. The colors are used to represent individual entries of the vector; this list of numbers can also be stored in a 2d matrix, as shown. (In this example, the matrix is layed out in column-major order, which is the opposite of that used by Python.) We can also reshape the vector into a 3d tensor, as shown.



Figure 7.2: Illustration of (a) row-major vs (b) column-major order. From https://commons.wikimedia.org/wiki/File:Row_and_column_major_order.svg. Used with kind permission of Wikipedia author Cmglee.

The number of dimensions is known as the **order** or **rank** of the tensor.[1] In mathematics, tensors can be viewed as a way to define multilinear maps, just as matrices can be used to define linear functions, although we will not need to use this interpretation.

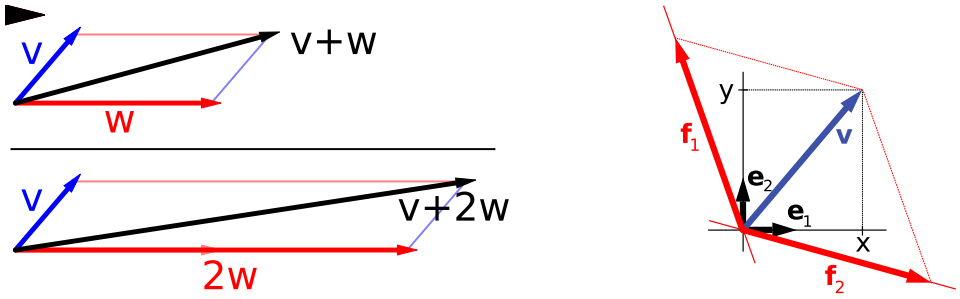
We can **reshape** a matrix into a vector by stacking its columns on top of each other, as shown in Figure 7.1. This is denoted by

$$\text{vec}(\mathbf{A}) = [\mathbf{A}_{:,1}; \cdots; \mathbf{A}_{:,n}] \in \mathbb{R}^{mn \times 1} \tag{7.12}$$

Conversely, we can reshape a vector into a matrix. There are two choices for how to do this, known as **row-major order** (used by languages such as Python and C++) and **column-major order** (used by languages such as Julia, Matlab, R and Fortran). See Figure 7.2 for an illustration of the difference.

1. Note, however, that the rank of a 2d matrix is a different concept, as discussed in Section 7.1.4.3.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 7.3: (a) Top: A vector $\boldsymbol{v}$ (blue) is added to another vector $\boldsymbol{w}$ (red). Bottom: $\boldsymbol{w}$ is stretched by a factor of 2, yielding the sum $\boldsymbol{v} + 2\boldsymbol{w}$. From https://en.wikipedia.org/wiki/Vector_space. Used with kind permission of Wikipedia author IkamusumeFan (b) A vector $\boldsymbol{v}$ in $\mathbb{R}^2$ (blue) expressed in terms of different bases: using the standard basis of $\mathbb{R}^2$, $\boldsymbol{v} = x\boldsymbol{e}_1 + y\boldsymbol{e}_2$ (black), and using a different, non-orthogonal basis: $\boldsymbol{v} = \boldsymbol{f}_1 + \boldsymbol{f}_2$ (red). From https://en.wikipedia.org/wiki/Vector_space. Used with kind permission of Wikiepdia author Jakob.scholbach.

7.1.2 Vector spaces

In this section, we discuss some fundamental concepts in linear algebra.

7.1.2.1 Vector addition and scaling

We can view a vector $\boldsymbol{x} \in \mathbb{R}^n$ as defining a point in n-dimensional Euclidean space. A **vector space** is a collection of such vectors, which can be added together, and scaled by **scalars** (1-dimensional numbers), in order to create new points. These operations are defined to operate elementwise, in the obvious way, namely $\boldsymbol{x} + \boldsymbol{y} = (x_1 + y_1, \ldots, x_n + y_n)$ and $c\boldsymbol{x} = (cx_1, \ldots, cx_n)$, where $c \in \mathbb{R}$. See Figure 7.3a for an illustration.

7.1.2.2 Linear independence, spans and basis sets

A set of vectors $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots \boldsymbol{x}_n\}$ is said to be **(linearly) independent** if no vector can be represented as a linear combination of the remaining vectors. Conversely, a vector which *can* be represented as a linear combination of the remaining vectors is said to be **(linearly) dependent**. For example, if

$$\boldsymbol{x}_n = \sum_{i=1}^{n-1} \alpha_i \boldsymbol{x}_i \tag{7.13}$$

for some $\{\alpha_1, \ldots, \alpha_{n-1}\}$ then $\boldsymbol{x}_n$ is dependent on $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1}\}$; otherwise, it is independent of $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1}\}$.

The **span** of a set of vectors $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n\}$ is the set of all vectors that can be expressed as a linear combination of $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$. That is,

$$\text{span}(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}) \triangleq \left\{ \boldsymbol{v} : \boldsymbol{v} = \sum_{i=1}^{n} \alpha_i \boldsymbol{x}_i, \quad \alpha_i \in \mathbb{R} \right\}. \tag{7.14}$$

It can be shown that if $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ is a set of n linearly independent vectors, where each $\boldsymbol{x}_i \in \mathbb{R}^n$, then $\text{span}(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}) = \mathbb{R}^n$. In other words, *any* vector $\boldsymbol{v} \in \mathbb{R}^n$ can be written as a linear combination of $\boldsymbol{x}_1$ through $\boldsymbol{x}_n$.

A **basis** $\mathcal{B}$ is a set of linearly independent vectors that spans the whole space, meaning that $\text{span}(\mathcal{B}) = \mathbb{R}^n$. There are often multiple bases to choose from, as illustrated in Figure 7.3b. The **standard basis** uses the **coordinate vectors** $\boldsymbol{e}_1 = (1, 0, \ldots, 0)$, up to $\boldsymbol{e}_n = (0, 0, \ldots, 0, 1)$. This lets us translate back and forth between viewing a vector in $\mathbb{R}^2$ as an either an "arrow in the plane", rooted at the origin, or as an ordered list of numbers (corresponding to the coefficients for each basis vector).

7.1.2.3 Linear maps and matrices

A **linear map** or **linear transformation** is any function $f : \mathcal{V} \to \mathcal{W}$ such that $f(\boldsymbol{v}+\boldsymbol{w}) = f(\boldsymbol{v})+f(\boldsymbol{w})$ and $f(a\ \boldsymbol{v}) = a\ f(\boldsymbol{v})$ for all $\boldsymbol{v}, \boldsymbol{w} \in \mathcal{V}$. Once the basis of $\mathcal{V}$ is chosen, a linear map $f : \mathcal{V} \to \mathcal{W}$ is completely determined by specifying the images of the basis vectors, because any element of $\mathcal{V}$ can be expressed uniquely as a linear combination of them.

Suppose $\mathcal{V} = \mathbb{R}^n$ and $\mathcal{W} = \mathbb{R}^m$. We can compute $f(\boldsymbol{v}_i) \in \mathbb{R}^m$ for each basis vector in $\mathcal{V}$, and store these along the columns of an $m \times n$ matrix $\mathbf{A}$. We can then compute $\boldsymbol{y} = f(\boldsymbol{x}) \in \mathbb{R}^m$ for any $\boldsymbol{x} \in \mathbb{R}^n$ as follows:

$$\boldsymbol{y} = \left(\sum_{j=1}^{n} a_{1j} x_j, \ldots, \sum_{j=1}^{n} a_{mj} x_j \right) \tag{7.15}$$

This corresponds to multiplying the vector $\boldsymbol{x}$ by the matrix $\mathbf{A}$:

$$\boldsymbol{y} = \mathbf{A}\boldsymbol{x} \tag{7.16}$$

See Section 7.2 for more details.

If the function is invertible, we can write

$$\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{y} \tag{7.17}$$

See Section 7.3 for details.

7.1.2.4 Range and nullspace of a matrix

Suppose we view a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ as a set of n vectors in $\mathbb{R}^m$. The **range** (sometimes also called the **column space**) of this matrix is the span of the columns of $\mathbf{A}$. In other words,

$$\text{range}(\mathbf{A}) \triangleq \{\boldsymbol{v} \in \mathbb{R}^m : \boldsymbol{v} = \mathbf{A}\boldsymbol{x}, \boldsymbol{x} \in \mathbb{R}^n\}. \tag{7.18}$$

This can be thought of as the set of vectors that can be "reached" or "generated" by $\mathbf{A}$; it is a subspace of $\mathbb{R}^m$ whose dimensionality is given by the rank of $\mathbf{A}$ (see Section 7.1.4.3). The **nullspace** of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the set of all vectors that get mapped to the null vector when multiplied by $\mathbf{A}$, i.e.,

$$\text{nullspace}(\mathbf{A}) \triangleq \{\boldsymbol{x} \in \mathbb{R}^n : \mathbf{A}\boldsymbol{x} = \mathbf{0}\}. \tag{7.19}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

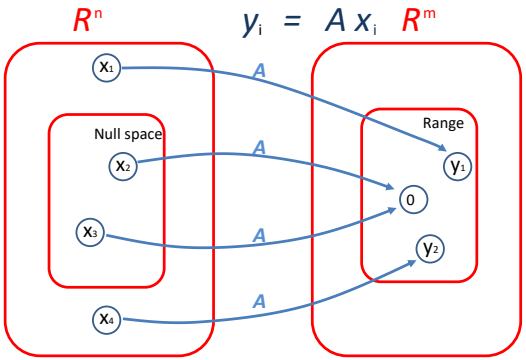


Figure 7.4: Visualization of the nullspace and range of an $m \times n$ matrix $\mathbf{A}$. Here $\boldsymbol{y}_1 = \mathbf{A}\boldsymbol{x}_1$ and $\boldsymbol{y}_2 = \mathbf{A}\boldsymbol{x}_4$, so $\boldsymbol{y}_1$ and $\boldsymbol{y}_2$ are in the range of $\mathbf{A}$ (are reachable from some $\boldsymbol{x}$). Also $\mathbf{A}\boldsymbol{x}_2 = \mathbf{0}$ and $\mathbf{A}\boldsymbol{x}_3 = \mathbf{0}$, so $\boldsymbol{x}_2$ and $\boldsymbol{x}_3$ are in the nullspace of $\mathbf{A}$ (get mapped to 0). We see that the range is often a subset of the input domain of the mapping.

The span of the rows of $\mathbf{A}$ is the complement to the nullspace of $\mathbf{A}$.

See Figure 7.4 for an illustration of the range and nullspace of a matrix. We shall discuss how to compute the range and nullspace of a matrix numerically in Section 7.5.4 below.

7.1.2.5 Linear projection

The **projection** of a vector $\boldsymbol{y} \in \mathbb{R}^m$ onto the span of $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ (here we assume $\boldsymbol{x}_i \in \mathbb{R}^m$) is the vector $\boldsymbol{v} \in \text{span}(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\})$, such that $\boldsymbol{v}$ is as close as possible to $\boldsymbol{y}$, as measured by the Euclidean norm $\|\boldsymbol{v} - \boldsymbol{y}\|_2$. We denote the projection as $\text{Proj}(\boldsymbol{y}; \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\})$ and can define it formally as

$$\text{Proj}(\boldsymbol{y}; \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}) = \text{argmin}_{\boldsymbol{v} \in \text{span}(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\})} \|\boldsymbol{y} - \boldsymbol{v}\|_2. \tag{7.20}$$

Given a (full rank) matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$, we can define the projection of a vector $\boldsymbol{y} \in \mathbb{R}^m$ onto the range of $\mathbf{A}$ as follows:

$$\text{Proj}(\boldsymbol{y}; \mathbf{A}) = \text{argmin}_{\boldsymbol{v} \in \mathcal{R}(\mathbf{A})} \|\boldsymbol{v} - \boldsymbol{y}\|_2 = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\boldsymbol{y} \ . \tag{7.21}$$

These are the same as the normal equations from Section 11.2.2.2.

7.1.3 Norms of a vector and matrix

In this section, we discuss ways of measuring the "size" of a vector and matrix.

7.1.3.1 Vector norms

A **norm** of a vector $\|\boldsymbol{x}\|$ is, informally, a measure of the "length" of the vector. More formally, a norm is any function $f : \mathbb{R}^n \to \mathbb{R}$ that satisfies 4 properties:

- For all $\boldsymbol{x} \in \mathbb{R}^n$, $f(\boldsymbol{x}) \geq 0$ (non-negativity).

- $f(\boldsymbol{x}) = 0$ if and only if $\boldsymbol{x} = \mathbf{0}$ (definiteness).
- For all $\boldsymbol{x} \in \mathbb{R}^n$, $t \in \mathbb{R}$, $f(t\boldsymbol{x}) = |t|f(x)$ (absolute value homogeneity).
- For all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, $f(\boldsymbol{x} + \boldsymbol{y}) \leq f(\boldsymbol{x}) + f(\boldsymbol{y})$ (triangle inequality).

Consider the following common examples:

p-norm $\|\boldsymbol{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$, for $p \geq 1$.

2-norm $\|\boldsymbol{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$, also called Euclidean norm. Note that $\|\boldsymbol{x}\|_2^2 = \boldsymbol{x}^\mathsf{T}\boldsymbol{x}$.

1-norm $\|\boldsymbol{x}\|_1 = \sum_{i=1}^n |x_i|$.

Max-norm $\|\boldsymbol{x}\|_\infty = \max_i |x_i|$.

0-norm $\|\boldsymbol{x}\|_0 = \sum_{i=1}^n \mathbb{I}(|x_i| > 0)$. This is a **pseudo norm**, since it does not satisfy homogeneity. It counts the number of non-zero elements in $\boldsymbol{x}$. If we define $0^0 = 0$, we can write this as $\|\boldsymbol{x}\|_0 = \sum_{i=1}^n x_i^0$.

7.1.3.2 Matrix norms

Suppose we think of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ as defining a linear function $f(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x}$. We define the **induced norm** of $\mathbf{A}$ as the maximum amount by which f can lengthen any unit-norm input:

$$||\mathbf{A}||_p = \max_{\boldsymbol{x} \neq \mathbf{0}} \frac{||\mathbf{A}\boldsymbol{x}||_p}{||\boldsymbol{x}||_p} = \max_{||\boldsymbol{x}||=1} ||\mathbf{A}\boldsymbol{x}||_p \tag{7.22}$$

Typically $p = 2$, in which case

$$||\mathbf{A}||_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\mathsf{T}\mathbf{A})} = \max_i \sigma_i \tag{7.23}$$

where $\lambda_{\max}(\mathbf{M})$ is the largest eigenvalue of $\mathbf{M}$, and σ_i is the i'th singular value.

The **nuclear norm**, also called the **trace norm**, is defined as

$$||\mathbf{A}||_* = \mathrm{tr}(\sqrt{\mathbf{A}^\mathsf{T}\mathbf{A}}) = \sum_i \sigma_i \tag{7.24}$$

where $\sqrt{\mathbf{A}^\mathsf{T}\mathbf{A}}$ is the matrix square root. Since the singular values are always non-negative, we have

$$||\mathbf{A}||_* = \sum_i |\sigma_i| = ||\boldsymbol{\sigma}||_1 \tag{7.25}$$

Using this as a regularizer encourages many singular values to become zero, resulting in a low rank matrix. More generally, we can define the **Schatten p-norm** as

$$||\mathbf{A}||_p = \left(\sum_i \sigma_i^p(\mathbf{A}) \right)^{1/p} \tag{7.26}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

If we think of a matrix as a vector, we can define the matrix norm in terms of a vector norm, $||\mathbf{A}|| = ||\text{vec}(\mathbf{A})||$. If the vector norm is the 2-norm, the corresponding matrix norm is the **Frobenius norm**:

$$||\mathbf{A}||_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2} = \sqrt{\text{tr}(\mathbf{A}^\mathsf{T}\mathbf{A})} = ||\text{vec}(\mathbf{A})||_2 \tag{7.27}$$

If $\mathbf{A}$ is expensive to evaluate, but $\mathbf{A}\boldsymbol{v}$ is cheap (for a random vector $\boldsymbol{v}$), we can create a stochastic approximation to the Frobenius norm by using the Hutchinson trace estimator from Equation (7.37) as follows:

$$||\mathbf{A}||_F^2 = \text{tr}(\mathbf{A}^\mathsf{T}\mathbf{A}) = \mathbb{E}\left[\boldsymbol{v}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{A}\boldsymbol{v}\right] = \mathbb{E}\left[||\mathbf{A}\boldsymbol{v}||_2^2\right] \tag{7.28}$$

where $\boldsymbol{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

7.1.4 Properties of a matrix

In this section, we discuss various scalar properties of matrices.

7.1.4.1 Trace of a square matrix

The **trace** of a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, denoted $\text{tr}(\mathbf{A})$, is the sum of diagonal elements in the matrix:

$$\text{tr}(\mathbf{A}) \triangleq \sum_{i=1}^{n} A_{ii}. \tag{7.29}$$

The trace has the following properties, where $c \in \mathbb{R}$ is a scalar, and $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ are square matrices:

$$\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}^\mathsf{T}) \tag{7.30}$$

$$\text{tr}(\mathbf{A}+\mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B}) \tag{7.31}$$

$$\text{tr}(c\mathbf{A}) = c\,\text{tr}(\mathbf{A}) \tag{7.32}$$

$$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA}) \tag{7.33}$$

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i \text{ where } \lambda_i \text{ are the eigenvalues of } \mathbf{A} \tag{7.34}$$

We also have the following important **cyclic permutation property**: For $\mathbf{A}, \mathbf{B}, \mathbf{C}$ such that $\mathbf{ABC}$ is square,

$$\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB}) \tag{7.35}$$

From this, we can derive the **trace trick**, which rewrites the scalar inner product $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}$ as follows

$$\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} = \text{tr}(\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}) = \text{tr}(\boldsymbol{x}\boldsymbol{x}^\mathsf{T}\mathbf{A}) \tag{7.36}$$

In some cases, it may be expensive to evaluate the matrix $\mathbf{A}$, but we may be able to cheaply evaluate matrix-vector products $\mathbf{A}\boldsymbol{v}$. Suppose $\boldsymbol{v}$ is a random vector such that $\mathbb{E}\left[\boldsymbol{v}\boldsymbol{v}^\mathsf{T}\right] = \mathbf{I}$. In this case, we can create a Monte Carlo approximation to $\mathrm{tr}(\mathbf{A})$ using the following identity:

$$\mathrm{tr}(\mathbf{A}) = \mathrm{tr}(\mathbf{A}\mathbb{E}\left[\boldsymbol{v}\boldsymbol{v}^\mathsf{T}\right]) = \mathbb{E}\left[\mathrm{tr}(\mathbf{A}\boldsymbol{v}\boldsymbol{v}^\mathsf{T})\right] = \mathbb{E}\left[\mathrm{tr}(\boldsymbol{v}^\mathsf{T}\mathbf{A}\boldsymbol{v})\right] = \mathbb{E}\left[\boldsymbol{v}^\mathsf{T}\mathbf{A}\boldsymbol{v}\right] \tag{7.37}$$

This is called the **Hutchinson trace estimator** [Hut90].

7.1.4.2 Determinant of a square matrix

The **determinant** of a square matrix, denoted $\det(\mathbf{A})$ or $|\mathbf{A}|$, is a measure of how much it changes a unit volume when viewed as a linear transformation. (The formal definition is rather complex and is not needed here.)

The determinant operator satisfies these properties, where $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$

$$|\mathbf{A}| = |\mathbf{A}^\mathsf{T}| \tag{7.38}$$

$$|c\mathbf{A}| = c^n|\mathbf{A}| \tag{7.39}$$

$$|\mathbf{A}\mathbf{B}| = |\mathbf{A}||\mathbf{B}| \tag{7.40}$$

$$|\mathbf{A}| = 0 \text{ iff } \mathbf{A} \text{ is singular} \tag{7.41}$$

$$|\mathbf{A}^{-1}| = 1/|\mathbf{A}| \text{ if } \mathbf{A} \text{ is not singular} \tag{7.42}$$

$$|\mathbf{A}| = \prod_{i=1}^{n} \lambda_i \text{ where } \lambda_i \text{ are the eigenvalues of } \mathbf{A} \tag{7.43}$$

For a positive definite matrix $\mathbf{A}$, we can write $\mathbf{A} = \mathbf{L}\mathbf{L}^\mathsf{T}$, where $\mathbf{L}$ is the lower triangular Cholesky decomposition. In this case, we have

$$\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{L}^\mathsf{T}) = \det(\mathbf{L})^2 \tag{7.44}$$

so

$$\log\det(\mathbf{A}) = 2\log\det(\mathbf{L}) = 2\log\prod_i L_{ii} = 2\mathrm{tr}(\log(\mathrm{diag}(\mathbf{L}))) \tag{7.45}$$

7.1.4.3 Rank of a matrix

The **column rank** of a matrix $\mathbf{A}$ is the dimension of the space spanned by its columns, and the **row rank** is the dimension of the space spanned by its rows. It is a basic fact of linear algebra (that can be shown using the SVD, discussed in Section 7.5) that for any matrix $\mathbf{A}$, $\mathrm{columnrank}(\mathbf{A}) = \mathrm{rowrank}(\mathbf{A})$, and so this quantity is simply referred to as the **rank** of $\mathbf{A}$, denoted as $\mathrm{rank}(\mathbf{A})$. The following are some basic properties of the rank:

- For $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathrm{rank}(\mathbf{A}) \leq \min(m, n)$. If $\mathrm{rank}(\mathbf{A}) = \min(m, n)$, then $\mathbf{A}$ is said to be **full rank**, otherwise it is called **rank deficient**.
- For $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathrm{rank}(\mathbf{A}) = \mathrm{rank}(\mathbf{A}^\mathsf{T}) = \mathrm{rank}(\mathbf{A}^\mathsf{T}\mathbf{A}) = \mathrm{rank}(\mathbf{A}\mathbf{A}^\mathsf{T})$.
- For $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{B} \in \mathbb{R}^{n\times p}$, $\mathrm{rank}(\mathbf{A}\mathbf{B}) \leq \min(\mathrm{rank}(\mathbf{A}), \mathrm{rank}(\mathbf{B}))$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

- For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$, $\text{rank}(\mathbf{A} + \mathbf{B}) \leq \text{rank}(\mathbf{A}) + \text{rank}(\mathbf{B})$.

One can show that a square matrix is invertible iff it is full rank.

7.1.4.4 Condition numbers

The **condition number** of a matrix $\mathbf{A}$ is a measure of how numerically stable any computations involving $\mathbf{A}$ will be. It is defined as follows:

$$\kappa(\mathbf{A}) \triangleq ||\mathbf{A}|| \cdot ||\mathbf{A}^{-1}|| \tag{7.46}$$

where $||\mathbf{A}||$ is the norm of the matrix. We can show that $\kappa(\mathbf{A}) \geq 1$. (The condition number depends on which norm we use; we will assume the ℓ_2-norm unless stated otherwise.)

We say $\mathbf{A}$ is **well-conditioned** if $\kappa(\mathbf{A})$ is small (close to 1), and **ill-conditioned** if $\kappa(\mathbf{A})$ is large. A large condition number means $\mathbf{A}$ is nearly singular. This is a better measure of nearness to singularity than the size of the determinant. For example, suppose $\mathbf{A} = 0.1\mathbf{I}_{100 \times 100}$. Then $\det(\mathbf{A}) = 10^{-100}$, which suggests $\mathbf{A}$ is nearly singular, but $\kappa(\mathbf{A}) = 1$, which means $\mathbf{A}$ is well-conditioned, reflecting the fact that $\mathbf{A}\boldsymbol{x}$ simply scales the entries of $\boldsymbol{x}$ by 0.1.

To get a better understanding of condition numbers, consider the linear system of equations $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. If $\mathbf{A}$ is non-singular, the unique solution is $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$. Suppose we change $\boldsymbol{b}$ to $\boldsymbol{b} + \Delta\boldsymbol{b}$; what effect will that have on $\boldsymbol{x}$? The new solution must satisify

$$\mathbf{A}(\boldsymbol{x} + \Delta\boldsymbol{x}) = \boldsymbol{b} + \Delta\boldsymbol{b} \tag{7.47}$$

where

$$\Delta\boldsymbol{x} = \mathbf{A}^{-1}\Delta\boldsymbol{b} \tag{7.48}$$

We say that $\mathbf{A}$ is well-conditioned if a small $\Delta\boldsymbol{b}$ results in a small $\Delta\boldsymbol{x}$; otherwise we say that $\mathbf{A}$ is ill-conditioned.

For example, suppose

$$\mathbf{A} = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 + 10^{-10} & 1 - 10^{-10} \end{pmatrix}, \quad \mathbf{A}^{-1} = \begin{pmatrix} 1 - 10^{10} & 10^{10} \\ 1 + 10^{10} & -10^{10} \end{pmatrix} \tag{7.49}$$

The solution for $\boldsymbol{b} = (1, 1)$ is $\boldsymbol{x} = (1, 1)$. If we change $\boldsymbol{b}$ by $\Delta\boldsymbol{b}$, the solution changes to

$$\Delta\boldsymbol{x} = \mathbf{A}^{-1}\Delta\boldsymbol{b} = \begin{pmatrix} \Delta b_1 - 10^{10}(\Delta b_1 - \Delta b_2) \\ \Delta b_1 + 10^{10}(\Delta b_1 - \Delta b_2) \end{pmatrix} \tag{7.50}$$

So a small change in $\boldsymbol{b}$ can lead to an extremely large change in $\boldsymbol{x}$, because $\mathbf{A}$ is ill-conditioned ($\kappa(\mathbf{A}) = 2 \times 10^{10}$).

In the case of the ℓ_2-norm, the condition number is equal to the ratio of the largest to smallest singular values (defined in Section 7.5); furthermore, the singular values of $\mathbf{A}$ are the square roots of the eigenvalues of $\mathbf{A}^\mathsf{T}\mathbf{A}$, and so

$$\kappa(\mathbf{A}) = \sigma_{max}/\sigma_{min} = \sqrt{\frac{\lambda_{\max}}{\lambda_{\min}}} \tag{7.51}$$

We can gain further insight into condition numbers by considering a quadratic objective function $f(\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}$. If we plot the level set of this function, it will be elliptical, as shown in Section 7.4.4. As we increase the condition number of $\mathbf{A}$, the ellipses become more and more elongated along certain directions, corresponding to a very narrow valley in function space. If $\kappa = 1$ (the minimum possible value), the level set will be circular.

7.1.5 Special types of matrices

In this section, we will list some common kinds of matrices with various forms of structure.

7.1.5.1 Diagonal matrix

A **diagonal matrix** is a matrix where all non-diagonal elements are 0. This is typically denoted $\mathbf{D} = \text{diag}(d_1, d_2, \ldots, d_n)$, with

$$\mathbf{D} = \begin{pmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_n \end{pmatrix} \tag{7.52}$$

The **identity matrix**, denoted $\mathbf{I} \in \mathbb{R}^{n \times n}$, is a square matrix with ones on the diagonal and zeros everywhere else, $\mathbf{I} = \text{diag}(1, 1, \ldots, 1)$. It has the property that for all $\mathbf{A} \in \mathbb{R}^{n \times n}$,

$$\mathbf{AI} = \mathbf{A} = \mathbf{IA} \tag{7.53}$$

where the size of $\mathbf{I}$ is determined by the dimensions of $\mathbf{A}$ so that matrix multiplication is possible.

We can extract the diagonal vector from a matrix using $\boldsymbol{d} = \text{diag}(\mathbf{D})$. We can convert a vector into a diagonal matrix by writing $\mathbf{D} = \text{diag}(\boldsymbol{d})$.

A **block diagonal** matrix is one which contains matrices on its main diagonal, and is 0 everywhere else, e.g.,

$$\begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{pmatrix} \tag{7.54}$$

A **band-diagonal matrix** only has non-zero entries along the diagonal, and on k sides of the diagonal, where k is the bandwidth. For example, a **tridiagonal** 6×6 matrix looks like this:

$$\begin{bmatrix} A_{11} & A_{12} & 0 & \cdots & \cdots & 0 \\ A_{21} & A_{22} & A_{23} & \ddots & \ddots & \vdots \\ 0 & A_{32} & A_{33} & A_{34} & \ddots & \vdots \\ \vdots & \ddots & A_{43} & A_{44} & A_{45} & 0 \\ \vdots & \ddots & \ddots & A_{54} & A_{55} & A_{56} \\ 0 & \cdots & \cdots & 0 & A_{65} & A_{66} \end{bmatrix} \tag{7.55}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.1.5.2 Triangular matrices

An **upper triangular matrix** only has non-zero entries on and above the diagonal. A **lower triangular matrix** only has non-zero entries on and below the diagonal.

Triangular matrices have the useful property that the diagonal entries of $\mathbf{A}$ are the eigenvalues of $\mathbf{A}$, and hence the determinant is the product of diagonal entries: $\det(\mathbf{A}) = \prod_i A_{ii}$.

7.1.5.3 Positive definite matrices

Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and a vector $\boldsymbol{x} \in \mathbb{R}^n$, the scalar value $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}$ is called a **quadratic form**. Written explicitly, we see that

$$\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} = \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}x_i x_j \quad . \tag{7.56}$$

Note that,

$$\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} = (\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x})^\mathsf{T} = \boldsymbol{x}^\mathsf{T}\mathbf{A}^\mathsf{T}\boldsymbol{x} = \boldsymbol{x}^\mathsf{T}(\frac{1}{2}\mathbf{A} + \frac{1}{2}\mathbf{A}^\mathsf{T})\boldsymbol{x} \tag{7.57}$$

For this reason, we often implicitly assume that the matrices appearing in a quadratic form are symmetric.

We give the following definitions:

- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **positive definite** iff for all non-zero vectors $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} > 0$. This is usually denoted $\mathbf{A} \succ 0$ (or just $\mathbf{A} > 0$). If it is possible that $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} = 0$, we say the matrix is **positive semidefinite** or **psd**. We denote the set of all positive definite matrices by $\mathbb{S}^n_{++}$.
- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **negative definite**, denoted $\mathbf{A} \prec 0$ (or just $\mathbf{A} < 0$) iff for all non-zero $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} < 0$. If it is possible that $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} = 0$, we say the matrix is **negative semidefinite**.
- A symmetric matrix $\mathbf{A} \in \mathbb{S}^n$ is **indefinite**, if it is neither positive semidefinite nor negative semidefinite — i.e., if there exists $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathbb{R}^n$ such that $\boldsymbol{x}_1^\mathsf{T}\mathbf{A}\boldsymbol{x}_1 > 0$ and $\boldsymbol{x}_2^\mathsf{T}\mathbf{A}\boldsymbol{x}_2 < 0$.

It should be obvious that if $\mathbf{A}$ is positive definite, then $-\mathbf{A}$ is negative definite and vice versa. Likewise, if $\mathbf{A}$ is positive semidefinite then $-\mathbf{A}$ is negative semidefinite and vice versa. If $\mathbf{A}$ is indefinite, then so is $-\mathbf{A}$. It can also be shown that positive definite and negative definite matrices are always invertible.

In Section 7.4.3.1, we show that a symmetric matrix is positive definite iff its eigenvalues are positive. Note that if all elements of $\mathbf{A}$ are positive, it does not mean $\mathbf{A}$ is necessarily positive definite. For example, $\mathbf{A} = \begin{pmatrix} 4 & 3 \\ 3 & 2 \end{pmatrix}$ is not positive definite. Conversely, a positive definite matrix can have negative entries e.g., $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$

A sufficient condition for a (real, symmetric) matrix to be positive definite is that it is **diagonally dominant**, i.e., if in every row of the matrix, the magnitude of the diagonal entry in that row is

larger than the sum of the magnitudes of all the other (non-diagonal) entries in that row. More precisely,

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for all } i \tag{7.58}$$

In 2d, any real, symmetric 2×2 matrix $\begin{pmatrix} a & b \\ b & d \end{pmatrix}$ is positive definite iff $a > 0$, $d > 0$ and $ad > b^2$.

Finally, there is one type of positive definite matrix that comes up frequently, and so deserves some special mention. Given any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ (not necessarily symmetric or even square), the **Gram matrix** $\mathbf{G} = \mathbf{A}^\mathsf{T}\mathbf{A}$ is always positive semidefinite. Further, if $m \geq n$ (and we assume for convenience that $\mathbf{A}$ is full rank), then $\mathbf{G} = \mathbf{A}^\mathsf{T}\mathbf{A}$ is positive definite.

7.1.5.4 Orthogonal matrices

Two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ are **orthogonal** if $\boldsymbol{x}^\mathsf{T}\boldsymbol{y} = 0$. A vector $\boldsymbol{x} \in \mathbb{R}^n$ is **normalized** if $\|\boldsymbol{x}\|_2 = 1$. A set of vectors that is pairwise orthogonal and normalized is called **orthonormal**. A square matrix $\mathbf{U} \in \mathbb{R}^{n \times n}$ is **orthogonal** if all its columns are orthonormal. (Note the different meaning of the term orthogonal when talking about vectors versus matrices.) If the entries of $\mathbf{U}$ are complex valued, we use the term **unitary** instead of orthogonal.

It follows immediately from the definition of orthogonality and normality that $\mathbf{U}$ is orthogonal iff

$$\mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{I} = \mathbf{U}\mathbf{U}^\mathsf{T}. \tag{7.59}$$

In other words, the inverse of an orthogonal matrix is its transpose. Note that if $\mathbf{U}$ is not square — i.e., $\mathbf{U} \in \mathbb{R}^{m \times n}$, $n < m$ — but its columns are still orthonormal, then $\mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{I}$, but $\mathbf{U}\mathbf{U}^\mathsf{T} \neq I$. We generally only use the term orthogonal to describe the previous case, where $\mathbf{U}$ is square.

An example of an orthogonal matrix is a **rotation matrix** (see Exercise 7.1). For example, a rotation in 3d by angle α about the z axis is given by

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{7.60}$$

If $\alpha = 45°$, this becomes

$$\mathbf{R}(45) = \begin{pmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{7.61}$$

where $\frac{1}{\sqrt{2}} = 0.7071$. We see that $\mathbf{R}(-\alpha) = \mathbf{R}(\alpha)^{-1} = \mathbf{R}(\alpha)^\mathsf{T}$, so this is an orthogonal matrix.

One nice property of orthogonal matrices is that operating on a vector with an orthogonal matrix will not change its Euclidean norm, i.e.,

$$\|\mathbf{U}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2 \tag{7.62}$$

for any nonzero $\boldsymbol{x} \in \mathbb{R}^n$, and orthogonal $\mathbf{U} \in \mathbb{R}^{n \times n}$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Similarly, one can show that the angle between two vectors is preserved after they are transformed by an orthogonal matrix. The cosine of the angle between $\boldsymbol{x}$ and $\boldsymbol{y}$ is given by

$$\cos(\alpha(\boldsymbol{x}, \boldsymbol{y})) = \frac{\boldsymbol{x}^\mathsf{T}\boldsymbol{y}}{||\boldsymbol{x}||||\boldsymbol{y}||} \tag{7.63}$$

so

$$\cos(\alpha(\mathbf{U}\boldsymbol{x}, \mathbf{U}\boldsymbol{y})) = \frac{(\mathbf{U}\boldsymbol{x})^\mathsf{T}(\mathbf{U}\boldsymbol{y})}{||\mathbf{U}\boldsymbol{x}||||\mathbf{U}\boldsymbol{y}||} = \frac{\boldsymbol{x}^\mathsf{T}\boldsymbol{y}}{||\boldsymbol{x}||||\boldsymbol{y}||} = \cos(\alpha(\boldsymbol{x}, \boldsymbol{y})) \tag{7.64}$$

In summary, transformations by orthogonal matrices are generalizations of rotations (if $\det(\mathbf{U}) = 1$) and reflections (if $\det(\mathbf{U}) = -1$), since they preserve lengths and angles.

Note that there is a technique called Gram Schmidt orthogonalization which is a way to make any square matrix orthogonal, but we will not cover it here.

7.2 Matrix multiplication

The product of two matrices $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$ is the matrix

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m\times p}, \tag{7.65}$$

where

$$C_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}. \tag{7.66}$$

Note that in order for the matrix product to exist, the number of columns in $\mathbf{A}$ must equal the number of rows in $\mathbf{B}$.

Matrix multiplication generally takes $O(mnp)$ time, although faster methods exist. In addition, specialized hardware, such as GPUs and TPUs, can be leveraged to speed up matrix multiplication significantly, by performing operations across the rows (or columns) in parallel.

It is useful to know a few basic properties of matrix multiplication:

- Matrix multiplication is **associative**: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$.
- Matrix multiplication is **distributive**: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$.
- Matrix multiplication is, in general, *not* **commutative**; that is, it can be the case that $\mathbf{AB} \neq \mathbf{BA}$.

(In each of the above cases, we are assuming that the dimensions match.)

There are many important special cases of matrix multiplication, as we discuss below.

7.2.1 Vector–vector products

Given two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, the quantity $\boldsymbol{x}^\mathsf{T}\boldsymbol{y}$, called the **inner product**, **dot product** or **scalar product** of the vectors, is a real number given by

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle \triangleq \boldsymbol{x}^\mathsf{T}\boldsymbol{y} = \sum_{i=1}^{n} x_i y_i. \tag{7.67}$$

Note that it is always the case that $\boldsymbol{x}^\mathsf{T}\boldsymbol{y} = \boldsymbol{y}^\mathsf{T}\boldsymbol{x}$.

Given vectors $\boldsymbol{x} \in \mathbb{R}^m$, $\boldsymbol{y} \in \mathbb{R}^n$ (they no longer have to be the same size), $\boldsymbol{x}\boldsymbol{y}^\mathsf{T}$ is called the **outer product** of the vectors. It is a matrix whose entries are given by $(\boldsymbol{x}\boldsymbol{y}^\mathsf{T})_{ij} = x_i y_j$, i.e.,

$$\boldsymbol{x}\boldsymbol{y}^\mathsf{T} \in \mathbb{R}^{m\times n} = \begin{bmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_n \\ x_2y_1 & x_2y_2 & \cdots & x_2y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_my_1 & x_my_2 & \cdots & x_my_n \end{bmatrix}. \tag{7.68}$$

7.2.2 Matrix–vector products

Given a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ and a vector $\boldsymbol{x} \in \mathbb{R}^n$, their product is a vector $\boldsymbol{y} = \mathbf{A}\boldsymbol{x} \in \mathbb{R}^m$. There are a couple of ways of looking at matrix-vector multiplication, and we will look at them both.

If we write $\mathbf{A}$ by rows, then we can express $\boldsymbol{y} = \mathbf{A}\boldsymbol{x}$ as follows:

$$\boldsymbol{y} = \mathbf{A}\boldsymbol{x} = \begin{bmatrix} — & \boldsymbol{a}_1^\mathsf{T} & — \\ — & \boldsymbol{a}_2^\mathsf{T} & — \\ & \vdots & \\ — & \boldsymbol{a}_m^\mathsf{T} & — \end{bmatrix} \boldsymbol{x} = \begin{bmatrix} \boldsymbol{a}_1^\mathsf{T}\boldsymbol{x} \\ \boldsymbol{a}_2^\mathsf{T}\boldsymbol{x} \\ \vdots \\ \boldsymbol{a}_m^\mathsf{T}\boldsymbol{x} \end{bmatrix}. \tag{7.69}$$

In other words, the ith entry of $\boldsymbol{y}$ is equal to the inner product of the ith *row* of $\mathbf{A}$ and $\boldsymbol{x}$, $y_i = \boldsymbol{a}_i^\mathsf{T}\boldsymbol{x}$.

Alternatively, let's write $\mathbf{A}$ in column form. In this case we see that

$$\boldsymbol{y} = \mathbf{A}\boldsymbol{x} = \begin{bmatrix} | & | & & | \\ \boldsymbol{a}_1 & \boldsymbol{a}_2 & \cdots & \boldsymbol{a}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} | \\ \boldsymbol{a}_1 \\ | \end{bmatrix} x_1 + \begin{bmatrix} | \\ \boldsymbol{a}_2 \\ | \end{bmatrix} x_2 + \ldots + \begin{bmatrix} | \\ \boldsymbol{a}_n \\ | \end{bmatrix} x_n. \tag{7.70}$$

In other words, $\boldsymbol{y}$ is a **linear combination** of the *columns* of $\mathbf{A}$, where the coefficients of the linear combination are given by the entries of $\boldsymbol{x}$. We can view the columns of $\mathbf{A}$ as a set of **basis vectors** defining a **linear subspace**. We can construct vectors in this subspace by taking linear combinations of the basis vectors. See Section 7.1.2 for details.

7.2.3 Matrix–matrix products

Below we look at four different (but, of course, equivalent) ways of viewing the matrix-matrix multiplication $\mathbf{C} = \mathbf{A}\mathbf{B}$.

First we can view matrix-matrix multiplication as a set of vector-vector products. The most obvious viewpoint, which follows immediately from the definition, is that the i, j entry of $\mathbf{C}$ is equal to the inner product of the ith row of $\mathbf{A}$ and the jth column of $\mathbf{B}$. Symbolically, this looks like the following,

$$\mathbf{C} = \mathbf{A}\mathbf{B} = \begin{bmatrix} — & \boldsymbol{a}_1^\mathsf{T} & — \\ — & \boldsymbol{a}_2^\mathsf{T} & — \\ & \vdots & \\ — & \boldsymbol{a}_m^\mathsf{T} & — \end{bmatrix} \begin{bmatrix} | & | & & | \\ \boldsymbol{b}_1 & \boldsymbol{b}_2 & \cdots & \boldsymbol{b}_p \\ | & | & & | \end{bmatrix} = \begin{bmatrix} \boldsymbol{a}_1^\mathsf{T}\boldsymbol{b}_1 & \boldsymbol{a}_1^\mathsf{T}\boldsymbol{b}_2 & \cdots & \boldsymbol{a}_1^\mathsf{T}\boldsymbol{b}_p \\ \boldsymbol{a}_2^\mathsf{T}\boldsymbol{b}_1 & \boldsymbol{a}_2^\mathsf{T}\boldsymbol{b}_2 & \cdots & \boldsymbol{a}_2^\mathsf{T}\boldsymbol{b}_p \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{a}_m^\mathsf{T}\boldsymbol{b}_1 & \boldsymbol{a}_m^\mathsf{T}\boldsymbol{b}_2 & \cdots & \boldsymbol{a}_m^\mathsf{T}\boldsymbol{b}_p \end{bmatrix}. \tag{7.71}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

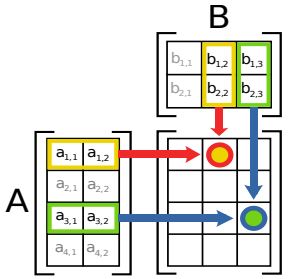


Figure 7.5: Illustration of matrix multiplication. From https://en.wikipedia.org/wiki/Matrix_multiplication. Used with kind permission of Wikipedia author Bilou.

Remember that since $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$, $\boldsymbol{a}_i \in \mathbb{R}^n$ and $\boldsymbol{b}_j \in \mathbb{R}^n$, so these inner products all make sense. This is the most "natural" representation when we represent $\mathbf{A}$ by rows and $\mathbf{B}$ by columns. See Figure 7.5 for an illustration.

Alternatively, we can represent $\mathbf{A}$ by columns, and $\mathbf{B}$ by rows, which leads to the interpretation of $\mathbf{AB}$ as a sum of outer products. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} | & | & & | \\ \boldsymbol{a}_1 & \boldsymbol{a}_2 & \cdots & \boldsymbol{a}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} — & \boldsymbol{b}_1^\mathsf{T} & — \\ — & \boldsymbol{b}_2^\mathsf{T} & — \\ & \vdots & \\ — & \boldsymbol{b}_n^\mathsf{T} & — \end{bmatrix} = \sum_{i=1}^{n} \boldsymbol{a}_i \boldsymbol{b}_i^\mathsf{T} \ . \tag{7.72}$$

Put another way, $\mathbf{AB}$ is equal to the sum, over all i, of the outer product of the ith column of $\mathbf{A}$ and the ith row of $\mathbf{B}$. Since, in this case, $\boldsymbol{a}_i \in \mathbb{R}^m$ and $\boldsymbol{b}_i \in \mathbb{R}^p$, the dimension of the outer product $\boldsymbol{a}_i \boldsymbol{b}_i^\mathsf{T}$ is $m \times p$, which coincides with the dimension of $\mathbf{C}$.

We can also view matrix-matrix multiplication as a set of matrix-vector products. Specifically, if we represent $\mathbf{B}$ by columns, we can view the columns of $\mathbf{C}$ as matrix-vector products between $\mathbf{A}$ and the columns of $\mathbf{B}$. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \mathbf{A} \begin{bmatrix} | & | & & | \\ \boldsymbol{b}_1 & \boldsymbol{b}_2 & \cdots & \boldsymbol{b}_p \\ | & | & & | \end{bmatrix} = \begin{bmatrix} | & | & & | \\ \mathbf{A}\boldsymbol{b}_1 & \mathbf{A}\boldsymbol{b}_2 & \cdots & \mathbf{A}\boldsymbol{b}_p \\ | & | & & | \end{bmatrix} . \tag{7.73}$$

Here the ith column of $\mathbf{C}$ is given by the matrix-vector product with the vector on the right, $\boldsymbol{c}_i = \mathbf{A}\boldsymbol{b}_i$. These matrix-vector products can in turn be interpreted using both viewpoints given in the previous subsection.

Finally, we have the analogous viewpoint, where we represent $\mathbf{A}$ by rows, and view the rows of $\mathbf{C}$ as the matrix-vector product between the rows of $\mathbf{A}$ and the matrix $\mathbf{B}$. Symbolically,

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} — & \boldsymbol{a}_1^\mathsf{T} & — \\ — & \boldsymbol{a}_2^\mathsf{T} & — \\ & \vdots & \\ — & \boldsymbol{a}_m^\mathsf{T} & — \end{bmatrix} \mathbf{B} = \begin{bmatrix} — & \boldsymbol{a}_1^\mathsf{T}\mathbf{B} & — \\ — & \boldsymbol{a}_2^\mathsf{T}\mathbf{B} & — \\ & \vdots & \\ — & \boldsymbol{a}_m^\mathsf{T}\mathbf{B} & — \end{bmatrix} . \tag{7.74}$$

Here the ith row of $\mathbf{C}$ is given by the matrix-vector product with the vector on the left, $\boldsymbol{c}_i^\mathsf{T} = \boldsymbol{a}_i^\mathsf{T}\mathbf{B}$.

It may seem like overkill to dissect matrix multiplication to such a large degree, especially when all these viewpoints follow immediately from the initial definition we gave (in about a line of math) at the beginning of this section. However, virtually all of linear algebra deals with matrix multiplications of some kind, and it is worthwhile to spend some time trying to develop an intuitive understanding of the viewpoints presented here.

Finally, a word on notation. We write $\mathbf{A}^2$ as shorthand for $\mathbf{AA}$, which is the matrix product. To denote elementwise squaring of the elements of a matrix, we write $\mathbf{A}^{\odot 2} = [A_{ij}^2]$. (If $\mathbf{A}$ is diagonal, then $\mathbf{A}^2 = \mathbf{A}^{\odot 2}$.)

We can also define the inverse of $\mathbf{A}^2$ using the **matrix square root**: we say $\mathbf{A} = \sqrt{\mathbf{M}}$ if $\mathbf{A}^2 = \mathbf{M}$. To denote elementwise square root of the elements of a matrix, we write $[\sqrt{M_{ij}}]$.

7.2.4 Application: manipulating data matrices

As an application of the above results, consider the case where $\mathbf{X}$ is the $N \times D$ design matrix, whose rows are the data cases. There are various common preprocessing operations that we apply to this matrix, which we summarize below. (Writing these operations in matrix form is useful because it is notationally compact, and it allows us to implement the methods quickly using fast matrix code.)

7.2.4.1 Summing slices of the matrix

Suppose $\mathbf{X}$ is an $N \times D$ matrix. We can sum across the rows by premultiplying by a $1 \times N$ matrix of ones to create a $1 \times D$ matrix:

$$\mathbf{1}_N^\mathsf{T}\mathbf{X} = \begin{pmatrix} \sum_n x_{n1} & \cdots & \sum_n x_{nD} \end{pmatrix} \tag{7.75}$$

Hence the mean of the data vectors is given by

$$\overline{\boldsymbol{x}}^\mathsf{T} = \frac{1}{N}\mathbf{1}_N^\mathsf{T}\mathbf{X} \tag{7.76}$$

We can sum across the columns by postmultiplying by a $D \times 1$ matrix of ones to create a $N \times 1$ matrix:

$$\mathbf{X}\mathbf{1}_D = \begin{pmatrix} \sum_d x_{1d} \\ \vdots \\ \sum_d x_{Nd} \end{pmatrix} \tag{7.77}$$

We can sum all entries in a matrix by pre and post multiplying by a vector of 1s:

$$\mathbf{1}_N^\mathsf{T}\mathbf{X}\mathbf{1}_D = \sum_{ij} X_{ij} \tag{7.78}$$

Hence the overall mean is given by

$$\overline{x} = \frac{1}{ND}\mathbf{1}_N^\mathsf{T}\mathbf{X}\mathbf{1}_D \tag{7.79}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.2.4.2 Scaling rows and columns of a matrix

We often want to scale rows or columns of a data matrix (e.g., to standardize them). We now show how to write this in matrix notation.

If we pre-multiply $\mathbf{X}$ by a diagonal matrix $\mathbf{S} = \text{diag}(\boldsymbol{s})$, where $\boldsymbol{s}$ is an N-vector, then we just scale each row of $\mathbf{X}$ by the corresponding scale factor in $\boldsymbol{s}$:

$$\text{diag}(\boldsymbol{s})\mathbf{X} = \begin{pmatrix} s_1 & \cdots & 0 \\ & \ddots & \\ 0 & \cdots & s_N \end{pmatrix} \begin{pmatrix} x_{1,1} & \cdots & x_{1,D} \\ & \ddots & \\ x_{N,1} & \cdots & x_{N,D} \end{pmatrix} = \begin{pmatrix} s_1 x_{1,1} & \cdots & s_1 x_{1,D} \\ & \ddots & \\ s_N x_{N,1} & \cdots & s_N x_{N,D} \end{pmatrix} \tag{7.80}$$

If we post-multiply $\mathbf{X}$ by a diagonal matrix $\mathbf{S} = \text{diag}(\boldsymbol{s})$, where $\boldsymbol{s}$ is a D-vector, then we just scale each column of $\mathbf{X}$ by the corresponding element in $\boldsymbol{s}$.

$$\mathbf{X}\text{diag}(\boldsymbol{s}) = \begin{pmatrix} x_{1,1} & \cdots & x_{1,D} \\ & \ddots & \\ x_{N,1} & \cdots & x_{N,D} \end{pmatrix} \begin{pmatrix} s_1 & \cdots & 0 \\ & \ddots & \\ 0 & \cdots & s_D \end{pmatrix} = \begin{pmatrix} s_1 x_{1,1} & \cdots & s_D x_{1,D} \\ & \ddots & \\ s_1 x_{N,1} & \cdots & s_D x_{N,D} \end{pmatrix} \tag{7.81}$$

Thus we can rewrite the standardization operation from Section 10.2.8 in matrix form as follows:

$$\text{standardize}(\mathbf{X}) = (\mathbf{X} - \mathbf{1}_N \boldsymbol{\mu}^T)\text{diag}(\boldsymbol{\sigma})^{-1} \tag{7.82}$$

where $\boldsymbol{\mu} = \overline{\boldsymbol{x}}$ is the empirical mean, and $\boldsymbol{\sigma}$ is a vector of the empirical standard deviations.

7.2.4.3 Sum of squares and scatter matrix

The **sum of squares matrix** is $D \times D$ matrix defined by

$$\mathbf{S}_0 \triangleq \mathbf{X}^\mathsf{T}\mathbf{X} = \sum_{n=1}^{N} \boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T} = \sum_{n=1}^{N} \begin{pmatrix} x_{n,1}^2 & \cdots & x_{n,1}x_{n,D} \\ & \ddots & \\ x_{n,D}x_{n,1} & \cdots & x_{n,D}^2 \end{pmatrix} \tag{7.83}$$

The **scatter matrix** is a $D \times D$ matrix defined by

$$\mathbf{S}_{\overline{\boldsymbol{x}}} \triangleq \sum_{n=1}^{N} (\boldsymbol{x}_n - \overline{\boldsymbol{x}})(\boldsymbol{x}_n - \overline{\boldsymbol{x}})^\mathsf{T} = \left(\sum_n \boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T} \right) - N\overline{\boldsymbol{x}}\overline{\boldsymbol{x}}^\mathsf{T} \tag{7.84}$$

We see that this is the sum of squares matrix applied to the mean-centered data. More precisely, define $\tilde{\mathbf{X}}$ to be a version of $\mathbf{X}$ where we subtract the mean $\overline{\boldsymbol{x}} = \frac{1}{N}\mathbf{X}^\mathsf{T}\mathbf{1}_N$ off every row. Hence we can compute the centered data matrix using

$$\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \overline{\boldsymbol{x}}^\mathsf{T} = \mathbf{X} - \frac{1}{N}\mathbf{1}_N \mathbf{1}_N^\mathsf{T}\mathbf{X} = \mathbf{C}_N\mathbf{X} \tag{7.85}$$

where

$$\mathbf{C}_N \triangleq \mathbf{I}_N - \frac{1}{N}\mathbf{J}_N \tag{7.86}$$

is the **centering matrix**, and $\mathbf{J}_N = \mathbf{1}_N \mathbf{1}_N^\mathsf{T}$ is a matrix of all 1s. The scatter matrix can now be computed as follows:

$$\mathbf{S}_{\bar{\boldsymbol{x}}} = \tilde{\mathbf{X}}^\mathsf{T}\tilde{\mathbf{X}} = \mathbf{X}^\mathsf{T}\mathbf{C}_N^\mathsf{T}\mathbf{C}_N\mathbf{X} = \mathbf{X}^\mathsf{T}\mathbf{C}_N\mathbf{X} \tag{7.87}$$

where we exploited the fact that $\mathbf{C}_N$ is symmetric and idempotent, i.e., $\mathbf{C}_N^k = \mathbf{C}_N$ for $k = 1, 2, \ldots$ (since once we subtract the mean, subtracting it again has no effect).

7.2.4.4 Gram matrix

The $N \times N$ matrix $\mathbf{X}\mathbf{X}^\mathsf{T}$ is a matrix of inner products called the **Gram matrix**:

$$\mathbf{K} \triangleq \mathbf{X}\mathbf{X}^\mathsf{T} = \begin{pmatrix} \boldsymbol{x}_1^\mathsf{T}\boldsymbol{x}_1 & \cdots & \boldsymbol{x}_1^\mathsf{T}\boldsymbol{x}_N \\ & \ddots & \\ \boldsymbol{x}_n^\mathsf{T}\boldsymbol{x}_1 & \cdots & \boldsymbol{x}_N^\mathsf{T}\boldsymbol{x}_N \end{pmatrix} \tag{7.88}$$

Sometimes we want to compute the inner products of the mean-centered data vectors, $\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\mathsf{T}$. However, if we are working with a feature similarity matrix instead of raw features, we will only have access to $\mathbf{K}$, not $\mathbf{X}$. (We will see examples of this in Section 20.4.4 and Section 20.4.6.) Fortunately, we can compute $\tilde{\mathbf{K}}$ from $\mathbf{K}$ using the **double centering trick**:

$$\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\mathsf{T} = \mathbf{C}_N\mathbf{K}\mathbf{C}_N = \mathbf{K} - \frac{1}{N}\mathbf{J}\mathbf{K} - \frac{1}{N}\mathbf{K}\mathbf{J} + \frac{1}{N^2}\mathbf{1}^\mathsf{T}\mathbf{K}\mathbf{1} \tag{7.89}$$

This subtracts the row means and column means from $\mathbf{K}$, and adds back the global mean that gets subtracted twice, so that both row means and column means of $\tilde{\mathbf{K}}$ are equal to zero.

To see why Equation (7.89) is true, consider the scalar form:

$$\tilde{K}_{ij} = \tilde{\boldsymbol{x}}_i^\mathsf{T}\tilde{\boldsymbol{x}}_j = (\boldsymbol{x}_i - \frac{1}{N}\sum_{k=1}^N \boldsymbol{x}_k)(\boldsymbol{x}_j - \frac{1}{N}\sum_{l=1}^N \boldsymbol{x}_l) \tag{7.90}$$

$$= \boldsymbol{x}_i^\mathsf{T}\boldsymbol{x}_j - \frac{1}{N}\sum_{k=1}^N \boldsymbol{x}_i^\mathsf{T}\boldsymbol{x}_k - \frac{1}{N}\sum_{k=1}^N \boldsymbol{x}_j^\mathsf{T}\boldsymbol{x}_k + \frac{1}{N^2}\sum_{k=1}^N\sum_{l=1}^N \boldsymbol{x}_k^\mathsf{T}\boldsymbol{x}_l \tag{7.91}$$

7.2.4.5 Distance matrix

Let $\mathbf{X}$ be $N_x \times D$ datamatrix, and $\mathbf{Y}$ be another $N_y \times D$ datamatrix. We can compute the squared pairwise distances between these using

$$\mathbf{D}_{ij} = (\boldsymbol{x}_i - \boldsymbol{y}_j)^\mathsf{T}(\boldsymbol{x}_i - \boldsymbol{y}_j) = ||\boldsymbol{x}_i||^2 - 2\boldsymbol{x}_i^\mathsf{T}\boldsymbol{y}_j + ||\boldsymbol{y}_j||^2 \tag{7.92}$$

Let us now write this in matrix form. Let $\hat{\boldsymbol{x}} = [||\boldsymbol{x}_1||^2; \cdots; ||\boldsymbol{x}_{N_x}||^2] = \text{diag}(\mathbf{X}\mathbf{X}^\mathsf{T})$ be a vector where each element is the squared norm of the examples in $\mathbf{X}$, and define $\hat{\boldsymbol{y}}$ similarly. Then we have

$$\mathbf{D} = \hat{\boldsymbol{x}}\mathbf{1}_{N_y}^\mathsf{T} - 2\mathbf{X}\mathbf{Y}^\mathsf{T} + \mathbf{1}_{N_x}\hat{\boldsymbol{y}}^\mathsf{T} \tag{7.93}$$

In the case that $\mathbf{X} = \mathbf{Y}$, we have

$$\mathbf{D} = \hat{\boldsymbol{x}}\mathbf{1}_N^\mathsf{T} - 2\mathbf{X}\mathbf{X}^\mathsf{T} + \mathbf{1}_N\hat{\boldsymbol{x}}^\mathsf{T} \tag{7.94}$$

This vectorized computation is often much faster than using for loops.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.2.5 Kronecker products *

If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{B}$ is a $p \times q$ matrix, then the **Kronecker product** $\mathbf{A} \otimes \mathbf{B}$ is the $mp \times nq$ block matrix

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \tag{7.95}$$

For example,

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \otimes \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{11}b_{12} & a_{11}b_{13} & a_{12}b_{11} & a_{12}b_{12} & a_{12}b_{13} \\ a_{11}b_{21} & a_{11}b_{22} & a_{11}b_{23} & a_{12}b_{21} & a_{12}b_{22} & a_{12}b_{23} \\ a_{21}b_{11} & a_{21}b_{12} & a_{21}b_{13} & a_{22}b_{11} & a_{22}b_{12} & a_{22}b_{13} \\ a_{21}b_{21} & a_{21}b_{22} & a_{21}b_{23} & a_{22}b_{21} & a_{22}b_{22} & a_{22}b_{23} \\ a_{31}b_{11} & a_{31}b_{12} & a_{31}b_{13} & a_{32}b_{11} & a_{32}b_{12} & a_{32}b_{13} \\ a_{31}b_{21} & a_{31}b_{22} & a_{31}b_{23} & a_{32}b_{21} & a_{32}b_{22} & a_{32}b_{23} \end{bmatrix} \tag{7.96}$$

Here are some useful identities:

$$(\mathbf{A} \otimes \mathbf{B})^{-1} = \mathbf{A}^{-1} \otimes \mathbf{B}^{-1} \tag{7.97}$$

$$(\mathbf{A} \otimes \mathbf{B})\text{vec}(\mathbf{C}) = \text{vec}(\mathbf{B}\mathbf{C}\mathbf{A}^{\mathsf{T}}) \tag{7.98}$$

where $\text{vec}(\mathbf{M})$ stacks the *columns* of $\mathbf{M}$. (If we stack along the rows, we get $(\mathbf{A} \otimes \mathbf{B})\text{vec}(\mathbf{C}) = \text{vec}(\mathbf{A}\mathbf{C}\mathbf{B}^{\mathsf{T}})$.) See [Loa00] for a list of other useful properties.

7.2.6 Einstein summation *

Einstein summation, or **einsum** for short, is a notational shortcut for working with tensors. The convention was introduced by Einstein [Ein16, sec 5], who later joked to a friend, "I have made a great discovery in mathematics; I have suppressed the summation sign every time that the summation must be made over an index which occurs twice..." [Pai05, p.216]. For example, instead of writing matrix multiplication as $C_{ij} = \sum_k A_{ik}B_{kj}$, we can just write it as $C_{ij} = A_{ik}B_{kj}$, where we drop the $\sum_k$.

As a more complex example, suppose we have a 3d tensor S_{ntk} where n indexes examples in the batch, t indexes locations in the sequence, and k indexes words in a one-hot representation. Let W_{kd} be an embedding matrix that maps sparse one-hot vectors $\mathbb{R}^k$ to dense vectors in $\mathbb{R}^d$. We can convert the batch of sequences of one-hots to a batch of sequences of embeddings as follows:

$$E_{ntd} = \sum_k S_{ntk}W_{kd} \tag{7.99}$$

We can compute the sum of the embedding vectors for each sequence (to get a global representation of each bag of words) as follows:

$$E_{nd} = \sum_k \sum_t S_{ntk}W_{kd} \tag{7.100}$$

Finally we can pass each sequence's vector representation through another linear transform V_{dc} to map to the logits over a classifier with c labels:

$$L_{nc} = \sum_d E_{nd} V_{dc} = \sum_d \sum_k \sum_t S_{ntk} W_{kd} V_{dc} \tag{7.101}$$

In einsum notation, we write $L_{nc} = S_{ntk} W_{kd} V_{dc}$. We sum over k and d because those indices occur twice on the RHS. We sum over t because that index does not occur on the LHS.

Einsum is implemented in NumPy, Tensorflow, PyTorch, etc. What makes it particularly useful is that it can perform the relevant tensor multiplications in complex expressions in an optimal order, so as to minimize time and intermediate memory allocation.[2] The library is best illustrated by the examples in einsum_demo.ipynb.

Note that the speed of einsum depends on the order in which the operations are performed, which depends on the shapes of the relevant arguments. The optimal ordering minimizes the treewidth of the resulting computation graph, as explained in [GASG18]. In general, the time to compute the optimal ordering is exponential in the number of arguments, so it is common to use a greedy approximation. However, if we expect to repeat the same calculation many times, using tensors of the same shape but potentially different content, we can compute the optimal ordering once and reuse it multiple times.

7.3 Matrix inversion

In this section, we discuss how to invert different kinds of matrices.

7.3.1 The inverse of a square matrix

The **inverse** of a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is denoted $\mathbf{A}^{-1}$, and is the unique matrix such that

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I} = \mathbf{A}\mathbf{A}^{-1}. \tag{7.102}$$

Note that $\mathbf{A}^{-1}$ exists if and only if $\det(\mathbf{A}) \neq 0$. If $\det(\mathbf{A}) = 0$, it is called a **singular** matrix.

The following are properties of the inverse; all assume that $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ are non-singular:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A} \tag{7.103}$$

$$(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1} \tag{7.104}$$

$$(\mathbf{A}^{-1})^{\mathsf{T}} = (\mathbf{A}^{\mathsf{T}})^{-1} \triangleq \mathbf{A}^{-T} \tag{7.105}$$

For the case of a 2×2 matrix, the expression for $\mathbf{A}^{-1}$ is simple enough to give explicitly. We have

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \tag{7.106}$$

For a block diagonal matrix, the inverse is obtained by simply inverting each block separately, e.g.,

$$\begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{A}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}^{-1} \end{pmatrix} \tag{7.107}$$

2. These optimizations are implemented in the **opt-einsum** library [GASG18]. Its core functionality is included in NumPy and JAX `einsum` functions, provided you set `optimize=True` parameter.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.3.2 Schur complements *

In this section, we review some useful results concerning block structured matrices.

Theorem 7.3.1 (Inverse of a partitioned matrix)**.** *Consider a general partitioned matrix*

$$\mathbf{M} = \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \tag{7.108}$$

where we assume $\mathbf{E}$ *and* $\mathbf{H}$ *are invertible. We have*

$$\mathbf{M}^{-1} = \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \end{pmatrix} \tag{7.109}$$

$$= \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & -\mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \tag{7.110}$$

where

$$\mathbf{M}/\mathbf{H} \triangleq \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} \tag{7.111}$$

$$\mathbf{M}/\mathbf{E} \triangleq \mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F} \tag{7.112}$$

We say that $\mathbf{M}/\mathbf{H}$ *is the* **Schur complement** *of* $\mathbf{M}$ *wrt* $\mathbf{H}$*, and* $\mathbf{M}/\mathbf{E}$ *is the Schur complement of* $\mathbf{M}$ *wrt* $\mathbf{E}$*.*

Equation (7.109) *and Equation* (7.110) *are called the* **partitioned inverse formulae***.*

Proof. If we could block diagonalize $\mathbf{M}$, it would be easier to invert. To zero out the top right block of $\mathbf{M}$ we can pre-multiply as follows

$$\begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \tag{7.113}$$

Similarly, to zero out the bottom left we can post-multiply as follows

$$\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{G} & \mathbf{H} \end{pmatrix} \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} = \begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix} \tag{7.114}$$

Putting it all together we get

$$\underbrace{\begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}}_{\mathbf{X}} \underbrace{\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}}_{\mathbf{M}} \underbrace{\begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix}}_{\mathbf{Z}} = \underbrace{\begin{pmatrix} \mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{H} \end{pmatrix}}_{\mathbf{W}} \tag{7.115}$$

Taking the inverse of both sides yields

$$\mathbf{Z}^{-1}\mathbf{M}^{-1}\mathbf{X}^{-1} = \mathbf{W}^{-1} \tag{7.116}$$

$$\mathbf{M}^{-1} = \mathbf{Z}\mathbf{W}^{-1}\mathbf{X} \tag{7.117}$$

Substituting in the definitions we get

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G} & \mathbf{I} \end{pmatrix} \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \tag{7.118}$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & \mathbf{0} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{F}\mathbf{H}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \tag{7.119}$$

$$= \begin{pmatrix} (\mathbf{M}/\mathbf{H})^{-1} & -(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \\ -\mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1} & \mathbf{H}^{-1} + \mathbf{H}^{-1}\mathbf{G}(\mathbf{M}/\mathbf{H})^{-1}\mathbf{F}\mathbf{H}^{-1} \end{pmatrix} \tag{7.120}$$

Alternatively, we could have decomposed the matrix $\mathbf{M}$ in terms of $\mathbf{E}$ and $\mathbf{M}/\mathbf{E} = (\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})$, yielding

$$\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & -\mathbf{E}^{-1}\mathbf{F}(\mathbf{M}/\mathbf{E})^{-1} \\ -(\mathbf{M}/\mathbf{E})^{-1}\mathbf{G}\mathbf{E}^{-1} & (\mathbf{M}/\mathbf{E})^{-1} \end{pmatrix} \tag{7.121}$$

□

7.3.3 The matrix inversion lemma *

Equating the top left block of the first matrix in Equation (7.119) with the top left block of the matrix in Equation (7.121)

$$(\mathbf{M}/\mathbf{H})^{-1} = (\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G})^{-1} = \mathbf{E}^{-1} + \mathbf{E}^{-1}\mathbf{F}(\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F})^{-1}\mathbf{G}\mathbf{E}^{-1} \tag{7.122}$$

This is known as the **matrix inversion lemma** or the **Sherman-Morrison-Woodbury formula**.

A typical application in machine learning is the following. Let $\mathbf{X}$ be an $N \times D$ data matrix, and $\mathbf{\Sigma}$ be $N \times N$ diagonal matrix. Then we have (using the substitutions $\mathbf{E} = \mathbf{\Sigma}$, $\mathbf{F} = \mathbf{G}^\mathsf{T} = \mathbf{X}$, and $\mathbf{H}^{-1} = -\mathbf{I}$) the following result:

$$(\mathbf{\Sigma} + \mathbf{X}\mathbf{X}^\mathsf{T})^{-1} = \mathbf{\Sigma}^{-1} - \mathbf{\Sigma}^{-1}\mathbf{X}(\mathbf{I} + \mathbf{X}^\mathsf{T}\mathbf{\Sigma}^{-1}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{\Sigma}^{-1} \tag{7.123}$$

The LHS takes $O(N^3)$ time to compute, the RHS takes time $O(D^3)$ to compute.

Another application concerns computing a **rank one update** of an inverse matrix. Let $\mathbf{E} = \mathbf{A}$, $\mathbf{F} = \boldsymbol{u}$, $\mathbf{G} = \boldsymbol{v}^\mathsf{T}$, and $H = -1$. Then we have

$$(\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^\mathsf{T})^{-1} = \mathbf{A}^{-1} + \mathbf{A}^{-1}\boldsymbol{u}(-1 - \boldsymbol{v}^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{u})^{-1}\boldsymbol{v}^\mathsf{T}\mathbf{A}^{-1} \tag{7.124}$$

$$= \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\boldsymbol{u}\boldsymbol{v}^\mathsf{T}\mathbf{A}^{-1}}{1 + \boldsymbol{v}^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{u}} \tag{7.125}$$

This is known as the **Sherman-Morrison formula**.

7.3.4 Matrix determinant lemma *

We now use the above results to derive an efficient way to compute the determinant of a block-structured matrix.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

From Equation (7.115), we have

$$|\mathbf{X}||\mathbf{M}||\mathbf{Z}| = |\mathbf{W}| = |\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}||\mathbf{H}| \tag{7.126}$$

$$|\begin{pmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{pmatrix}| = |\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}||\mathbf{H}| \tag{7.127}$$

$$|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}| \tag{7.128}$$

$$|\mathbf{M}/\mathbf{H}| = \frac{|\mathbf{M}|}{|\mathbf{H}|} \tag{7.129}$$

So we can see that $\mathbf{M}/\mathbf{H}$ acts somewhat like a division operator (hence the notation).

Furthermore, we have

$$|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}| = |\mathbf{M}/\mathbf{E}||\mathbf{E}| \tag{7.130}$$

$$|\mathbf{M}/\mathbf{H}| = \frac{|\mathbf{M}/\mathbf{E}||\mathbf{E}|}{|\mathbf{H}|} \tag{7.131}$$

$$|\mathbf{E} - \mathbf{F}\mathbf{H}^{-1}\mathbf{G}| = |\mathbf{H} - \mathbf{G}\mathbf{E}^{-1}\mathbf{F}||\mathbf{H}^{-1}||\mathbf{E}| \tag{7.132}$$

Hence (setting $\mathbf{E} = \mathbf{A}$, $\mathbf{F} = -\boldsymbol{u}$, $\mathbf{G} = \boldsymbol{v}^\mathsf{T}$, $\mathbf{H} = 1$) we have

$$|\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^\mathsf{T}| = (1 + \boldsymbol{v}^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{u})|\mathbf{A}| \tag{7.133}$$

This is known as the **matrix determinant lemma**.

7.3.5 Application: deriving the conditionals of an MVN *

Consider a joint Gaussian of the form $p(\boldsymbol{x}_1, \boldsymbol{x}_2) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, where

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix} \tag{7.134}$$

In Section 3.2.3, we claimed that

$$p(\boldsymbol{x}_1|\boldsymbol{x}_2) = \mathcal{N}(\boldsymbol{x}_1|\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\boldsymbol{x}_2 - \boldsymbol{\mu}_2),\ \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}) \tag{7.135}$$

In this section, we derive this result using Schur complements.

Let us factor the joint $p(\boldsymbol{x}_1, \boldsymbol{x}_2)$ as $p(\boldsymbol{x}_2)p(\boldsymbol{x}_1|\boldsymbol{x}_2)$ as follows:

$$p(\boldsymbol{x}_1, \boldsymbol{x}_2) \propto \exp\left\{-\frac{1}{2}\begin{pmatrix} \boldsymbol{x}_1 - \boldsymbol{\mu}_1 \\ \boldsymbol{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix}^\mathsf{T} \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix}^{-1} \begin{pmatrix} \boldsymbol{x}_1 - \boldsymbol{\mu}_1 \\ \boldsymbol{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix}\right\} \tag{7.136}$$

Using Equation (7.118) the above exponent becomes

$$p(\boldsymbol{x}_1, \boldsymbol{x}_2) \propto \exp\left\{ -\frac{1}{2} \begin{pmatrix} \boldsymbol{x}_1 - \boldsymbol{\mu}_1 \\ \boldsymbol{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix}^\mathsf{T} \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21} & \mathbf{I} \end{pmatrix} \begin{pmatrix} (\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22})^{-1} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{22}^{-1} \end{pmatrix} \right. \tag{7.137}$$

$$\left. \times \begin{pmatrix} \mathbf{I} & -\boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \boldsymbol{x}_1 - \boldsymbol{\mu}_1 \\ \boldsymbol{x}_2 - \boldsymbol{\mu}_2 \end{pmatrix} \right\} \tag{7.138}$$

$$= \exp\left\{ -\frac{1}{2} (\boldsymbol{x}_1 - \boldsymbol{\mu}_1 - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\boldsymbol{x}_2 - \boldsymbol{\mu}_2))^\mathsf{T} (\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22})^{-1} \right. \tag{7.139}$$

$$\left. (\boldsymbol{x}_1 - \boldsymbol{\mu}_1 - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\boldsymbol{x}_2 - \boldsymbol{\mu}_2)) \right\} \times \exp\left\{ -\frac{1}{2} (\boldsymbol{x}_2 - \boldsymbol{\mu}_2)^\mathsf{T} \boldsymbol{\Sigma}_{22}^{-1} (\boldsymbol{x}_2 - \boldsymbol{\mu}_2) \right\} \tag{7.140}$$

This is of the form

$$\exp(\text{quadratic form in } \boldsymbol{x}_1, \boldsymbol{x}_2) \times \exp(\text{quadratic form in } \boldsymbol{x}_2) \tag{7.141}$$

Hence we have successfully factorized the joint as

$$p(\boldsymbol{x}_1, \boldsymbol{x}_2) = p(\boldsymbol{x}_1|\boldsymbol{x}_2)p(\boldsymbol{x}_2) \tag{7.142}$$

$$= \mathcal{N}(\boldsymbol{x}_1|\boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2})\mathcal{N}(\boldsymbol{x}_2|\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_{22}) \tag{7.143}$$

where the parameters of the conditional distribution can be read off from the above equations using

$$\boldsymbol{\mu}_{1|2} = \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\boldsymbol{x}_2 - \boldsymbol{\mu}_2) \tag{7.144}$$

$$\boldsymbol{\Sigma}_{1|2} = \boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21} \tag{7.145}$$

We can also use the fact that $|\mathbf{M}| = |\mathbf{M}/\mathbf{H}||\mathbf{H}|$ to check the normalization constants are correct:

$$(2\pi)^{(d_1+d_2)/2}|\boldsymbol{\Sigma}|^{\frac{1}{2}} = (2\pi)^{(d_1+d_2)/2}(|\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22}|\ |\boldsymbol{\Sigma}_{22}|)^{\frac{1}{2}} \tag{7.146}$$

$$= (2\pi)^{d_1/2}|\boldsymbol{\Sigma}/\boldsymbol{\Sigma}_{22}|^{\frac{1}{2}}\ (2\pi)^{d_2/2}|\boldsymbol{\Sigma}_{22}|^{\frac{1}{2}} \tag{7.147}$$

where $d_1 = \dim(\boldsymbol{x}_1)$ and $d_2 = \dim(\boldsymbol{x}_2)$.

7.4 Eigenvalue decomposition (EVD)

In this section, we review some standard material on the **eigenvalue decomposition** or **EVD** of square (real-valued) matrices.

7.4.1 Basics

Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, we say that $\lambda \in \mathbb{R}$ is an **eigenvalue** of $\mathbf{A}$ and $\boldsymbol{u} \in \mathbb{R}^n$ is the corresponding **eigenvector** if

$$\mathbf{A}\boldsymbol{u} = \lambda \boldsymbol{u}, \quad \boldsymbol{u} \neq 0\ . \tag{7.148}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Intuitively, this definition means that multiplying $\mathbf{A}$ by the vector $\boldsymbol{u}$ results in a new vector that points in the same direction as $\boldsymbol{u}$, but is scaled by a factor λ. For example, if $\mathbf{A}$ is a rotation matrix, then $\boldsymbol{u}$ is the axis of rotation and $\lambda = 1$.

Note that for any eigenvector $\boldsymbol{u} \in \mathbb{R}^n$, and scalar $c \in \mathbb{R}$,

$$\mathbf{A}(c\boldsymbol{u}) = c\mathbf{A}\boldsymbol{u} = c\lambda\boldsymbol{u} = \lambda(c\boldsymbol{u}) \tag{7.149}$$

Hence $c\boldsymbol{u}$ is also an eigenvector. For this reason when we talk about "the" eigenvector associated with λ, we usually assume that the eigenvector is normalized to have length 1 (this still creates some ambiguity, since $\boldsymbol{u}$ and $-\boldsymbol{u}$ will both be eigenvectors, but we will have to live with this).

We can rewrite the equation above to state that $(\lambda, \boldsymbol{x})$ is an eigenvalue-eigenvector pair of $\mathbf{A}$ if

$$(\lambda\mathbf{I} - \mathbf{A})\boldsymbol{u} = \mathbf{0}, \quad \boldsymbol{u} \neq 0 \ . \tag{7.150}$$

Now $(\lambda\mathbf{I} - \mathbf{A})\boldsymbol{u} = \mathbf{0}$ has a non-zero solution to $\boldsymbol{u}$ if and only if $(\lambda\mathbf{I} - \mathbf{A})$ has a non-empty nullspace, which is only the case if $(\lambda\mathbf{I} - \mathbf{A})$ is singular, i.e.,

$$\det(\lambda\mathbf{I} - \mathbf{A}) = 0 \ . \tag{7.151}$$

This is called the **characteristic equation** of $\mathbf{A}$. (See Exercise 7.2.) The n solutions of this equation are the n (possibly complex-valued) eigenvalues λ_i, and $\boldsymbol{u}_i$ are the corresponding eigenvectors. It is standard to sort the eigenvectors in order of their eigenvalues, with the largest magnitude ones first.

The following are properties of eigenvalues and eigenvectors.

- The trace of a matrix is equal to the sum of its eigenvalues,

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i \ . \tag{7.152}$$

- The determinant of $\mathbf{A}$ is equal to the product of its eigenvalues,

$$\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i \ . \tag{7.153}$$

- The rank of $\mathbf{A}$ is equal to the number of non-zero eigenvalues of $\mathbf{A}$.
- If $\mathbf{A}$ is non-singular then $1/\lambda_i$ is an eigenvalue of $\mathbf{A}^{-1}$ with associated eigenvector $\boldsymbol{u}_i$, i.e., $\mathbf{A}^{-1}\boldsymbol{u}_i = (1/\lambda_i)\boldsymbol{u}_i$.
- The eigenvalues of a diagonal or triangular matrix are just the diagonal entries.

7.4.2 Diagonalization

We can write all the eigenvector equations simultaneously as

$$\mathbf{AU} = \mathbf{U}\boldsymbol{\Lambda} \tag{7.154}$$

where the columns of $\mathbf{U} \in \mathbb{R}^{n \times n}$ are the eigenvectors of $\mathbf{A}$ and $\mathbf{\Lambda}$ is a diagonal matrix whose entries are the eigenvalues of $\mathbf{A}$, i.e.,

$$\mathbf{U} \in \mathbb{R}^{n \times n} = \begin{bmatrix} | & | & & | \\ \boldsymbol{u}_1 & \boldsymbol{u}_2 & \cdots & \boldsymbol{u}_n \\ | & | & & | \end{bmatrix}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_n) \ . \tag{7.155}$$

If the eigenvectors of $\mathbf{A}$ are linearly independent, then the matrix $\mathbf{U}$ will be invertible, so

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}. \tag{7.156}$$

A matrix that can be written in this form is called **diagonalizable**.

7.4.3 Eigenvalues and eigenvectors of symmetric matrices

When $\mathbf{A}$ is real and symmetric, it can be shown that all the eigenvalues are real, and the eigenvectors are **orthonormal**, i.e., $\boldsymbol{u}_i^\mathsf{T}\boldsymbol{u}_j = 0$ if $i \neq j$, and $\boldsymbol{u}_i^\mathsf{T}\boldsymbol{u}_i = 1$, where $\boldsymbol{u}_i$ are the eigenvectors. In matrix form, this becomes $\mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{U}\mathbf{U}^\mathsf{T} = \mathbf{I}$; hence we see that $\mathbf{U}$ is an orthogonal matrix.

We can therefore represent $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\mathsf{T} = \begin{pmatrix} | & | & & | \\ \boldsymbol{u}_1 & \boldsymbol{u}_2 & \cdots & \boldsymbol{u}_n \\ | & | & & | \end{pmatrix} \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{pmatrix} \begin{pmatrix} - & \boldsymbol{u}_1^\mathsf{T} & - \\ - & \boldsymbol{u}_2^\mathsf{T} & - \\ & \vdots & \\ - & \boldsymbol{u}_n^\mathsf{T} & - \end{pmatrix} \tag{7.157}$$

$$= \lambda_1 \begin{pmatrix} | \\ \boldsymbol{u}_1 \\ | \end{pmatrix} \begin{pmatrix} - & \boldsymbol{u}_1^\mathsf{T} & - \end{pmatrix} + \cdots + \lambda_n \begin{pmatrix} | \\ \boldsymbol{u}_n \\ | \end{pmatrix} \begin{pmatrix} - & \boldsymbol{u}_n^\mathsf{T} & - \end{pmatrix} = \sum_{i=1}^{n} \lambda_i \boldsymbol{u}_i \boldsymbol{u}_i^\mathsf{T} \tag{7.158}$$

Thus multiplying by any symmetric matrix $\mathbf{A}$ can be interpreted as multiplying by a rotation matrix $\mathbf{U}^\mathsf{T}$, a scaling matrix $\mathbf{\Lambda}$, followed by an inverse rotation $\mathbf{U}$.

Once we have diagonalized a matrix, it is easy to invert. Since $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\mathsf{T}$, where $\mathbf{U}^\mathsf{T} = \mathbf{U}^{-1}$, we have

$$\mathbf{A}^{-1} = \mathbf{U}\mathbf{\Lambda}^{-1}\mathbf{U}^\mathsf{T} = \sum_{i=1}^{d} \frac{1}{\lambda_i} \boldsymbol{u}_i \boldsymbol{u}_i^\mathsf{T} \tag{7.159}$$

This corresponds to rotating, unscaling, and then rotating back.

7.4.3.1 Checking for positive definiteness

We can also use the diagonalization property to show that a symmetric matrix is positive definite iff all its eigenvalues are positive. To see this, note that

$$\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} = \boldsymbol{x}^\mathsf{T}\mathbf{U}\mathbf{\Lambda}\mathbf{U}^\mathsf{T}\boldsymbol{x} = \boldsymbol{y}^\mathsf{T}\mathbf{\Lambda}\boldsymbol{y} = \sum_{i=1}^{n} \lambda_i y_i^2 \tag{7.160}$$

where $\boldsymbol{y} = \mathbf{U}^\mathsf{T}\boldsymbol{x}$. Because y_i^2 is always nonnegative, the sign of this expression depends entirely on the λ_i's. If all $\lambda_i > 0$, then the matrix is positive definite; if all $\lambda_i \geq 0$, it is positive semidefinite. Likewise, if all $\lambda_i < 0$ or $\lambda_i \leq 0$, then $\mathbf{A}$ is negative definite or negative semidefinite respectively. Finally, if $\mathbf{A}$ has both positive and negative eigenvalues, it is indefinite.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

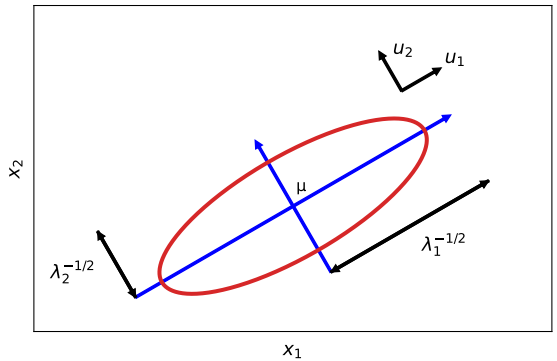


Figure 7.6: Visualization of a level set of the quadratic form $(\boldsymbol{x}-\boldsymbol{\mu})^\mathsf{T}\mathbf{A}(\boldsymbol{x}-\boldsymbol{\mu})$ in 2d. The major and minor axes of the ellipse are defined by the first two eigenvectors of $\mathbf{A}$, namely $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$. Adapted from Figure 2.7 of [Bis06]. Generated by gaussEvec.ipynb.

7.4.4 Geometry of quadratic forms

A **quadratic form** is a function that can be written as

$$f(\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} \tag{7.161}$$

where $\boldsymbol{x} \in \mathbb{R}^n$ and $\mathbf{A}$ is a positive definite, symmetric n-by-n matrix. Let $\mathbf{A} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\mathsf{T}$ be a diagonalization of $\mathbf{A}$ (see Section 7.4.3). Hence we can write

$$f(\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} = \boldsymbol{x}^\mathsf{T}\mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\mathsf{T}\boldsymbol{x} = \boldsymbol{y}^\mathsf{T}\boldsymbol{\Lambda}\boldsymbol{y} = \sum_{i=1}^{n}\lambda_i y_i^2 \tag{7.162}$$

where $y_i = \boldsymbol{x}^\mathsf{T}\boldsymbol{u}_i$ and $\lambda_i > 0$ (since $\mathbf{A}$ is positive definite). The level sets of $f(\boldsymbol{x})$ define hyper-ellipsoids. For example, in 2d, we have

$$\lambda_1 y_1^2 + \lambda_2 y_2^2 = r \tag{7.163}$$

which is the equation of a 2d ellipse. This is illustrated in Figure 7.6. The eigenvectors determine the orientation of the ellipse, and the eigenvalues determine how elongated it is. In particular, the major and minor semi-axes of the ellipse satisfy $a^{-2} = \lambda_1$ and $b^{-2} = \lambda_2$. In the case of a Gaussian distribution, we have $\mathbf{A} = \boldsymbol{\Sigma}^{-1}$, so small values of λ_i correspond to directions where the posterior has low precision and hence high variance.

7.4.5 Standardizing and whitening data

Suppose we have a dataset $\mathbf{X} \in \mathbb{R}^{N\times D}$. It is common to preprocess the data so that each column has zero mean and unit variance. This is called standardizing the data, as we discuss in Section 10.2.8. Although standardizing forces the variance to be 1, it does not remove correlation between the columns. To do that, we must **whiten** the data. To define this, let the empirical covariance matrix

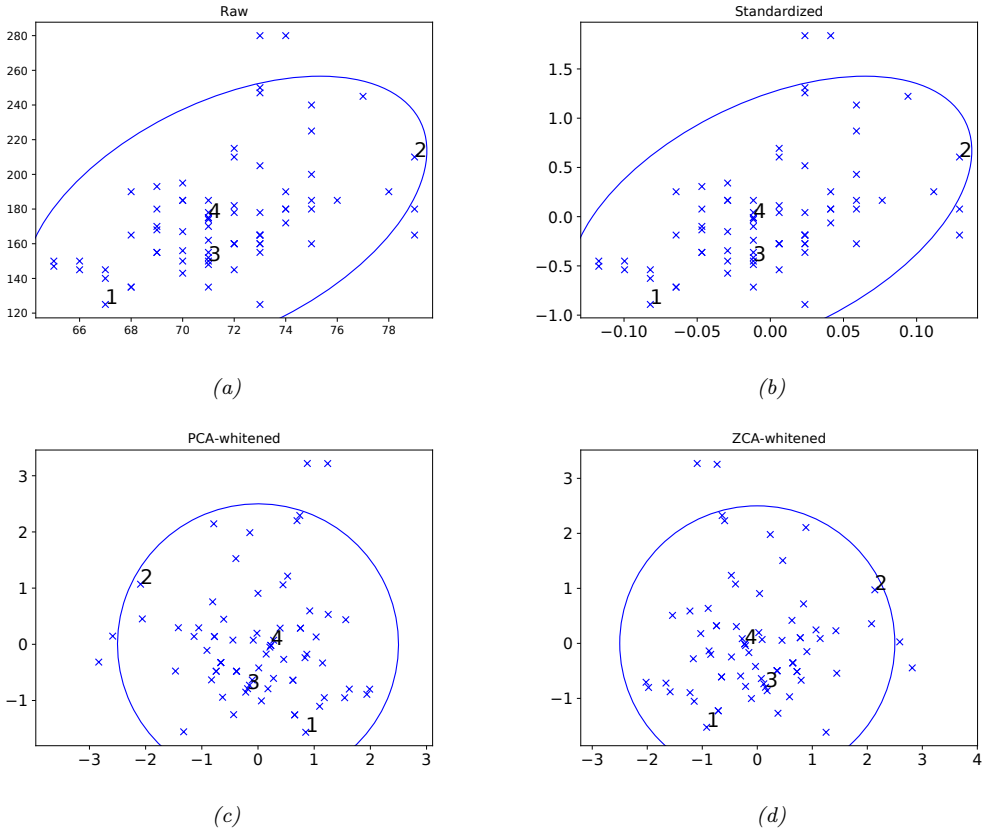


Figure 7.7: (a) Height/weight data. (b) Standardized. (c) PCA Whitening. (d) ZCA whitening. Numbers refer to the first 4 datapoints, but there are 73 datapoints in total. Generated by height_weight_whiten_plot.ipynb.

be $\mathbf{\Sigma} = \frac{1}{N}\mathbf{X}^\mathsf{T}\mathbf{X}$, and let $\mathbf{\Sigma} = \mathbf{E}\mathbf{D}\mathbf{E}^\mathsf{T}$ be its diagonalization. Equivalently, let $[\mathbf{U}, \mathbf{S}, \mathbf{V}]$ be the SVD of $\frac{1}{\sqrt{N}}\mathbf{X}$ (so $\mathbf{E} = \mathbf{V}$ and $\mathbf{D} = \mathbf{S}^2$, as we discuss in Section 20.1.3.3.) Now define

$$\mathbf{W}_{pca} = \mathbf{D}^{-\frac{1}{2}}\mathbf{E}^\mathsf{T} \tag{7.164}$$

This is called the **PCA whitening** matrix. (We discuss PCA in Section 20.1.) Let $\boldsymbol{y} = \mathbf{W}_{pca}\boldsymbol{x}$ be a transformed vector. We can check that its covariance is white as follows:

$$\text{Cov}\left[\boldsymbol{y}\right] = \mathbf{W}\mathbb{E}\left[\boldsymbol{x}\boldsymbol{x}^\mathsf{T}\right]\mathbf{W}^\mathsf{T} = \mathbf{W}\mathbf{\Sigma}\mathbf{W}^\mathsf{T} = (\mathbf{D}^{-\frac{1}{2}}\mathbf{E}^\mathsf{T})(\mathbf{E}\mathbf{D}\mathbf{E}^\mathsf{T})(\mathbf{E}\mathbf{D}^{-\frac{1}{2}}) = \mathbf{I} \tag{7.165}$$

The whitening matrix is not unique, since any rotation of it, $\mathbf{W} = \mathbf{R}\mathbf{W}_{pca}$, will still maintain the whitening property, i.e., $\mathbf{W}^\mathsf{T}\mathbf{W} = \mathbf{\Sigma}^{-1}$. For example, if we take $\mathbf{R} = \mathbf{E}$, we get

$$\mathbf{W}_{zca} = \mathbf{E}\mathbf{D}^{-\frac{1}{2}}\mathbf{E}^\mathsf{T} = \mathbf{\Sigma}^{-\frac{1}{2}} = \mathbf{V}\mathbf{S}^{-1}\mathbf{V}^\mathsf{T} \tag{7.166}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

This is called **Mahalanobis whitening** or **ZCA**. (ZCA stands for "zero-phase component analysis", and was introduced in [BS97].) The advantage of ZCA whitening over PCA whitening is that the resulting transformed data is as close as possible to the original data (in the least squares sense) [Amo17]. This is illustrated in Figure 7.7. When applied to images, the ZCA transformed data vectors still look like images. This is useful when the method is used inside a deep learning system [KH09].

7.4.6 Power method

We now describe a simple iterative method for computing the eigenvector corresponding to the largest eigenvalue of a real, symmetric matrix; this is called the **power method**. This can be useful when the matrix is very large but sparse. For example, it is used by Google's **PageRank** to compute the stationary distribution of the transition matrix of the world wide web (a matrix of size about 3 billion by 3 billion!). In Section 7.4.7, we will see how to use this method to compute subsequent eigenvectors and values.

Let $\mathbf{A}$ be a matrix with orthonormal eigenvectors $\boldsymbol{u}_i$ and eigenvalues $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_m| \geq 0$, so $\mathbf{A} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\mathsf{T}$. Let $\boldsymbol{v}_{(0)}$ be an arbitrary vector in the range of $\mathbf{A}$, so $\mathbf{A}\boldsymbol{x} = \boldsymbol{v}_{(0)}$ for some $\boldsymbol{x}$. Hence we can write $\boldsymbol{v}_{(0)}$ as

$$\boldsymbol{v}_0 = \mathbf{U}(\boldsymbol{\Lambda}\mathbf{U}^\mathsf{T}\boldsymbol{x}) = a_1\boldsymbol{u}_1 + \cdots + a_m\boldsymbol{u}_m \tag{7.167}$$

for some constants a_i. We can now repeatedly multiply $\boldsymbol{v}$ by $\mathbf{A}$ and renormalize:

$$\boldsymbol{v}_t \propto \mathbf{A}\boldsymbol{v}_{t-1} \tag{7.168}$$

(We normalize at each iteration for numerical stability.)

Since $\boldsymbol{v}_t$ is a multiple of $\mathbf{A}^t\boldsymbol{v}_0$, we have

$$\boldsymbol{v}_t \propto a_1\lambda_1^t\boldsymbol{u}_1 + a_2\lambda_2^t\boldsymbol{u}_2 + \cdots + a_m\lambda_m^t\boldsymbol{u}_m \tag{7.169}$$

$$= \lambda_1^t\left(a_1\boldsymbol{u}_1 + a_1(\lambda_2/\lambda_1)^t\boldsymbol{u}_2 + \cdots + a_m(\lambda_m/\lambda_1)^t\boldsymbol{u}_m\right) \tag{7.170}$$

$$\rightarrow \lambda_1^t a_1\boldsymbol{u}_1 \tag{7.171}$$

since $\frac{|\lambda_k|}{|\lambda_1|} < 1$ for $k > 1$ (assuming the eigenvalues are sorted in descending order). So we see that this converges to $\boldsymbol{u}_1$, although not very quickly (the error is reduced by approximately $|\lambda_2/\lambda_1|$ at each iteration). The only requirement is that the initial guess satisfy $\boldsymbol{v}_0^\mathsf{T}\boldsymbol{u}_1 \neq 0$, which will be true for a random $\boldsymbol{v}_0$ with high probability.

We now discuss how to compute the corresponding eigenvalue, λ_1. Define the **Rayleigh quotient** to be

$$R(\mathbf{A}, \boldsymbol{x}) \triangleq \frac{\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}}{\boldsymbol{x}^\mathsf{T}\boldsymbol{x}} \tag{7.172}$$

Hence

$$R(\mathbf{A}, \boldsymbol{u}_i) = \frac{\boldsymbol{u}_i^\mathsf{T}\mathbf{A}\boldsymbol{u}_i}{\boldsymbol{u}_i^\mathsf{T}\boldsymbol{u}_i} = \frac{\lambda_i\boldsymbol{u}_i^\mathsf{T}\boldsymbol{u}_i}{\boldsymbol{u}_i^\mathsf{T}\boldsymbol{u}_i} = \lambda_i \tag{7.173}$$

Thus we can easily compute λ_1 from $\boldsymbol{u}_1$ and $\mathbf{A}$. See power_method_demo.ipynb for a demo.

7.4.7 Deflation

Suppose we have computed the first eigenvector and value $\boldsymbol{u}_1, \lambda_1$ by the power method. We now describe how to compute subsequent eigenvectors and values. Since the eigenvectors are orthonormal, and the eigenvalues are real, we can project out the $\boldsymbol{u}_1$ component from the matrix as follows:

$$\mathbf{A}^{(2)} = (\mathbf{I} - \boldsymbol{u}_1\boldsymbol{u}_1^\mathsf{T})\mathbf{A}^{(1)} = \mathbf{A}^{(1)} - \boldsymbol{u}_1\boldsymbol{u}_1^\mathsf{T}\mathbf{A}^{(1)} = \mathbf{A}^{(1)} - \lambda_1\boldsymbol{u}_1\boldsymbol{u}_1^\mathsf{T} \tag{7.174}$$

This is called matrix **deflation**. We can then apply the power method to $\mathbf{A}^{(2)}$, which will find the largest eigenvector/value in the subspace orthogonal to $\boldsymbol{u}_1$.

In Section 20.1.2, we show that the optimal estimate $\hat{\mathbf{W}}$ for the PCA model (described in Section 20.1) is given by the first K eigenvectors of the empirical covariance matrix. Hence deflation can be used to implement PCA. It can also be modified to implement sparse PCA [Mac09].

7.4.8 Eigenvectors optimize quadratic forms

We can use matrix calculus to solve an optimization problem in a way that leads directly to eigenvalue/eigenvector analysis. Consider the following, equality constrained optimization problem:

$$\max_{\boldsymbol{x}\in\mathbb{R}^n} \ \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} \quad \text{subject to } \|\boldsymbol{x}\|_2^2 = 1 \tag{7.175}$$

for a symmetric matrix $\mathbf{A} \in \mathbb{S}^n$. A standard way of solving optimization problems with equality constraints is by forming the Lagrangian, an objective function that includes the equality constraints (see Section 8.5.1). The Lagrangian in this case can be given by

$$\mathcal{L}(\boldsymbol{x}, \lambda) = \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} + \lambda(1 - \boldsymbol{x}^\mathsf{T}\boldsymbol{x}) \tag{7.176}$$

where λ is called the Lagrange multiplier associated with the equality constraint. It can be established that for x^* to be a optimal point to the problem, the gradient of the Lagrangian has to be zero at x^* (this is not the only condition, but it is required). That is,

$$\nabla_{\boldsymbol{x}}\mathcal{L}(\boldsymbol{x}, \lambda) = 2\mathbf{A}^\mathsf{T}\boldsymbol{x} - 2\lambda\boldsymbol{x} = \mathbf{0}. \tag{7.177}$$

Notice that this is just the linear equation $\mathbf{A}\boldsymbol{x} = \lambda\boldsymbol{x}$. This shows that the only points which can possibly maximize (or minimize) $\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}$ assuming $\boldsymbol{x}^\mathsf{T}\boldsymbol{x} = 1$ are the eigenvectors of $\mathbf{A}$.

7.5 Singular value decomposition (SVD)

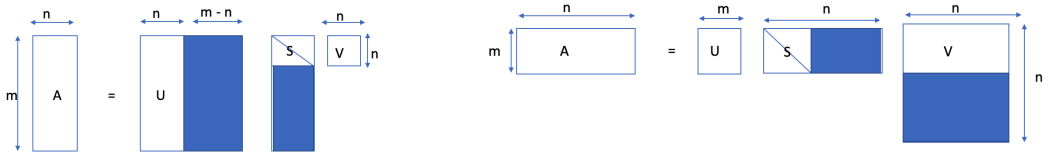
We now discuss the SVD, which generalizes EVD to rectangular matrices.

7.5.1 Basics

Any (real) $m \times n$ matrix $\mathbf{A}$ can be decomposed as

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} = \sigma_1 \begin{pmatrix} | \\ \boldsymbol{u}_1 \\ | \end{pmatrix} \begin{pmatrix} - & \boldsymbol{v}_1^\mathsf{T} & - \end{pmatrix} + \cdots + \sigma_r \begin{pmatrix} | \\ \boldsymbol{u}_r \\ | \end{pmatrix} \begin{pmatrix} - & \boldsymbol{v}_r^\mathsf{T} & - \end{pmatrix} \tag{7.178}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 7.8: SVD decomposition of a matrix, $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$. The shaded parts of each matrix are not computed in the economy-sized version. (a) Tall skinny matrix. (b) Short wide matrix.

where $\mathbf{U}$ is an $m \times m$ whose columns are orthornormal (so $\mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{I}_m$), $\mathbf{V}$ is $n \times n$ matrix whose rows and columns are orthonormal (so $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{I}_n$), and $\mathbf{S}$ is a $m \times n$ matrix containing the $r = \min(m, n)$ **singular values** $\sigma_i \geq 0$ on the main diagonal, with 0s filling the rest of the matrix. The columns of $\mathbf{U}$ are the left **singular vectors**, and the columns of $\mathbf{V}$ are the right singular vectors. This is called the **singular value decomposition** or **SVD** of the matrix. See Figure 7.8 for an example.

As is apparent from Figure 7.8a, if $m > n$, there are at most n singular values, so the last $m - n$ columns of $\mathbf{U}$ are irrelevant (since they will be multiplied by 0). The **economy sized SVD**, also called a **thin SVD**, avoids computing these unnecessary elements. In other words, if we write the $\mathbf{U}$ matrix as $\mathbf{U} = [\mathbf{U}_1, \mathbf{U}_2]$, we only compute $\mathbf{U}_1$. Figure 7.8b shows the opposite case, where $m < n$, where we represent $\mathbf{V} = [\mathbf{V}_1; \mathbf{V}_2]$, and only compute $\mathbf{V}_1$.

The cost of computing the SVD is $O(\min(mn^2, m^2n))$. Details on how it works can be found in standard linear algebra textbooks.

7.5.2 Connection between SVD and EVD

If $\mathbf{A}$ is real, symmetric and positive definite, then the singular values are equal to the eigenvalues, and the left and right singular vectors are equal to the eigenvectors (up to a sign change):

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} = \mathbf{U}\mathbf{S}\mathbf{U}^\mathsf{T} = \mathbf{U}\mathbf{S}\mathbf{U}^{-1} \tag{7.179}$$

Note, however, that NumPy always returns the singular values in decreasing order, whereas the eigenvalues need not necessarily be sorted.

In general, for an arbitrary real matrix $\mathbf{A}$, if $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$, we have

$$\mathbf{A}^\mathsf{T}\mathbf{A} = \mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T}\ \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} = \mathbf{V}(\mathbf{S}^\mathsf{T}\mathbf{S})\mathbf{V}^\mathsf{T} \tag{7.180}$$

Hence

$$(\mathbf{A}^\mathsf{T}\mathbf{A})\mathbf{V} = \mathbf{V}\mathbf{D}_n \tag{7.181}$$

so the eigenvectors of $\mathbf{A}^\mathsf{T}\mathbf{A}$ are equal to $\mathbf{V}$, the right singular vectors of $\mathbf{A}$, and the eigenvalues of $\mathbf{A}^\mathsf{T}\mathbf{A}$ are equal to $\mathbf{D}_n = \mathbf{S}^\mathsf{T}\mathbf{S}$, which is an $n \times n$ diagonal matrix containing the squared singular values. Similarly

$$\mathbf{A}\mathbf{A}^\mathsf{T} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}\ \mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T} = \mathbf{U}(\mathbf{S}\mathbf{S}^\mathsf{T})\mathbf{U}^\mathsf{T} \tag{7.182}$$

$$(\mathbf{A}\mathbf{A}^\mathsf{T})\mathbf{U} = \mathbf{U}\mathbf{D}_m \tag{7.183}$$

so the eigenvectors of $\mathbf{A}\mathbf{A}^\mathsf{T}$ are equal to $\mathbf{U}$, the left singular vectors of $\mathbf{A}$, and the eigenvalues of $\mathbf{A}\mathbf{A}^\mathsf{T}$ are equal to $\mathbf{D}_m = \mathbf{S}\mathbf{S}^\mathsf{T}$, which is an $m \times m$ diagonal matrix containing the squared singular values. In summary,

$$\mathbf{U} = \text{evec}(\mathbf{A}\mathbf{A}^\mathsf{T}),\ \mathbf{V} = \text{evec}(\mathbf{A}^\mathsf{T}\mathbf{A}),\ \mathbf{D}_m = \text{eval}(\mathbf{A}\mathbf{A}^\mathsf{T}), \mathbf{D}_n = \text{eval}(\mathbf{A}^\mathsf{T}\mathbf{A}) \tag{7.184}$$

If we just use the computed (non-zero) parts in the economy-sized SVD, then we can define

$$\mathbf{D} = \mathbf{S}^2 = \mathbf{S}^\mathsf{T}\mathbf{S} = \mathbf{S}\mathbf{S}^\mathsf{T} \tag{7.185}$$

Note also that an EVD does not always exist, even for square $\mathbf{A}$, whereas an SVD always exists.

7.5.3 Pseudo inverse

The **Moore-Penrose pseudo-inverse** of $\mathbf{A}$, pseudo inverse denoted $\mathbf{A}^\dagger$, is defined as the unique matrix that satisfies the following 4 properties:

$$\mathbf{A}\mathbf{A}^\dagger\mathbf{A} = \mathbf{A},\ \mathbf{A}^\dagger\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}^\dagger,\ (\mathbf{A}\mathbf{A}^\dagger)^\mathsf{T} = \mathbf{A}\mathbf{A}^\dagger,\ (\mathbf{A}^\dagger\mathbf{A})^\mathsf{T} = \mathbf{A}^\dagger\mathbf{A} \tag{7.186}$$

If $\mathbf{A}$ is square and non-singular, then $\mathbf{A}^\dagger = \mathbf{A}^{-1}$.

If $m > n$ (tall, skinny) and the columns of $\mathbf{A}$ are linearly independent (so $\mathbf{A}$ is full rank), then

$$\mathbf{A}^\dagger = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T} \tag{7.187}$$

which is the same expression as arises in the normal equations (see Section 11.2.2.1). In this case, $\mathbf{A}^\dagger$ is a left inverse of $\mathbf{A}$ because

$$\mathbf{A}^\dagger\mathbf{A} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{A} = \mathbf{I} \tag{7.188}$$

but is not a right inverse because

$$\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}(\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T} \tag{7.189}$$

only has rank n, and so cannot be the $m \times m$ identity matrix.

If $m < n$ (short, fat) and the rows of $\mathbf{A}$ are linearly independent (so $\mathbf{A}^\mathsf{T}$ is full rank), then the pseudo inverse is

$$\mathbf{A}^\dagger = \mathbf{A}^\mathsf{T}(\mathbf{A}\mathbf{A}^\mathsf{T})^{-1} \tag{7.190}$$

In this case, $\mathbf{A}^\dagger$ is a right inverse of $\mathbf{A}$.

We can compute the pseudo inverse using the SVD decomposition $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$. In particular, one can show that

$$\mathbf{A}^\dagger = \mathbf{V}[\text{diag}(1/\sigma_1, \cdots, 1/\sigma_r, 0, \cdots, 0)]\mathbf{U}^\mathsf{T} = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\mathsf{T} \tag{7.191}$$

where r is the rank of the matrix, and where we define $\mathbf{S}^{-1} = \text{diag}(\sigma_1^{-1}, \ldots, \sigma_r^{-1}, 0, \ldots, 0)$. Indeed if the matrices were square and full rank we would have

$$(\mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T})^{-1} = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\mathsf{T} \tag{7.192}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.5.4 SVD and the range and null space of a matrix *

In this section, we show that the left and right singular vectors form an orthonormal basis for the range and null space.

From Equation (7.178) we have

$$\mathbf{A}\boldsymbol{x} = \sum_{j:\sigma_j>0} \sigma_j(\boldsymbol{v}_j^\mathsf{T}\boldsymbol{x})\boldsymbol{u}_j = \sum_{j=1}^{r} \sigma_j(\boldsymbol{v}_j^\mathsf{T}\boldsymbol{x})\boldsymbol{u}_j \tag{7.193}$$

where r is the rank of $\mathbf{A}$. Thus any $\mathbf{A}\boldsymbol{x}$ can be written as a linear combination of the left singular vectors $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$, so the range of $\mathbf{A}$ is given by

$$\text{range}(\mathbf{A}) = \text{span}\left(\{\boldsymbol{u}_j : \sigma_j > 0\}\right) \tag{7.194}$$

with dimension r.

To find a basis for the null space, let us now define a second vector $\boldsymbol{y} \in \mathbb{R}^n$ that is a linear combination solely of the right singular vectors for the zero singular values,

$$\boldsymbol{y} = \sum_{j:\sigma_j=0} c_j\boldsymbol{v}_j = \sum_{j=r+1}^{n} c_j\boldsymbol{v}_j \tag{7.195}$$

Since the $\boldsymbol{v}_j$'s are orthonormal, we have

$$\mathbf{A}\boldsymbol{y} = \mathbf{U}\begin{pmatrix} \sigma_1\boldsymbol{v}_1^\mathsf{T}\boldsymbol{y} \\ \vdots \\ \sigma_r\boldsymbol{v}_r^\mathsf{T}\boldsymbol{y} \\ \sigma_{r+1}\boldsymbol{v}_{r+1}^\mathsf{T}\boldsymbol{y} \\ \vdots \\ \sigma_n\boldsymbol{v}_n^\mathsf{T}\boldsymbol{y} \end{pmatrix} = \mathbf{U}\begin{pmatrix} \sigma_1 0 \\ \vdots \\ \sigma_r 0 \\ 0\boldsymbol{v}_{r+1}^\mathsf{T}\boldsymbol{y} \\ \vdots \\ 0\boldsymbol{v}_n^\mathsf{T}\boldsymbol{y} \end{pmatrix} = \mathbf{U}\mathbf{0} = \mathbf{0} \tag{7.196}$$

Hence the right singular vectors form an orthonormal basis for the null space:

$$\text{nullspace}(\mathbf{A}) = \text{span}\left(\{\boldsymbol{v}_j : \sigma_j = 0\}\right) \tag{7.197}$$

with dimension $n - r$. We see that

$$\dim(\text{range}(\mathbf{A})) + \dim(\text{nullspace}(\mathbf{A})) = r + (n - r) = n \tag{7.198}$$

In words, this is often written as

$$\text{rank} + \text{nullity} = n \tag{7.199}$$

This is called the **rank-nullity theorem**. It follows from this that the rank of a matrix is the number of nonzero singular values.

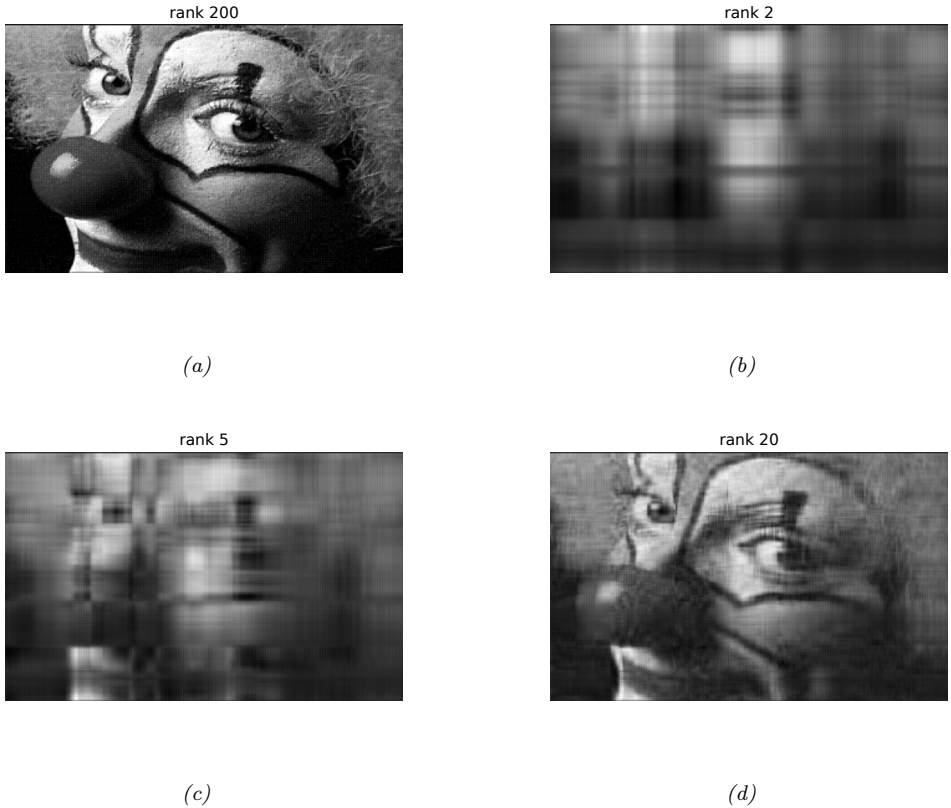


Figure 7.9: Low rank approximations to an image. Top left: The original image is of size 200×320*, so has rank 200. Subsequent images have ranks 2, 5, and 20. Generated by svd_image_demo.ipynb.*

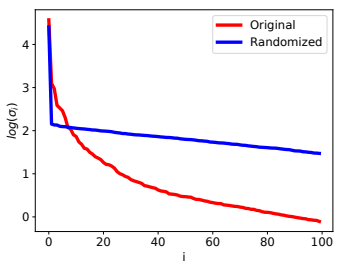


Figure 7.10: First 100 log singular values for the clown image (red line), and for a data matrix obtained by randomly shuffling the pixels (blue line). Generated by svd_image_demo.ipynb. Adapted from Figure 14.24 of [HTF09].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.5.5 Truncated SVD

Let $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be the SVD of $\mathbf{A}$, and let $\hat{\mathbf{A}}_K = \mathbf{U}_K\mathbf{S}_K\mathbf{V}_K^{\mathsf{T}}$, where we use the first K columns of $\mathbf{U}$ and $\mathbf{V}$. This can be shown to be the optimal rank K approximation, in the sense that it minimizes $||\mathbf{A} - \hat{\mathbf{A}}_K||_F^2$.

If $K = r = \text{rank}(\mathbf{A})$, there is no error introduced by this decomposition. But if $K < r$, we incur some error. This is called a **truncated SVD**. If the singular values die off quickly, as is typical in natural data (see e.g., Figure 7.10), the error will be small. The total number of parameters needed to represent an $N \times D$ matrix using a rank K approximation is

$$NK + KD + K = K(N + D + 1) \tag{7.200}$$

As an example, consider the 200×320 pixel image in Figure 7.9(top left). This has 64,000 numbers in it. We see that a rank 20 approximation, with only $(200 + 320 + 1) \times 20 = 10,420$ numbers is a very good approximation.

One can show that the error in this rank-K approximation is given by

$$||\mathbf{A} - \hat{\mathbf{A}}||_F = \sum_{k=K+1}^{r} \sigma_k \tag{7.201}$$

where σ_k is the k'th singular value of $\mathbf{A}$.

7.6 Other matrix decompositions *

In this section, we briefly review some other useful matrix decompositions.

7.6.1 LU factorization

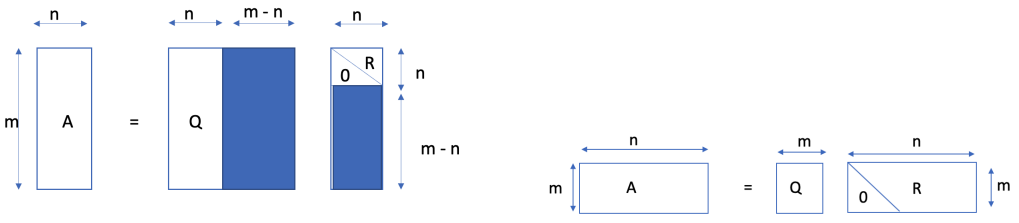
We can factorize any square matrix $\mathbf{A}$ into a product of a lower triangular matrix $\mathbf{L}$ and an upper triangular matrix $\mathbf{U}$. For example,

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}. \tag{7.202}$$

In general we may need to permute the entries in the matrix before creating this decomposition. To see this, suppose $a_{11} = 0$. Since $a_{11} = l_{11}u_{11}$, this means either l_{11} or u_{11} or both must be zero, but that would imply $\mathbf{L}$ or $\mathbf{U}$ are singular. To avoid this, the first step of the algorithm can simply reorder the rows so that the first element is nonzero. This is repeated for subsequent steps. We can denote this process by

$$\mathbf{P}\mathbf{A} = \mathbf{L}\mathbf{U} \tag{7.203}$$

where $\mathbf{P}$ is a permutation matrix, i.e., a square binary matrix where $P_{ij} = 1$ if row j gets permuted to row i. This is called **partial pivoting**.



(a) (b)

Figure 7.11: Illustration of QR decomposition, $\mathbf{A} = \mathbf{QR}$, where $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$ and $\mathbf{R}$ is upper triangular. (a) Tall, skinny matrix. The shaded parts are not computed in the economy-sized version, since they are not needed. (b) Short, wide matrix.

7.6.2 QR decomposition

Suppose we have $\mathbf{A} \in \mathbb{R}^{m \times n}$ representing a set of linearly independent basis vectors (so $m \geq n$), and we want to find a series of orthonormal vectors $\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots$ that span the successive subspaces of $\text{span}(\boldsymbol{a}_1)$, $\text{span}(\boldsymbol{a}_1, \boldsymbol{a}_2)$, etc. In other words, we want to find vectors $\boldsymbol{q}_j$ and coefficients r_{ij} such that

$$\begin{pmatrix} | & | & & | \\ \boldsymbol{a}_1 & \boldsymbol{a}_2 & \cdots & \boldsymbol{a}_n \\ | & | & & | \end{pmatrix} = \begin{pmatrix} | & | & & | \\ \boldsymbol{q}_1 & \boldsymbol{q}_2 & \cdots & \boldsymbol{q}_n \\ | & | & & | \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \\ & & & r_{nn} \end{pmatrix} \tag{7.204}$$

We can write this

$$\boldsymbol{a}_1 = r_{11}\boldsymbol{q}_1 \tag{7.205}$$

$$\boldsymbol{a}_2 = r_{12}\boldsymbol{q}_1 + r_{22}\boldsymbol{q}_2 \tag{7.206}$$

$$\vdots$$

$$\boldsymbol{a}_n = r_{1n}\boldsymbol{q}_1 + \cdots + r_{nn}\boldsymbol{q}_n \tag{7.207}$$

so we see $\boldsymbol{q}_1$ spans the space of $\boldsymbol{a}_1$, and $\boldsymbol{q}_1$ and $\boldsymbol{q}_2$ span the space of $\{\boldsymbol{a}_1, \boldsymbol{a}_2\}$, etc.

In matrix notation, we have

$$\mathbf{A} = \hat{\mathbf{Q}}\hat{\mathbf{R}} \tag{7.208}$$

where $\hat{\mathbf{Q}}$ is $m \times n$ with orthonormal columns and $\hat{\mathbf{R}}$ is $n \times n$ and upper triangular. This is called a **reduced QR** or **economy sized QR** factorization of $\mathbf{A}$; see Figure 7.11.

A full QR factorization appends an additional $m - n$ orthonormal columns to $\hat{\mathbf{Q}}$ so it becomes a square, orthogonal matrix $\mathbf{Q}$, which satisfies $\mathbf{QQ}^\mathsf{T} = \mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$. Also, we append rows made of zero to $\hat{\mathbf{R}}$ so it becomes an $m \times n$ matrix that is still upper triangular, called $\mathbf{R}$: see Figure 7.11. The zero entries in $\mathbf{R}$ "kill off" the new columns in $\mathbf{Q}$, so the result is the same as $\hat{\mathbf{Q}}\hat{\mathbf{R}}$.

QR decomposition is commonly used to solve systems of linear equations, as we discuss in Section 11.2.2.3.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

7.6.3 Cholesky decomposition

Any symmetric positive definite matrix can be factorized as $\mathbf{A} = \mathbf{R}^\mathsf{T}\mathbf{R}$, where $\mathbf{R}$ is upper triangular with real, positive diagonal elements. (This can also be written as $\mathbf{A} = \mathbf{L}\mathbf{L}^\mathsf{T}$, where $\mathbf{L} = \mathbf{R}^\mathsf{T}$ is lower triangular.) This is called a **Cholesky factorization** or **matrix square root**. In NumPy, this is implemented by `np.linalg.cholesky`. The computational complexity of this operation is $O(V^3)$, where V is the number of variables, but can be less for sparse matrices. Below we give some applications of this factorization.

7.6.3.1 Application: Sampling from an MVN

The Cholesky decomposition of a covariance matrix can be used to sample from a multivariate Gaussian. Let $\boldsymbol{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\mathsf{T}$. We first sample $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, which is easy because it just requires sampling from d separate 1d Gaussians. We then set $\boldsymbol{y} = \mathbf{L}\boldsymbol{x} + \boldsymbol{\mu}$. This is valid since

$$\text{Cov}\left[\boldsymbol{y}\right] = \mathbf{L}\text{Cov}\left[\boldsymbol{x}\right]\mathbf{L}^\mathsf{T} = \mathbf{L}\,\mathbf{I}\,\mathbf{L}^\mathsf{T} = \boldsymbol{\Sigma} \tag{7.209}$$

See cholesky_demo.ipynb for some code.

7.7 Solving systems of linear equations *

An important application of linear algebra is the study of systems of linear equations. For example, consider the following set of 3 equations:

$$3x_1 + 2x_2 - x_3 = 1 \tag{7.210}$$
$$2x_1 - 2x_2 + 4x_3 = -2 \tag{7.211}$$
$$-x_1 + \frac{1}{2}x_2 - x_3 = 0 \tag{7.212}$$

We can represent this in matrix-vector form as follows:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \tag{7.213}$$

where

$$\mathbf{A} = \begin{pmatrix} 3 & 2 & -1 \\ 2 & -2 & 4 \\ -1 & \frac{1}{2} & -1 \end{pmatrix}, \; \boldsymbol{b} = \begin{pmatrix} 1 \\ -2 \\ 0 \end{pmatrix} \tag{7.214}$$

The solution is $\boldsymbol{x} = [1, -2, -2]$.

In general, if we have m equations and n unknowns, then $\mathbf{A}$ will be a $m \times n$ matrix, and $\boldsymbol{b}$ will be a $m \times 1$ vector. If $m = n$ (and $\mathbf{A}$ is full rank), there is a single unique solution. If $m < n$, the system is **underdetermined**, so there is not a unique solution. If $m > n$, the system is **overdetermined**, since there are more constraints than unknowns, and not all the lines intersect at the same point. See Figure 7.12 for an illustration. We discuss how to compute solutions in each of these cases below.

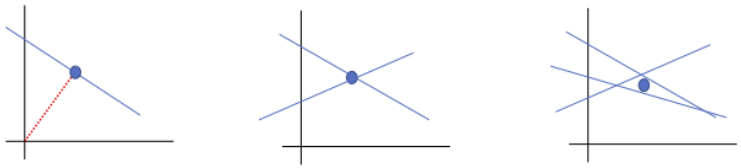


Figure 7.12: Solution of a set of m linear equations in $n = 2$ variables. (a) $m = 1 < n$ so the system is underdetermined. We show the minimal norm solution as a blue circle. (The dotted red line is orthogonal to the line, and its length is the distance to the origin.) (b) $m = n = 2$, so there is a unique solution. (c) $m = 3 > n$, so there is no unique solution. We show the least squares solution.

7.7.1 Solving square systems

In the case where $m = n$, we can solve for $\boldsymbol{x}$ by computing an LU decomposition, $\mathbf{A} = \mathbf{LU}$, and then proceeding as follows:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \tag{7.215}$$

$$\mathbf{LU}\boldsymbol{x} = \boldsymbol{b} \tag{7.216}$$

$$\mathbf{U}\boldsymbol{x} = \mathbf{L}^{-1}\boldsymbol{b} \triangleq \boldsymbol{y} \tag{7.217}$$

$$\boldsymbol{x} = \mathbf{U}^{-1}\boldsymbol{y} \tag{7.218}$$

The crucial point is that $\mathbf{L}$ and $\mathbf{U}$ are both triangular matrices, so we can avoid taking matrix inverses, and use a method known as **backsubstitution** instead.

In particular, we can solve $\boldsymbol{y} = \mathbf{L}^{-1}\boldsymbol{b}$ without taking inverses as follows. First we write

$$\begin{pmatrix} L_{11} & & & \\ L_{21} & L_{22} & & \\ & & \ddots & \\ L_{n1} & L_{n2} & \cdots & L_{nn} \end{pmatrix} \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} \tag{7.219}$$

We start by solving $L_{11}y_1 = b_1$ to find y_1 and then substitute this in to solve

$$L_{21}y_1 + L_{22}y_2 = b_2 \tag{7.220}$$

for y_2. We repeat this recursively. This process is often denoted by the **backslash operator**, $\boldsymbol{y} = \mathbf{L} \setminus \boldsymbol{b}$. Once we have $\boldsymbol{y}$, we can solve $\boldsymbol{x} = \mathbf{U}^{-1}\boldsymbol{y}$ using backsubstitution in a similar manner.

7.7.2 Solving underconstrained systems (least norm estimation)

In this section, we consider the underconstrained setting, where $m < n$.[3] We assume the rows are linearly independent, so $\mathbf{A}$ is full rank.

3. Our presentation is based in part on lecture notes by Stephen Boyd at http://ee263.stanford.edu/lectures/min-norm.pdf.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

When $m < n$, there are multiple possible solutions, which have the form

$$\{\boldsymbol{x} : \mathbf{A}\boldsymbol{x} = \boldsymbol{b}\} = \{\boldsymbol{x}_p + \boldsymbol{z} : \boldsymbol{z} \in \text{nullspace}(\mathbf{A})\} \tag{7.221}$$

where $\boldsymbol{x}_p$ is any particular solution. It is standard to pick the particular solution with minimal ℓ_2 norm, i.e.,

$$\hat{\boldsymbol{x}} = \underset{\boldsymbol{x}}{\text{argmin}} \, ||\boldsymbol{x}||_2^2 \;\; \text{s.t.} \;\; \mathbf{A}\boldsymbol{x} = \boldsymbol{b} \tag{7.222}$$

We can compute the minimal norm solution using the **right pseudo inverse**:

$$\boldsymbol{x}_{\text{pinv}} = \mathbf{A}^\mathsf{T}(\mathbf{A}\mathbf{A}^\mathsf{T})^{-1}\boldsymbol{b} \tag{7.223}$$

(See Section 7.5.3 for more details.)

To see this, suppose $\boldsymbol{x}$ is some other solution, so $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, and $\mathbf{A}(\boldsymbol{x} - \boldsymbol{x}_{\text{pinv}}) = \mathbf{0}$. Thus

$$(\boldsymbol{x} - \boldsymbol{x}_{\text{pinv}})^\mathsf{T}\boldsymbol{x}_{\text{pinv}} = (\boldsymbol{x} - \boldsymbol{x}_{\text{pinv}})^\mathsf{T}\mathbf{A}^\mathsf{T}(\mathbf{A}\mathbf{A}^\mathsf{T})^{-1}\boldsymbol{b} = (\mathbf{A}(\boldsymbol{x} - \boldsymbol{x}_{\text{pinv}}))^\mathsf{T}(\mathbf{A}\mathbf{A}^\mathsf{T})^{-1}\boldsymbol{b} = 0 \tag{7.224}$$

and hence $(\boldsymbol{x} - \boldsymbol{x}_{\text{pinv}}) \perp \boldsymbol{x}_{\text{pinv}}$. By **Pythagoras's theorem**, the norm of $\boldsymbol{x}$ is

$$||\boldsymbol{x}||^2 = ||\boldsymbol{x}_{\text{pinv}} + \boldsymbol{x} - \boldsymbol{x}_{\text{pinv}}||^2 = ||\boldsymbol{x}_{\text{pinv}}||^2 + ||\boldsymbol{x} - \boldsymbol{x}_{\text{pinv}}||^2 \geq ||\boldsymbol{x}_{\text{pinv}}||^2 \tag{7.225}$$

Thus any solution apart from $\boldsymbol{x}_{\text{pinv}}$ has larger norm.

We can also solve the constrained optimization problem in Equation (7.222) by minimizing the following unconstrained objective

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}) = \boldsymbol{x}^\mathsf{T}\boldsymbol{x} + \boldsymbol{\lambda}^\mathsf{T}(\mathbf{A}\boldsymbol{x} - \boldsymbol{b}) \tag{7.226}$$

From Section 8.5.1, the optimality conditions are

$$\nabla_{\boldsymbol{x}}\mathcal{L} = 2\boldsymbol{x} + \mathbf{A}^\mathsf{T}\boldsymbol{\lambda} = \mathbf{0}, \;\; \nabla_{\boldsymbol{\lambda}}\mathcal{L} = \mathbf{A}\boldsymbol{x} - \boldsymbol{b} = \mathbf{0} \tag{7.227}$$

From the first condition we have $\boldsymbol{x} = -\mathbf{A}^\mathsf{T}\boldsymbol{\lambda}/2$. Subsituting into the second we get

$$\mathbf{A}\boldsymbol{x} = -\frac{1}{2}\mathbf{A}\mathbf{A}^\mathsf{T}\boldsymbol{\lambda} = \boldsymbol{b} \tag{7.228}$$

which implies $\boldsymbol{\lambda} = -2(\mathbf{A}\mathbf{A}^\mathsf{T})^{-1}\boldsymbol{b}$. Hence $\boldsymbol{x} = \mathbf{A}^\mathsf{T}(\mathbf{A}\mathbf{A}^\mathsf{T})^{-1}\boldsymbol{b}$, which is the right pseudo inverse solution.

7.7.3 Solving overconstrained systems (least squares estimation)

If $m > n$, we have an overdetermined solution, which typically does not have an exact solution, but we will try to find the solution that gets as close as possible to satisfying all of the constraints specified by $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. We can do this by minimizing the following cost function, known as the **least squares objective**:[4]

$$f(\boldsymbol{x}) = \frac{1}{2}||\mathbf{A}\boldsymbol{x} - \boldsymbol{b}||_2^2 \tag{7.233}$$

4. Note that some equation numbers have been skipped. This is intentional. The reason is that I have omitted some erroneous material from an earlier version (described in https://github.com/probml/pml-book/issues/266), but want to make sure the equation numbering is consistent across different versions of the book.

Using matrix calculus results from Section 7.8 we have that the gradient is given by

$$\boldsymbol{g}(\boldsymbol{x}) = \frac{\partial}{\partial \boldsymbol{x}} f(\boldsymbol{x}) = \mathbf{A}^\mathsf{T}\mathbf{A}\boldsymbol{x} - \mathbf{A}^\mathsf{T}\boldsymbol{b} \tag{7.234}$$

The optimum can be found by solving $\boldsymbol{g}(\boldsymbol{x}) = \mathbf{0}$. This gives

$$\mathbf{A}^\mathsf{T}\mathbf{A}\boldsymbol{x} = \mathbf{A}^\mathsf{T}\boldsymbol{b} \tag{7.235}$$

These are known as the **normal equations**, since, at the optimal solution, $\boldsymbol{b} - \mathbf{A}\boldsymbol{x}$ is normal (orthogonal) to the range of $\mathbf{A}$, as we explain in Section 11.2.2.2. The corresponding solution $\hat{\boldsymbol{x}}$ is the **ordinary least squares** (**OLS**) solution, which is given by

$$\hat{\boldsymbol{x}} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\boldsymbol{b} \tag{7.236}$$

The quantity $\mathbf{A}^\dagger = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ is the **left pseudo inverse** of the (non-square) matrix $\mathbf{A}$ (see Section 7.5.3 for more details).

We can check that the solution is unique by showing that the Hessian is positive definite. In this case, the Hessian is given by

$$\mathbf{H}(\boldsymbol{x}) = \frac{\partial^2}{\partial \boldsymbol{x}^2} f(\boldsymbol{x}) = \mathbf{A}^\mathsf{T}\mathbf{A} \tag{7.237}$$

If $\mathbf{A}$ is full rank (so the columns of $\mathbf{A}$ are linearly independent), then $\mathbf{H}$ is positive definite, since for any $\boldsymbol{v} > \mathbf{0}$, we have

$$\boldsymbol{v}^\mathsf{T}(\mathbf{A}^\mathsf{T}\mathbf{A})\boldsymbol{v} = (\mathbf{A}\boldsymbol{v})^\mathsf{T}(\mathbf{A}\boldsymbol{v}) = ||\mathbf{A}\boldsymbol{v}||^2 > 0 \tag{7.238}$$

Hence in the full rank case, the least squares objective has a unique global minimum.

7.8 Matrix calculus

The topic of **calculus** concerns computing "rates of change" of functions as we vary their inputs. It is of vital importance to machine learning, as well as almost every other numerical discipline. In this section, we review some standard results. In some cases, we use some concepts and notation from matrix algebra, which we cover in Chapter 7. For more details on these results from a deep learning perspective, see [PH18].

7.8.1 Derivatives

Consider a scalar-argument function $f : \mathbb{R} \to \mathbb{R}$. We define its **derivative** at a point x to be the quantity

$$f'(x) \triangleq \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{7.239}$$

assuming the limit exists. This measures how quickly the output changes when we move a small distance in input space away from x (i.e., the "rate of change" of the function). We can interpret $f'(x)$ as the slope of the tangent line at $f(x)$, and hence

$$f(x+h) \approx f(x) + f'(x)h \tag{7.240}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

for small h.

We can compute a **finite difference** approximation to the derivative by using a finite step size h, as follows:

$$f'(x) \equiv \underbrace{\lim_{h\to 0} \frac{f(x+h)-f(x)}{h}}_{\text{forward difference}} = \underbrace{\lim_{h\to 0} \frac{f(x+h/2)-f(x-h/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h\to 0} \frac{f(x)-f(x-h)}{h}}_{\text{backward difference}} \tag{7.241}$$

The smaller the step size h, the better the estimate, although if h is too small, there can be errors due to numerical cancellation.

We can think of differentiation as an operator that maps functions to functions, $D(f) = f'$, where $f'(x)$ computes the derivative at x (assuming the derivative exists at that point). The use of the prime symbol f' to denote the derivative is called **Lagrange notation**. The second derivative function, which measures how quickly the gradient is changing, is denoted by f''. The n'th derivative function is denoted $f^{(n)}$.

Alternatively, we can use **Leibniz notation**, in which we denote the function by $y = f(x)$, and its derivative by $\frac{dy}{dx}$ or $\frac{d}{dx}f(x)$. To denote the evaluation of the derivative at a point a, we write $\left.\frac{df}{dx}\right|_{x=a}$.

7.8.2 Gradients

We can extend the notion of derivatives to handle vector-argument functions, $f : \mathbb{R}^n \to \mathbb{R}$, by defining the **partial derivative** of f with respect to x_i to be

$$\frac{\partial f}{\partial x_i} = \lim_{h\to 0} \frac{f(\boldsymbol{x}+h\boldsymbol{e}_i)-f(\boldsymbol{x})}{h} \tag{7.242}$$

where $\boldsymbol{e}_i$ is the i'th unit vector.

The **gradient** of a function at a point $\boldsymbol{x}$ is the vector of its partial derivatives:

$$\boldsymbol{g} = \frac{\partial f}{\partial \boldsymbol{x}} = \nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \tag{7.243}$$

To emphasize the point at which the gradient is evaluated, we can write

$$\boldsymbol{g}(\boldsymbol{x}^*) \triangleq \left.\frac{\partial f}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}^*} \tag{7.244}$$

We see that the operator ∇ (pronounced "nabla") maps a function $f : \mathbb{R}^n \to \mathbb{R}$ to another function $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^n$. Since $\boldsymbol{g}()$ is a vector-valued function, it is known as a **vector field**. By contrast, the derivative function f' is a **scalar field**.

7.8.3 Directional derivative

The **directional derivative** measures how much the function $f : \mathbb{R}^n \to \mathbb{R}$ changes along a direction $\boldsymbol{v}$ in space. It is defined as follows

$$D_{\boldsymbol{v}} f(\boldsymbol{x}) = \lim_{h\to 0} \frac{f(\boldsymbol{x}+h\boldsymbol{v})-f(\boldsymbol{x})}{h} \tag{7.245}$$

We can approximate this numerically using 2 function calls to f, regardless of n. By contrast, a numerical approximation to the standard gradient vector takes $n+1$ calls (or $2n$ if using central differences).

Note that the directional derivative along $\boldsymbol{v}$ is the scalar product of the gradient $\boldsymbol{g}$ and the vector $\boldsymbol{v}$:

$$D_{\boldsymbol{v}} f(\boldsymbol{x}) = \nabla f(\boldsymbol{x}) \cdot \boldsymbol{v} \tag{7.246}$$

7.8.4 Total derivative *

Suppose that some of the arguments to the function depend on each other. Concretely, suppose the function has the form $f(t, x(t), y(t))$. We define the **total derivative** of f wrt t as follows:

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt} \tag{7.247}$$

If we multiply both sides by the differential dt, we get the **total differential**

$$df = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial x}dx + \frac{\partial f}{\partial y}dy \tag{7.248}$$

This measures how much f changes when we change t, both via the direct effect of t on f, but also indirectly, via the effects of t on x and y.

7.8.5 Jacobian

Consider a function that maps a vector to another vector, $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$. The **Jacobian matrix** of this function is an $m \times n$ matrix of partial derivatives:

$$\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x}) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}^\mathsf{T}} \triangleq \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\boldsymbol{x})^\mathsf{T} \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\mathsf{T} \end{pmatrix} \tag{7.249}$$

Note that we lay out the results in the same orientation as the output $\boldsymbol{f}$; this is sometimes called numerator layout or the Jacobian formulation.[5]

7.8.5.1 Multiplying Jacobians and vectors

The **Jacobian vector product** or **JVP** is defined to be the operation that corresponds to right-multiplying the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ by a vector $\boldsymbol{v} \in \mathbb{R}^n$:

$$\mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x})\boldsymbol{v} = \begin{pmatrix} \nabla f_1(\boldsymbol{x})^\mathsf{T} \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\mathsf{T} \end{pmatrix} \boldsymbol{v} = \begin{pmatrix} \nabla f_1(\boldsymbol{x})^\mathsf{T}\boldsymbol{v} \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\mathsf{T}\boldsymbol{v} \end{pmatrix} \tag{7.250}$$

5. For a much more detailed discussion of notation, see https://en.wikipedia.org/wiki/Matrix_calculus.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

So we can see that we can approximate this numerically using just 2 calls to $\boldsymbol{f}$.

The **vector Jacobian product** or **VJP** is defined to be the operation that corresponds to left-multiplying the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ by a vector $\boldsymbol{u} \in \mathbb{R}^m$:

$$\boldsymbol{u}^\mathsf{T} \mathbf{J}_{\boldsymbol{f}}(\boldsymbol{x}) = \boldsymbol{u}^\mathsf{T} \left(\frac{\partial \boldsymbol{f}}{\partial x_1}, \cdots, \frac{\partial \boldsymbol{f}}{\partial x_n} \right) = \left(\boldsymbol{u} \cdot \frac{\partial \boldsymbol{f}}{\partial x_1}, \cdots, \boldsymbol{u} \cdot \frac{\partial \boldsymbol{f}}{\partial x_n} \right) \tag{7.251}$$

The JVP is more efficient if $m \geq n$, and the VJP is more efficient if $m \leq n$. See Section 13.3 for details on how this can be used to perform automatic differentiation in a computation graph such as a DNN.

7.8.5.2 Jacobian of a composition

Sometimes it is useful to take the Jacobian of the composition of two functions. Let $h(\boldsymbol{x}) = g(f(\boldsymbol{x}))$. By the chain rule of calculus, we have

$$\mathbf{J}_h(\boldsymbol{x}) = \mathbf{J}_g(f(\boldsymbol{x})) \mathbf{J}_f(\boldsymbol{x}) \tag{7.252}$$

For example, suppose $f : \mathbb{R} \to \mathbb{R}^2$ and $g : \mathbb{R}^2 \to \mathbb{R}^2$. We have

$$\frac{\partial \boldsymbol{g}}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x} g_1(f_1(x), f_2(x)) \\ \frac{\partial}{\partial x} g_2(f_1(x), f_2(x)) \end{pmatrix} = \begin{pmatrix} \frac{\partial g_1}{\partial f_1}\frac{\partial f_1}{\partial x} + \frac{\partial g_1}{\partial f_2}\frac{\partial f_2}{\partial x} \\ \frac{\partial g_2}{\partial f_1}\frac{\partial f_1}{\partial x} + \frac{\partial g_2}{\partial f_2}\frac{\partial f_2}{\partial x} \end{pmatrix} \tag{7.253}$$

$$= \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{f}^\mathsf{T}} \frac{\partial \boldsymbol{f}}{\partial x} = \begin{pmatrix} \frac{\partial g_1}{\partial f_1} & \frac{\partial g_1}{\partial f_2} \\ \frac{\partial g_2}{\partial f_1} & \frac{\partial g_2}{\partial f_2} \end{pmatrix} \begin{pmatrix} \frac{\partial f_1}{\partial x} \\ \frac{\partial f_2}{\partial x} \end{pmatrix} \tag{7.254}$$

7.8.6 Hessian

For a function $f : \mathbb{R}^n \to \mathbb{R}$ that is twice differentiable, we define the **Hessian matrix** as the (symmetric) $n \times n$ matrix of second partial derivatives:

$$\mathbf{H}_f = \frac{\partial^2 f}{\partial \boldsymbol{x}^2} = \nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ & \vdots & \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix} \tag{7.255}$$

We see that the Hessian is the Jacobian of the gradient.

7.8.7 Gradients of commonly used functions

In this section, we list without proof the gradients of certain widely used functions.

7.8.7.1 Functions that map scalars to scalars

Consider a differentiable function $f : \mathbb{R} \to \mathbb{R}$. Here are some useful identities from scalar calculus, which you should already be familiar with.

$$\frac{d}{dx}cx^n = cnx^{n-1} \tag{7.256}$$
$$\frac{d}{dx}\log(x) = 1/x \tag{7.257}$$
$$\frac{d}{dx}\exp(x) = \exp(x) \tag{7.258}$$
$$\frac{d}{dx}[f(x) + g(x)] = \frac{df(x)}{dx} + \frac{dg(x)}{dx} \tag{7.259}$$
$$\frac{d}{dx}[f(x)g(x)] = f(x)\frac{dg(x)}{dx} + g(x)\frac{df(x)}{dx} \tag{7.260}$$
$$\frac{d}{dx}f(u(x)) = \frac{du}{dx}\frac{df(u)}{du} \tag{7.261}$$

Equation (7.261) is known as the **chain rule of calculus**.

7.8.7.2 Functions that map vectors to scalars

Consider a differentiable function $f : \mathbb{R}^n \to \mathbb{R}$. Here are some useful identities:[6]

$$\frac{\partial(\boldsymbol{a}^\mathsf{T}\boldsymbol{x})}{\partial \boldsymbol{x}} = \boldsymbol{a} \tag{7.262}$$
$$\frac{\partial(\boldsymbol{b}^\mathsf{T}\mathbf{A}\boldsymbol{x})}{\partial \boldsymbol{x}} = \mathbf{A}^\mathsf{T}\boldsymbol{b} \tag{7.263}$$
$$\frac{\partial(\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x})}{\partial \boldsymbol{x}} = (\mathbf{A} + \mathbf{A}^\mathsf{T})\boldsymbol{x} \tag{7.264}$$

It is fairly easy to prove these identities by expanding out the quadratic form, and applying scalar calculus.

7.8.7.3 Functions that map matrices to scalars

Consider a function $f : \mathbb{R}^{m\times n} \to \mathbb{R}$ which maps a matrix to a scalar. We are using the following natural layout for the derivative matrix:

$$\frac{\partial f}{\partial \mathbf{X}} = \begin{pmatrix} \frac{\partial f}{\partial x_{11}} & \cdots & \frac{\partial f}{\partial x_{1n}} \\ & \vdots & \\ \frac{\partial f}{\partial x_{m1}} & \cdots & \frac{\partial f}{\partial x_{mn}} \end{pmatrix} \tag{7.265}$$

Below are some useful identities.

6. Some of the identities are taken from the list at http://www.cs.nyu.edu/~roweis/notes/matrixid.pdf.

Identities involving quadratic forms

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}}(\boldsymbol{a}^\mathsf{T}\mathbf{X}\boldsymbol{b}) = \boldsymbol{a}\boldsymbol{b}^\mathsf{T} \tag{7.266}$$

$$\frac{\partial}{\partial \mathbf{X}}(\boldsymbol{a}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{b}) = \boldsymbol{b}\boldsymbol{a}^\mathsf{T} \tag{7.267}$$

Identities involving matrix trace

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{AXB}) = \mathbf{A}^\mathsf{T}\mathbf{B}^\mathsf{T} \tag{7.268}$$

$$\frac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{X}^\mathsf{T}\mathbf{A}) = \mathbf{A} \tag{7.269}$$

$$\frac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{X}^{-1}\mathbf{A}) = -\mathbf{X}^{-\mathsf{T}}\mathbf{A}^\mathsf{T}\mathbf{X}^{-\mathsf{T}} \tag{7.270}$$

$$\frac{\partial}{\partial \mathbf{X}}\mathrm{tr}(\mathbf{X}^\mathsf{T}\mathbf{AX}) = (\mathbf{A} + \mathbf{A}^\mathsf{T})\mathbf{X} \tag{7.271}$$

Identities involving matrix determinant

One can show the following results.

$$\frac{\partial}{\partial \mathbf{X}}\det(\mathbf{AXB}) = \det(\mathbf{AXB})\mathbf{X}^{-\mathsf{T}} \tag{7.272}$$

$$\frac{\partial}{\partial \mathbf{X}}\log(\det(\mathbf{X})) = \mathbf{X}^{-\mathsf{T}} \tag{7.273}$$

7.9 Exercises

Exercise 7.1 [Orthogonal matrices]

a. A rotation in 3d by angle α about the z axis is given by the following matrix:

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{7.274}$$

Prove that $\mathbf{R}$ is an orthogonal matrix, i.e., $\mathbf{R}^T\mathbf{R} = \mathbf{I}$, for any α.

b. What is the only eigenvector $\boldsymbol{v}$ of $\mathbf{R}$ with an eigenvalue of 1.0 and of unit norm (i.e., $||\boldsymbol{v}||^2 = 1$)? (Your answer should be the same for any α.) Hint: think about the geometrical interpretation of eigenvectors.

Exercise 7.2 [Eigenvectors by hand †]

Find the eigenvalues and eigenvectors of the following matrix

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \tag{7.275}$$

Compute your result by hand and check it with Python.

8 Optimization

Parts of this chapter were written by Frederik Kunstner, Si Yi Meng, Aaron Mishkin, Sharan Vaswani, and Mark Schmidt.

8.1 Introduction

We saw in Chapter 4 that the core problem in machine learning is parameter estimation (aka model fitting). This requires solving an **optimization problem**, where we try to find the values for a set of variables $\boldsymbol{\theta} \in \Theta$, that minimize a scalar-valued **loss function** or **cost function** $\mathcal{L} : \Theta \to \mathbb{R}$:

$$\boldsymbol{\theta}^* \in \operatorname*{argmin}_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta}) \tag{8.1}$$

We will assume that the **parameter space** is given by $\Theta \subseteq \mathbb{R}^D$, where D is the number of variables being optimized over. Thus we are focusing on **continuous optimization**, rather than **discrete optimization**.

If we want to *maximize* a **score function** or **reward function** $R(\boldsymbol{\theta})$, we can equivalently minimize $\mathcal{L}(\boldsymbol{\theta}) = -R(\boldsymbol{\theta})$. We will use the term **objective function** to refer generically to a function we want to maximize or minimize. An algorithm that can find an optimum of an objective function is often called a **solver**.

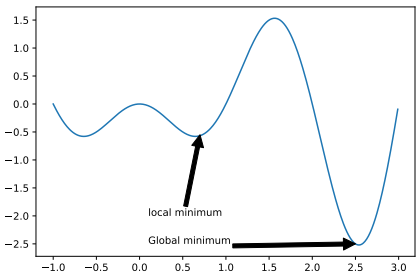
In the rest of this chapter, we discuss different kinds of solvers for different kinds of objective functions, with a focus on methods used in the machine learning community. For more details on optimization, please consult some of the many excellent textbooks, such as [KW19b; BV04; NW06; Ber15; Ber16] as well as various review articles, such as [BCN18; Sun+19b; PPS18; Pey20]. A visualization of the taxonomy of optimization algorithms can be found at https://neos-guide.org/guide/types.

8.1.1 Local vs global optimization

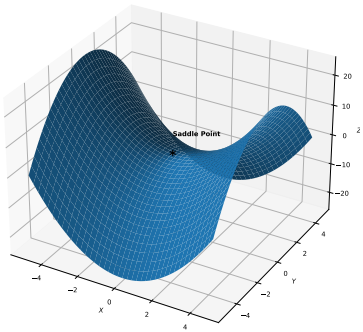
A point that satisfies Equation (8.1) is called a **global optimum**. Finding such a point is called **global optimization**.

In general, finding global optima is computationally intractable [Neu04]. In such cases, we will just try to find a **local optimum**. For continuous problems, this is defined to be a point $\boldsymbol{\theta}^*$ which has lower (or equal) cost than "nearby" points. Formally, we say $\boldsymbol{\theta}^*$ is a **local minimum** if

$$\exists \delta > 0,\ \forall \boldsymbol{\theta} \in \Theta \ \text{ s.t. } \ ||\boldsymbol{\theta} - \boldsymbol{\theta}^*|| < \delta,\ \mathcal{L}(\boldsymbol{\theta}^*) \leq \mathcal{L}(\boldsymbol{\theta}) \tag{8.2}$$



(a)



(b)

Figure 8.1: (a) Illustration of local and global minimum in 1d. Generated by extrema_fig_1d.ipynb. (b) Illustration of a saddle point in 2d. Generated by saddle.ipynb.

A local minimum could be surrounded by other local minima with the same objective value; this is known as a **flat local minimum**. A point is said to be a **strict local minimum** if its cost is strictly lower than those of neighboring points:

$$\exists \delta > 0,\ \forall \boldsymbol{\theta} \in \Theta, \boldsymbol{\theta} \neq \boldsymbol{\theta}^* : ||\boldsymbol{\theta} - \boldsymbol{\theta}^*|| < \delta,\ \mathcal{L}(\boldsymbol{\theta}^*) < \mathcal{L}(\boldsymbol{\theta}) \tag{8.3}$$

We can define a (strict) **local maximum** analogously. See Figure 8.1a for an illustration.

A final note on terminology; if an algorithm is guaranteed to converge to a stationary point from any starting point, it is called **globally convergent**. However, this does not mean (rather confusingly) that it will converge to a global optimum; instead, it just means it will converge to some stationary point.

8.1.1.1 Optimality conditions for local vs global optima

For continuous, twice differentiable functions, we can precisely characterize the points which correspond to local minima. Let $\boldsymbol{g}(\boldsymbol{\theta}) = \nabla \mathcal{L}(\boldsymbol{\theta})$ be the gradient vector, and $\mathbf{H}(\boldsymbol{\theta}) = \nabla^2 \mathcal{L}(\boldsymbol{\theta})$ be the Hessian matrix. (See Section 7.8 for a refresher on these concepts, if necessary.) Consider a point $\boldsymbol{\theta}^* \in \mathbb{R}^D$, and let $\boldsymbol{g}^* = \boldsymbol{g}(\boldsymbol{\theta})|_{\boldsymbol{\theta}^*}$ be the gradient at that point, and $\mathbf{H}^* = \mathbf{H}(\boldsymbol{\theta})|_{\boldsymbol{\theta}^*}$ be the corresponding Hessian. One can show that the following conditions characterize every local minimum:

- Necessary condition: If $\boldsymbol{\theta}^*$ is a local minimum, then we must have $\boldsymbol{g}^* = \mathbf{0}$ (i.e., $\boldsymbol{\theta}^*$ must be a **stationary point**), and $\mathbf{H}^*$ must be positive semi-definite.
- Sufficient condition: If $\boldsymbol{g}^* = \mathbf{0}$ and $\mathbf{H}^*$ is positive definite, then $\boldsymbol{\theta}^*$ is a local optimum.

To see why the first condition is necessary, suppose we were at a point $\boldsymbol{\theta}^*$ at which the gradient is non-zero: at such a point, we could decrease the function by following the negative gradient a small distance, so this would not be optimal. So the gradient must be zero. (In the case of nonsmooth

functions, the necessary condition is that the zero is a local subgradient at the minimum.) To see why a zero gradient is not sufficient, note that the stationary point could be a local minimum, maximum or **saddle point**, which is a point where some directions point downhill, and some uphill (see Figure 8.1b). More precisely, at a saddle point, the eigenvalues of the Hessian will be both positive and negative. However, if the Hessian at a point is positive semi-definite, then some directions may point uphill, while others are flat. Moreover, if the Hessian is strictly positive definite, then we are at the bottom of a "bowl", and all directions point uphill, which is sufficient for this to be a minimum.

8.1.2 Constrained vs unconstrained optimization

In **unconstrained optimization**, we define the optimization task as finding any value in the parameter space Θ that minimizes the loss. However, we often have a set of **constraints** on the allowable values. It is standard to partition the set of constraints $\mathcal{C}$ into **inequality constraints**, $g_j(\boldsymbol{\theta}) \leq 0$ for $j \in \mathcal{I}$ and **equality constraints**, $h_k(\boldsymbol{\theta}) = 0$ for $k \in \mathcal{E}$. For example, we can represent a sum-to-one constraint as an equality constraint $h(\boldsymbol{\theta}) = (1 - \sum_{i=1}^{D} \theta_i) = 0$, and we can represent a non-negativity constraint on the parameters by using D inequality constraints of the form $g_i(\boldsymbol{\theta}) = -\theta_i \leq 0$.

We define the **feasible set** as the subset of the parameter space that satisfies the constraints:

$$\mathcal{C} = \{\boldsymbol{\theta} : g_j(\boldsymbol{\theta}) \leq 0 : j \in \mathcal{I}, h_k(\boldsymbol{\theta}) = 0 : k \in \mathcal{E}\} \subseteq \mathbb{R}^D \tag{8.4}$$

Our **constrained optimization** problem now becomes

$$\boldsymbol{\theta}^* \in \operatorname*{argmin}_{\boldsymbol{\theta} \in \mathcal{C}} \mathcal{L}(\boldsymbol{\theta}) \tag{8.5}$$

If $\mathcal{C} = \mathbb{R}^D$, it is called **unconstrained optimization**.

The addition of constraints can change the number of optima of a function. For example, a function that was previously unbounded (and hence had no well-defined global maximum or minimum) can "acquire" multiple maxima or minima when we add constraints, as illustrated in Figure 8.2. However, if we add too many constraints, we may find that the feasible set becomes empty. The task of finding any point (regardless of its cost) in the feasible set is called a **feasibility problem**; this can be a hard subproblem in itself.

A common strategy for solving constrained problems is to create penalty terms that measure how much we violate each constraint. We then add these terms to the objective and solve an unconstrained optimization problem. The **Lagrangian** is a special case of such a combined objective (see Section 8.5 for details).

8.1.3 Convex vs nonconvex optimization

In **convex optimization**, we require the objective to be a convex function defined over a convex set (we define these terms below). In such problems, every local minimum is also a global minimum. Thus many models are designed so that their training objectives are convex.

8.1.3.1 Convex sets

We say $\mathcal{S}$ is a **convex set** if, for any $\boldsymbol{x}, \boldsymbol{x}' \in \mathcal{S}$, we have

$$\lambda \boldsymbol{x} + (1 - \lambda)\boldsymbol{x}' \in \mathcal{S}, \ \ \forall\, \lambda \in [0, 1] \tag{8.6}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

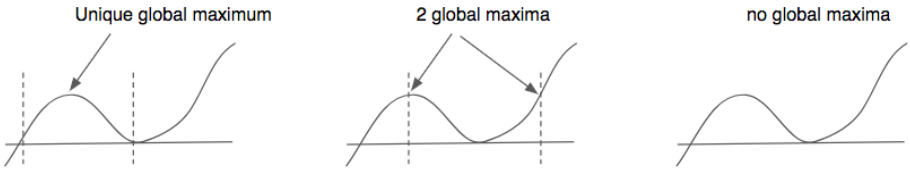


Figure 8.2: Illustration of constrained maximization of a nonconvex 1d function. The area between the dotted vertical lines represents the feasible set. (a) There is a unique global maximum since the function is concave within the support of the feasible set. (b) There are two global maxima, both occuring at the boundary of the feasible set. (c) In the unconstrained case, this function has no global maximum, since it is unbounded.

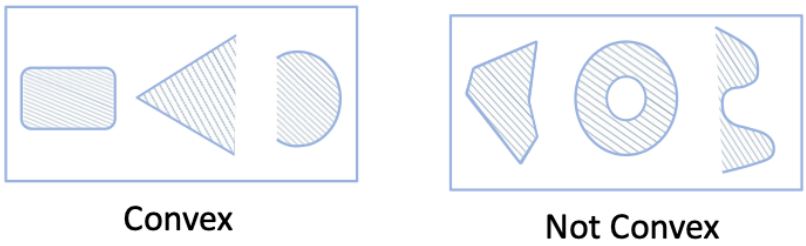


Figure 8.3: Illustration of some convex and non-convex sets.

That is, if we draw a line from $\boldsymbol{x}$ to $\boldsymbol{x}'$, all points on the line lie inside the set. See Figure 8.3 for some illustrations of convex and non-convex sets.

8.1.3.2 Convex functions

We say f is a **convex function** if its **epigraph** (the set of points above the function, illustrated in Figure 8.4a) defines a convex set. Equivalently, a function $f(\boldsymbol{x})$ is called convex if it is defined on a convex set and if, for any $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{S}$, and for any $0 \leq \lambda \leq 1$, we have

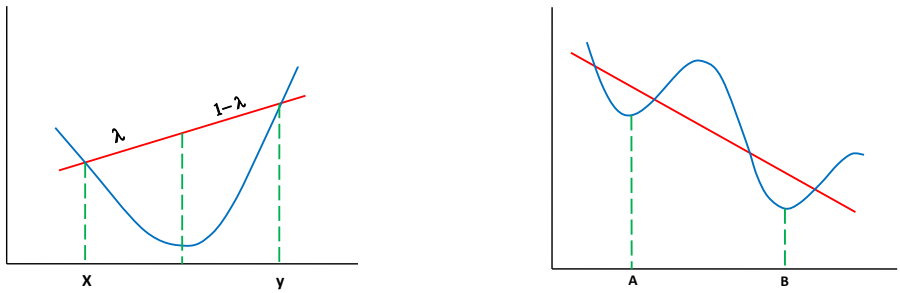
$$f(\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}) \leq \lambda f(\boldsymbol{x}) + (1-\lambda) f(\boldsymbol{y}) \tag{8.7}$$

See Figure 8.5(a) for a 1d example of a convex function. A function is called **strictly convex** if the inequality is strict. A function $f(\boldsymbol{x})$ is **concave** if $-f(\boldsymbol{x})$ is convex, and **strictly concave** if $-f(\boldsymbol{x})$ is strictly convex. See Figure 8.5(b) for a 1d example of a function that is neither convex nor concave.



(a) (b)

Figure 8.4: (a) Illustration of the epigraph of a function. (b) For a convex function $f(x)$, its epipgraph can be represented as the intersection of half-spaces defined by linear lower bounds derived from the **conjugate function** $f^*(\lambda) = \max_x \lambda x - f(x)$.



(a) (b)

Figure 8.5: (a) Illustration of a convex function. We see that the chord joining $(x, f(x))$ to $(y, f(y))$ lies above the function. (b) A function that is neither convex nor concave. **A** *is a local minimum,* **B** *is a global minimum.*

Here are some examples of 1d convex functions:

$$\begin{aligned} x^2 \\ e^{ax} \\ -\log x \\ x^a,\ a > 1, x > 0 \\ |x|^a,\ a \geq 1 \\ x \log x,\ x > 0 \end{aligned}$$

8.1.3.3 Characterization of convex functions

Intuitively, a convex function is shaped like a bowl. Formally, one can prove the following important result:

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

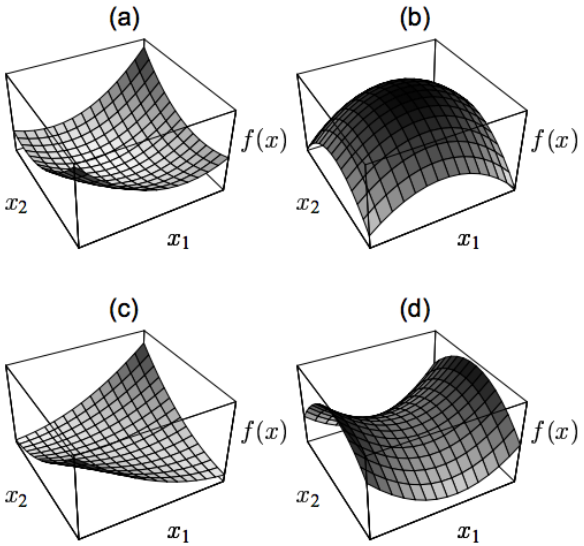


Figure 8.6: The quadratic form $f(\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x}$ in 2d. (a) $\mathbf{A}$ is positive definite, so f is convex. (b) $\mathbf{A}$ is negative definite, so f is concave. (c) $\mathbf{A}$ is positive semidefinite but singular, so f is convex, but not strictly. Notice the valley of constant height in the middle. (d) $\mathbf{A}$ is indefinite, so f is neither convex nor concave. The stationary point in the middle of the surface is a saddle point. From Figure 5 of [She94].

Theorem 8.1.1. *Suppose $f : \mathbb{R}^n \to \mathbb{R}$ is twice differentiable over its domain. Then f is convex iff $\mathbf{H} = \nabla^2 f(\boldsymbol{x})$ is positive semi definite (Section 7.1.5.3) for all $\boldsymbol{x} \in \text{dom}(f)$. Furthermore, f is strictly convex if $\mathbf{H}$ is positive definite.*

For example, consider the quadratic form

$$f(\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} \tag{8.8}$$

This is convex if $\mathbf{A}$ is positive semi definite, and is strictly convex if $\mathbf{A}$ is positive definite. It is neither convex nor concave if $\mathbf{A}$ has eigenvalues of mixed sign. See Figure 8.6.

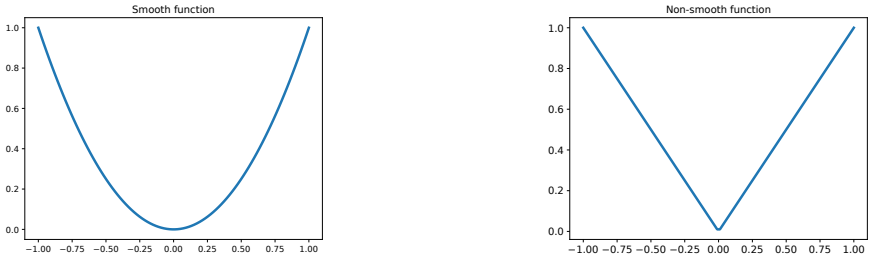
8.1.3.4 Strongly convex functions

We say a function f is **strongly convex** with parameter $m > 0$ if the following holds for all $\boldsymbol{x}$, $\boldsymbol{y}$ in f's domain:

$$(\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y}))^\mathsf{T}(\boldsymbol{x} - \boldsymbol{y}) \geq m||\boldsymbol{x} - \boldsymbol{y}||_2^2 \tag{8.9}$$

A strongly convex function is also strictly convex, but not vice versa.

If the function f is twice continuously differentiable, then it is strongly convex with parameter m if and only if $\nabla^2 f(\boldsymbol{x}) \succeq m\mathbf{I}$ for all $\boldsymbol{x}$ in the domain, where $\mathbf{I}$ is the identity and $\nabla^2 f$ is the Hessian matrix, and the inequality $\succeq$ means that $\nabla^2 f(\boldsymbol{x}) - m\mathbf{I}$ is positive semi-definite. This is equivalent



(a) (b)

Figure 8.7: (a) Smooth 1d function. (b) Non-smooth 1d function. (There is a discontinuity at the origin.) Generated by smooth-vs-nonsmooth-1d.ipynb.

to requiring that the minimum eigenvalue of $\nabla^2 f(\boldsymbol{x})$ be at least m for all $\boldsymbol{x}$. If the domain is just the real line, then $\nabla^2 f(x)$ is just the second derivative $f''(x)$, so the condition becomes $f''(x) \geq m$. If $m = 0$, then this means the Hessian is positive semidefinite (or if the domain is the real line, it means that $f''(x) \geq 0$), which implies the function is convex, and perhaps strictly convex, but not strongly convex.

The distinction between convex, strictly convex, and strongly convex is rather subtle. To better understand this, consider the case where f is twice continuously differentiable and the domain is the real line. Then we can characterize the differences as follows:

- f is convex if and only if $f''(x) \geq 0$ for all x.
- f is strictly convex if $f''(x) > 0$ for all x (note: this is sufficient, but not necessary).
- f is strongly convex if and only if $f''(x) \geq m > 0$ for all x.

Note that it can be shown that a function f is strongly convex with parameter m iff the function

$$J(\boldsymbol{x}) = f(\boldsymbol{x}) - \frac{m}{2}||\boldsymbol{x}||^2 \tag{8.10}$$

is convex.

8.1.4 Smooth vs nonsmooth optimization

In **smooth optimization**, the objective and constraints are continuously differentiable functions. For smooth functions, we can quantify the degree of smoothness using the **Lipschitz constant**. In the 1d case, this is defined as any constant $L \geq 0$ such that, for all real x_1 and x_2, we have

$$|f(x_1) - f(x_2)| \leq L|x_1 - x_2| \tag{8.11}$$

This is illustrated in Figure 8.8: for a given constant L, the function output cannot change by more than L if we change the function input by 1 unit. This can be generalized to vector inputs using a suitable norm.

In **nonsmooth optimization**, there are at least some points where the gradient of the objective function or the constraints is not well-defined. See Figure 8.7 for an example. In some optimization

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

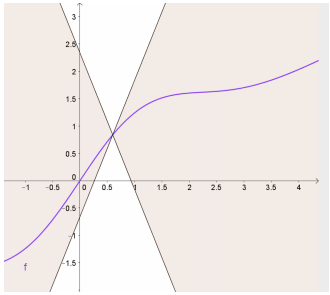


Figure 8.8: For a Lipschitz continuous function f, there exists a double cone (white) whose origin can be moved along the graph of f so that the whole graph always stays outside the double cone. From https://en.wikipedia.org/wiki/Lipschitz_continuity. Used with kind permission of Wikipedia author Taschee.

problems, we can partition the objective into a part that only contains smooth terms, and a part that contains the nonsmooth terms:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_s(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) \tag{8.12}$$

where $\mathcal{L}_s$ is smooth (differentiable), and $\mathcal{L}_r$ is nonsmooth ("rough"). This is often referred to as a **composite objective**. In machine learning applications, $\mathcal{L}_s$ is usually the training set loss, and $\mathcal{L}_r$ is a regularizer, such as the ℓ_1 norm of $\boldsymbol{\theta}$. This composite structure can be exploited by various algorithms.

8.1.4.1 Subgradients

In this section, we generalize the notion of a derivative to work with functions which have local discontinuities. In particular, for a convex function of several variables, $f : \mathbb{R}^n \to \mathbb{R}$, we say that $\boldsymbol{g} \in \mathbb{R}^n$ is a **subgradient** of f at $\boldsymbol{x} \in \text{dom}(f)$ if for all $\boldsymbol{z} \in \text{dom}(f)$,

$$f(\boldsymbol{z}) \geq f(\boldsymbol{x}) + \boldsymbol{g}^\mathsf{T}(\boldsymbol{z} - \boldsymbol{x}) \tag{8.13}$$

Note that a subgradient can exist even when f is not differentiable at a point, as shown in Figure 8.9.

A function f is called **subdifferentiable** at $\boldsymbol{x}$ if there is at least one subgradient at $\boldsymbol{x}$. The set of such subgradients is called the **subdifferential** of f at $\boldsymbol{x}$, and is denoted $\partial f(\boldsymbol{x})$.

For example, consider the absolute value function $f(x) = |x|$. Its subdifferential is given by

$$\partial f(x) = \begin{cases} \{-1\} & \text{if } x < 0 \\ [-1, 1] & \text{if } x = 0 \\ \{+1\} & \text{if } x > 0 \end{cases} \tag{8.14}$$

where the notation $[-1, 1]$ means any value between -1 and 1 inclusive. See Figure 8.10 for an illustration.

8.2 First-order methods

In this section, we consider iterative optimization methods that leverage **first-order** derivatives of the objective function, i.e., they compute which directions point "downhill", but they ignore curvature

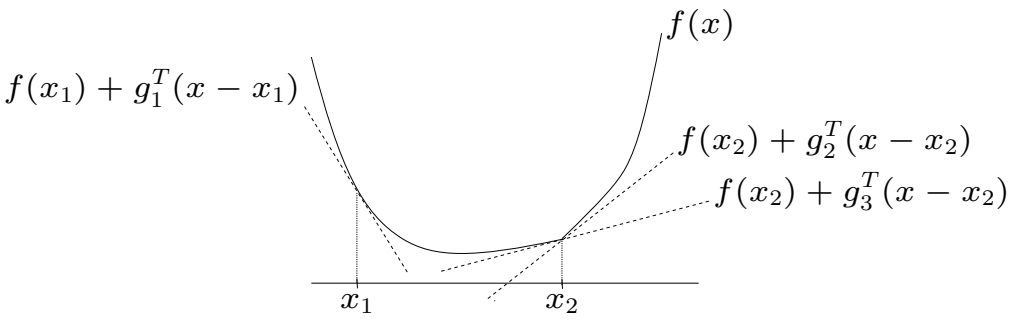


Figure 8.9: Illustration of subgradients. At $\boldsymbol{x}_1$, the convex function f is differentiable, and $\boldsymbol{g}_1$ (which is the derivative of f at $\boldsymbol{x}_1$) is the unique subgradient at $\boldsymbol{x}_1$. At the point $\boldsymbol{x}_2$, f is not differentiable, because of the "kink". However, there are many subgradients at this point, of which two are shown. From https://web.stanford.edu/class/ee364b/lectures/subgradients_slides.pdf. Used with kind permission of Stephen Boyd.

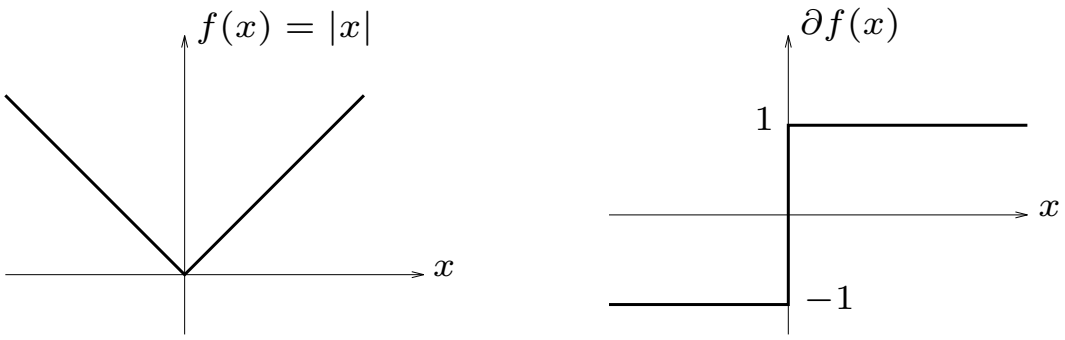


Figure 8.10: The absolute value function (left) and its subdifferential (right). From https://web.stanford.edu/class/ee364b/lectures/subgradients_slides.pdf. Used with kind permission of Stephen Boyd.

information. All of these algorithms require that the user specify a starting point $\boldsymbol{\theta}_0$. Then at each iteration t, they perform an update of the following form:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \eta_t \boldsymbol{d}_t \tag{8.15}$$

where η_t is known as the **step size** or **learning rate**, and $\boldsymbol{d}_t$ is a **descent direction**, such as the negative of the **gradient**, given by $\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t}$. These update steps are continued until the method reaches a stationary point, where the gradient is zero.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.2.1 Descent direction

We say that a direction $\boldsymbol{d}$ is a **descent direction** if there is a small enough (but nonzero) amount η we can move in direction $\boldsymbol{d}$ and be guaranteed to decrease the function value. Formally, we require that there exists an $\eta_{\max} > 0$ such that

$$\mathcal{L}(\boldsymbol{\theta} + \eta \boldsymbol{d}) < \mathcal{L}(\boldsymbol{\theta}) \tag{8.16}$$

for all $0 < \eta < \eta_{\max}$. The gradient at the current iterate, $\boldsymbol{\theta}_t$, is given by

$$\boldsymbol{g}_t \triangleq \nabla \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t} = \nabla \mathcal{L}(\boldsymbol{\theta}_t) = \boldsymbol{g}(\boldsymbol{\theta}_t) \tag{8.17}$$

This points in the direction of maximal increase in f, so the negative gradient is a descent direction. It can be shown that any direction $\boldsymbol{d}$ is also a descent direction if the angle θ between $\boldsymbol{d}$ and $-\boldsymbol{g}_t$ is less than 90 degrees and satisfies

$$\boldsymbol{d}^{\mathsf{T}} \boldsymbol{g}_t = ||\boldsymbol{d}||\ ||\boldsymbol{g}_t||\ \cos(\theta) < 0 \tag{8.18}$$

It seems that the best choice would be to pick $\boldsymbol{d}_t = -\boldsymbol{g}_t$. This is known as the direction of **steepest descent**. However, this can be quite slow. We consider faster versions later.

8.2.2 Step size (learning rate)

In machine learning, the sequence of step sizes $\{\eta_t\}$ is called the **learning rate schedule**. There are several widely used methods for picking this, some of which we discuss below. (See also Section 8.4.3, where we discuss schedules for stochastic optimization.)

8.2.2.1 Constant step size

The simplest method is to use a constant step size, $\eta_t = \eta$. However, if it is too large, the method may fail to converge, and if it is too small, the method will converge but very slowly.

For example, consider the convex function

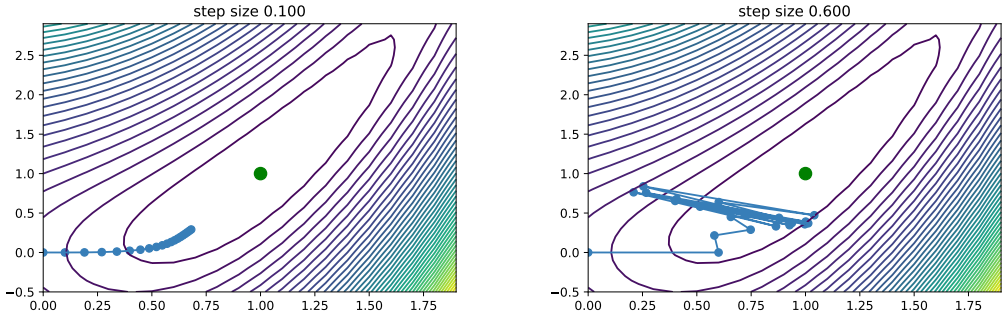
$$\mathcal{L}(\boldsymbol{\theta}) = 0.5(\theta_1^2 - \theta_2)^2 + 0.5(\theta_1 - 1)^2 \tag{8.19}$$

Let us pick as our descent direction $\boldsymbol{d}_t = -\boldsymbol{g}_t$. Figure 8.11 shows what happens if we use this descent direction with a fixed step size, starting from $(0, 0)$. In Figure 8.11(a), we use a small step size of $\eta = 0.1$; we see that the iterates move slowly along the valley. In Figure 8.11(b), we use a larger step size $\eta = 0.6$; we see that the iterates start oscillating up and down the sides of the valley and never converge to the optimum, even though this is a convex problem.

In some cases, we can derive a theoretical upper bound on the maximum step size we can use. For example, consider a quadratic objective, $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^{\mathsf{T}}\mathbf{A}\boldsymbol{\theta} + \boldsymbol{b}^{\mathsf{T}}\boldsymbol{\theta} + c$ with $\mathbf{A} \succeq \mathbf{0}$. One can show that steepest descent will have global convergence iff the step size satisfies

$$\eta < \frac{2}{\lambda_{\max}(\mathbf{A})} \tag{8.20}$$

where $\lambda_{\max}(\mathbf{A})$ is the largest eigenvalue of $\mathbf{A}$. The intuitive reason for this can be understood by thinking of a ball rolling down a valley. We want to make sure it doesn't take a step that is larger than the slope of the steepest direction, which is what the largest eigenvalue measures (see Section 3.2.2).



(a) *(b)*

Figure 8.11: Steepest descent on a simple convex function, starting from $(0, 0)$*, for 20 steps, using a fixed step size. The global minimum is at* $(1, 1)$*. (a)* $\eta = 0.1$*. (b)* $\eta = 0.6$*. Generated by steepestDescentDemo.ipynb.*

More generally, setting $\eta < 2/L$, where L is the Lipschitz constant of the gradient (Section 8.1.4), ensures convergence. Since this constant is generally unknown, we usually need to adapt the step size, as we discuss below.

8.2.2.2 Line search

The optimal step size can be found by finding the value that maximally decreases the objective along the chosen direction by solving the 1d minimization problem

$$\eta_t = \underset{\eta>0}{\operatorname{argmin}}\, \phi_t(\eta) = \underset{\eta>0}{\operatorname{argmin}}\, \mathcal{L}(\boldsymbol{\theta}_t + \eta \boldsymbol{d}_t) \tag{8.21}$$

This is known as **line search**, since we are searching along the line defined by $\boldsymbol{d}_t$.

If the loss is convex, this subproblem is also convex, because $\phi_t(\eta) = \mathcal{L}(\boldsymbol{\theta}_t + \eta \boldsymbol{d}_t)$ is a convex function of an affine function of η, for fixed $\boldsymbol{\theta}_t$ and $\boldsymbol{d}_t$. For example, consider the quadratic loss

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\mathbf{A}\boldsymbol{\theta} + \boldsymbol{b}^\mathsf{T}\boldsymbol{\theta} + c \tag{8.22}$$

Computing the derivative of ϕ gives

$$\frac{d\phi(\eta)}{d\eta} = \frac{d}{d\eta}\left[\frac{1}{2}(\boldsymbol{\theta} + \eta\boldsymbol{d})^\mathsf{T}\mathbf{A}(\boldsymbol{\theta} + \eta\boldsymbol{d}) + \boldsymbol{b}^\mathsf{T}(\boldsymbol{\theta} + \eta\boldsymbol{d}) + c\right] \tag{8.23}$$

$$= \boldsymbol{d}^\mathsf{T}\mathbf{A}(\boldsymbol{\theta} + \eta\boldsymbol{d}) + \boldsymbol{d}^\mathsf{T}\boldsymbol{b} \tag{8.24}$$

$$= \boldsymbol{d}^\mathsf{T}(\mathbf{A}\boldsymbol{\theta} + \boldsymbol{b}) + \eta\boldsymbol{d}^\mathsf{T}\mathbf{A}\boldsymbol{d} \tag{8.25}$$

Solving for $\frac{d\phi(\eta)}{d\eta} = 0$ gives

$$\eta = -\frac{\boldsymbol{d}^\mathsf{T}(\mathbf{A}\boldsymbol{\theta} + \boldsymbol{b})}{\boldsymbol{d}^\mathsf{T}\mathbf{A}\boldsymbol{d}} \tag{8.26}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Using the optimal step size is known as **exact line search**. However, it is not usually necessary to be so precise. There are several methods, such as the **Armijo backtracking method**, that try to ensure sufficient reduction in the objective function without spending too much time trying to solve Equation (8.21). In particular, we can start with the current stepsize (or some maximum value), and then reduce it by a factor $0 < c < 1$ at each step until we satisfy the following condition, known as the **Armijo-Goldstein** test:

$$\mathcal{L}(\boldsymbol{\theta}_t + \eta \boldsymbol{d}_t) \leq \mathcal{L}(\boldsymbol{\theta}_t) + c\eta \boldsymbol{d}_t^\mathsf{T} \nabla \mathcal{L}(\boldsymbol{\theta}_t) \tag{8.27}$$

where $c \in [0, 1]$ is a constant, typically $c = 10^{-4}$. In practice, the initialization of the line-search and how to backtrack can significantly affect performance. See [NW06, Sec 3.1] for details.

8.2.3 Convergence rates

We want to find optimization algorithms that converge quickly to a (local) optimum. For certain convex problems, with a gradient with bounded Lipschitz constant, one can show that gradient descent converges at a **linear rate**. This means that there exists a number $0 < \mu < 1$ such that

$$|\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_*)| \leq \mu |\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)| \tag{8.28}$$

Here μ is called the **rate of convergence**.

For some simple problems, we can derive the convergence rate explicitly, For example, consider a quadratic objective $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\mathbf{A}\boldsymbol{\theta} + \boldsymbol{b}^\mathsf{T}\boldsymbol{\theta} + c$ with $\mathbf{A} \succ 0$. Suppose we use steepest descent with exact line search. One can show (see e.g., [Ber15]) that the convergence rate is given by

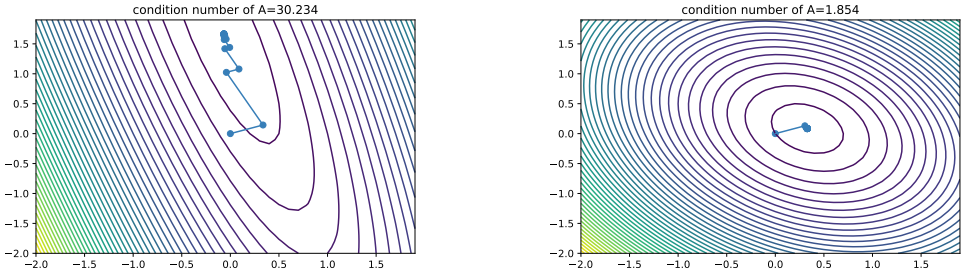
$$\mu = \left(\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} \right)^2 \tag{8.29}$$

where $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{A}$ and $\lambda_{\min}$ is the smallest eigenvalue. We can rewrite this as $\mu = \left(\frac{\kappa - 1}{\kappa + 1}\right)^2$, where $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the condition number of $\mathbf{A}$. Intuitively, the condition number measures how "skewed" the space is, in the sense of being far from a symmetrical "bowl". (See Section 7.1.4.4 for more information on condition numbers.)

Figure 8.12 illustrates the effect of the condition number on the convergence rate. On the left we show an example where $\mathbf{A} = [20, 5; 5, 2]$, $\boldsymbol{b} = [-14; -6]$ and $c = 10$, so $\kappa(\mathbf{A}) = 30.234$. On the right we show an example where $\mathbf{A} = [20, 5; 5, 16]$, $\boldsymbol{b} = [-14; -6]$ and $c = 10$, so $\kappa(\mathbf{A}) = 1.8541$. We see that steepest descent converges much more quickly for the problem with the smaller condition number.

In the more general case of non-quadratic functions, the objective will often be locally quadratic around a local optimum. Hence the convergence rate depends on the condition number of the Hessian, $\kappa(\mathbf{H})$, at that point. We can often improve the convergence speed by optimizing a surrogate objective (or model) at each step which has a Hessian that is close to the Hessian of the objective function as we discuss in Section 8.3.

Although line search works well, we see from Figure 8.12 that the path of steepest descent with an exact line-search exhibits a characteristic **zig-zag** behavior, which is inefficient. This problem can be overcome using a method called **conjugate gradient** descent (see e.g., [She94]).



(a) (b)

Figure 8.12: Illustration of the effect of condition number κ on the convergence speed of steepest descent with exact line searches. (a) Large κ. (b) Small κ. Generated by lineSearchConditionNum.ipynb.

8.2.4 Momentum methods

Gradient descent can move very slowly along flat regions of the loss landscape, as we illustrated in Figure 8.11. We discuss some solutions to this below.

8.2.4.1 Momentum

One simple heuristic, known as the **heavy ball** or **momentum** method [Ber99], is to move faster along directions that were previously good, and to slow down along directions where the gradient has suddenly changed, just like a ball rolling downhill. This can be implemented as follows:

$$\boldsymbol{m}_t = \beta \boldsymbol{m}_{t-1} + \boldsymbol{g}_{t-1} \tag{8.30}$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta_t \boldsymbol{m}_t \tag{8.31}$$

where $\boldsymbol{m}_t$ is the momentum (mass times velocity) and $0 < \beta < 1$. A typical value of β is 0.9. For $\beta = 0$, the method reduces to gradient descent.

We see that $\boldsymbol{m}_t$ is like an exponentially weighted moving average of the past gradients (see Section 4.4.2.2):

$$\boldsymbol{m}_t = \beta \boldsymbol{m}_{t-1} + \boldsymbol{g}_{t-1} = \beta^2 \boldsymbol{m}_{t-2} + \beta \boldsymbol{g}_{t-2} + \boldsymbol{g}_{t-1} = \cdots = \sum_{\tau=0}^{t-1} \beta^\tau \boldsymbol{g}_{t-\tau-1} \tag{8.32}$$

If all the past gradients are a constant, say $\boldsymbol{g}$, this simplifies to

$$\boldsymbol{m}_t = \boldsymbol{g} \sum_{\tau=0}^{t-1} \beta^\tau \tag{8.33}$$

The scaling factor is a geometric series, whose infinite sum is given by

$$1 + \beta + \beta^2 + \cdots = \sum_{i=0}^{\infty} \beta^i = \frac{1}{1-\beta} \tag{8.34}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

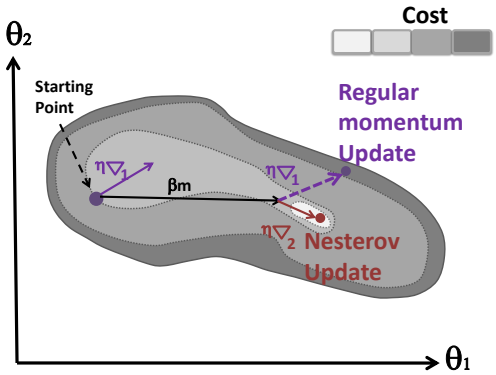


Figure 8.13: Illustration of the Nesterov update. Adapted from Figure 11.6 of [Gér19].

Thus in the limit, we multiply the gradient by $1/(1-\beta)$. For example, if $\beta = 0.9$, we scale the gradient up by 10.

Since we update the parameters using the gradient average $\boldsymbol{m}_{t-1}$, rather than just the most recent gradient, $\boldsymbol{g}_{t-1}$, we see that past gradients can exhibit some influence on the present. Furthermore, when momentum is combined with SGD, discussed in Section 8.4, we will see that it can simulate the effects of a larger minibatch, without the computational cost.

8.2.4.2 Nesterov momentum

One problem with the standard momentum method is that it may not slow down enough at the bottom of a valley, causing oscillation. The **Nesterov accelerated gradient** method of [Nes04] instead modifies the gradient descent to include an extrapolation step, as follows:

$$\tilde{\boldsymbol{\theta}}_{t+1} = \boldsymbol{\theta}_t + \beta(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}) \tag{8.35}$$

$$\boldsymbol{\theta}_{t+1} = \tilde{\boldsymbol{\theta}}_{t+1} - \eta_t \nabla \mathcal{L}(\tilde{\boldsymbol{\theta}}_{t+1}) \tag{8.36}$$

This is essentially a form of one-step "look ahead", that can reduce the amount of oscillation, as illustrated in Figure 8.13.

Nesterov accelerated gradient can also be rewritten in the same format as standard momentum. In this case, the momentum term is updated using the gradient at the predicted new location,

$$\boldsymbol{m}_{t+1} = \beta \boldsymbol{m}_t - \eta_t \nabla \mathcal{L}(\boldsymbol{\theta}_t + \beta \boldsymbol{m}_t) \tag{8.37}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \boldsymbol{m}_{t+1} \tag{8.38}$$

This explains why the Nesterov accelerated gradient method is sometimes called Nesterov momentum. It also shows how this method can be faster than standard momentum: the momentum vector is already roughly pointing in the right direction, so measuring the gradient at the new location, $\boldsymbol{\theta}_t + \beta \boldsymbol{m}_t$, rather than the current location, $\boldsymbol{\theta}_t$, can be more accurate.

The Nesterov accelerated gradient method is provably faster than steepest descent for convex functions when β and η_t are chosen appropriately. It is called "accelerated" because of this improved

convergence rate, which is optimal for gradient-based methods using only first-order information when the objective function is convex and has Lipschitz-continuous gradients. In practice, however, using Nesterov momentum can be slower than steepest descent, and can even unstable if β or η_t are misspecified.

8.3 Second-order methods

Optimization algorithms that only use the gradient are called **first-order** methods. They have the advantage that the gradient is cheap to compute and to store, but they do not model the curvature of the space, and hence they can be slow to converge, as we have seen in Figure 8.12. **Second-order** optimization methods incorporate curvature in various ways (e.g., via the Hessian), which may yield faster convergence. We discuss some of these methods below.

8.3.1 Newton's method

The classic second-order method is **Newton's method**. This consists of updates of the form

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{H}_t^{-1} \boldsymbol{g}_t \tag{8.39}$$

where

$$\mathbf{H}_t \triangleq \nabla^2 \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t} = \nabla^2 \mathcal{L}(\boldsymbol{\theta}_t) = \mathbf{H}(\boldsymbol{\theta}_t) \tag{8.40}$$

is assumed to be positive-definite to ensure the update is well-defined. The pseudo-code for Newton's method is given in Algorithm 8.1. The intuition for why this is faster than gradient descent is that the matrix inverse $\mathbf{H}^{-1}$ "undoes" any skew in the local curvature, converting a topology like Figure 8.12a to one like Figure 8.12b.

Algorithm 8.1: Newton's method for minimizing a function

Initialize $\boldsymbol{\theta}_0$
for $t = 0, 1, 2, \ldots$ *until convergence* **do**
  Evaluate $\boldsymbol{g}_t = \nabla \mathcal{L}(\boldsymbol{\theta}_t)$
  Evaluate $\mathbf{H}_t = \nabla^2 \mathcal{L}(\boldsymbol{\theta}_t)$
  Solve $\mathbf{H}_t \boldsymbol{d}_t = -\boldsymbol{g}_t$ for $\boldsymbol{d}_t$
  Use line search to find stepsize η_t along $\boldsymbol{d}_t$
  $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \eta_t \boldsymbol{d}_t$

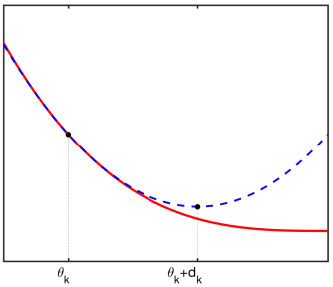
This algorithm can be derived as follows. Consider making a second-order Taylor series approximation of $\mathcal{L}(\boldsymbol{\theta})$ around $\boldsymbol{\theta}_t$:

$$\mathcal{L}_{\text{quad}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}_t) + \boldsymbol{g}_t^{\mathsf{T}}(\boldsymbol{\theta} - \boldsymbol{\theta}_t) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_t)^{\mathsf{T}} \mathbf{H}_t (\boldsymbol{\theta} - \boldsymbol{\theta}_t) \tag{8.41}$$

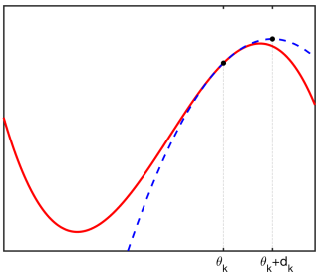
The minimum of $\mathcal{L}_{\text{quad}}$ is at

$$\boldsymbol{\theta} = \boldsymbol{\theta}_t - \mathbf{H}_t^{-1} \boldsymbol{g}_t \tag{8.42}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a)



(b)

Figure 8.14: Illustration of Newton's method for minimizing a 1d function. (a) The solid curve is the function $\mathcal{L}(x)$. The dotted line $\mathcal{L}_{\text{quad}}(\theta)$ is its second order approximation at θ_t. The Newton step d_t is what must be added to θ_t to get to the minimum of $\mathcal{L}_{\text{quad}}(\theta)$. Adapted from Figure 13.4 of [Van06]. Generated by newtonsMethodMinQuad.ipynb. (b) Illustration of Newton's method applied to a nonconvex function. We fit a quadratic function around the current point θ_t and move to its stationary point, $\theta_{t+1} = \theta_t + d_t$. Unfortunately, this takes us near a local maximum of f, not minimum. This means we need to be careful about the extent of our quadratic approximation. Adapted from Figure 13.11 of [Van06]. Generated by newtonsMethodNonConvex.ipynb.

So if the quadratic approximation is a good one, we should pick $\boldsymbol{d}_t = -\mathbf{H}_t^{-1}\boldsymbol{g}_t$ as our descent direction. See Figure 8.14(a) for an illustration. Note that, in a "pure" Newton method, we use $\eta_t = 1$ as our stepsize. However, we can also use linesearch to find the best stepsize; this tends to be more robust as using $\eta_t = 1$ may not always converge globally.

If we apply this method to linear regression, we get to the optimum in one step, since (as we show in Section 11.2.2.1) we have $\mathbf{H} = \mathbf{X}^\mathsf{T}\mathbf{X}$ and $\boldsymbol{g} = \mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w} - \mathbf{X}^\mathsf{T}\boldsymbol{y}$, so the Newton update becomes

$$\boldsymbol{w}_1 = \boldsymbol{w}_0 - \mathbf{H}^{-1}\boldsymbol{g} = \boldsymbol{w}_0 - (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}(\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w}_0 - \mathbf{X}^\mathsf{T}\boldsymbol{y}) = \boldsymbol{w}_0 - \boldsymbol{w}_0 + (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} \tag{8.43}$$

which is the OLS estimate. However, when we apply this method to logistic regression, it may take multiple iterations to converge to the global optimum, as we discuss in Section 10.2.6.

8.3.2 BFGS and other quasi-Newton methods

Quasi-Newton methods, sometimes called **variable metric** methods, iteratively build up an approximation to the Hessian using information gleaned from the gradient vector at each step. The most common method is called **BFGS** (named after its simultaneous inventors, Broyden, Fletcher, Goldfarb and Shanno), which updates the approximation to the Hessian $\mathbf{B}_t \approx \mathbf{H}_t$ as follows:

$$\mathbf{B}_{t+1} = \mathbf{B}_t + \frac{\boldsymbol{y}_t\boldsymbol{y}_t^\mathsf{T}}{\boldsymbol{y}_t^\mathsf{T}\boldsymbol{s}_t} - \frac{(\mathbf{B}_t\boldsymbol{s}_t)(\mathbf{B}_t\boldsymbol{s}_t)^\mathsf{T}}{\boldsymbol{s}_t^\mathsf{T}\mathbf{B}_t\boldsymbol{s}_t} \tag{8.44}$$

$$\boldsymbol{s}_t = \boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1} \tag{8.45}$$

$$\boldsymbol{y}_t = \boldsymbol{g}_t - \boldsymbol{g}_{t-1} \tag{8.46}$$

This is a rank-two update to the matrix. If $\mathbf{B}_0$ is positive-definite, and the step size η is chosen via line search satisfying both the Armijo condition in Equation (8.27) and the following curvature

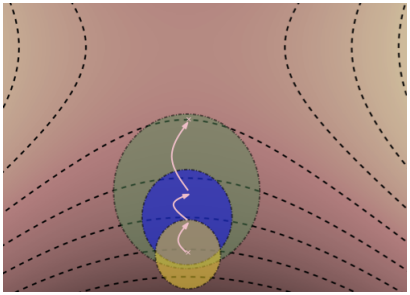


Figure 8.15: Illustration of the trust region approach. The dashed lines represents contours of the original nonconvex objective. The circles represent successive quadratic approximations. From Figure 4.2 of [Pas14]. Used with kind permission of Razvan Pascanu.

condition

$$\nabla \mathcal{L}(\boldsymbol{\theta}_t + \eta \boldsymbol{d}_t) \geq c_2 \eta \boldsymbol{d}_t^\mathsf{T} \nabla \mathcal{L}(\boldsymbol{\theta}_t) \tag{8.47}$$

then $\mathbf{B}_{t+1}$ will remain positive definite. The constant c_2 is chosen within $(c, 1)$ where c is the tunable parameter in Equation (8.27). The two step size conditions are together known as the **Wolfe conditions**. We typically start with a diagonal approximation, $\mathbf{B}_0 = \mathbf{I}$. Thus BFGS can be thought of as a "diagonal plus low-rank" approximation to the Hessian.

Alternatively, BFGS can iteratively update an approximation to the inverse Hessian, $\mathbf{C}_t \approx \mathbf{H}_t^{-1}$, as follows:

$$\mathbf{C}_{t+1} = \left(\mathbf{I} - \frac{\boldsymbol{s}_t \boldsymbol{y}_t^\mathsf{T}}{\boldsymbol{y}_t^\mathsf{T} \boldsymbol{s}_t}\right) \mathbf{C}_t \left(\mathbf{I} - \frac{\boldsymbol{y}_t \boldsymbol{s}_t^\mathsf{T}}{\boldsymbol{y}_t^\mathsf{T} \boldsymbol{s}_t}\right) + \frac{\boldsymbol{s}_t \boldsymbol{s}_t^\mathsf{T}}{\boldsymbol{y}_t^\mathsf{T} \boldsymbol{s}_t} \tag{8.48}$$

Since storing the Hessian approximation still takes $O(D^2)$ space, for very large problems, one can use **limited memory BFGS**, or **L-BFGS**, where we control the rank of the approximation by only using the M most recent $(\boldsymbol{s}_t, \boldsymbol{y}_t)$ pairs while ignoring older information. Rather than storing $\mathbf{B}_t$ explicitly, we just store these vectors in memory, and then approximate $\mathbf{H}_t^{-1}\boldsymbol{g}_t$ by performing a sequence of inner products with the stored $\boldsymbol{s}_t$ and $\boldsymbol{y}_t$ vectors. The storage requirements are therefore $O(MD)$. Typically choosing M to be between 5–20 suffices for good performance [NW06, p177].

Note that sklearn uses LBFGS as its default solver for logistic regression.[1]

8.3.3 Trust region methods

If the objective function is nonconvex, then the Hessian $\mathbf{H}_t$ may not be positive definite, so $\boldsymbol{d}_t = -\mathbf{H}_t^{-1}\boldsymbol{g}_t$ may not be a descent direction. This is illustrated in 1d in Figure 8.14(b), which shows that Newton's method can end up in a local maximum rather than a local minimum.

In general, any time the quadratic approximation made by Newton's method becomes invalid, we are in trouble. However, there is usually a local region around the current iterate where we can safely

1. See https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

approximate the objective by a quadratic. Let us call this region $\mathcal{R}_t$, and let us call $M(\boldsymbol{\delta})$ the model (or approximation) to the objective, where $\boldsymbol{\delta} = \boldsymbol{\theta} - \boldsymbol{\theta}_t$. Then at each step we can solve

$$\boldsymbol{\delta}^* = \underset{\boldsymbol{\delta} \in \mathcal{R}_t}{\operatorname{argmin}} \, M_t(\boldsymbol{\delta}) \tag{8.49}$$

This is called **trust-region optimization**. (This can be seen as the "opposite" of line search, in the sense that we pick a distance we want to travel, determined by $\mathcal{R}_t$, and then solve for the optimal direction, rather than picking the direction and then solving for the optimal distance.)

We usually assume that $M_t(\boldsymbol{\delta})$ is a quadratic approximation:

$$M_t(\boldsymbol{\delta}) = \mathcal{L}(\boldsymbol{\theta}_t) + \boldsymbol{g}_t^\mathsf{T} \boldsymbol{\delta} + \frac{1}{2} \boldsymbol{\delta}^\mathsf{T} \mathbf{H}_t \boldsymbol{\delta} \tag{8.50}$$

where $\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t}$ is the gradient, and $\mathbf{H}_t = \nabla^2_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})|_{\boldsymbol{\theta}_t}$ is the Hessian. Furthermore, it is common to assume that $\mathcal{R}_t$ is a ball of radius r, i.e., $\mathcal{R}_t = \{\boldsymbol{\delta} : ||\boldsymbol{\delta}||_2 \leq r\}$. Using this, we can convert the constrained problem into an unconstrained one as follows:

$$\boldsymbol{\delta}^* = \underset{\boldsymbol{\delta}}{\operatorname{argmin}} \, M(\boldsymbol{\delta}) + \lambda ||\boldsymbol{\delta}||_2^2 = \underset{\boldsymbol{\delta}}{\operatorname{argmin}} \, \boldsymbol{g}^\mathsf{T} \boldsymbol{\delta} + \frac{1}{2} \boldsymbol{\delta}^\mathsf{T} (\mathbf{H} + \lambda \mathbf{I}) \boldsymbol{\delta} \tag{8.51}$$

for some Lagrange multiplier $\lambda > 0$ which depends on the radius r (see Section 8.5.1 for a discussion of Lagrange multipliers). We can solve this using

$$\boldsymbol{\delta} = -(\mathbf{H} + \lambda \mathbf{I})^{-1} \boldsymbol{g} \tag{8.52}$$

This is called **Tikhonov damping** or **Tikhonov regularization**. See Figure 8.15 for an illustration.

Note that adding a sufficiently large $\lambda \mathbf{I}$ to $\mathbf{H}$ ensures the resulting matrix is always positive definite. As $\lambda \to 0$, this trust method reduces to Newton's method, but for λ large enough, it will make all the negative eigenvalues positive (and all the 0 eigenvalues become equal to λ).

8.4 Stochastic gradient descent

In this section, we consider **stochastic optimization**, where the goal is to minimize the average value of a function:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{z})} \left[\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{z}) \right] \tag{8.53}$$

where $\boldsymbol{z}$ is a random input to the objective. This could be a "noise" term, coming from the environment, or it could be a training example drawn randomly from the training set, as we explain below.

At each iteration, we assume we observe $\mathcal{L}_t(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{z}_t)$, where $\boldsymbol{z}_t \sim q$. We also assume a way to compute an unbiased estimate of the gradient of $\mathcal{L}$. If the distribution $q(\boldsymbol{z})$ is independent of the parameters we are optimizing, we can use $\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_t(\boldsymbol{\theta}_t)$. In this case, the resulting algorithm can be written as follows:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t, \boldsymbol{z}_t) = \boldsymbol{\theta}_t - \eta_t \boldsymbol{g}_t \tag{8.54}$$

This method is known as **stochastic gradient descent** or **SGD**. As long as the gradient estimate is unbiased, then this method will converge to a stationary point, providing we decay the step size η_t at a certain rate, as we discuss in Section 8.4.3.

8.4.1 Application to finite sum problems

SGD is very widely used in machine learning. To see why, recall from Section 4.3 that many model fitting procedures are based on empirical risk minimization, which involve minimizing the following loss:

$$\mathcal{L}(\boldsymbol{\theta}_t) = \frac{1}{N}\sum_{n=1}^{N} \ell(\boldsymbol{y}_n, f(\boldsymbol{x}_n; \boldsymbol{\theta}_t)) = \frac{1}{N}\sum_{n=1}^{N} \mathcal{L}_n(\boldsymbol{\theta}_t) \tag{8.55}$$

This is called a **finite sum problem**. The gradient of this objective has the form

$$\boldsymbol{g}_t = \frac{1}{N}\sum_{n=1}^{N} \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}_t) = \frac{1}{N}\sum_{n=1}^{N} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{y}_n, f(\boldsymbol{x}_n; \boldsymbol{\theta}_t)) \tag{8.56}$$

This requires summing over all N training examples, and thus can be slow if N is large. Fortunately we can approximate this by sampling a **minibatch** of $B \ll N$ samples to get

$$\boldsymbol{g}_t \approx \frac{1}{|\mathcal{B}_t|}\sum_{n\in\mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}_t) = \frac{1}{|\mathcal{B}_t|}\sum_{n\in\mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{y}_n, f(\boldsymbol{x}_n; \boldsymbol{\theta}_t)) \tag{8.57}$$

where $\mathcal{B}_t$ is a set of randomly chosen examples to use at iteration t.[2] This is an unbiased approximation to the empirical average in Equation (8.56). Hence we can safely use this with SGD.

Although the theoretical rate of convergence of SGD is slower than batch GD (in particular, SGD has a sublinear convergence rate), in practice SGD is often faster, since the per-step time is much lower [BB08; BB11]. To see why SGD can make faster progress than full batch GD, suppose we have a dataset consisting of a single example duplicated K times. Batch training will be (at least) K times slower than SGD, since it will waste time computing the gradient for the repeated examples. Even if there are no duplicates, batch training can be wasteful, since early on in training the parameters are not well estimated, so it is not worth carefully evaluating the gradient.

8.4.2 Example: SGD for fitting linear regression

In this section, we show how to use SGD to fit a linear regression model. Recall from Section 4.2.7 that the objective has the form

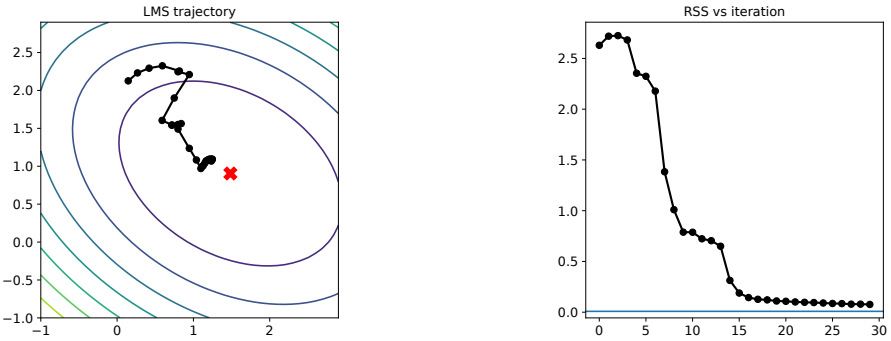
$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2N}\sum_{n=1}^{N} (\boldsymbol{x}_n^\mathsf{T}\boldsymbol{\theta} - y_n)^2 = \frac{1}{2N}||\mathbf{X}\boldsymbol{\theta} - \boldsymbol{y}||_2^2 \tag{8.58}$$

The gradient is

$$\boldsymbol{g}_t = \frac{1}{N}\sum_{n=1}^{N} (\boldsymbol{\theta}_t^\mathsf{T}\boldsymbol{x}_n - y_n)\boldsymbol{x}_n \tag{8.59}$$

2. In practice we usually sample $\mathcal{B}_t$ without replacement. However, once we reach the end of the dataset (i.e., after a single training **epoch**), we can perform a random shuffling of the examples, to ensure that each minibatch on the next epoch is different from the last. This version of SGD is analyzed in [HS19].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 8.16: Illustration of the LMS algorithm. Left: we start from $\boldsymbol{\theta} = (-0.5, 2)$ and slowly converging to the least squares solution of $\hat{\boldsymbol{\theta}} = (1.45, 0.93)$ (red cross). Right: plot of objective function over time. Note that it does not decrease monotonically. Generated by lms_ demo.ipynb.

Now consider using SGD with a minibatch size of $B = 1$. The update becomes

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t(\boldsymbol{\theta}_t^\mathsf{T}\boldsymbol{x}_n - y_n)\boldsymbol{x}_n \tag{8.60}$$

where $n = n(t)$ is the index of the example chosen at iteration t. The overall algorithm is called the **least mean squares** (**LMS**) algorithm, and is also known as the **delta rule**, or the **Widrow-Hoff rule**.

Figure 8.16 shows the results of applying this algorithm to the data shown in Figure 11.2. We start at $\boldsymbol{\theta} = (-0.5, 2)$ and converge (in the sense that $||\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}||_2^2$ drops below a threshold of 10^{-2}) in about 26 iterations. Note that SGD (and hence LMS) may require multiple passes through the data to find the optimum.

8.4.3 Choosing the step size (learning rate)

When using SGD, we need to be careful in how we choose the learning rate in order to achieve convergence. For example, in Figure 8.17 we plot the loss vs the learning rate when we apply SGD to a deep neural network classifier (see Chapter 13 for details). We see a U-shaped curve, where an overly small learning rate results in underfitting, and overly large learning rate results in instability of the model (c.f., Figure 8.11(b)); in both cases, we fail to converge to a local optimum.

One heuristic for choosing a good learning rate, proposed in [Smi18], is to start with a small learning rate and gradually increase it, evaluating performance using a small number of minibatches. We then make a plot like the one in Figure 8.17, and pick the learning rate with the lowest loss. (In practice, it is better to pick a rate that is slightly smaller than (i.e., to the left of) the one with the lowest loss, to ensure stability.)

Rather than choosing a single constant learning rate, we can use a **learning rate schedule**, in which we adjust the step size over time. Theoretically, a sufficient condition for SGD to achieve

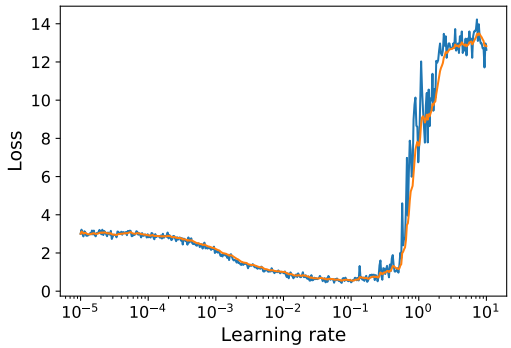


Figure 8.17: Loss vs learning rate (horizontal axis). Training loss vs learning rate for a small MLP fit to FashionMNIST using vanilla SGD. (Raw loss in blue, EWMA smoothed version in orange). Generated by lrschedule_tf.ipynb.

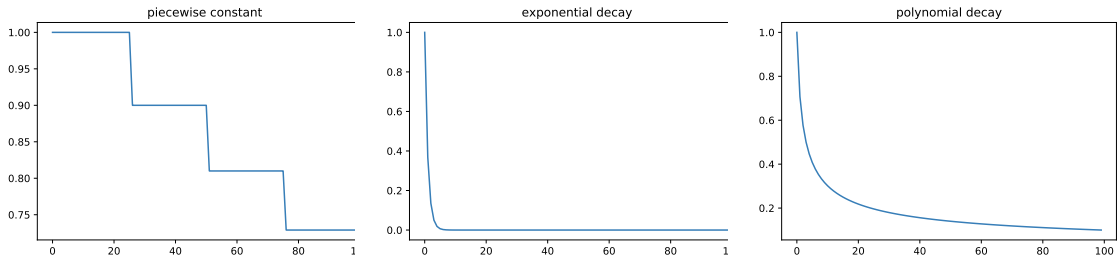


Figure 8.18: Illustration of some common learning rate schedules. (a) Piecewise constant. (b) Exponential decay. (c) Polynomial decay. Generated by learning_rate_plot.ipynb.

convergence is if the learning rate schedule satisfies the **Robbins-Monro conditions**:

$$\eta_t \to 0, \ \frac{\sum_{t=1}^{\infty} \eta_t^2}{\sum_{t=1}^{\infty} \eta_t} \to 0 \tag{8.61}$$

Some common examples of learning rate schedules are listed below:

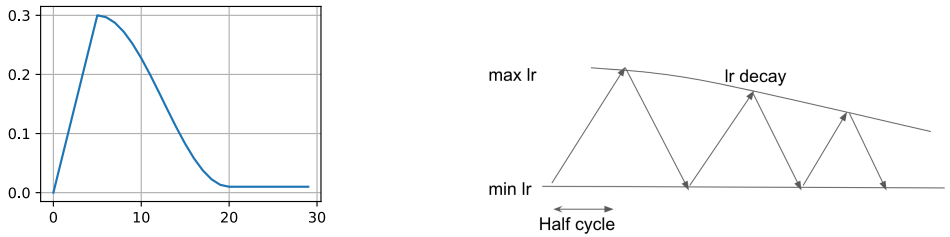
$$\eta_t = \eta_i \text{ if } t_i \leq t \leq t_{i+1} \quad \text{piecewise constant} \tag{8.62}$$

$$\eta_t = \eta_0 e^{-\lambda t} \text{ exponential decay} \tag{8.63}$$

$$\eta_t = \eta_0 (\beta t + 1)^{-\alpha} \text{ polynomial decay} \tag{8.64}$$

In the piecewise constant schedule, t_i are a set of time points at which we adjust the learning rate to a specified value. For example, we may set $\eta_i = \eta_0 \gamma^i$, which reduces the initial learning rate by a factor of γ for each threshold (or milestone) that we pass. Figure 8.18a illustrates this for $\eta_0 = 1$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 8.19: (a) Linear warm-up followed by cosine cool-down. (b) Cyclical learning rate schedule.

and $\gamma = 0.9$. This is called **step decay**. Sometimes the threshold times are computed adaptively, by estimating when the train or validation loss has plateaued; this is called **reduce-on-plateau**. Exponential decay is typically too fast, as illustrated in Figure 8.18b. A common choice is polynomial decay, with $\alpha = 0.5$ and $\beta = 1$, as illustrated in Figure 8.18c; this corresponds to a **square-root schedule**, $\eta_t = \eta_0 \frac{1}{\sqrt{t+1}}$.

In the deep learning community, another common schedule is to quickly increase the learning rate and then gradually decrease it again, as shown in Figure 8.19a. This is called **learning rate warmup**, or the **one-cycle learning rate schedule** [Smi18]. The motivation for this is the following: initially the parameters may be in a part of the loss landscape that is poorly conditioned, so a large step size will "bounce around" too much (c.f., Figure 8.11(b)) and fail to make progress downhill. However, with a slow learning rate, the algorithm can discover flatter regions of space, where a larger step size can be used. Once there, fast progress can be made. However, to ensure convergence to a point, we must reduce the learning rate to 0. See [Got+19; Gil+21] for more details.

It is also possible to increase and decrease the learning rate multiple times, in a cyclical fashion. This is called a **cyclical learning rate** [Smi18], and was popularized by the `fast.ai` course. See Figure 8.19b for an illustration using triangular shapes. The motivation behind this approach is to escape local minima. The minimum and maximum learning rates can be found based on the initial "dry run" described above, and the half-cycle can be chosen based on how many restarts you want to do with your training budget. A related approach, known as **stochastic gradient descent with warm restarts**, was proposed in [LH17]; they proposed storing all the checkpoints visited after each cool down, and using all of them as members of a model ensemble. (See Section 18.2 for a discussion of ensemble learning.)

An alternative to using heuristics for estimating the learning rate is to use line search (Section 8.2.2.2). This is tricky when using SGD, because the noisy gradients make the computation of the Armijo condition difficult [CS20]. However, [Vas+19] show that it can be made to work if the variance of the gradient noise goes to zero over time. This can happen if the model is sufficiently flexible that it can perfectly interpolate the training set.

8.4.4 Iterate averaging

The parameter estimates produced by SGD can be very unstable over time. To reduce the variance of the estimate, we can compute the average using

$$\overline{\boldsymbol{\theta}}_t = \frac{1}{t}\sum_{i=1}^{t}\boldsymbol{\theta}_i = \frac{1}{t}\boldsymbol{\theta}_t + \frac{t-1}{t}\overline{\boldsymbol{\theta}}_{t-1} \tag{8.65}$$

where $\boldsymbol{\theta}_t$ are the usual SGD iterates. This is called **iterate averaging** or **Polyak-Ruppert averaging** [Rup88].

In [PJ92], they prove that the estimate $\overline{\boldsymbol{\theta}}_t$ achieves the best possible asymptotic convergence rate among SGD algorithms, matching that of variants using second-order information, such as Hessians.

This averaging can also have statistical benefits. For example, in [NR18], they prove that, in the case of linear regression, this method is equivalent to ℓ_2 regularization (i.e., ridge regression).

Rather than an exponential moving average of SGD iterates, **Stochastic Weight Averaging** (SWA) [Izm+18] uses an *equal* average in conjunction with a modified learning rate schedule. In contrast to standard Polyak-Ruppert averaging, which was motivated for faster convergence rates, SWA exploits the flatness in objectives used to train deep neural networks, to find solutions which provide better generalization.

8.4.5 Variance reduction *

In this section, we discuss various ways to reduce the variance in SGD. In some cases, this can improve the theoretical convergence rate from sublinear to linear (i.e., the same as full-batch gradient descent) [SLRB17; JZ13; DBLJ14]. These methods reduce the variance of the gradients, rather than the parameters themselves and are designed to work for finite sum problems.

8.4.5.1 SVRG

The basic idea of **stochastic variance reduced gradient** (**SVRG**) [JZ13] is to use a control variate, in which we estimate a baseline value of the gradient based on the full batch, which we then use to compare the stochastic gradients to.

More precisely, ever so often (e.g., once per epoch), we compute the full gradient at a "snapshot" of the model parameters $\tilde{\boldsymbol{\theta}}$; the corresponding "exact" gradient is therefore $\nabla\mathcal{L}(\tilde{\boldsymbol{\theta}})$. At step t, we compute the usual stochastic gradient at the current parameters, $\nabla\mathcal{L}_t(\boldsymbol{\theta}_t)$, but also at the snapshot parameters, $\nabla\mathcal{L}_t(\tilde{\boldsymbol{\theta}})$, which we use as a baseline. We can then use the following improved gradient estimate

$$\boldsymbol{g}_t = \nabla\mathcal{L}_t(\boldsymbol{\theta}_t) - \nabla\mathcal{L}_t(\tilde{\boldsymbol{\theta}}) + \nabla\mathcal{L}(\tilde{\boldsymbol{\theta}}) \tag{8.66}$$

to compute $\boldsymbol{\theta}_{t+1}$. This is unbiased because $\mathbb{E}\left[\nabla\mathcal{L}_t(\tilde{\boldsymbol{\theta}})\right] = \nabla\mathcal{L}(\tilde{\boldsymbol{\theta}})$. Furthermore, the update only involves two gradient computations, since we can compute $\nabla\mathcal{L}(\tilde{\boldsymbol{\theta}})$ once per epoch. At the end of the epoch, we update the snapshot parameters, $\tilde{\boldsymbol{\theta}}$, based on the most recent value of $\boldsymbol{\theta}_t$, or a running average of the iterates, and update the expected baseline. (We can compute snapshots less often, but then the baseline will not be correlated with the objective and can hurt performance, as shown in [DB18].)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Iterations of SVRG are computationally faster than those of full-batch GD, but SVRG can still match the theoretical convergence rate of GD.

8.4.5.2 SAGA

In this section, we describe the **stochastic averaged gradient accelerated** (**SAGA**) algorithm of [DBLJ14]. Unlike SVRG, it only requires one full batch gradient computation, at the start of the algorithm. However, it "pays" for this saving in time by using more memory. In particular, it must store N gradient vectors. This enables the method to maintain an approximation of the global gradient by removing the old local gradient from the overall sum and replacing it with the new local gradient. This is called an **aggregated gradient** method.

More precisely, we first initialize by computing $\boldsymbol{g}_n^{\text{local}} = \nabla \mathcal{L}_n(\boldsymbol{\theta}_0)$ for all n, and the average, $\boldsymbol{g}^{\text{avg}} = \frac{1}{N}\sum_{n=1}^{N} \boldsymbol{g}_n^{\text{local}}$. Then, at iteration t, we use the gradient estimate

$$\boldsymbol{g}_t = \nabla \mathcal{L}_n(\boldsymbol{\theta}_t) - \boldsymbol{g}_n^{\text{local}} + \boldsymbol{g}^{\text{avg}} \tag{8.67}$$

where $n \sim \text{Unif}\{1, \ldots, N\}$ is the example index sampled at iteration t. We then update $\boldsymbol{g}_n^{\text{local}} = \nabla \mathcal{L}_n(\boldsymbol{\theta}_t)$ and $\boldsymbol{g}^{\text{avg}}$ by replacing the old $\boldsymbol{g}_n^{\text{local}}$ by its new value.

This has an advantage over SVRG since it only has to do one full batch sweep at the start. (In fact, the initial sweep is not necessary, since we can compute $\boldsymbol{g}^{\text{avg}}$ "lazily", by only incorporating gradients we have seen so far.) The downside is the large extra memory cost. However, if the features (and hence gradients) are sparse, the memory cost can be reasonable. Indeed, the SAGA algorithm is recommended for use in the sklearn logistic regression code when N is large and $\boldsymbol{x}$ is sparse.[3]

8.4.5.3 Application to deep learning

Variance reduction methods are widely used for fitting ML models with convex objectives, such as linear models. However, there are various difficulties associated with using SVRG with conventional deep learning training practices. For example, the use of batch normalization (Section 14.2.4.1), data augmentation (Section 19.1) and dropout (Section 13.5.4) all break the assumptions of the method, since the loss will differ randomly in ways that depend not just on the parameters and the data index n. For more details, see e.g., [DB18; Arn+19].

8.4.6 Preconditioned SGD

In this section, we consider **preconditioned SGD**, which involves the following update:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{M}_t^{-1} \boldsymbol{g}_t, \tag{8.68}$$

where $\mathbf{M}_t$ is a **preconditioning matrix**, or simply the **preconditioner**, typically chosen to be positive-definite. Unfortunately the noise in the gradient estimates make it difficult to reliably estimate the Hessian, which makes it difficult to use the methods from Section 8.3. In addition, it is expensive to solve for the update direction with a full preconditioning matrix. Therefore most practitioners use a diagonal preconditioner $\mathbf{M}_t$. Such preconditioners do not necessarily use second-order information, but often result in speedups compared to vanilla SGD. See also [Roo+21]

3. See https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.

for a probabilitic interpretation of these heuristics, and sgd_comparison.ipynb for an empirical comparison on some simple datasets.

8.4.6.1 AdaGrad

The **AdaGrad** (short for "adaptive gradient") method of [DHS11] was originally designed for optimizing convex objectives where many elements of the gradient vector are zero; these might correspond to features that are rarely present in the input, such as rare words. The update has the following form

$$\theta_{t+1,d} = \theta_{t,d} - \eta_t \frac{1}{\sqrt{s_{t,d} + \epsilon}} g_{t,d} \tag{8.69}$$

where $d = 1 : D$ indexes the dimensions of the parameter vector, and

$$s_{t,d} = \sum_{i=1}^{t} g_{i,d}^2 \tag{8.70}$$

is the sum of the squared gradients and $\epsilon > 0$ is a small term to avoid dividing by zero. Equivalently we can write the update in vector form as follows:

$$\Delta \boldsymbol{\theta}_t = -\eta_t \frac{1}{\sqrt{\boldsymbol{s}_t + \epsilon}} \boldsymbol{g}_t \tag{8.71}$$

where the square root and division is performed elementwise. Viewed as preconditioned SGD, this is equivalent to taking $\mathbf{M}_t = \text{diag}(\boldsymbol{s}_t + \boldsymbol{\epsilon})^{1/2}$. This is an example of an **adaptive learning rate**; the overall stepsize η_t still needs to be chosen, but the results are less sensitive to it compared to vanilla GD. In particular, we usually fix $\eta_t = \eta_0$.

8.4.6.2 RMSProp and AdaDelta

A defining feature of AdaGrad is that the term in the denominator gets larger over time, so the effective learning rate drops. While it is necessary to ensure convergence, it might hurt performance as the denominator gets large too fast.

An alternative is to use an exponentially weighted moving average (EWMA, Section 4.4.2.2) of the past squared gradients, rather than their sum:

$$s_{t+1,d} = \beta s_{t,d} + (1 - \beta) g_{t,d}^2 \tag{8.72}$$

In practice we usually use $\beta \sim 0.9$, which puts more weight on recent examples. In this case,

$$\sqrt{s_{t,d}} \approx \text{RMS}(\boldsymbol{g}_{1:t,d}) = \sqrt{\frac{1}{t} \sum_{\tau=1}^{t} g_{\tau,d}^2} \tag{8.73}$$

where RMS stands for "root mean squared". Hence this method, (which is based on the earlier **RPROP** method of [RB93]) is known as **RMSProp** [Hin14]. The overall update of RMSProp is

$$\Delta \boldsymbol{\theta}_t = -\eta_t \frac{1}{\sqrt{\boldsymbol{s}_t + \epsilon}} \boldsymbol{g}_t. \tag{8.74}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

The **AdaDelta** method was independently introduced in [Zei12], and is similar to RMSprop. However, in addition to accumulating an EWMA of the gradients in $\hat{\boldsymbol{s}}$, it also keeps an EWMA of the updates $\boldsymbol{\delta}_t$ to obtain an update of the form

$$\Delta\boldsymbol{\theta}_t = -\eta_t \frac{\sqrt{\boldsymbol{\delta}_{t-1} + \epsilon}}{\sqrt{\boldsymbol{s}_t + \epsilon}} \boldsymbol{g}_t \tag{8.75}$$

where

$$\boldsymbol{\delta}_t = \beta \boldsymbol{\delta}_{t-1} + (1-\beta)(\Delta\boldsymbol{\theta}_t)^2 \tag{8.76}$$

and $\boldsymbol{s}_t$ is the same as in RMSProp. This has the advantage that the "units" of the numerator and denominator cancel, so we are just elementwise-multiplying the gradient by a scalar. This eliminates the need to tune the learning rate η_t, which means one can simply set $\eta_t = 1$, although popular implementations of AdaDelta still keep η_t as a tunable hyperparameter. However, since these adaptive learning rates need not decrease with time (unless we choose η_t to explicitly do so), these methods are not guaranteed to converge to a solution.

8.4.6.3 Adam

It is possible to combine RMSProp with momentum. In particular, let us compute an EWMA of the gradients (as in momentum) and squared gradients (as in RMSProp)

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t \tag{8.77}$$

$$\boldsymbol{s}_t = \beta_2 \boldsymbol{s}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2 \tag{8.78}$$

We then perform the following update:

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_t - \eta_t \frac{\boldsymbol{m}_t}{\sqrt{\boldsymbol{s}_t} + \epsilon} \tag{8.79}$$

The resulting method is known as **Adam**, which stands for "adaptive moment estimation" [KB15].

The standard values for the various constants are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-6}$. (If we set $\beta_1 = 0$ and no bias-correction, we recover RMSProp, which does not use momentum.) For the overall learning rate, it is common to use a fixed value such as $\eta_t = 0.001$. Again, as the adaptive learning rate may not decrease over time, convergence is not guaranteed (see Section 8.4.6.4).

If we initialize with $\boldsymbol{m}_0 = \boldsymbol{s}_0 = \mathbf{0}$, then initial estimates will be biased towards small values. The authors therefore recommend using the bias-corrected moments, which increase the values early in the optimization process. These estimates are given by

$$\hat{\boldsymbol{m}}_t = \boldsymbol{m}_t / (1-\beta_1^t) \tag{8.80}$$

$$\hat{\boldsymbol{s}}_t = \boldsymbol{s}_t / (1-\beta_2^t) \tag{8.81}$$

The advantage of bias-correction is shown in Figure 4.3.

8.4.6.4 Issues with adaptive learning rates

When using diagonal scaling methods, the overall learning rate is determined by $\eta_0 \mathbf{M}_t^{-1}$, which changes with time. Hence these methods are often called **adaptive learning rate** methods. However, they still require setting the base learning rate η_0.

Since the EWMA methods are typically used in the stochastic setting where the gradient estimates are noisy, their learning rate adaptation can result in non-convergence even on convex problems [RKK18]. In [Zha+22] they show experimentally that vanilla Adam can be made to converge provided the β_1 and β_2 parameters are tuned on a per-dataset basis, but it is better to find an automatic, robust method. Various solutions to this problem have been proposed, including AMSGRAD [RKK18], PADAM [CG18; Zho+18], and YOGI [Zah+18]. For example, the YOGI update modifies ADAM by replacing

$$\boldsymbol{s}_t = \beta_2 \boldsymbol{s}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2 = \boldsymbol{s}_{t-1} + (1-\beta_2)(\boldsymbol{g}_t^2 - \boldsymbol{s}_{t-1}) \tag{8.82}$$

with

$$\boldsymbol{s}_t = \boldsymbol{s}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2 \odot \operatorname{sgn}(\boldsymbol{g}_t^2 - \boldsymbol{s}_{t-1}) \tag{8.83}$$

More recently, [Tan+24] proposed **ADOPT**, which not only results in provable convergence, but also seems to work better in practice. The basic idea is to normalize (precondition) the gradient before computing the momentum update, i.e., we use $\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\frac{\boldsymbol{g}_t}{\sqrt{\boldsymbol{s}_{t-1}+\epsilon}}$ instead of $\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t$. We also now update the parameters using $\boldsymbol{\theta}_t = \boldsymbol{\theta}_t - \eta_t \boldsymbol{m}_t$ instead of $\boldsymbol{\theta}_t = \boldsymbol{\theta}_t - \eta_t \frac{\boldsymbol{m}_t}{\sqrt{\boldsymbol{s}_t+\epsilon}}$.

8.4.6.5 Non-diagonal preconditioning matrices

Although the methods we have discussed above can adapt the learning rate of each parameter, they do not solve the more fundamental problem of ill-conditioning due to correlation of the parameters, and hence do not always provide as much of a speed boost over vanilla SGD as one may hope.

One way to get faster convergence is to use the following preconditioning matrix, known as **full-matrix Adagrad** [DHS11]:

$$\mathbf{M}_t = [(\mathbf{G}_t \mathbf{G}_t^\mathsf{T})^{\frac{1}{2}} + \epsilon \mathbf{I}_D]^{-1} \tag{8.84}$$

where

$$\mathbf{G}_t = [\boldsymbol{g}_t, \ldots, \boldsymbol{g}_1] \tag{8.85}$$

Here $\boldsymbol{g}_i = \nabla_{\boldsymbol{\psi}} c(\boldsymbol{\psi}_i)$ is the D-dimensional gradient vector computed at step i. Unfortunately, $\mathbf{M}_t$ is a $D \times D$ matrix, which is expensive to store and invert.

The **Shampoo** algorithm [GKS18] makes a block diagonal approximation to $\mathbf{M}$, one per layer of the model, and then exploits Kronecker product structure to efficiently invert it. (It is called "shampoo" because it uses a conditioner.) Recently, [Ani+20] scaled this method up to fit very large deep models in record time.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

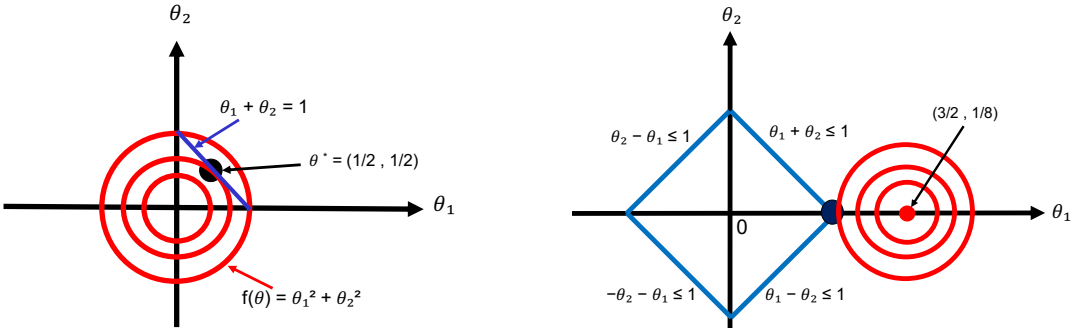


Figure 8.20: Illustration of some constrained optimization problems. Red contours are the level sets of the objective function $\mathcal{L}(\boldsymbol{\theta})$. Optimal constrained solution is the black dot, (a) Blue line is the equality constraint $h(\boldsymbol{\theta}) = 0$. (b) Blue lines denote the inequality constraints $|\theta_1| + |\theta_2| \leq 1$. (Compare to Figure 11.8 (left).)

8.5 Constrained optimization

In this section, we consider the following **constrained optimization problem**:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta} \in \mathcal{C}} \mathcal{L}(\boldsymbol{\theta}) \tag{8.86}$$

where the feasible set, or constraint set, is

$$\mathcal{C} = \{\boldsymbol{\theta} \in \mathbb{R}^D : h_i(\boldsymbol{\theta}) = 0, i \in \mathcal{E},\ g_j(\boldsymbol{\theta}) \leq 0, j \in \mathcal{I}\} \tag{8.87}$$

where $\mathcal{E}$ is the set of **equality constraints**, and $\mathcal{I}$ is the set of **inequality constraints**.

For example, suppose we have a quadratic objective, $\mathcal{L}(\boldsymbol{\theta}) = \theta_1^2 + \theta_2^2$, subject to a linear equality constraint, $h(\boldsymbol{\theta}) = 1 - \theta_1 - \theta_2 = 0$. Figure 8.20(a) plots the level sets of $\mathcal{L}$, as well as the constraint surface. What we are trying to do is find the point $\boldsymbol{\theta}^*$ that lives on the line, but which is closest to the origin. It is clear from the geometry that the optimal solution is $\boldsymbol{\theta} = (0.5, 0.5)$, indicated by the solid black dot.

In the following sections, we briefly describe some of the theory and algorithms underlying constrained optimization. More details can be found in other books, such as [BV04; NW06; Ber15; Ber16].

8.5.1 Lagrange multipliers

In this section, we discuss how to solve equality contrained optimization problems. We initially assume that we have just one equality constraint, $h(\boldsymbol{\theta}) = 0$.

First note that for any point on the constraint surface, $\nabla h(\boldsymbol{\theta})$ will be orthogonal to the constraint surface. To see why, consider another point nearby, $\boldsymbol{\theta} + \boldsymbol{\epsilon}$, that also lies on the surface. If we make a first-order Taylor expansion around $\boldsymbol{\theta}$ we have

$$h(\boldsymbol{\theta} + \boldsymbol{\epsilon}) \approx h(\boldsymbol{\theta}) + \boldsymbol{\epsilon}^\mathsf{T} \nabla h(\boldsymbol{\theta}) \tag{8.88}$$

Since both $\boldsymbol{\theta}$ and $\boldsymbol{\theta} + \boldsymbol{\epsilon}$ are on the constraint surface, we must have $h(\boldsymbol{\theta}) = h(\boldsymbol{\theta} + \boldsymbol{\epsilon})$ and hence $\boldsymbol{\epsilon}^{\mathsf{T}} \nabla h(\boldsymbol{\theta}) \approx 0$. Since $\boldsymbol{\epsilon}$ is parallel to the constraint surface, $\nabla h(\boldsymbol{\theta})$ must be perpendicular to it.

We seek a point $\boldsymbol{\theta}^*$ on the constraint surface such that $\mathcal{L}(\boldsymbol{\theta})$ is minimized. We just showed that it must satisfy the condition that $\nabla h(\boldsymbol{\theta}^*)$ is orthogonal to the constraint surface. In addition, such a point must have the property that $\nabla \mathcal{L}(\boldsymbol{\theta})$ is also orthogonal to the constraint surface, as otherwise we could decrease $\mathcal{L}(\boldsymbol{\theta})$ by moving a short distance along the constraint surface. Since both $\nabla h(\boldsymbol{\theta})$ and $\nabla \mathcal{L}(\boldsymbol{\theta})$ are orthogonal to the constraint surface at $\boldsymbol{\theta}^*$, they must be parallel (or anti-parallel) to each other. Hence there must exist a constant $\lambda^* \in \mathbb{R}$ such that

$$\nabla \mathcal{L}(\boldsymbol{\theta}^*) = \lambda^* \nabla h(\boldsymbol{\theta}^*) \tag{8.89}$$

(We cannot just equate the gradient vectors, since they may have different magnitudes.) The constant λ^* is called a **Lagrange multiplier**, and can be positive, negative, or zero. This latter case occurs when $\nabla \mathcal{L}(\boldsymbol{\theta}^*) = 0$.

We can convert Equation (8.89) into an objective, known as the **Lagrangian**, that we should find a stationary point of the following:

$$L(\boldsymbol{\theta}, \lambda) \triangleq \mathcal{L}(\boldsymbol{\theta}) + \lambda h(\boldsymbol{\theta}) \tag{8.90}$$

At a stationary point of the Lagrangian, we have

$$\nabla_{\boldsymbol{\theta},\lambda} L(\boldsymbol{\theta}, \lambda) = \mathbf{0} \iff \lambda \nabla_{\boldsymbol{\theta}} h(\boldsymbol{\theta}) = \nabla \mathcal{L}(\boldsymbol{\theta}),\ h(\boldsymbol{\theta}) = 0 \tag{8.91}$$

This is called a **critical point**, and satisfies the original constraint $h(\boldsymbol{\theta}) = 0$ and Equation (8.89).

If we have $m > 1$ constraints, we can form a new constraint function by addition, as follows:

$$L(\boldsymbol{\theta}, \boldsymbol{\lambda}) = \mathcal{L}(\boldsymbol{\theta}) + \sum_{j=1}^{m} \lambda_j h_j(\boldsymbol{\theta}) \tag{8.92}$$

We now have $D+m$ equations in $D+m$ unknowns and we can use standard unconstrained optimization methods to find a stationary point. We give some examples below.

8.5.1.1 Example: 2d Quadratic objective with one linear equality constraint

Consider minimizing $\mathcal{L}(\boldsymbol{\theta}) = \theta_1^2 + \theta_2^2$ subject to the constraint that $\theta_1 + \theta_2 = 1$.

(This is the problem illustrated in Figure 8.20(a).) The Lagrangian is

$$L(\theta_1, \theta_2, \lambda) = \theta_1^2 + \theta_2^2 + \lambda(\theta_1 + \theta_2 - 1) \tag{8.93}$$

We have the following conditions for a stationary point:

$$\frac{\partial}{\partial \theta_1} L(\theta_1, \theta_2, \lambda) = 2\theta_1 + \lambda = 0 \tag{8.94}$$

$$\frac{\partial}{\partial \theta_2} L(\theta_1, \theta_2, \lambda) = 2\theta_2 + \lambda = 0 \tag{8.95}$$

$$\frac{\partial}{\partial \lambda} L(\theta_1, \theta_2, \lambda) = \theta_1 + \theta_2 - 1 = 0 \tag{8.96}$$

From Equations 8.94 and 8.95 we find $2\theta_1 = -\lambda = 2\theta_2$, so $\theta_1 = \theta_2$. Also, from Equation (8.96), we find $2\theta_1 = 1$. So $\boldsymbol{\theta}^* = (0.5, 0.5)$, as we claimed earlier. Furthermore, this is the global minimum since the objective is convex and the constraint is affine.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.5.2 The KKT conditions

In this section, we generalize the concept of Lagrange multipliers to additionally handle inequality constraints.

First consider the case where we have a single inequality constraint $g(\boldsymbol{\theta}) \leq 0$. To find the optimum, one approach would be to consider an unconstrained problem where we add the penalty as an infinite step function:

$$\hat{\mathcal{L}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \infty \mathbb{I}(g(\boldsymbol{\theta}) > 0) \tag{8.97}$$

However, this is a discontinuous function that is hard to optimize.

Instead, we create a lower bound of the form $\mu g(\boldsymbol{\theta})$, where $\mu \geq 0$. This gives us the following Lagrangian:

$$L(\boldsymbol{\theta}, \mu) = \mathcal{L}(\boldsymbol{\theta}) + \mu g(\boldsymbol{\theta}) \tag{8.98}$$

Note that the step function can be recovered using

$$\hat{\mathcal{L}}(\boldsymbol{\theta}) = \max_{\mu \geq 0} L(\boldsymbol{\theta}, \mu) = \begin{cases} \infty & \text{if } g(\boldsymbol{\theta}) > 0, \\ \mathcal{L}(\boldsymbol{\theta}) & \text{otherwise} \end{cases} \tag{8.99}$$

Thus our optimization problem becomes

$$\min_{\boldsymbol{\theta}} \max_{\mu \geq 0} L(\boldsymbol{\theta}, \mu) \tag{8.100}$$

Now consider the general case where we have multiple inequality constraints, $\boldsymbol{g}(\boldsymbol{\theta}) \leq \mathbf{0}$, and multiple equality constraints, $\boldsymbol{h}(\boldsymbol{\theta}) = \mathbf{0}$. The **generalized Lagrangian** becomes

$$L(\boldsymbol{\theta}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \mathcal{L}(\boldsymbol{\theta}) + \sum_i \mu_i g_i(\boldsymbol{\theta}) + \sum_j \lambda_j h_j(\boldsymbol{\theta}) \tag{8.101}$$

(We are free to change $-\lambda_j h_j$ to $+\lambda_j h_j$ since the sign is arbitrary.) Our optimization problem becomes

$$\min_{\boldsymbol{\theta}} \max_{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}} L(\boldsymbol{\theta}, \boldsymbol{\mu}, \boldsymbol{\lambda}) \tag{8.102}$$

When $\mathcal{L}$ and g are convex, then all critical points of this problem must satisfy the following criteria (under some conditions [BV04, Sec.5.2.3]):

- All constraints are satisfied (this is called **feasibility**):

$$\boldsymbol{g}(\boldsymbol{\theta}) \leq \mathbf{0}, \ \boldsymbol{h}(\boldsymbol{\theta}) = \mathbf{0} \tag{8.103}$$

- The solution is a stationary point:

$$\nabla \mathcal{L}(\boldsymbol{\theta}^*) + \sum_i \mu_i \nabla g_i(\boldsymbol{\theta}^*) + \sum_j \lambda_j \nabla h_j(\boldsymbol{\theta}^*) = \mathbf{0} \tag{8.104}$$

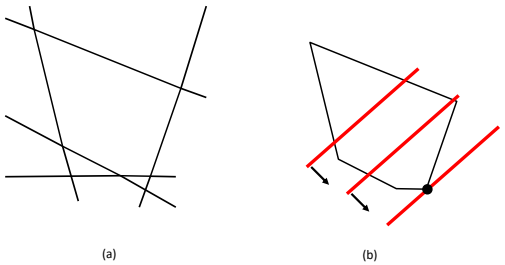


Figure 8.21: (a) A convex polytope in 2d defined by the intersection of linear constraints. (b) Depiction of the feasible set as well as the linear objective function. The red line is a level set of the objective, and the arrow indicates the direction in which it is improving. We see that the optimal solution lies at a vertex of the polytope.

- The penalty for the inequality constraint points in the right direction (this is called **dual feasibility**):

$$\boldsymbol{\mu} \geq \mathbf{0} \tag{8.105}$$

- The Lagrange multipliers pick up any slack in the inactive constraints, i.e., either $\mu_i = 0$ or $g_i(\boldsymbol{\theta}^*) = 0$, so

$$\boldsymbol{\mu} \odot \boldsymbol{g} = \mathbf{0} \tag{8.106}$$

  This is called **complementary slackness**.

To see why the last condition holds, consider (for simplicity) the case of a single inequality constraint, $g(\boldsymbol{\theta}) \leq 0$. Either it is **active**, meaning $g(\boldsymbol{\theta}) = 0$, or it is inactive, meaning $g(\boldsymbol{\theta}) < 0$. In the active case, the solution lies on the constraint boundary, and $g(\boldsymbol{\theta}) = 0$ becomes an equality constraint; then we have $\nabla \mathcal{L} = \mu \nabla g$ for some constant $\mu \neq 0$, because of Equation (8.89). In the inactive case, the solution is not on the constraint boundary; we still have $\nabla \mathcal{L} = \mu \nabla g$, but now $\mu = 0$.

These are called the **Karush-Kuhn-Tucker** (**KKT**) conditions. If $\mathcal{L}$ is a convex function, and the constraints define a convex set, the KKT conditions are sufficient for (global) optimality, as well as necessary.

8.5.3 Linear programming

Consider optimizing a linear function subject to linear constraints. When written in **standard form**, this can be represented as

$$\min_{\boldsymbol{\theta}} \boldsymbol{c}^\mathsf{T} \boldsymbol{\theta} \qquad \text{s.t.} \quad \mathbf{A}\boldsymbol{\theta} \leq \boldsymbol{b}, \ \boldsymbol{\theta} \geq 0 \tag{8.107}$$

The feasible set defines a convex **polytope**, which is a convex set defined as the intersection of half spaces. See Figure 8.21(a) for a 2d example. Figure 8.21(b) shows a linear cost function that

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

decreases as we move to the bottom right. We see that the lowest point that is in the feasible set is a vertex. In fact, it can be proved that the optimum point always occurs at a vertex of the polytope, assuming the solution is unique. If there are multiple solutions, the line will be parallel to a face. There may also be no optima inside the feasible set; in this case, the problem is said to be infeasible.

8.5.3.1 The simplex algorithm

It can be shown that the optima of an LP occur at vertices of the polytope defining the feasible set (see Figure 8.21(b) for an example). The **simplex algorithm** solves LPs by moving from vertex to vertex, each time seeking the edge which most improves the objective.

In the worst-case scenario, the simplex algorithm can take time exponential in D, although in practice it is usually very efficient. There are also various polynomial-time algorithms, such as the interior point method, although these are often slower in practice.

8.5.3.2 Applications

There are many applications of linear programming in science, engineering and business. It is also useful in some machine learning problems. For example, Section 11.6.1.1 shows how to use it to solve robust linear regression. It is also useful for state estimation in graphical models (see e.g., [SGJ11]).

8.5.4 Quadratic programming

Consider minimizing a quadratic objective subject to linear equality and inequality constraints. This kind of problem is known as a **quadratic program** or **QP**, and can be written as follows:

$$\min_{\boldsymbol{\theta}} \frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\mathbf{H}\boldsymbol{\theta} + \boldsymbol{c}^\mathsf{T}\boldsymbol{\theta} \quad \text{s.t.} \quad \mathbf{A}\boldsymbol{\theta} \leq \boldsymbol{b},\ \mathbf{C}\boldsymbol{\theta} = \boldsymbol{d} \tag{8.108}$$

If $\mathbf{H}$ is positive semidefinite, then this is a convex optimization problem.

8.5.4.1 Example: 2d quadratic objective with linear inequality constraints

As a concrete example, suppose we want to minimize

$$\mathcal{L}(\boldsymbol{\theta}) = (\theta_1 - \frac{3}{2})^2 + (\theta_2 - \frac{1}{8})^2 = \frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\mathbf{H}\boldsymbol{\theta} + \boldsymbol{c}^\mathsf{T}\boldsymbol{\theta} + \text{const} \tag{8.109}$$

where $\mathbf{H} = 2\mathbf{I}$ and $\boldsymbol{c} = -(3, 1/4)$, subject to

$$|\theta_1| + |\theta_2| \leq 1 \tag{8.110}$$

See Figure 8.20(b) for an illustration.

We can rewrite the constraints as

$$\theta_1 + \theta_2 \leq 1,\ \ \theta_1 - \theta_2 \leq 1,\ \ -\theta_1 + \theta_2 \leq 1,\ \ -\theta_1 - \theta_2 \leq 1 \tag{8.111}$$

which we can write more compactly as

$$\mathbf{A}\boldsymbol{\theta} \leq \boldsymbol{b} \tag{8.112}$$

where $\boldsymbol{b} = (1, 1, 1, 1)^\mathsf{T} = \mathbf{1}_4$ and

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \\ -1 & 1 \\ -1 & -1 \end{pmatrix} \tag{8.113}$$

This is now in the standard QP form.

From the geometry of the problem, shown in Figure 8.20(b), we see that the constraints corresponding to the two left faces of the diamond) are inactive (since we are trying to get as close to the center of the circle as possible, which is outside of, and to the right of, the constrained feasible region). Denoting $g_i(\boldsymbol{\theta})$ as the inequality constraint corresponding to row i of $\mathbf{A}$, this means $g_3(\boldsymbol{\theta}^*) > 0$ and $g_4(\boldsymbol{\theta}^*) > 0$, and hence, by complementarity, $\mu_3^* = \mu_4^* = 0$. We can therefore remove these inactive constraints.

From the KKT conditions we know that

$$\mathbf{H}\boldsymbol{\theta} + \boldsymbol{c} + \mathbf{A}^\mathsf{T}\boldsymbol{\mu} = \mathbf{0} \tag{8.114}$$

Using these for the actively constrained subproblem, we get

$$\begin{pmatrix} 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & -1 \\ 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \end{pmatrix} \begin{pmatrix} \theta_1 \\ \theta_2 \\ \mu_1 \\ \mu_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 1/4 \\ 1 \\ 1 \end{pmatrix} \tag{8.115}$$

Hence the solution is

$$\boldsymbol{\theta}_* = (1, 0)^\mathsf{T}, \boldsymbol{\mu}_* = (0.625, 0.375, 0, 0)^\mathsf{T} \tag{8.116}$$

Notice that the optimal value of $\boldsymbol{\theta}$ occurs at one of the vertices of the ℓ_1 "ball" (the diamond shape).

8.5.4.2 Applications

There are several applications of quadratic programming in ML. For example, in Section 11.4, we discuss the lasso method for sparse linear regression, which amounts to optimizing $\mathcal{L}(\boldsymbol{w}) = ||\mathbf{X}\boldsymbol{w} - \boldsymbol{y}||_2^2 + \lambda||\boldsymbol{w}||_1$, which can be reformulated into a QP. And in Section 17.3, we show how to use QP for SVMs (support vector machines).

8.5.5 Mixed integer linear programming *

Integer linear programming or **ILP** corresponds to minimizing a linear objective, subject to linear constraints, where the optimization variables are discrete integers instead of reals. In standard form, the problem is as follows:

$$\min_{\boldsymbol{\theta}} \boldsymbol{c}^\mathsf{T}\boldsymbol{\theta} \qquad \text{s.t.} \quad \mathbf{A}\boldsymbol{\theta} \leq \boldsymbol{b},\ \boldsymbol{\theta} \geq 0, \boldsymbol{\theta} \in \mathbb{Z}^D \tag{8.117}$$

where $\mathbb{Z}$ is the set of integers. If some of the optimization variables are real-valued, it is called a **mixed ILP**, often called a **MIP** for short. (If all of the variables are real-valued, it becomes a standard LP.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

MIPs have a large number of applications, such as in vehicle routing, scheduling and packing. They are also useful for some ML applications, such as formally verifying the behavior of certain kinds of deep neural networks [And+18], and proving robustness properties of DNNs to adversarial (worst-case) perturbations [TXT19].

8.6 Proximal gradient method *

We are often interested in optimizing an objective of the form

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_s(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) \tag{8.118}$$

where $\mathcal{L}_s$ is differentiable (smooth), and $\mathcal{L}_r$ is convex but not necessarily differentiable (i.e., it may be non-smooth or "rough"). For example, $\mathcal{L}_s$ might be the negative log likelihood (NLL), and $\mathcal{L}_r$ might be an indicator function that is infinite if a constraint is violated (see Section 8.6.1), or $\mathcal{L}_r$ might be the ℓ_1 norm of some parameters (see Section 8.6.2), or $\mathcal{L}_r$ might measure how far the parameters are from a set of allowed quantized values (see Section 8.6.3).

One way to tackle such problems is to use the **proximal gradient method** (see e.g., [PB+14; PSW15]). Roughly speaking, this takes a step of size η in the direction of the gradient, and then projects the resulting parameter update into a space that respects $\mathcal{L}_r$. More precisely, the update is as follows

$$\boldsymbol{\theta}_{t+1} = \text{prox}_{\eta_t \mathcal{L}_r}(\boldsymbol{\theta}_t - \eta_t \nabla \mathcal{L}_s(\boldsymbol{\theta}_t)) \tag{8.119}$$

where $\text{prox}_{\eta \mathcal{L}_r}(\boldsymbol{\theta})$ is the **proximal operator** of $\mathcal{L}_r$ (scaled by η) evaluated at $\boldsymbol{\theta}$:

$$\text{prox}_{\eta \mathcal{L}_r}(\boldsymbol{\theta}) \triangleq \underset{\boldsymbol{z}}{\text{argmin}} \left(\mathcal{L}_r(\boldsymbol{z}) + \frac{1}{2\eta} ||\boldsymbol{z} - \boldsymbol{\theta}||_2^2 \right) \tag{8.120}$$

(The factor of $\frac{1}{2}$ is an arbitrary convention.) We can rewrite the proximal operator as solving a constrained optimization problem, as follows:

$$\text{prox}_{\eta \mathcal{L}_r}(\boldsymbol{\theta}) = \underset{\boldsymbol{z}}{\text{argmin}}\, \mathcal{L}_r(\boldsymbol{z}) \;\; \text{s.t.} \;\; ||\boldsymbol{z} - \boldsymbol{\theta}||_2 \leq \rho \tag{8.121}$$

where the bound ρ depends on the scaling factor η. Thus we see that the proximal projection minimizes the function while staying close to (i.e., proximal to) the current iterate. We give some examples below.

8.6.1 Projected gradient descent

Suppose we want to solve the problem

$$\underset{\boldsymbol{\theta}}{\text{argmin}}\, \mathcal{L}_s(\boldsymbol{\theta}) \;\; \text{s.t.} \;\; \boldsymbol{\theta} \in \mathcal{C} \tag{8.122}$$

where $\mathcal{C}$ is a convex set. For example, we may have the **box constraints** $\mathcal{C} = \{\boldsymbol{\theta} : \boldsymbol{l} \leq \boldsymbol{\theta} \leq \boldsymbol{u}\}$, where we specify lower and upper bounds on each element. These bounds can be infinite for certain

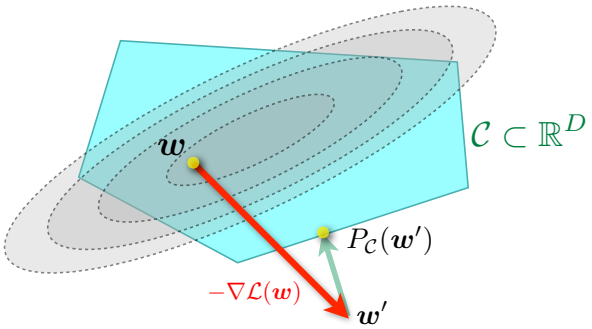


Figure 8.22: Illustration of projected gradient descent. $\boldsymbol{w}$ is the current parameter estimate, $\boldsymbol{w}'$ is the update after a gradient step, and $P_{\mathcal{C}}(\boldsymbol{w}')$ projects this onto the constraint set $\mathcal{C}$. From https://bit.ly/3eJ3BhZ Used with kind permission of Martin Jaggi.

elements if we don't want to constrain values along that dimension. For example, if we just want to ensure the parameters are non-negative, we set $l_d = 0$ and $u_d = \infty$ for each dimension d.

We can convert the constrained optimization problem into an unconstrained one by adding a **penalty term** to the original objective:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_s(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) \tag{8.123}$$

where $\mathcal{L}_r(\boldsymbol{\theta})$ is the indicator function for the convex set $\mathcal{C}$, i.e.,

$$\mathcal{L}_r(\boldsymbol{\theta}) = I_{\mathcal{C}}(\boldsymbol{\theta}) = \begin{cases} 0 & \text{if } \boldsymbol{\theta} \in \mathcal{C} \\ \infty & \text{if } \boldsymbol{\theta} \notin \mathcal{C} \end{cases} \tag{8.124}$$

We can use proximal gradient descent to solve Equation (8.123). The proximal operator for the indicator function is equivalent to projection onto the set $\mathcal{C}$:

$$\text{proj}_{\mathcal{C}}(\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}' \in \mathcal{C}}{\text{argmin}} \, ||\boldsymbol{\theta}' - \boldsymbol{\theta}||_2 \tag{8.125}$$

This method is known as **projected gradient descent**. See Figure 8.22 for an illustration.

For example, consider the box constraints $\mathcal{C} = \{\boldsymbol{\theta} : \boldsymbol{l} \leq \boldsymbol{\theta} \leq \boldsymbol{u}\}$. The projection operator in this case can be computed elementwise by simply thresholding at the boundaries:

$$\text{proj}_{\mathcal{C}}(\boldsymbol{\theta})_d = \begin{cases} l_d & \text{if } \theta_d \leq l_d \\ \theta_d & \text{if } l_d \leq \theta_d \leq u_d \\ u_d & \text{if } \theta_d \geq u_d \end{cases} \tag{8.126}$$

For example, if we want to ensure all elements are non-negative, we can use

$$\text{proj}_{\mathcal{C}}(\boldsymbol{\theta}) = \boldsymbol{\theta}_+ = [\max(\theta_1, 0), \ldots, \max(\theta_D, 0)] \tag{8.127}$$

See Section 11.4.9.2 for an application of this method to sparse linear regression.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.6.2 Proximal operator for ℓ_1-norm regularizer

Consider a linear predictor of the form $f(\boldsymbol{x};\boldsymbol{\theta}) = \sum_{d=1}^{D} \theta_d x_d$. If we have $\theta_d = 0$ for any dimension d, we ignore the corresponding feature x_d. This is a form of **feature selection**, which can be useful both as a way to reduce overfitting as well as way to improve model interpretability. We can encourage weights to be zero (and not just small) by penalizing the ℓ_1 norm,

$$||\boldsymbol{\theta}||_1 = \sum_{d=1}^{D} |\theta_d| \tag{8.128}$$

This is called a **sparsity inducing regularizer**.

To see why this induces sparsity, consider two possible parameter vectors, one which is sparse, $\boldsymbol{\theta} = (1, 0)$, and one which is non-sparse, $\boldsymbol{\theta}' = (1/\sqrt{2}, 1/\sqrt{2})$. Both have the same ℓ_2 norm

$$||(1,0)||_2^2 = ||(1/\sqrt{2}, 1/\sqrt{2})||_2^2 = 1 \tag{8.129}$$

Hence ℓ_2 regularization (Section 4.5.3) will not favor the sparse solution over the dense solution. However, when using ℓ_1 regularization, the sparse solution is cheaper, since

$$||(1,0)||_1 = 1 < ||(1/\sqrt{2}, 1/\sqrt{2})||_1 = \sqrt{2} \tag{8.130}$$

See Section 11.4 for more details on sparse regression.

If we combine this regularizer with our smooth loss, we get

$$\mathcal{L}(\boldsymbol{\theta}) = \text{NLL}(\boldsymbol{\theta}) + \lambda ||\boldsymbol{\theta}||_1 \tag{8.131}$$

We can optimize this objective using proximal gradient descent. The key question is how to compute the prox operator for the function $f(\boldsymbol{\theta}) = ||\boldsymbol{\theta}||_1$. Since this function decomposes over dimensions d, the proximal projection can be computed componentwise. From Equation (8.120), with $\eta = 1$, we have

$$\text{prox}_{\lambda f}(\theta) = \underset{z}{\text{argmin}}\, |z| + \frac{1}{2\lambda}(z-\theta)^2 = \underset{z}{\text{argmin}}\, \lambda|z| + \frac{1}{2}(z-\theta)^2 \tag{8.132}$$

In Section 11.4.3, we show that the solution to this is given by

$$\text{prox}_{\lambda f}(\theta) = \begin{cases} \theta - \lambda & \text{if } \theta \geq \lambda \\ 0 & \text{if } |\theta| \leq \lambda \\ \theta + \lambda & \text{if } \theta \leq -\lambda \end{cases} \tag{8.133}$$

This is known as the **soft thresholding operator**, since values less than λ in absolute value are set to 0 (thresholded), but in a continuous way. Note that soft thresholding can be written more compactly as

$$\text{SoftThreshold}(\theta, \lambda) = \text{sign}(\theta)\,(|\theta| - \lambda)_+ \tag{8.134}$$

where $\theta_+ = \max(\theta, 0)$ is the positive part of θ. In the vector case, we perform this elementwise:

$$\text{SoftThreshold}(\boldsymbol{\theta}, \lambda) = \text{sign}(\boldsymbol{\theta}) \odot (|\boldsymbol{\theta}| - \lambda)_+ \tag{8.135}$$

See Section 11.4.9.3 for an application of this method to sparse linear regression.

8.6.3 Proximal operator for quantization

In some applications (e.g., when training deep neural networks to run on memory-limited **edge devices**, such as mobile phones) we want to ensure that the parameters are **quantized**. For example, in the extreme case where each parameter can only be -1 or +1, the state space becomes $\mathcal{C} = \{-1, +1\}^D$.

Let us define a regularizer that measures distance to the nearest quantized version of the parameter vector:

$$\mathcal{L}_r(\boldsymbol{\theta}) = \inf_{\boldsymbol{\theta}_0 \in \mathcal{C}} ||\boldsymbol{\theta} - \boldsymbol{\theta}_0||_1 \tag{8.136}$$

(We could also use the ℓ_2 norm.) In the case of $\mathcal{C} = \{-1, +1\}^D$, this becomes

$$\mathcal{L}_r(\boldsymbol{\theta}) = \sum_{d=1}^{D} \inf_{[\theta_0]_d \in \{\pm 1\}} |\theta_d - [\theta_0]_d| = \sum_{d=1}^{D} \min\{|\theta_d - 1|, |\theta_d + 1|\} = ||\boldsymbol{\theta} - \text{sign}(\boldsymbol{\theta})||_1 \tag{8.137}$$

Let us define the corresponding quantization operator to be

$$q(\boldsymbol{\theta}) = \text{proj}_{\mathcal{C}}(\boldsymbol{\theta}) = \text{argmin}\, \mathcal{L}_r(\boldsymbol{\theta}) = \text{sign}(\boldsymbol{\theta}) \tag{8.138}$$

The core difficulty with quantized learning is that quantization is not a differentiable operation. A popular solution to this is to use the **straight-through estimator**, which uses the approximation $\frac{\partial \mathcal{L}}{\partial q(\boldsymbol{\theta})} \approx \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$ (see e.g., [Yin+19]). The corresponding update can be done in two steps: first compute the gradient vector at the quantized version of the current parameters, and then update the unconstrained parameters using this approximate gradient:

$$\tilde{\boldsymbol{\theta}}_t = \text{proj}_{\mathcal{C}}(\boldsymbol{\theta}_t) = q(\boldsymbol{\theta}_t) \tag{8.139}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla \mathcal{L}_s(\tilde{\boldsymbol{\theta}}_t) \tag{8.140}$$

When applied to $\mathcal{C} = \{-1, +1\}^D$, this is known as the **binary connect** method [CBD15].

We can get better results using proximal gradient descent, in which we treat quantization as a regularizer, rather than a hard constraint; this is known as **ProxQuant** [BWL19]. The update becomes

$$\tilde{\boldsymbol{\theta}}_t = \text{prox}_{\lambda \mathcal{L}_r}\left(\boldsymbol{\theta}_t - \eta_t \nabla \mathcal{L}_s(\boldsymbol{\theta}_t)\right) \tag{8.141}$$

In the case that $\mathcal{C} = \{-1, +1\}^D$, one can show that the proximal operator is a generalization of the soft thresholding operator in Equation (8.135):

$$\text{prox}_{\lambda \mathcal{L}_r}(\boldsymbol{\theta}) = \text{SoftThreshold}(\boldsymbol{\theta}, \lambda, \text{sign}(\boldsymbol{\theta})) \tag{8.142}$$

$$= \text{sign}(\boldsymbol{\theta}) + \text{sign}(\boldsymbol{\theta} - \text{sign}(\boldsymbol{\theta})) \odot (|\boldsymbol{\theta} - \text{sign}(\boldsymbol{\theta})| - \lambda)_+ \tag{8.143}$$

This can be generalized to other forms of quantization; see [Yin+19] for details.

8.6.4 Incremental (online) proximal methods

Many ML problems have an objective function which is a sum of losses, one per example. Such problems can be solved incrementally; this is a special case of **online learning**. It is possible to extend proximal methods to this setting. For a probabilistic perspective on such methods (in terms of Kalman filtering), see [AEM18; Aky+19].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.7 Bound optimization *

In this section, we consider a class of algorithms known as **bound optimization** or **MM** algorithms. In the context of minimization, MM stands for **majorize-minimize**. In the context of maximization, MM stands for **minorize-maximize**. We will discuss a special case of MM, known as **expectation maximization** or **EM**, in Section 8.7.2.

8.7.1 The general algorithm

In this section, we give a brief outline of MM methods. (More details can be found in e.g., [HL04; Mai15; SBP17; Nad+19].) To be consistent with the literature, we assume our goal is to *maximize* some function $\ell(\boldsymbol{\theta})$, such as the log likelihood, wrt its parameters $\boldsymbol{\theta}$. The basic approach in MM algorithms is to construct a **surrogate function** $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$ which is a tight lowerbound to $\ell(\boldsymbol{\theta})$ such that $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \leq \ell(\boldsymbol{\theta})$ and $Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \ell(\boldsymbol{\theta}^t)$. If these conditions are met, we say that Q minorizes ℓ. We then perform the following update at each step:

$$\boldsymbol{\theta}^{t+1} = \operatorname*{argmax}_{\boldsymbol{\theta}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \tag{8.144}$$

This guarantees us monotonic increases in the original objective:

$$\ell(\boldsymbol{\theta}^{t+1}) \geq Q(\boldsymbol{\theta}^{t+1}, \boldsymbol{\theta}^t) \geq Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \ell(\boldsymbol{\theta}^t) \tag{8.145}$$

where the first inequality follows since $Q(\boldsymbol{\theta}^{t+1}, \boldsymbol{\theta}')$ is a lower bound on $\ell(\boldsymbol{\theta}^{t+1})$ for any $\boldsymbol{\theta}'$; the second inequality follows from Equation (8.144); and the final equality follows the tightness property. As a consequence of this result, if you do not observe monotonic increase of the objective, you must have an error in your math and/or code. This is a surprisingly powerful debugging tool.

This process is sketched in Figure 8.23. The dashed red curve is the original function (e.g., the log-likelihood of the observed data). The solid blue curve is the lower bound, evaluated at $\boldsymbol{\theta}^t$; this touches the objective function at $\boldsymbol{\theta}^t$. We then set $\boldsymbol{\theta}^{t+1}$ to the maximum of the lower bound (blue curve), and fit a new bound at that point (dotted green curve). The maximum of this new bound becomes $\boldsymbol{\theta}^{t+2}$, etc.

If Q is a quadratic lower bound, the overall method is similar to Newton's method, which repeatedly fits and then optimizes a quadratic approximation, as shown in Figure 8.14(a). The difference is that optimizing Q is guaranteed to lead to an improvement in the objective, even if it is not convex, whereas Newton's method may overshoot or lead to a decrease in the objective, as shown in Figure 8.24, since it is a quadratic approximation and not a bound.

8.7.2 The EM algorithm

In this section, we discuss the **expectation maximization** (**EM**) algorithm [DLR77; MK97], which is a bound optimization algorithm designed to compute the MLE or MAP parameter estimate for probability models that have **missing data** and/or **hidden variables**. We let $\boldsymbol{y}_n$ be the visible data for example n, and $\boldsymbol{z}_n$ be the hidden data.

The basic idea behind EM is to alternate between estimating the hidden variables (or missing values) during the **E step** (expectation step), and then using the fully observed data to compute the MLE during the **M step** (maximization step). Of course, we need to iterate this process, since the expected values depend on the parameters, but the parameters depend on the expected values.

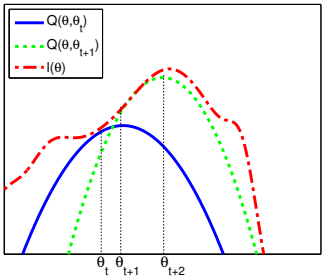
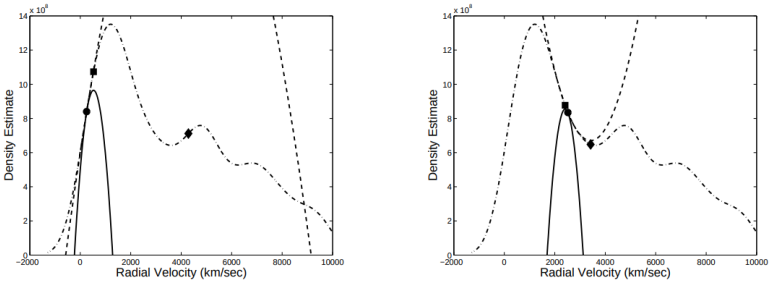


Figure 8.23: Illustration of a bound optimization algorithm. Adapted from Figure 9.14 of [Bis06]. Generated by emLogLikelihoodMax.ipynb.



(a) Overshooting. (b) Seeking the wrong root.

Figure 8.24: The quadratic lower bound of an MM algorithm (solid) and the quadratic approximation of Newton's method (dashed) superimposed on an empirical density esitmate (dotted). The starting point of both algorithms is the circle. The square denotes the outcome of one MM update. The diamond denotes the outcome of one Newton update. (a) Newton's method overshoots the global maximum. (b) Newton's method results in a reduction of the objective. From Figure 4 of [FT05]. Used with kind permission of Carlo Tomasi.

In Section 8.7.2.1, we show that EM is an MM algorithm, which implies that this iterative procedure will converge to a local maximum of the log likelihood. The speed of convergence depends on the amount of missing data, which affects the tightness of the bound [XJ96; MD97; SRG03; KKS20].

8.7.2.1 Lower bound

The goal of EM is to maximize the log likelihood of the observed data:

$$\ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log p(\boldsymbol{y}_n|\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{\boldsymbol{z}_n} p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta}) \right] \tag{8.146}$$

where $\boldsymbol{y}_n$ are the visible variables and $\boldsymbol{z}_n$ are the hidden variables. Unfortunately this is hard to optimize, since the log cannot be pushed inside the sum.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

EM gets around this problem as follows. First, consider a set of arbitrary distributions $q_n(\boldsymbol{z}_n)$ over each hidden variable $\boldsymbol{z}_n$. The observed data log likelihood can be written as follows:

$$\ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{\boldsymbol{z}_n} q_n(\boldsymbol{z}_n) \frac{p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta})}{q_n(\boldsymbol{z}_n)} \right] \tag{8.147}$$

Using Jensen's inequality (Equation (6.34)), we can push the log (which is a concave function) inside the expectation to get the following lower bound on the log likelihood:

$$\ell(\boldsymbol{\theta}) \geq \sum_n \sum_{\boldsymbol{z}_n} q_n(\boldsymbol{z}_n) \log \frac{p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta})}{q_n(\boldsymbol{z}_n)} \tag{8.148}$$

$$= \sum_n \underbrace{\mathbb{E}_{q_n}\left[\log p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta})\right] + \mathbb{H}(q_n)}_{Ł(\boldsymbol{\theta}, q_n)} \tag{8.149}$$

$$= \sum_n Ł(\boldsymbol{\theta}, q_n) \triangleq Ł(\boldsymbol{\theta}, \{q_n\}) = Ł(\boldsymbol{\theta}, q_{1:N}) \tag{8.150}$$

where $\mathbb{H}(q)$ is the entropy of probability distribution q, and $Ł(\boldsymbol{\theta}, \{q_n\})$ is called the **evidence lower bound** or **ELBO**, since it is a lower bound on the log marginal likelihood, $\log p(\boldsymbol{y}_{1:N}|\boldsymbol{\theta})$, also called the evidence. Optimizing this bound is the basis of variational inference, which we discuss in Section 4.6.8.3.

8.7.2.2 E step

We see that the lower bound is a sum of N terms, each of which has the following form:

$$Ł(\boldsymbol{\theta}, q_n) = \sum_{\boldsymbol{z}_n} q_n(\boldsymbol{z}_n) \log \frac{p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta})}{q_n(\boldsymbol{z}_n)} \tag{8.151}$$

$$= \sum_{\boldsymbol{z}_n} q_n(\boldsymbol{z}_n) \log \frac{p(\boldsymbol{z}_n|\boldsymbol{y}_n, \boldsymbol{\theta}) p(\boldsymbol{y}_n|\boldsymbol{\theta})}{q_n(\boldsymbol{z}_n)} \tag{8.152}$$

$$= \sum_{\boldsymbol{z}_n} q_n(\boldsymbol{z}_n) \log \frac{p(\boldsymbol{z}_n|\boldsymbol{y}_n, \boldsymbol{\theta})}{q_n(\boldsymbol{z}_n)} + \sum_{\boldsymbol{z}_n} q_n(\boldsymbol{z}_n) \log p(\boldsymbol{y}_n|\boldsymbol{\theta}) \tag{8.153}$$

$$= -D_{\mathbb{KL}}\left(q_n(\boldsymbol{z}_n) \parallel p(\boldsymbol{z}_n|\boldsymbol{y}_n, \boldsymbol{\theta})\right) + \log p(\boldsymbol{y}_n|\boldsymbol{\theta}) \tag{8.154}$$

where $D_{\mathbb{KL}}\left(q \parallel p\right) \triangleq \sum_z q(z) \log \frac{q(z)}{p(z)}$ is the Kullback-Leibler divergence (or KL divergence for short) between probability distributions q and p. We discuss this in more detail in Section 6.2, but the key property we need here is that $D_{\mathbb{KL}}\left(q \parallel p\right) \geq 0$ and $D_{\mathbb{KL}}\left(q \parallel p\right) = 0$ iff $q = p$. Hence we can maximize the lower bound $Ł(\boldsymbol{\theta}, \{q_n\})$ wrt $\{q_n\}$ by setting each one to $q_n^* = p(\boldsymbol{z}_n|\boldsymbol{y}_n, \boldsymbol{\theta})$. This is called the **E step**. This ensures the ELBO is a tight lower bound:

$$Ł(\boldsymbol{\theta}, \{q_n^*\}) = \sum_n \log p(\boldsymbol{y}_n|\boldsymbol{\theta}) = \ell(\boldsymbol{\theta}) \tag{8.155}$$

To see how this connects to bound optimization, let us define

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) = Ł(\boldsymbol{\theta}, \{p(\boldsymbol{z}_n|\boldsymbol{y}_n; \boldsymbol{\theta}^t)\}) \tag{8.156}$$

Then we have $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t) \leq \ell(\boldsymbol{\theta})$ and $Q(\boldsymbol{\theta}^t, \boldsymbol{\theta}^t) = \ell(\boldsymbol{\theta}^t)$, as required.

However, if we cannot compute the posteriors $p(\boldsymbol{z}_n|\boldsymbol{y}_n; \boldsymbol{\theta}^t)$ exactly, we can still use an approximate distribution $q(\boldsymbol{z}_n|\boldsymbol{y}_n; \boldsymbol{\theta}^t)$; this will yield a non-tight lower-bound on the log-likelihood. This generalized version of EM is known as **variational EM** [NH98]. See the sequel to this book, [Mur23], for details.

8.7.2.3 M step

In the M step, we need to maximize $Ł(\boldsymbol{\theta}, \{q_n^t\})$ wrt $\boldsymbol{\theta}$, where the q_n^t are the distributions computed in the E step at iteration t. Since the entropy terms $\mathbb{H}(q_n)$ are constant wrt $\boldsymbol{\theta}$, so we can drop them in the M step. We are left with

$$\ell^t(\boldsymbol{\theta}) = \sum_n \mathbb{E}_{q_n^t(\boldsymbol{z}_n)} \left[\log p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta})\right] \tag{8.157}$$

This is called the **expected complete data log likelihood**. If the joint probability is in the exponential family (Section 3.4), we can rewrite this as

$$\ell^t(\boldsymbol{\theta}) = \sum_n \mathbb{E}\left[\mathcal{T}(\boldsymbol{y}_n, \boldsymbol{z}_n)^\mathsf{T}\boldsymbol{\theta} - A(\boldsymbol{\theta})\right] = \sum_n (\mathbb{E}\left[\mathcal{T}(\boldsymbol{y}_n, \boldsymbol{z}_n)\right]^\mathsf{T}\boldsymbol{\theta} - A(\boldsymbol{\theta})) \tag{8.158}$$

where $\mathbb{E}\left[\mathcal{T}(\boldsymbol{y}_n, \boldsymbol{z}_n)\right]$ are called the **expected sufficient statistics**.

In the M step, we maximize the expected complete data log likelihood to get

$$\boldsymbol{\theta}^{t+1} = \arg\max_{\boldsymbol{\theta}} \sum_n \mathbb{E}_{q_n^t} \left[\log p(\boldsymbol{y}_n, \boldsymbol{z}_n|\boldsymbol{\theta})\right] \tag{8.159}$$

In the case of the exponential family, the maximization can be solved in closed-form by matching the moments of the expected sufficient statistics.

We see from the above that the E step does not in fact need to return the full set of posterior distributions $\{q(\boldsymbol{z}_n)\}$, but can instead just return the sum of the expected sufficient statistics, $\sum_n \mathbb{E}_{q(\boldsymbol{z}_n)}\left[\mathcal{T}(\boldsymbol{y}_n, \boldsymbol{z}_n)\right]$. This will become clearer in the examples below.

8.7.3 Example: EM for a GMM

In this section, we show how to use the EM algorithm to compute MLE and MAP estimates of the parameters for a Gaussian mixture model (GMM).

8.7.3.1 E step

The E step simply computes the **responsibility** of cluster k for generating data point n, as estimated using the current parameter estimates $\boldsymbol{\theta}^{(t)}$:

$$r_{nk}^{(t)} = p^*(z_n = k|\boldsymbol{y}_n, \boldsymbol{\theta}^{(t)}) = \frac{\pi_k^{(t)} p(\boldsymbol{y}_n|\boldsymbol{\theta}_k^{(t)})}{\sum_{k'} \pi_{k'}^{(t)} p(\boldsymbol{y}_n|\boldsymbol{\theta}_{k'}^{(t)})} \tag{8.160}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

8.7.3.2 M step

The M step maximizes the expected complete data log likelihood, given by

$$\ell^t(\boldsymbol{\theta}) = \mathbb{E}\left[\sum_n \log p(z_n|\boldsymbol{\pi}) + \sum_n \log p(\boldsymbol{y}_n|z_n,\boldsymbol{\theta})\right] \tag{8.161}$$

$$= \mathbb{E}\left[\sum_n \log\left(\prod_k \pi_k^{z_{nk}}\right) + \sum_n \log\left(\prod_k \mathcal{N}(\boldsymbol{y}_n|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)^{z_{nk}}\right)\right] \tag{8.162}$$

$$= \sum_n \sum_k \mathbb{E}\left[z_{nk}\right] \log \pi_k + \sum_n \sum_k \mathbb{E}\left[z_{nk}\right] \log \mathcal{N}(\boldsymbol{y}_n|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k) \tag{8.163}$$

$$= \sum_n \sum_k r_{nk}^{(t)} \log(\pi_k) - \frac{1}{2}\sum_n \sum_k r_{nk}^{(t)}\left[\log|\boldsymbol{\Sigma}_k| + (\boldsymbol{y}_n - \boldsymbol{\mu}_k)^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}(\boldsymbol{y}_n - \boldsymbol{\mu}_k)\right] + \text{const} \tag{8.164}$$

where $z_{nk} = \mathbb{I}(z_n = k)$ is a one-hot encoding of the categorical value z_n. This objective is just a weighted version of the standard problem of computing the MLEs of an MVN (see Section 4.2.6). One can show that the new parameter estimates are given by

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n r_{nk}^{(t)} \boldsymbol{y}_n}{r_k^{(t)}} \tag{8.165}$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_n r_{nk}^{(t)} (\boldsymbol{y}_n - \boldsymbol{\mu}_k^{(t+1)})(\boldsymbol{y}_n - \boldsymbol{\mu}_k^{(t+1)})^\mathsf{T}}{r_k^{(t)}}$$

$$= \frac{\sum_n r_{nk}^{(t)} \boldsymbol{y}_n \boldsymbol{y}_n^\mathsf{T}}{r_k^{(t)}} - \boldsymbol{\mu}_k^{(t+1)}(\boldsymbol{\mu}_k^{(t+1)})^\mathsf{T} \tag{8.166}$$

where $r_k^{(t)} \triangleq \sum_n r_{nk}^{(t)}$ is the weighted number of points assigned to cluster k. The mean of cluster k is just the weighted average of all points assigned to cluster k, and the covariance is proportional to the weighted empirical scatter matrix.

The M step for the mixture weights is simply a weighted form of the usual MLE:

$$\pi_k^{(t+1)} = \frac{1}{N}\sum_n r_{nk}^{(t)} = \frac{r_k^{(t)}}{N} \tag{8.167}$$

8.7.3.3 Example

An example of the algorithm in action is shown in Figure 8.25 where we fit some 2d data with a 2 component GMM. The data set, from [Bis06], is derived from measurements of the Old Faithful geyser in Yellowstone National Park. In particular, we plot the time to next eruption in minutes versus the duration of the eruption in minutes. The data was standardized, by removing the mean and dividing by the standard deviation, before processing; this often helps convergence. We start with $\boldsymbol{\mu}_1 = (-1, 1)$, $\boldsymbol{\Sigma}_1 = \mathbf{I}$, $\boldsymbol{\mu}_2 = (1, -1)$, $\boldsymbol{\Sigma}_2 = \mathbf{I}$. We then show the cluster assignments, and corresponding mixture components, at various iterations.

For more details on applying GMMs for clustering, see Section 21.4.1.

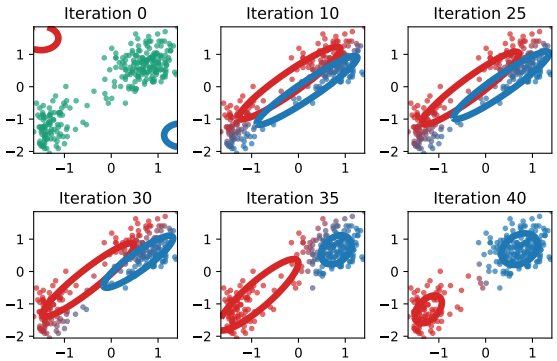


Figure 8.25: Illustration of the EM for a GMM applied to the Old Faithful data. The degree of redness indicates the degree to which the point belongs to the red cluster, and similarly for blue; thus purple points have a roughly 50/50 split in their responsibilities to the two clusters. Adapted from [Bis06] Figure 9.8. Generated by mix_gauss_demo_faithful.ipynb.

8.7.3.4 MAP estimation

Computing the MLE of a GMM often suffers from numerical problems and overfitting. To see why, suppose for simplicity that $\boldsymbol{\Sigma}_k = \sigma_k^2 \mathbf{I}$ for all k. It is possible to get an infinite likelihood by assigning one of the centers, say $\boldsymbol{\mu}_k$, to a single data point, say $\boldsymbol{y}_n$, since then the likelihood of that data point is given by

$$\mathcal{N}(\boldsymbol{y}_n | \boldsymbol{\mu}_k = \boldsymbol{y}_n, \sigma_k^2 \mathbf{I}) = \frac{1}{\sqrt{2\pi\sigma_k^2}} e^0 \tag{8.168}$$

Hence we can drive this term to infinity by letting $\sigma_k \to 0$, as shown in Figure 8.26(a). We call this the "collapsing variance problem".

An easy solution to this is to perform MAP estimation. Fortunately, we can still use EM to find this MAP estimate. Our goal is now to maximize the expected complete data log-likelihood plus the log prior:

$$\ell^t(\boldsymbol{\theta}) = \left[\sum_n \sum_k r_{nk}^{(t)} \log \pi_{nk} + \sum_n \sum_k r_{nk}^{(t)} \log p(\boldsymbol{y}_n | \boldsymbol{\theta}_k) \right] + \log p(\boldsymbol{\pi}) + \sum_k \log p(\boldsymbol{\theta}_k) \tag{8.169}$$

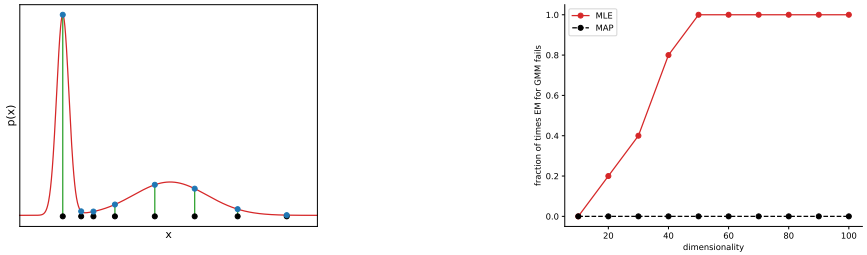
Note that the E step remains unchanged, but the M step needs to be modified, as we now explain.

For the prior on the mixture weights, it is natural to use a Dirichlet prior (Section 4.6.3.2), $\boldsymbol{\pi} \sim \text{Dir}(\boldsymbol{\alpha})$, since this is conjugate to the categorical distribution. The MAP estimate is given by

$$\tilde{\pi}_k^{(t+1)} = \frac{r_k^{(t)} + \alpha_k - 1}{N + \sum_k \alpha_k - K} \tag{8.170}$$

If we use a uniform prior, $\alpha_k = 1$, this reduces to the MLE.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 8.26: (a) Illustration of how singularities can arise in the likelihood function of GMMs. Here $K = 2$, but the first mixture component is a narrow spike (with $\sigma_1 \approx 0$) centered on a single data point x_1. Adapted from Figure 9.7 of [Bis06]. Generated by mix_gauss_singularity.ipynb. (b) Illustration of the benefit of MAP estimation vs ML estimation when fitting a Gaussian mixture model. We plot the fraction of times (out of 5 random trials) each method encounters numerical problems vs the dimensionality of the problem, for $N = 100$ samples. Solid red (upper curve): MLE. Dotted black (lower curve): MAP. Generated by mix_gauss_mle_vs_map.ipynb.

For the prior on the mixture components, let us consider a conjugate prior of the form

$$p(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \text{NIW}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k | \breve{\boldsymbol{m}}, \breve{\kappa}, \breve{\nu}, \breve{\mathbf{S}}) \tag{8.171}$$

This is called the **Normal-Inverse-Wishart distribution** (see the sequel to this book, [Mur23], for details.) Suppose we set the hyper-parameters for $\boldsymbol{\mu}$ to be $\breve{\kappa} = 0$, so that the $\boldsymbol{\mu}_k$ are unregularized; thus the prior will only influence our estimate of $\boldsymbol{\Sigma}_k$. In this case, the MAP estimates are given by

$$\tilde{\boldsymbol{\mu}}_k^{(t+1)} = \hat{\boldsymbol{\mu}}_k^{(t+1)} \tag{8.172}$$

$$\tilde{\boldsymbol{\Sigma}}_k^{(t+1)} = \frac{\breve{\mathbf{S}} + \hat{\boldsymbol{\Sigma}}_k^{(t+1)}}{\breve{\nu} + r_k^{(t)} + D + 2} \tag{8.173}$$

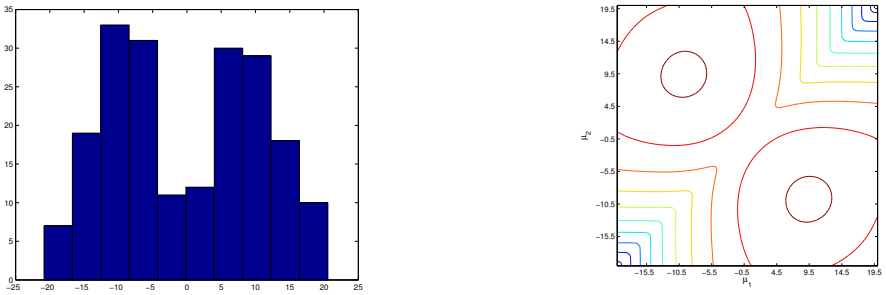
where $\hat{\boldsymbol{\mu}}_k$ is the MLE for $\boldsymbol{\mu}_k$ from Equation (8.165), and $\hat{\boldsymbol{\Sigma}}_k$ is the MLE for $\boldsymbol{\Sigma}_k$ from Equation (8.166).

Now we discuss how to set the prior covariance, $\breve{\mathbf{S}}$. One possibility (suggested in [FR07, p163]) is to use

$$\breve{\mathbf{S}} = \frac{1}{K^{2/D}} \text{diag}(s_1^2, \ldots, s_D^2) \tag{8.174}$$

where $s_d^2 = (1/N) \sum_{n=1}^N (x_{nd} - \overline{x}_d)^2$ is the pooled variance for dimension d. The parameter $\breve{\nu}$ controls how strongly we believe this prior. The weakest prior we can use, while still being proper, is to set $\breve{\nu} = D + 2$, so this is a common choice.

We now illustrate the benefits of using MAP estimation instead of ML estimation in the context of GMMs. We apply EM to some synthetic data with $N = 100$ samples in D dimensions, using either ML or MAP estimation. We count the trial as a "failure" if there are numerical issues involving singular matrices. For each dimensionality, we conduct 5 random trials. The results are illustrated in Figure 8.26(b). We see that as soon as D becomes even moderately large, ML estimation crashes and burns, whereas MAP with an appropriate prior estimation rarely encounters numerical problems.



(a) (b)

Figure 8.27: Left: $N = 200$ data points sampled from a mixture of 2 Gaussians in 1d, with $\pi_k = 0.5$, $\sigma_k = 5$, $\mu_1 = -10$ and $\mu_2 = 10$. Right: Likelihood surface $p(\mathcal{D}|\mu_1, \mu_2)$, with all other parameters set to their true values. We see the two symmetric modes, reflecting the unidentifiability of the parameters. Generated by gmm_lik_surface_plot.ipynb.

8.7.3.5 Nonconvexity of the NLL

The likelihood for a mixture model is given by

$$\ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{z_n=1}^{K} p(\boldsymbol{y}_n, z_n|\boldsymbol{\theta}) \right] \tag{8.175}$$

In general, this will have multiple modes, and hence there will not be a unique global optimum.

Figure 8.27 illustrates this for a mixture of 2 Gaussians in 1d. We see that there are two equally good global optima, corresponding to two different labelings of the clusters, one in which the left peak corresponds to $z = 1$, and one in which the left peak corresponds to $z = 2$. This is called the **label switching problem**; see Section 21.4.1.2 for more details.

The question of how many modes there are in the likelihood function is hard to answer. There are $K!$ possible labelings, but some of the peaks might get merged, depending on how far apart the μ_k are. Nevertheless, there can be an exponential number of modes. Consequently, finding any global optimum is NP-hard [Alo+09; Dri+04]. We will therefore have to be satisfied with finding a local optimum. To find a good local optimum, we can use Kmeans++ (Section 21.3.4) to initialize EM.

8.8 Blackbox and derivative free optimization

In some optimization problems, the objective function is a **blackbox**, meaning that its functional form is unknown. This means we cannot use gradient-based methods to optimize it. Instead, solving such problems require **blackbox optimization** (**BBO**) methods, also called **derivative free optimization** (**DFO**).

In ML, this kind of problem often arises when performing model selection. For example, suppose we have some hyper-parameters, $\boldsymbol{\lambda} \in \boldsymbol{\Lambda}$, which control the type or complexity of a model. We often define the objective function $\mathcal{L}(\boldsymbol{\lambda})$ to be the loss on a validation set (see Section 4.5.4). Since the validation loss depends on the optimal model parameters, which are computed using a complex

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

algorithm, this objective function is effectively a blackbox.[4]

A simple approach to such problems is to use **grid search**, where we evaluate each point in the parameter space, and pick the one with the lowest loss. Unfortunately, this does not scale to high dimensions, because of the curse of dimensionality. In addition, even in low dimensions this can be expensive if evaluating the blackbox objective is expensive (e.g., if it first requires training the model before computing the validation loss). Various solutions to this problem have been proposed. See the sequel to this book, [Mur23], for details.

8.9 Exercises

Exercise 8.1 [Subderivative of the hinge loss function †]

Let $f(x) = (1 - x)_+$ be the hinge loss function, where $(z)_+ = \max(0, z)$. What are $\partial f(0)$, $\partial f(1)$, and $\partial f(2)$?

Exercise 8.2 [EM for the Student distribution]

Derive the EM equations for computing the MLE for a multivariate Student distribution. Consider the case where the dof parameter is known and unknown separately. Hint: write the Student distribution as a scale mixture of Gaussians.

4. If the optimal parameters are computed using a gradient-based optimizer, we can "unroll" the gradient steps, to create a deep circuit that maps from the training data to the optimal parameters and hence to the validation loss. We can then optimize through the optimizer (see e.g., [Fra+17]). However, this technique can only be applied in limited settings.

Part II

Linear Models

9 Linear Discriminant Analysis

9.1 Introduction

In this chapter, we consider classification models of the following form:

$$p(y = c|\boldsymbol{x}, \boldsymbol{\theta}) = \frac{p(\boldsymbol{x}|y = c, \boldsymbol{\theta})p(y = c|\boldsymbol{\theta})}{\sum_{c'} p(\boldsymbol{x}|y = c', \boldsymbol{\theta})p(y = c'|\boldsymbol{\theta})} \tag{9.1}$$

The term $p(y = c|\boldsymbol{\theta})$ is the prior over class labels, and the term $p(\boldsymbol{x}|y = c, \boldsymbol{\theta})$ is called the **class conditional density** for class c.

The overall model is called a **generative classifier**, since it specifies a way to *generate* the features $\boldsymbol{x}$ for each class c, by sampling from $p(\boldsymbol{x}|y = c, \boldsymbol{\theta})$. By contrast, a **discriminative classifier** directly models the class posterior $p(y|\boldsymbol{x}, \boldsymbol{\theta})$. We discuss the pros and cons of these two approaches to classification in Section 9.4.

If we choose the class conditional densities in a special way, we will see that the resulting posterior over classes is a linear function of $\boldsymbol{x}$, i.e., $\log p(y = c|\boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + \text{const}$, where $\boldsymbol{w}$ is derived from $\boldsymbol{\theta}$. Thus the overall method is called **linear discriminant analysis** or **LDA**.[1]

9.2 Gaussian discriminant analysis

In this section, we consider a generative classifier where the class conditional densities are multivariate Gaussians:

$$p(\boldsymbol{x}|y = c, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \tag{9.2}$$

The corresponding class posterior therefore has the form

$$p(y = c|\boldsymbol{x}, \boldsymbol{\theta}) \propto \pi_c \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \tag{9.3}$$

where $\pi_c = p(y = c|\boldsymbol{\theta})$ is the prior probability of label c. (Note that we can ignore the normalization constant in the denominator of the posterior, since it is independent of c.) We call this model **Gaussian discriminant analysis** or **GDA**.

1. This term is rather confusing for two reasons. First, LDA is a generative, not discriminative, classifier. Second, LDA also stands for "latent Dirichlet allocation", which is a popular unsupervised generative model for bags of words [BNJ03].



(a) (b)

Figure 9.1: (a) Some 2d data from 3 different classes. (b) Fitting 2d Gaussians to each class. Generated by discrim_analysis_dboundaries_plot2.ipynb.



(a) (b)

Figure 9.2: Gaussian discriminant analysis fit to data in Figure 9.1. (a) Unconstrained covariances induce quadratic decision boundaries. (b) Tied covariances induce linear decision boundaries. Generated by discrim_analysis_dboundaries_plot2.ipynb.

9.2.1 Quadratic decision boundaries

From Equation (9.3), we see that the log posterior over class labels is given by

$$\log p(y=c|\boldsymbol{x},\boldsymbol{\theta}) = \log \pi_c - \frac{1}{2}\log|2\pi\boldsymbol{\Sigma}_c| - \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_c)^\mathsf{T}\boldsymbol{\Sigma}_c^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_c) + \text{const} \tag{9.4}$$

This is called the **discriminant function**. We see that the decision boundary between any two classes, say c and c', will be a quadratic function of $\boldsymbol{x}$. Hence this is known as **quadratic discriminant analysis** (QDA).

For example, consider the 2d data from 3 different classes in Figure 9.1a. We fit full covariance Gaussian class-conditionals (using the method explained in Section 9.2.4), and plot the results in Figure 9.1b. We see that the features for the blue class are somewhat correlated, whereas the features for the green class are independent, and the features for the red class are independent and isotropic (spherical covariance). In Figure 9.2a, we see that the resulting decision boundaries are quadratic functions of $\boldsymbol{x}$.

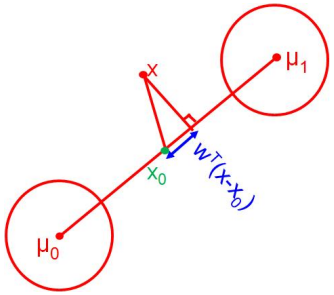


Figure 9.3: Geometry of LDA in the 2 class case where $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = \mathbf{I}$.

9.2.2 Linear decision boundaries

Now we consider a special case of Gaussian discriminant analysis in which the covariance matrices are **tied** or **shared** across classes, so $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$. If $\boldsymbol{\Sigma}$ is independent of c, we can simplify Equation (9.4) as follows:

$$\log p(y=c|\boldsymbol{x},\boldsymbol{\theta}) = \log \pi_c - \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_c)^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_c) + \text{const} \tag{9.5}$$

$$= \underbrace{\log \pi_c - \frac{1}{2}\boldsymbol{\mu}_c^\mathsf{T}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c}_{\gamma_c} + \boldsymbol{x}^\mathsf{T}\underbrace{\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_c}_{\boldsymbol{\beta}_c} \underbrace{+\text{const} - \frac{1}{2}\boldsymbol{x}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\boldsymbol{x}}_{\kappa} \tag{9.6}$$

$$= \gamma_c + \boldsymbol{x}^\mathsf{T}\boldsymbol{\beta}_c + \kappa \tag{9.7}$$

The final term is independent of c, and hence is an irrelevant additive constant that can be dropped. Hence we see that the discriminant function is a linear function of $\boldsymbol{x}$, so the decision boundaries will be linear. Hence this method is called **linear discriminant analysis** or **LDA**. See Figure 9.2b for an example.

9.2.3 The connection between LDA and logistic regression

In this section, we derive an interesting connection between LDA and logistic regression, which we introduced in Section 2.5.3. From Equation (9.7) we can write

$$p(y=c|\boldsymbol{x},\boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_c^\mathsf{T}\boldsymbol{x}+\gamma_c}}{\sum_{c'} e^{\boldsymbol{\beta}_{c'}^\mathsf{T}\boldsymbol{x}+\gamma_{c'}}} = \frac{e^{\boldsymbol{w}_c^\mathsf{T}[1,\boldsymbol{x}]}}{\sum_{c'} e^{\boldsymbol{w}_{c'}^\mathsf{T}[1,\boldsymbol{x}]}} \tag{9.8}$$

where $\boldsymbol{w}_c = [\gamma_c, \boldsymbol{\beta}_c]$. We see that Equation (9.8) has the same form as the multinomial logistic regression model. The key difference is that in LDA, we first fit the Gaussians (and class prior) to maximize the joint likelihood $p(\boldsymbol{x}, y|\boldsymbol{\theta})$, as discussed in Section 9.2.4, and then we derive $\boldsymbol{w}$ from $\boldsymbol{\theta}$. By contrast, in logistic regression, we estimate $\boldsymbol{w}$ directly to maximize the conditional likelihood $p(y|\boldsymbol{x},\boldsymbol{w})$. In general, these can give different results (see Exercise 10.3).

To gain further insight into Equation (9.8), let us consider the binary case. In this case, the

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

posterior is given by

$$p(y=1|\boldsymbol{x},\boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_1^\mathsf{T}\boldsymbol{x}+\gamma_1}}{e^{\boldsymbol{\beta}_1^\mathsf{T}\boldsymbol{x}+\gamma_1} + e^{\boldsymbol{\beta}_0^\mathsf{T}\boldsymbol{x}+\gamma_0}} = \frac{1}{1+e^{(\boldsymbol{\beta}_0-\boldsymbol{\beta}_1)^\mathsf{T}\boldsymbol{x}+(\gamma_0-\gamma_1)}} \tag{9.9}$$

$$= \sigma\left((\boldsymbol{\beta}_1-\boldsymbol{\beta}_0)^\mathsf{T}\boldsymbol{x}+(\gamma_1-\gamma_0)\right) \tag{9.10}$$

where $\sigma(\eta)$ refers to the sigmoid function.

Now

$$\gamma_1-\gamma_0 = -\frac{1}{2}\boldsymbol{\mu}_1^\mathsf{T}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_0^\mathsf{T}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_0 + \log(\pi_1/\pi_0) \tag{9.11}$$

$$= -\frac{1}{2}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0) + \log(\pi_1/\pi_0) \tag{9.12}$$

So if we define

$$\boldsymbol{w} = \boldsymbol{\beta}_1-\boldsymbol{\beta}_0 = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0) \tag{9.13}$$

$$\boldsymbol{x}_0 = \frac{1}{2}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0) - (\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)\frac{\log(\pi_1/\pi_0)}{(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)} \tag{9.14}$$

then we have $\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_0 = -(\gamma_1-\gamma_0)$, and hence

$$p(y=1|\boldsymbol{x},\boldsymbol{\theta}) = \sigma(\boldsymbol{w}^\mathsf{T}(\boldsymbol{x}-\boldsymbol{x}_0)) \tag{9.15}$$

This has the same form as binary logistic regression. Hence the MAP decision rule is

$$\hat{y}(\boldsymbol{x}) = 1 \text{ iff } \boldsymbol{w}^\mathsf{T}\boldsymbol{x} > c \tag{9.16}$$

where $c = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_0$. If $\pi_0 = \pi_1 = 0.5$, then the threshold simplifies to $c = \frac{1}{2}\boldsymbol{w}^\mathsf{T}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0)$.

To interpret this equation geometrically, suppose $\boldsymbol{\Sigma} = \sigma^2\mathbf{I}$. In this case, $\boldsymbol{w} = \sigma^{-2}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_0)$, which is parallel to a line joining the two centroids, $\boldsymbol{\mu}_0$ and $\boldsymbol{\mu}_1$. So we can classify a point by projecting it onto this line, and then checking if the projection is closer to $\boldsymbol{\mu}_0$ or $\boldsymbol{\mu}_1$, as illustrated in Figure 9.3. The question of how close it has to be depends on the prior over classes. If $\pi_1 = \pi_0$, then $\boldsymbol{x}_0 = \frac{1}{2}(\boldsymbol{\mu}_1+\boldsymbol{\mu}_0)$, which is halfway between the means. If we make $\pi_1 > \pi_0$, we have to be closer to $\boldsymbol{\mu}_0$ than halfway in order to pick class 0. And vice versa if $\pi_0 > \pi_1$. Thus we see that the class prior just changes the decision threshold, but not the overall shape of the decision boundary. (A similar argument applies in the multi-class case.)

9.2.4 Model fitting

We now discuss how to fit a GDA model using maximum likelihood estimation. The likelihood function is as follows

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^{N} \text{Cat}(y_n|\boldsymbol{\pi}) \prod_{c=1}^{C} \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_c,\boldsymbol{\Sigma}_c)^{\mathbb{I}(y_n=c)} \tag{9.17}$$

Hence the log-likelihood is given by

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \left[\sum_{n=1}^{N}\sum_{c=1}^{C}\mathbb{I}\left(y_n = c\right)\log \pi_c\right] + \sum_{c=1}^{C}\left[\sum_{n:y_n=c}\log \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)\right] \tag{9.18}$$

Thus we see that we can optimize $\boldsymbol{\pi}$ and the $(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ terms separately.

From Section 4.2.4, we have that the MLE for the class prior is $\hat{\pi}_c = \frac{N_c}{N}$. Using the results from Section 4.2.6, we can derive the MLEs for the Gaussians as follows:

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c}\sum_{n:y_n=c}\boldsymbol{x}_n \tag{9.19}$$

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c}\sum_{n:y_n=c}(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)^{\mathsf{T}} \tag{9.20}$$

Unfortunately the MLE for $\hat{\boldsymbol{\Sigma}}_c$ can easily overfit (i.e., the estimate may not be well-conditioned) if N_c is small compared to D, the dimensionality of the input features. We discuss some solutions to this below.

9.2.4.1 Tied covariances

If we force $\boldsymbol{\Sigma}_c = \boldsymbol{\Sigma}$ to be tied, we will get linear decision boundaries, as we have seen. This also usually results in a more reliable parameter estimate, since we can pool all the samples across classes:

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\sum_{c=1}^{C}\sum_{n:y_n=c}(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)(\boldsymbol{x}_n - \hat{\boldsymbol{\mu}}_c)^{\mathsf{T}} \tag{9.21}$$

9.2.4.2 Diagonal covariances

If we force $\boldsymbol{\Sigma}_c$ to be diagonal, we reduce the number of parameters from $O(CD^2)$ to $O(CD)$, which avoids the overfitting problem. However, this loses the ability to capture correlations between the features. (This is known as the naive Bayes assumption, which we discuss further in Section 9.3.) Despite this approximation, this approach scales well to high dimensions.

We can further restrict the model capacity by using a shared (tied) diagonal covariace matrix. This is called "diagonal LDA" [BL04].

9.2.4.3 MAP estimation

Forcing the covariance matrix to be diagonal is a rather strong assumption. An alternative approach is to perform MAP estimation of a (shared) full covariance Gaussian, rather than using the MLE. Based on the results of Section 4.5.2, we find that the MAP estimate is

$$\hat{\boldsymbol{\Sigma}}_{\text{map}} = \lambda\text{diag}(\hat{\boldsymbol{\Sigma}}_{\text{mle}}) + (1-\lambda)\hat{\boldsymbol{\Sigma}}_{\text{mle}} \tag{9.22}$$

where λ controls the amount of regularization. This technique is known as **regularized discriminant analysis** or RDA [HTF09, p656].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

9.2.5 Nearest centroid classifier

If we assume a uniform prior over classes, we can compute the most probable class label as follows:

$$\hat{y}(\boldsymbol{x}) = \operatorname*{argmax}_c \log p(y=c|\boldsymbol{x},\boldsymbol{\theta}) = \operatorname*{argmin}_c (\boldsymbol{x}-\boldsymbol{\mu}_c)^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_c) \tag{9.23}$$

This is called the **nearest centroid classifier**, or **nearest class mean classifier** (**NCM**), since we are assigning $\boldsymbol{x}$ to the class with the closest $\boldsymbol{\mu}_c$, where distance is measured using (squared) Mahalanobis distance.

We can replace this with any other distance metric to get the decision rule

$$\hat{y}(\boldsymbol{x}) = \operatorname*{argmin}_c d^2(\boldsymbol{x},\boldsymbol{\mu}_c) \tag{9.24}$$

We discuss how to learn distance metrics in Section 16.2, but one simple approach is to use

$$d^2(\boldsymbol{x},\boldsymbol{\mu}_c) = ||\boldsymbol{x}-\boldsymbol{\mu}_c||^2_{\mathbf{W}} = (\boldsymbol{x}-\boldsymbol{\mu}_c)^\mathsf{T}(\mathbf{W}\mathbf{W}^\mathsf{T})(\boldsymbol{x}-\boldsymbol{\mu}_c) = ||\mathbf{W}(\boldsymbol{x}-\boldsymbol{\mu}_c)||^2 \tag{9.25}$$

The corresponding class posterior becomes

$$p(y=c|\boldsymbol{x},\boldsymbol{\mu},\mathbf{W}) = \frac{\exp(-\frac{1}{2}||\mathbf{W}(\boldsymbol{x}-\boldsymbol{\mu}_c)||_2^2)}{\sum_{c'=1}^{C}\exp(-\frac{1}{2}||\mathbf{W}(\boldsymbol{x}-\boldsymbol{\mu}_{c'})||_2^2)} \tag{9.26}$$

We can optimize $\mathbf{W}$ using gradient descent applied to the discriminative loss. This is called **nearest class mean metric learning** [Men+12]. The advantage of this technique is that it can be used for **one-shot learning** of new classes, since we just need to see a single labeled prototype $\boldsymbol{\mu}_c$ per class (assuming we have learned a good $\mathbf{W}$ already).

9.2.6 Fisher's linear discriminant analysis *

Discriminant analysis is a generative approach to classification, which requires fitting an MVN to the features. As we have discussed, this can be problematic in high dimensions. An alternative approach is to reduce the dimensionality of the features $\boldsymbol{x} \in \mathbb{R}^D$ and then fit an MVN to the resulting low-dimensional features $\boldsymbol{z} \in \mathbb{R}^K$. The simplest approach is to use a linear projection matrix, $\boldsymbol{z} = \mathbf{W}\boldsymbol{x}$, where $\mathbf{W}$ is a $K \times D$ matrix. One approach to finding $\mathbf{W}$ would be to use principal components analysis or PCA (Section 20.1). However, PCA is an unsupervised technique that does not take class labels into account. Thus the resulting low dimensional features are not necessarily optimal for classification, as illustrated in Figure 9.4.

An alternative approach is to use gradient based methods to optimize the log likelihood, derived from the class posterior in the low dimensional space, as we discussed in Section 9.2.5.

A third approach (which relies on an eigendecomposition, rather than a gradient-based optimizer) is to find the matrix $\mathbf{W}$ such that the low-dimensional data can be classified as well as possible using a Gaussian class-conditional density model. The assumption of Gaussianity is reasonable since we are computing linear combinations of (potentially non-Gaussian) features. This approach is called **Fisher's linear discriminant analysis**, or **FLDA**.

FLDA is an interesting hybrid of discriminative and generative techniques. The drawback of this technique is that it is restricted to using $K \leq C-1$ dimensions, regardless of D, for reasons that we will explain below. In the two-class case, this means we are seeking a single vector $\boldsymbol{w}$ onto which we can project the data. Below we derive the optimal $\boldsymbol{w}$ in the two-class case. We then generalize to the multi-class case, and finally we give a probabilistic interpretation of this technique.

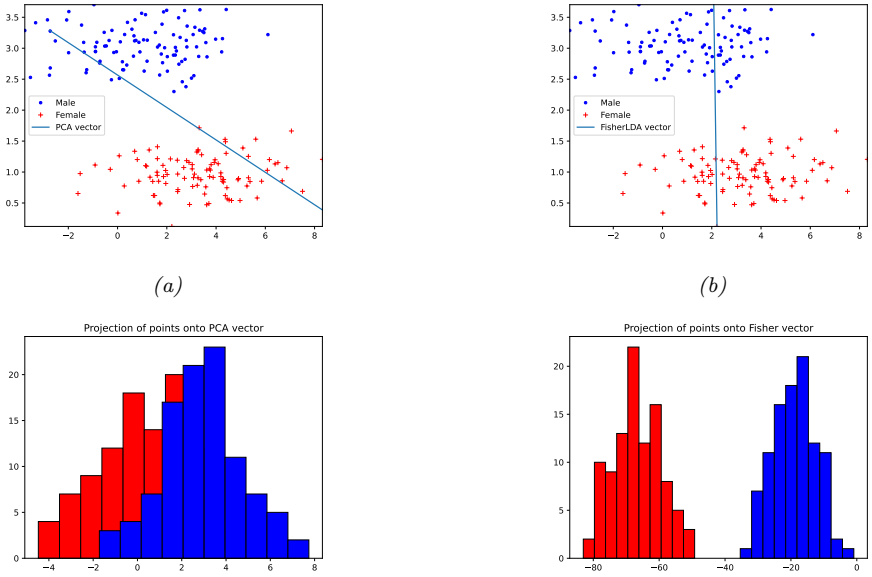


Figure 9.4: Linear disciminant analysis applied to two class dataset in 2d, representing (standardized) height and weight for male and female adults (a) PCA direction. (b) FLDA direction. (c) Projection onto PCA direction shows poor class separation. (d) Projection onto FLDA direction shows good class separation. Generated by fisher_lda_demo.ipynb.

9.2.6.1 Derivation of the optimal 1d projection

We now derive this optimal direction $\boldsymbol{w}$, for the two-class case, following the presentation of [Bis06, Sec 4.1.4]. Define the class-conditional means as

$$\boldsymbol{\mu}_1 = \frac{1}{N_1} \sum_{n:y_n=1} \boldsymbol{x}_n, \ \boldsymbol{\mu}_2 = \frac{1}{N_2} \sum_{n:y_n=2} \boldsymbol{x}_n \tag{9.27}$$

Let $m_k = \boldsymbol{w}^\mathsf{T} \boldsymbol{\mu}_k$ be the projection of each mean onto the line $\boldsymbol{w}$. Also, let $z_n = \boldsymbol{w}^\mathsf{T} \boldsymbol{x}_n$ be the projection of the data onto the line. The variance of the projected points is proportional to

$$s_k^2 = \sum_{n:y_n=k} (z_n - m_k)^2 \tag{9.28}$$

The goal is to find $\boldsymbol{w}$ such that we maximize the distance between the means, $m_2 - m_1$, while also ensuring the projected clusters are "tight", which we can do by minimizing their variance. This suggests the following objective:

$$J(\boldsymbol{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2} \tag{9.29}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We can rewrite the right hand side of the above in terms of $\boldsymbol{w}$ as follows

$$J(\boldsymbol{w}) = \frac{\boldsymbol{w}^\mathsf{T}\mathbf{S}_B\boldsymbol{w}}{\boldsymbol{w}^\mathsf{T}\mathbf{S}_W\boldsymbol{w}} \tag{9.30}$$

where $\mathbf{S}_B$ is the between-class scatter matrix given by

$$\mathbf{S}_B = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\mathsf{T} \tag{9.31}$$

and $\mathbf{S}_W$ is the within-class scatter matrix, given by

$$\mathbf{S}_W = \sum_{n:y_n=1} (\boldsymbol{x}_n - \boldsymbol{\mu}_1)(\boldsymbol{x}_n - \boldsymbol{\mu}_1)^\mathsf{T} + \sum_{n:y_n=2} (\boldsymbol{x}_n - \boldsymbol{\mu}_2)(\boldsymbol{x}_n - \boldsymbol{\mu}_2)^\mathsf{T} \tag{9.32}$$

To see this, note that

$$\boldsymbol{w}^\mathsf{T}\mathbf{S}_B\boldsymbol{w} = \boldsymbol{w}^\mathsf{T}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\mathsf{T}\boldsymbol{w} = (m_2 - m_1)(m_2 - m_1) \tag{9.33}$$

and

$$\begin{aligned} \boldsymbol{w}^\mathsf{T}\mathbf{S}_W\boldsymbol{w} &= \sum_{n:y_n=1} \boldsymbol{w}^\mathsf{T}(\boldsymbol{x}_n - \boldsymbol{\mu}_1)(\boldsymbol{x}_n - \boldsymbol{\mu}_1)^\mathsf{T}\boldsymbol{w} + \\ &\quad \sum_{n:y_n=2} \boldsymbol{w}^\mathsf{T}(\boldsymbol{x}_n - \boldsymbol{\mu}_2)(\boldsymbol{x}_n - \boldsymbol{\mu}_2)^\mathsf{T}\boldsymbol{w} \qquad (9.34) \\ &= \sum_{n:y_n=1} (z_n - m_1)^2 + \sum_{n:y_n=2} (z_n - m_2)^2 \qquad (9.35) \end{aligned}$$

Equation (9.30) is a ratio of two scalars; we can take its derivative with respect to $\boldsymbol{w}$ and equate to zero. One can show (Exercise 9.1) that $J(\boldsymbol{w})$ is maximized when

$$\mathbf{S}_B\boldsymbol{w} = \lambda\mathbf{S}_W\boldsymbol{w} \tag{9.36}$$

where

$$\lambda = \frac{\boldsymbol{w}^\mathsf{T}\mathbf{S}_B\boldsymbol{w}}{\boldsymbol{w}^\mathsf{T}\mathbf{S}_W\boldsymbol{w}} \tag{9.37}$$

Equation (9.36) is called a **generalized eigenvalue** problem. If $\mathbf{S}_W$ is invertible, we can convert it to a regular eigenvalue problem:

$$\mathbf{S}_W^{-1}\mathbf{S}_B\boldsymbol{w} = \lambda\boldsymbol{w} \tag{9.38}$$

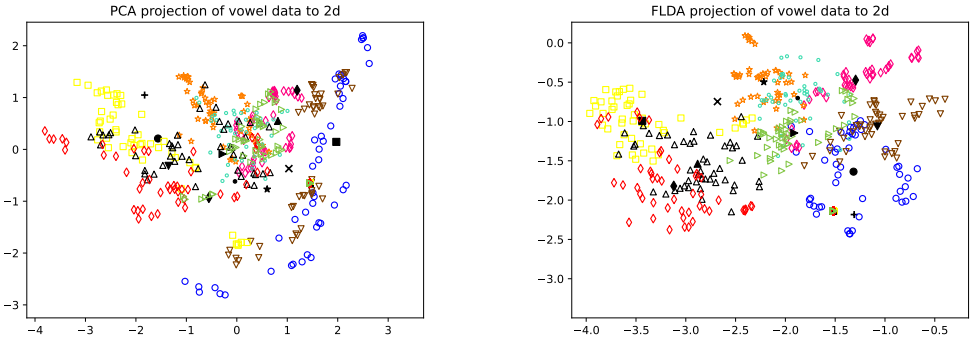
However, in the two class case, there is a simpler solution. In particular, since

$$\mathbf{S}_B\boldsymbol{w} = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\mathsf{T}\boldsymbol{w} = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(m_2 - m_1) \tag{9.39}$$

then, from Equation (9.38) we have

$$\lambda\,\boldsymbol{w} = \mathbf{S}_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(m_2 - m_1) \tag{9.40}$$

$$\boldsymbol{w} \propto \mathbf{S}_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \tag{9.41}$$



(a) (b)

Figure 9.5: (a) PCA projection of vowel data to 2d. (b) FLDA projection of vowel data to 2d. We see there is better class separation in the FLDA case. Adapted from Figure 4.11 of [HTF09]. Generated by fisher_discrim_vowel.ipynb.

Since we only care about the directionality, and not the scale factor, we can just set

$$\boldsymbol{w} = \mathbf{S}_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \tag{9.42}$$

This is the optimal solution in the two-class case. If $\mathbf{S}_W \propto \mathbf{I}$, meaning the pooled covariance matrix is isotropic, then $\boldsymbol{w}$ is proportional to the vector that joins the class means. This is an intuitively reasonable direction to project onto, as shown in Figure 9.3.

9.2.6.2 Extension to higher dimensions and multiple classes

We can extend the above idea to multiple classes, and to higher dimensional subspaces, by finding a projection *matrix* $\mathbf{W}$ which maps from D to K. Let $\boldsymbol{z}_n = \mathbf{W}\boldsymbol{x}_n$ be the low dimensional projection of the n'th data point. Let $\boldsymbol{m}_c = \frac{1}{N_c}\sum_{n:y_n=c} \boldsymbol{z}_n$ be the corresponding mean for the c'th class and $\boldsymbol{m} = \frac{1}{N}\sum_{c=1}^{C} N_c \boldsymbol{m}_c$ be the overall mean, both in the low dimensional space. We define the following scatter matrices:

$$\tilde{\mathbf{S}}_W = \sum_{c=1}^{C} \sum_{n:y_n=c} (\boldsymbol{z}_n - \boldsymbol{m}_c)(\boldsymbol{z}_n - \boldsymbol{m}_c)^\mathsf{T} \tag{9.43}$$

$$\tilde{\mathbf{S}}_B = \sum_{c=1}^{C} N_c (\boldsymbol{m}_c - \boldsymbol{m})(\boldsymbol{m}_c - \boldsymbol{m})^\mathsf{T} \tag{9.44}$$

Finally, we define the objective function as maximizing the following:[2]

$$J(\mathbf{W}) = \frac{|\tilde{\mathbf{S}}_B|}{|\tilde{\mathbf{S}}_W|} = \frac{|\mathbf{W}^\mathsf{T}\mathbf{S}_B\mathbf{W}|}{|\mathbf{W}^\mathsf{T}\mathbf{S}_W\mathbf{W}|} \tag{9.45}$$

2. An alternative criterion that is sometimes used [Fuk90] is $J(\mathbf{W}) = \mathrm{tr}\left\{\tilde{\mathbf{S}}_W^{-1}\tilde{\mathbf{S}}_B\right\} = \mathrm{tr}\left\{(\mathbf{W}\mathbf{S}_W\mathbf{W}^\mathsf{T})^{-1}(\mathbf{W}\mathbf{S}_B\mathbf{W}^\mathsf{T})\right\}$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\mathbf{S}_W$ and $\mathbf{S}_B$ are defined in the original high dimensional space in the obvious way (namely using $\boldsymbol{x}_n$ instead of $\boldsymbol{z}_n$, $\boldsymbol{\mu}_c$ instead of $\boldsymbol{m}_c$, and $\boldsymbol{\mu}$ instead of $\boldsymbol{m}$). The solution can be shown [DHS01] to be $\mathbf{W} = \mathbf{S}_W^{-\frac{1}{2}}\mathbf{U}$, where $\mathbf{U}$ are the K leading eigenvectors of $\mathbf{S}_W^{-\frac{1}{2}}\mathbf{S}_B\mathbf{S}_W^{-\frac{1}{2}}$, assuming $\mathbf{S}_W$ is non-singular. (If it is singular, we can first perform PCA on all the data.)

Figure 9.5 gives an example of this method applied to some $D = 10$ dimensional speech data, representing $C = 11$ different vowel sounds. We project to $K = 2$ dimensions in order to visualize the data. We see that FLDA gives better class separation than PCA.

Note that FLDA is restricted to finding at most a $K \leq C - 1$ dimensional linear subspace, no matter how large D, because the rank of the between class scatter matrix $\mathbf{S}_B$ is $C - 1$. (The -1 term arises because of the $\boldsymbol{\mu}$ term, which is a linear function of the $\boldsymbol{\mu}_c$.) This is a rather severe restriction which limits the usefulness of FLDA.

9.3 Naive Bayes classifiers

In this section, we discuss a simple generative approach to classification in which we assume the features are conditionally independent given the class label. This is called the **naive Bayes assumption**. The model is called "naive" since we do not expect the features to be independent, even conditional on the class label. However, even if the naive Bayes assumption is not true, it often results in classifiers that work well [DP97; HY01a]. One reason for this is that the model is quite simple (it only has $O(CD)$ parameters, for C classes and D features), and hence it is relatively immune to overfitting.

More precisely, the naive Bayes assumption corresponds to using a class conditional density of the following form:

$$p(\boldsymbol{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} p(x_d|y = c, \boldsymbol{\theta}_{dc}) \tag{9.46}$$

where $\boldsymbol{\theta}_{dc}$ are the parameters for the class conditional density for class c and feature d. Hence the posterior over class labels is given by

$$p(y = c|\boldsymbol{x}, \boldsymbol{\theta}) = \frac{p(y = c|\boldsymbol{\pi}) \prod_{d=1}^{D} p(x_d|y = c, \boldsymbol{\theta}_{dc})}{\sum_{c'} p(y = c'|\boldsymbol{\pi}) \prod_{d=1}^{D} p(x_d|y = c', \boldsymbol{\theta}_{dc'})} \tag{9.47}$$

where π_c is the prior probability of class c, and $\boldsymbol{\theta} = (\boldsymbol{\pi}, \{\boldsymbol{\theta}_{dc}\})$ are all the parameters. This is known as a **naive Bayes classifier** or **NBC**.

9.3.1 Example models

We still need to specify the form of the probability distributions in Equation (9.46). This depends on what type of feature x_d is. We give some examples below:

- In the case of binary features, $x_d \in \{0, 1\}$, we can use the Bernoulli distribution: $p(\boldsymbol{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Ber}(x_d|\theta_{dc})$, where θ_{dc} is the probability that $x_d = 1$ in class c. This is sometimes called the **multivariate Bernoulli naive Bayes** model. For example, Figure 9.6 shows the estimated parameters for each class when we fit this model to a binarized version of MNIST. This approach

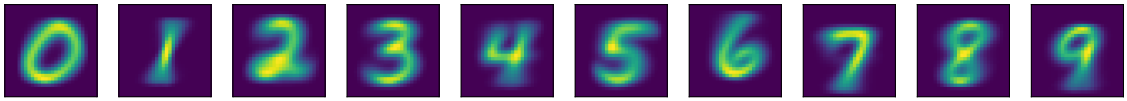


Figure 9.6: Visualization of the Bernoulli class conditional densities for a naive Bayes classifier fit to a binarized version of the MNIST dataset. Generated by naive_bayes_mnist_jax.ipynb.



Figure 9.7: Visualization of the predictions made by the model in Figure 9.6 when applied to some binarized MNIST test images. The title shows the most probable predicted class. Generated by naive_bayes_mnist_jax.ipynb.

does surprisingly well, and has a test set accuracy of 84.3%. (See Figure 9.7 for some sample predictions.)

- In the case of categorical features, $x_d \in \{1, \ldots, K\}$, we can use the categorical distribution: $p(\boldsymbol{x}|y=c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \text{Cat}(x_d|\boldsymbol{\theta}_{dc})$, where θ_{dck} is the probability that $x_d = k$ given that $y = c$.
- In the case of real-valued features, $x_d \in \mathbb{R}$, we can use the univariate Gaussian distribution: $p(\boldsymbol{x}|y=c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \mathcal{N}(x_d|\mu_{dc}, \sigma_{dc}^2)$, where μ_{dc} is the mean of feature d when the class label is c, and σ_{dc}^2 is its variance. (This is equivalent to Gaussian discriminant analysis using diagonal covariance matrices.)

9.3.2 Model fitting

In this section, we discuss how to fit a naive Bayes classifier using maximum likelihood estimation.

We can write the likelihood as follows:

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^{N} \left[\text{Cat}(y_n|\boldsymbol{\pi}) \prod_{d=1}^{D} p(x_{nd}|y_n, \boldsymbol{\theta}_d) \right] \tag{9.48}$$

$$= \prod_{n=1}^{N} \left[\text{Cat}(y_n|\boldsymbol{\pi}) \prod_{d=1}^{D} \prod_{c=1}^{C} p(x_{nd}|\boldsymbol{\theta}_{dc})^{\mathbb{I}(y_n=c)} \right] \tag{9.49}$$

so the log-likelihood is given by

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \left[\sum_{n=1}^{N} \sum_{c=1}^{C} \mathbb{I}\left(y_n = c\right) \log \pi_c \right] + \sum_{c=1}^{C} \sum_{d=1}^{D} \left[\sum_{n:y_n=c} \log p(x_{nd}|\boldsymbol{\theta}_{dc}) \right] \tag{9.50}$$

We see that this decomposes into a term for $\boldsymbol{\pi}$, and CD terms for each $\boldsymbol{\theta}_{dc}$:

$$\log p(\mathcal{D}|\boldsymbol{\theta}) = \log p(\mathcal{D}_y|\boldsymbol{\pi}) + \sum_{c} \sum_{d} \log p(\mathcal{D}_{dc}|\boldsymbol{\theta}_{dc}) \tag{9.51}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\mathcal{D}_y = \{y_n : n = 1 : N\}$ are all the labels, and $\mathcal{D}_{dc} = \{x_{nd} : y_n = c\}$ are all the values of feature d for examples from class c. Hence we can estimate these parameters separately.

In Section 4.2.4, we show that the MLE for $\boldsymbol{\pi}$ is the vector of empirical counts, $\hat{\pi}_c = \frac{N_c}{N}$. The MLEs for $\boldsymbol{\theta}_{dc}$ depend on the choice of the class conditional density for feature d. We discuss some common choices below.

- In the case of discrete features, we can use a categorical distribution. A straightforward extension of the results in Section 4.2.4 gives the following expression for the MLE:

$$\hat{\theta}_{dck} = \frac{N_{dck}}{\sum_{k'=1}^{K} N_{dck'}} = \frac{N_{dck}}{N_c} \tag{9.52}$$

  where $N_{dck} = \sum_{n=1}^{N} \mathbb{I}(x_{nd} = k, y_n = c)$ is the number of times that feature d had value k in examples of class c.

- In the case of binary features, the categorical distribution becomes the Bernoulli, and the MLE becomes

$$\hat{\theta}_{dc} = \frac{N_{dc}}{N_c} \tag{9.53}$$

  which is the empirical fraction of times that feature d is on in examples of class c.

- In the case of real-valued features, we can use a Gaussian distribution. A straightforward extension of the results in Section 4.2.5 gives the following expression for the MLE:

$$\hat{\mu}_{dc} = \frac{1}{N_c} \sum_{n:y_n=c} x_{nd} \tag{9.54}$$

$$\hat{\sigma}^2_{dc} = \frac{1}{N_c} \sum_{n:y_n=c} (x_{nd} - \hat{\mu}_{dc})^2 \tag{9.55}$$

  Thus we see that fitting a naive Bayes classifier is extremely simple and efficient.

9.3.3 Bayesian naive Bayes

In this section, we extend our discussion of MLE estimation for naive Bayes classifiers from Section 9.3.2 to compute the posterior distribution over the parameters. For simplicity, let us assume we have categorical features, so $p(x_d|\boldsymbol{\theta}_{dc}) = \text{Cat}(x_d|\boldsymbol{\theta}_{dc})$, where $\theta_{dck} = p(x_d = k|y = c)$. In Section 4.6.3.2, we show that the conjugate prior for the categorical likelihood is the Dirichlet distribution, $p(\boldsymbol{\theta}_{dc}) = \text{Dir}(\boldsymbol{\theta}_{dc}|\boldsymbol{\beta}_{dc})$, where β_{dck} can be interpereted as a set of "**pseudo counts**", corresponding to counts N_{dck} that come from prior data. Similarly we use a Dirichlet prior for the label frequencies, $p(\boldsymbol{\pi}) = \text{Dir}(\boldsymbol{\pi}|\boldsymbol{\alpha})$. By using a conjugate prior, we can compute the posterior in closed form, as we explain in Section 4.6.3. In particular, we have

$$p(\boldsymbol{\theta}|\mathcal{D}) = \text{Dir}(\boldsymbol{\pi}|\,\widehat{\boldsymbol{\alpha}}) \prod_{d=1}^{D} \prod_{c=1}^{C} \text{Dir}(\boldsymbol{\theta}_{dc}|\,\widehat{\boldsymbol{\beta}}_{dc}) \tag{9.56}$$

where $\widehat{\alpha}_c = \breve{\alpha}_c + N_c$ and $\widehat{\beta}_{dck} = \breve{\beta}_{dck} + N_{dck}$.

Using the results from Section 4.6.3.4, we can derive the posterior predictive distribution as follows. For the label prior (before seeing $\boldsymbol{x}$, but after seeing $\mathcal{D}$), we have $p(y|\mathcal{D}) = \mathrm{Cat}(y|\overline{\boldsymbol{\pi}})$, where $\overline{\pi}_c = \widehat{\alpha}_c / \sum_{c'} \widehat{\alpha}_{c'}$. For the feature likelihood of $\boldsymbol{x}$ (given y and $\mathcal{D}$), we have $p(x_d = k|y = c, \mathcal{D}) = \overline{\theta}_{dck}$, where

$$\overline{\theta}_{dck} = \frac{\widehat{\beta}_{dck}}{\sum_{k'=1}^{K} \widehat{\beta}_{dck'}} = \frac{\breve{\beta}_{dck} + N_{dck}}{\sum_{k'=1}^{K} \breve{\beta}_{dck'} + N_{dck'}} \tag{9.57}$$

is the posterior mean of the parameters. (Note that $\sum_{k'=1}^{K} N_{dck'} = N_{dc} = N_c$ is the number of examples for class c.)

If $\breve{\beta}_{dck} = 0$, this reduces to the MLE in Equation (9.52). By contrast, if we set $\breve{\beta}_{dck} = 1$, we add 1 to all the empirical counts before normalizing. This is called **add-one smoothing** or **Laplace smoothing**. For example, in the binary case, this gives

$$\overline{\theta}_{dc} = \frac{\breve{\beta}_{dc1} + N_{dc1}}{\breve{\beta}_{dc0} + N_{dc0} + \breve{\beta}_{dc1} + N_{dc1}} = \frac{1 + N_{dc1}}{2 + N_{dc}} \tag{9.58}$$

We can finally compute the posterior predictive distribution over the label as follows:

$$p(y = c|\boldsymbol{x}, \mathcal{D}) \propto p(y = c|\mathcal{D}) \prod_d p(x_d|y = c, \mathcal{D}) = \overline{\pi}_c \prod_d \prod_k \overline{\theta}_{dck}^{\mathbb{I}(x_d = k)} \tag{9.59}$$

This gives us a fully Bayesian form of naive Bayes, in which we have integrated out all the parameters. (In this case, the predictive distribution can be obtained merely by plugging in the posterior mean parameters.)

9.3.4 The connection between naive Bayes and logistic regression

In this section, we show that the class posterior $p(y|\boldsymbol{x}, \boldsymbol{\theta})$ for a NBC model has the same form as multinomial logistic regression. For simplicity, we assume that the features are all discrete, and each has K states, although the result holds for arbitrary feature distributions in the exponential family.

Let $x_{dk} = \mathbb{I}(x_d = k)$, so $\boldsymbol{x}_d$ is a one-hot encoding of feature d. Then the class conditional density can be written as follows:

$$p(\boldsymbol{x}|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \mathrm{Cat}(x_d|y = c, \boldsymbol{\theta}) = \prod_{d=1}^{D} \prod_{k=1}^{K} \theta_{dck}^{x_{dk}} \tag{9.60}$$

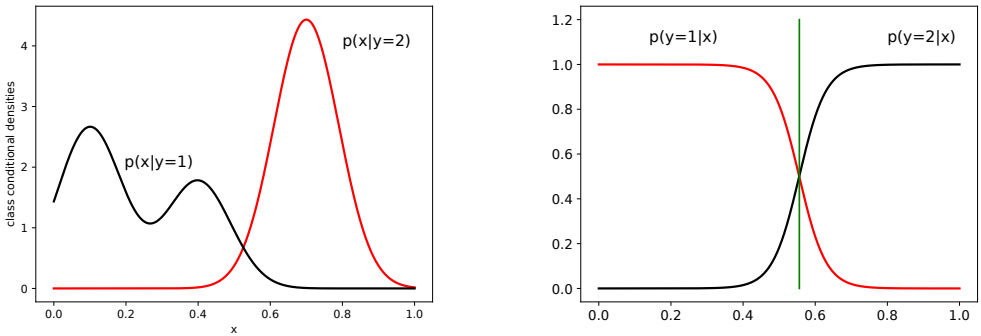
Hence the posterior over classes is given by

$$p(y = c|\boldsymbol{x}, \boldsymbol{\theta}) = \frac{\pi_c \prod_d \prod_k \theta_{dck}^{x_{dk}}}{\sum_{c'} \pi_{c'} \prod_d \prod_k \theta_{dc'k}^{x_{dk}}} = \frac{\exp[\log \pi_c + \sum_d \sum_k x_{dk} \log \theta_{dck}]}{\sum_{c'} \exp[\log \pi_{c'} + \sum_d \sum_k x_{dk} \log \theta_{dc'k}]} \tag{9.61}$$

This can be written as a softmax

$$p(y = c|\boldsymbol{x}, \boldsymbol{\theta}) = \frac{e^{\boldsymbol{\beta}_c^{\mathsf{T}} \boldsymbol{x} + \gamma_c}}{\sum_{c'=1}^{C} e^{\boldsymbol{\beta}_{c'}^{\mathsf{T}} \boldsymbol{x} + \gamma_{c'}}} \tag{9.62}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 9.8: The class-conditional densities $p(x|y=c)$ (left) may be more complex than the class posteriors $p(y=c|x)$ (right). Adapted from Figure 1.27 of [Bis06]. Generated by generativeVsDiscrim.ipynb.

by suitably defining $\boldsymbol{\beta}_c$ and γ_c. This has exactly the same form as multinomial logistic regression in Section 2.5.3. The difference is that with naive Bayes we optimize the joint likelihood $\prod_n p(y_n, \boldsymbol{x}_n|\boldsymbol{\theta})$, whereas with logistic regression, we optimize the conditional likelihood $\prod_n p(y_n|\boldsymbol{x}_n, \boldsymbol{\theta})$. In general, these can give different results (see Exercise 10.3).

9.4 Generative vs discriminative classifiers

A model of the form $p(\boldsymbol{x}, y) = p(y)p(\boldsymbol{x}|y)$ is called a **generative classifier**, since it can be used to generate examples $\boldsymbol{x}$ from each class y. By contrast, a model of the form $p(y|\boldsymbol{x})$ is called a **discriminative classifier**, since it can only be used to discriminate between different classes. Below we discuss various pros and cons of the generative and discriminative approaches to classification. (See also [BT04; UB05; LBM06; BL07a; Rot+18].)

9.4.1 Advantages of discriminative classifiers

The main advantages of discriminative classifiers are as follows:

- **Better predictive accuracy**. Discriminative classifiers are often much more accurate than generative classifiers [NJ02]. The reason is that the conditional distribution $p(y|\boldsymbol{x})$ is often much simpler (and therefore easier to learn) than the joint distribution $p(y, \boldsymbol{x})$, as illustrated in Figure 9.8. In particular, discriminative models do not need to "waste effort" modeling the distribution of the input features.
- **Can handle feature preprocessing**. A big advantage of discriminative methods is that they allow us to preprocess the input in arbitrary ways. For example, we can perform a polynomial expansion of the input features, and we can replace a string of words with embedding vectors (see Section 20.5). It is often hard to define a generative model on such pre-processed data, since the new features can be correlated in complex ways which are hard to model.

- **Well-calibrated probabilities**. Some generative classifiers, such as naive Bayes (described in Section 9.3), make strong independence assumptions which are often not valid. This can result in very extreme posterior class probabilities (very near 0 or 1). Discriminative models, such as logistic regression, are often better calibrated in terms of their probability estimates, although they also sometimes need adjustment (see e.g., [NMC05]).

9.4.2 Advantages of generative classifiers

The main advantages of generative classifiers are as follows:

- **Easy to fit.** Generative classifiers are often very easy to fit. For example, in Section 9.3.2, we show how to fit a naive Bayes classifier by simple counting and averaging. By contrast, logistic regression requires solving a convex optimization problem (see Section 10.2.3 for the details), and neural nets require solving a non-convex optimization problem, both of which are much slower.

- **Can easily handle missing input features.** Sometimes some of the inputs (components of $\boldsymbol{x}$) are not observed. In a generative classifier, there is a simple method for dealing with this, as we show in Section 1.5.5. However, in a discriminative classifier, there is no principled solution to this problem, since the model assumes that $\boldsymbol{x}$ is always available to be conditioned on.

- **Can fit classes separately.** In a generative classifier, we estimate the parameters of each class conditional density independently (as we show in Section 9.3.2), so we do not have to retrain the model when we add more classes. In contrast, in discriminative models, all the parameters interact, so the whole model must be retrained if we add a new class.

- **Can handle unlabeled training data.** It is easy to use generative models for semi-supervised learning, in which we combine labeled data $\mathcal{D}_{xy} = \{(\boldsymbol{x}_n, y_n)\}$ and unlabeled data, $\mathcal{D}_x = \{\boldsymbol{x}_n\}$. However, this is harder to do with discriminative models, since there is no uniquely optimal way to exploit $\mathcal{D}_x$.

- **May be more robust to spurious features**. A discriminative model $p(y|\boldsymbol{x})$ may pick up on features of the input $\boldsymbol{x}$ that can discriminate different values of y in the training set, but which are not robust and do not generalize beyond the training set. These are called **spurious features** (see e.g., [Arj21; Zho+21]). By contrast, a generative model $p(\boldsymbol{x}|y)$ may be better able to capture the causal mechanisms of the underlying data generating process; such causal models can be more robust to distribution shift (see e.g., [Sch19; LBS19; LN81]).

9.4.3 Handling missing features

Sometimes we are missing parts of the input $\boldsymbol{x}$ during training and/or testing. In a generative classifier, we can handle this situation by marginalizing out the missing values. (We assume that the missingness of a feature is not informative about its potential value.) By contrast, when using a discriminative model, there is no unique best way to handle missing inputs, as we discuss in Section 1.5.5.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

For example, suppose we are missing the value of x_1. We just have to compute

$$p(y=c|\boldsymbol{x}_{2:D},\boldsymbol{\theta}) \propto p(y=c|\boldsymbol{\pi})p(\boldsymbol{x}_{2:D}|y=c,\boldsymbol{\theta}) \tag{9.63}$$

$$= p(y=c|\boldsymbol{\pi})\sum_{x_1} p(x_1,\boldsymbol{x}_{2:D}|y=c,\boldsymbol{\theta}) \tag{9.64}$$

In Gaussian discriminant analysis, we can marginalize out x_1 using the equations from Section 3.2.3.

If we make the naive Bayes assumption, things are even easier, since we can just ignore the likelihood term for x_1. This follows because

$$\sum_{x_1} p(x_1,x_{2:D}|y=c,\boldsymbol{\theta}) = \left[\sum_{x_1} p(x_1|\boldsymbol{\theta}_{1c})\right]\prod_{d=2}^{D} p(x_d|\boldsymbol{\theta}_{dc}) = \prod_{d=2}^{D} p(x_d|\boldsymbol{\theta}_{dc}) \tag{9.65}$$

where we exploited the fact that $p(x_d|y=c,\boldsymbol{\theta}) = p(x_d|\boldsymbol{\theta}_{dc})$ and $\sum_{x_1} p(x_1|\boldsymbol{\theta}_{1c}) = 1$.

9.5 Exercises

Exercise 9.1 [Derivation of Fisher's linear discriminant]

Show that the maximum of $J(\boldsymbol{w}) = \frac{\boldsymbol{w}^T\mathbf{S}_B\boldsymbol{w}}{\boldsymbol{w}^T\mathbf{S}_W\boldsymbol{w}}$ is given by $\mathbf{S}_B\boldsymbol{w} = \lambda\mathbf{S}_W\boldsymbol{w}$ where $\lambda = \frac{\boldsymbol{w}^T\mathbf{S}_B\boldsymbol{w}}{\boldsymbol{w}^T\mathbf{S}_W\boldsymbol{w}}$. Hint: recall that the derivative of a ratio of two scalars is given by $\frac{d}{dx}\frac{f(x)}{g(x)} = \frac{f'g-fg'}{g^2}$, where $f' = \frac{d}{dx}f(x)$ and $g' = \frac{d}{dx}g(x)$. Also, recall that $\frac{d}{d\boldsymbol{x}}\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = (\mathbf{A}+\mathbf{A}^T)\boldsymbol{x}$.

10 Logistic Regression

10.1 Introduction

Logistic regression is a widely used discriminative classification model $p(y|\boldsymbol{x};\boldsymbol{\theta})$, where $\boldsymbol{x} \in \mathbb{R}^D$ is a fixed-dimensional input vector, $y \in \{1,\ldots,C\}$ is the class label, and $\boldsymbol{\theta}$ are the parameters. If $C = 2$, this is known as **binary logistic regression**, and if $C > 2$, it is known as **multinomial logistic regression**, or alternatively, **multiclass logistic regression**. We give the details below.

10.2 Binary logistic regression

Binary logistic regression corresponds to the following model

$$p(y|\boldsymbol{x},\boldsymbol{\theta}) = \text{Ber}(y|\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b)) \tag{10.1}$$

where σ is the sigmoid function defined in Section 2.4.2, $\boldsymbol{w}$ are the weights, b is the bias, and $\boldsymbol{\theta} = (\boldsymbol{w}, b)$ are all the parameters. In other words,

$$p(y = 1|\boldsymbol{x},\boldsymbol{\theta}) = \sigma(a) = \frac{1}{1 + e^{-a}} \tag{10.2}$$

where $a = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b = \log(\frac{p}{1-p})$, is the log-odds, and $p = p(y = 1|\boldsymbol{x},\boldsymbol{\theta})$. (In ML, the quantity a is usually called the **logit** or the **pre-activation**.)

Sometimes we choose to use the labels $\tilde{y} \in \{-1, +1\}$ instead of $y \in \{0, 1\}$. We can compute the probability of these alternative labels using

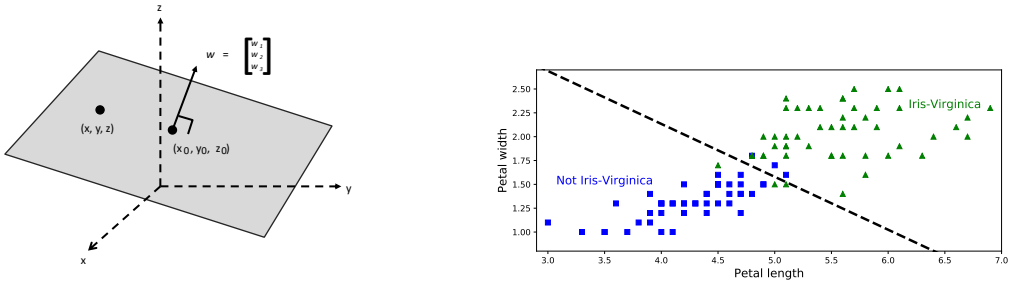
$$p(\tilde{y}|\boldsymbol{x},\boldsymbol{\theta}) = \sigma(\tilde{y}a) \tag{10.3}$$

since $\sigma(-a) = 1 - \sigma(a)$. This slightly more compact notation is widely used in the ML literature.

10.2.1 Linear classifiers

The sigmoid gives the probability that the class label is $y = 1$. If the loss for misclassifying each class is the same, then the optimal decision rule is to predict $y = 1$ iff class 1 is more likely than class 0, as we explained in Section 5.1.2.2. Thus

$$f(\boldsymbol{x}) = \mathbb{I}\left(p(y = 1|\boldsymbol{x}) > p(y = 0|\boldsymbol{x})\right) = \mathbb{I}\left(\log \frac{p(y = 1|\boldsymbol{x})}{p(y = 0|\boldsymbol{x})} > 0\right) = \mathbb{I}\left(a > 0\right) \tag{10.4}$$



(a) (b)

Figure 10.1: (a) Visualization of a 2d plane in a 3d space with surface normal $\boldsymbol{w}$ going through point $\boldsymbol{x}_0 = (x_0, y_0, z_0)$. See text for details. (b) Visualization of optimal linear decision boundary induced by logistic regression on a 2-class, 2-feature version of the iris dataset. Generated by iris_logreg.ipynb. Adapted from Figure 4.24 of [Gér19].

where $a = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b$.

Thus we can write the prediction function as follows:

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = b + \boldsymbol{w}^\mathsf{T}\boldsymbol{x} = b + \sum_{d=1}^{D} w_d x_d \tag{10.5}$$

where $\boldsymbol{w}^\mathsf{T}\boldsymbol{x} = \langle \boldsymbol{w}, \boldsymbol{x} \rangle$ is the inner product between the weight vector $\boldsymbol{w}$ and the feature vector $\boldsymbol{x}$. This function defines a linear **hyperplane**, with normal vector $\boldsymbol{w} \in \mathbb{R}^D$ and an offset $b \in \mathbb{R}$ from the origin.

Equation (10.5) can be understood by looking at Figure 10.1a. Here we show a plane in a 3d feature space going through the point $\boldsymbol{x}_0$ with surface normal $\boldsymbol{w}$. Points on the surface satisfy $\boldsymbol{w}^\mathsf{T}(\boldsymbol{x} - \boldsymbol{x}_0) = 0$. If we define $b = -\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_0$, we can rewrite this as $\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b = 0$. This plane separates 3d space into two **half spaces**. This linear plane is known as a **decision boundary**. If we can perfectly separate the training examples by such a linear boundary (without making any classification errors on the training set), we say the data is **linearly separable**. From Figure 10.1b, we see that the two-class, two-feature version of the iris dataset is not linearly separable.

In general, there will be uncertainty about the correct class label, so we need to predict a probability distribution over labels, and not just decide which side of the decision boundary we are on. In Figure 10.2, we plot $p(y = 1|(x_1, x_2), \boldsymbol{w}) = \sigma(w_1 x_1 + w_2 x_2)$ for different weight vectors $\boldsymbol{w}$. The vector $\boldsymbol{w}$ defines the orientation of the decision boundary, and its magnitude, $||\boldsymbol{w}|| = \sqrt{\sum_{d=1}^{D} w_d^2}$, controls the steepness of the sigmoid, and hence the confidence of the predictions.

10.2.2 Nonlinear classifiers

We can often make a problem linearly separable by preprocessing the inputs in a suitable way. In particular, let $\boldsymbol{\phi}(\boldsymbol{x})$ be a transformed version of the input feature vector. For example, suppose we use $\boldsymbol{\phi}(x_1, x_2) = [1, x_1^2, x_2^2]$, and we let $\boldsymbol{w} = [-R^2, 1, 1]$. Then $\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x}) = x_1^2 + x_2^2 - R^2$, so the decision boundary (where $f(\boldsymbol{x}) = 0$) defines a circle with radius R, as shown in Figure 10.3. The resulting

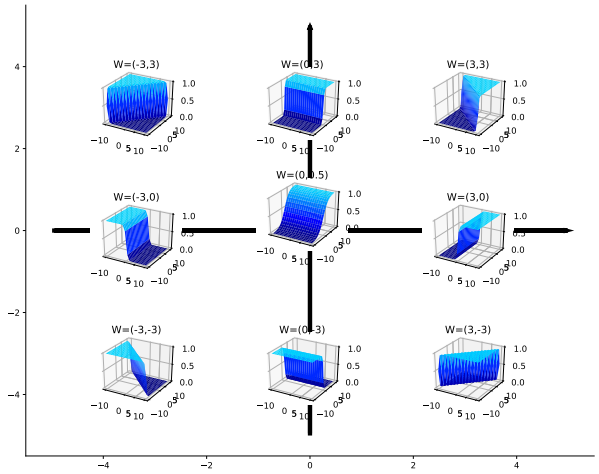


Figure 10.2: Plots of $\sigma(w_1 x_1 + w_2 x_2)$. Here $\boldsymbol{w} = (w_1, w_2)$ defines the normal to the decision boundary. Points to the right of this have $\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}) > 0.5$, and points to the left have $\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}) < 0.5$. Adapted from Figure 39.3 of [Mac03]. Generated by sigmoid_2d_plot.ipynb.

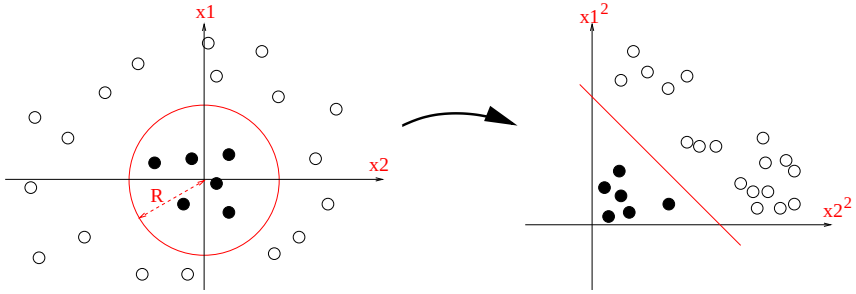


Figure 10.3: Illustration of how we can transform a quadratic decision boundary into a linear one by transforming the features from $\boldsymbol{x} = (x_1, x_2)$ to $\boldsymbol{\phi}(\boldsymbol{x}) = (x_1^2, x_2^2)$. Used with kind permission of Jean-Philippe Vert.

function f is still linear in the parameters $\boldsymbol{w}$, which is important for simplifying the learning problem, as we will see in Section 10.2.3. However, we can gain even more power by learning the parameters of the feature extractor $\boldsymbol{\phi}(\boldsymbol{x})$ in addition to linear weights $\boldsymbol{w}$; we discuss how to do this in Part III.

In Figure 10.3, we used a quadratic expansion of the features. We can also use a higher order polynomial, as in Section 1.2.2.2. In Figure 10.4, we show the effects of using polynomial expansion up to degree K on a 2d logistic regression problem. As in Figure 1.7, we see that the model becomes more complex as the number of parameters increases, and eventually results in overfitting. We discuss ways to reduce overfitting in Section 10.2.7.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

10.2.3 Maximum likelihood estimation

In this section, we discuss how to estimate the parameters of a logistic regression model using maximum likelihood estimation.

10.2.3.1 Objective function

The negative log likelihood (scaled by the dataset size N) is given by the following (we assume the bias term b is absorbed into the weight vector $\boldsymbol{w}$):

$$\text{NLL}(\boldsymbol{w}) = -\frac{1}{N}\log p(\mathcal{D}|\boldsymbol{w}) = -\frac{1}{N}\log\prod_{n=1}^{N}\text{Ber}(y_n|\mu_n) \tag{10.6}$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\log[\mu_n^{y_n}\times(1-\mu_n)^{1-y_n}] \tag{10.7}$$

$$= -\frac{1}{N}\sum_{n=1}^{N}[y_n\log\mu_n + (1-y_n)\log(1-\mu_n)] \tag{10.8}$$

$$= \frac{1}{N}\sum_{n=1}^{N}\mathbb{H}_{ce}(y_n,\mu_n) \tag{10.9}$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, $a_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n$ is the **logit**, and $\mathbb{H}_{ce}(y_n,\mu_n)$ is the **binary cross entropy** defined by

$$\mathbb{H}_{ce}(p,q) = -[p\log q + (1-p)\log(1-q)] \tag{10.10}$$

If we use $\tilde{y}_n \in \{-1,+1\}$ instead of $y_n \in \{0,1\}$, then we can rewrite this as follows:

$$\text{NLL}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N}[\mathbb{I}(\tilde{y}_n=1)\log(\sigma(a_n)) + \mathbb{I}(\tilde{y}_n=-1)\log(\sigma(-a_n))] \tag{10.11}$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\log(\sigma(\tilde{y}_n a_n)) \tag{10.12}$$

$$= \frac{1}{N}\sum_{n=1}^{N}\log(1+\exp(-\tilde{y}_n a_n)) \tag{10.13}$$

However, in this book, we will mostly use the $y_n \in \{0,1\}$ notation, since it is easier to generalize to the multiclass case (Section 10.3), and makes the connection with cross-entropy easier to see.

10.2.3.2 Optimizing the objective

To find the MLE, we must solve

$$\nabla_{\boldsymbol{w}}\text{NLL}(\boldsymbol{w}) = \boldsymbol{g}(\boldsymbol{w}) = \boldsymbol{0} \tag{10.14}$$

We can use any gradient-based optimization algorithm to solve this, such as those we discuss in Chapter 8. We give a specific example in Section 10.2.4. But first we must derive the gradient, as we explain below.

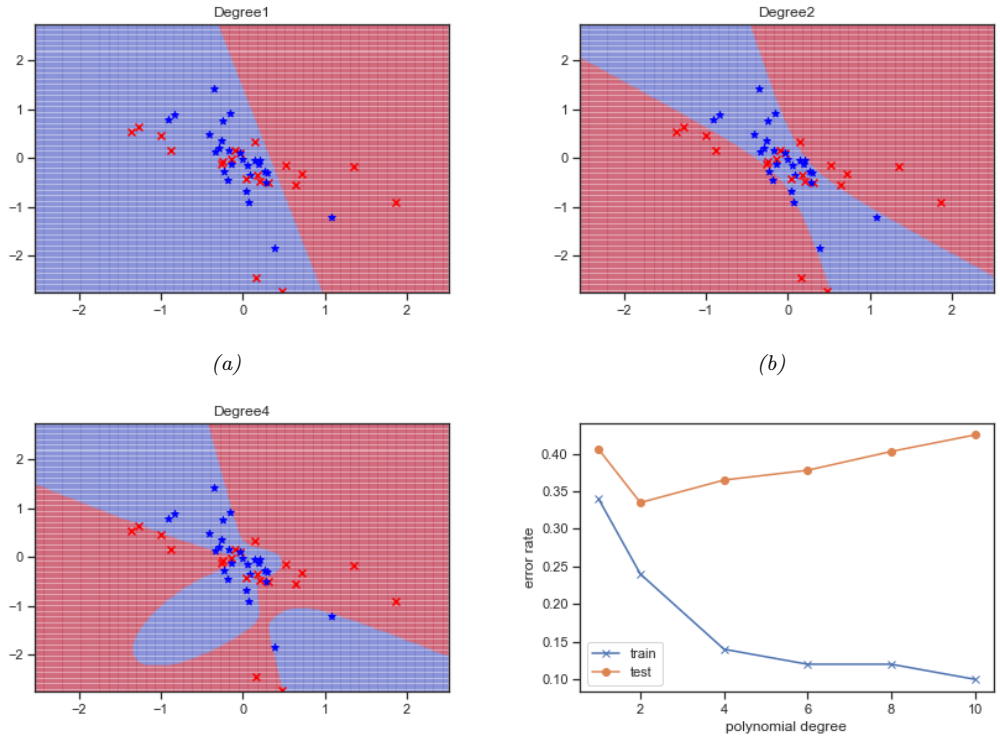


Figure 10.4: Polynomial feature expansion applied to a two-class, two-dimensional logistic regression problem. (a) Degree $K = 1$. (b) Degree $K = 2$. (c) Degree $K = 4$. (d) Train and test error vs degree. Generated by logreg_poly_demo.ipynb.

10.2.3.3 Deriving the gradient

Although we can use automatic differentiation methods (Section 13.3) to compute the gradient of the NLL, it is also easy to do explicitly, as we show below. Fortunately the resulting equations will turn out to have a simple and intuitive interpretation, which can be used to derive other methods, as we will see.

To start, note that

$$\frac{d\mu_n}{da_n} = \sigma(a_n)(1 - \sigma(a_n)) \tag{10.15}$$

where $a_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n$ and $\mu_n = \sigma(a_n)$. Hence by the chain rule (and the rules of vector calculus, discussed in Section 7.8) we have

$$\frac{\partial}{\partial w_d}\mu_n = \frac{\partial}{\partial w_d}\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n) = \frac{\partial}{\partial a_n}\sigma(a_n)\frac{\partial a_n}{\partial w_d} = \mu_n(1 - \mu_n)x_{nd} \tag{10.16}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

The gradient for the bias term can be derived in the same way, by using the input $x_{n0} = 1$ in the above equation. However, we will ignore the bias term for simplicity. Hence

$$\nabla_{\boldsymbol{w}} \log(\mu_n) = \frac{1}{\mu_n} \nabla_{\boldsymbol{w}} \mu_n = (1 - \mu_n)\boldsymbol{x}_n \tag{10.17}$$

Similarly,

$$\nabla_{\boldsymbol{w}} \log(1 - \mu_n) = \frac{-\mu_n(1 - \mu_n)\boldsymbol{x}_n}{1 - \mu_n} = -\mu_n \boldsymbol{x}_n \tag{10.18}$$

Thus the gradient vector of the NLL is given by

$$\nabla_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) = -\frac{1}{N} \sum_{n=1}^{N} [y_n(1 - \mu_n)\boldsymbol{x}_n - (1 - y_n)\mu_n \boldsymbol{x}_n] \tag{10.19}$$

$$= -\frac{1}{N} \sum_{n=1}^{N} [y_n \boldsymbol{x}_n - y_n \boldsymbol{x}_n \mu_n - \boldsymbol{x}_n \mu_n + y_n \boldsymbol{x}_n \mu_n)] \tag{10.20}$$

$$= \frac{1}{N} \sum_{n=1}^{N} (\mu_n - y_n)\boldsymbol{x}_n \tag{10.21}$$

If we interpret $e_n = \mu_n - y_n$ as an error signal, we can see that the gradient weights each input $\boldsymbol{x}_n$ by its error, and then averages the result. Note that we can rewrite the gradient in matrix form as follows:

$$\nabla_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) = \frac{1}{N} (\mathbf{1}_N^{\mathsf{T}} (\text{diag}(\boldsymbol{\mu} - \boldsymbol{y})\mathbf{X}))^{\mathsf{T}} \tag{10.22}$$

where $\mathbf{X}$ is the $N \times D$ design matrix containing the examples $\boldsymbol{x}_n$ in each row.

10.2.3.4 Deriving the Hessian

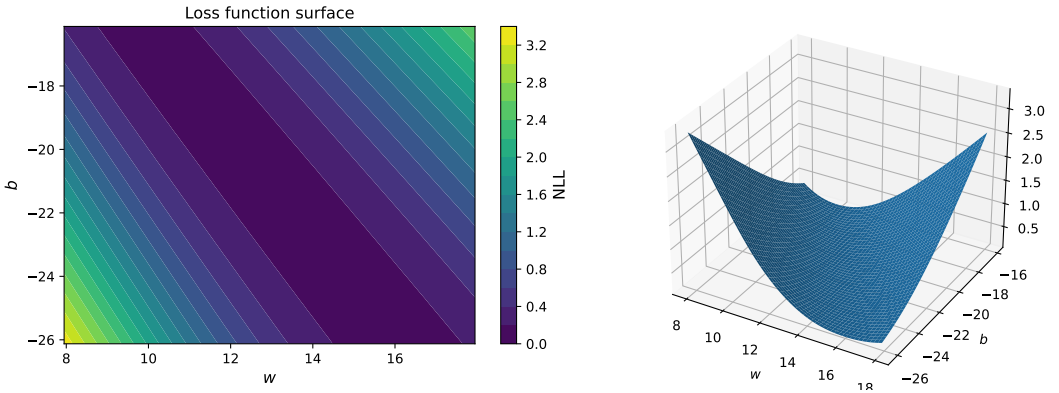
Gradient-based optimizers will find a stationary point where $\boldsymbol{g}(\boldsymbol{w}) = \mathbf{0}$. This could either be a global optimum or a local optimum. To be sure the stationary point is the global optimum, we must show that the objective is **convex**, for reasons we explain in Section 8.1.1.1. Intuitvely this means that the NLL has a **bowl shape**, with a unique lowest point, which is indeed the case, as illustrated in Figure 10.5b.

More formally, we must prove that the Hessian is positive semi-definite, which we now do. (See Chapter 7 for relevant background information on linear algebra.) One can show that the Hessian is given by

$$\mathbf{H}(\boldsymbol{w}) = \nabla_{\boldsymbol{w}} \nabla_{\boldsymbol{w}}^{\mathsf{T}} \text{NLL}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} (\mu_n(1 - \mu_n)\boldsymbol{x}_n)\boldsymbol{x}_n^{\mathsf{T}} = \frac{1}{N} \mathbf{X}^{\mathsf{T}} \mathbf{S} \mathbf{X} \tag{10.23}$$

where

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \ldots, \mu_N(1 - \mu_N)) \tag{10.24}$$



(a) (b)

Figure 10.5: NLL loss surface for binary logistic regression applied to Iris dataset with 1 feature and 1 bias term. The goal is to minimize the function. The global MLE is at the center of the plot. Generated by iris_logreg_loss_surface.ipynb.

We see that $\mathbf{H}$ is positive definite, since for any nonzero vector $\boldsymbol{v}$, we have

$$\boldsymbol{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{S}\mathbf{X}\boldsymbol{v} = (\boldsymbol{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{S}^{\frac{1}{2}})(\mathbf{S}^{\frac{1}{2}}\mathbf{X}\boldsymbol{v}) = ||\boldsymbol{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{S}^{\frac{1}{2}}||_2^2 > 0 \tag{10.25}$$

This follows since $\mu_n > 0$ for all n, because of the use of the sigmoid function. Consequently the NLL is strictly convex. However, in practice, values of μ_n which are close to 0 or 1 might cause the Hessian to be close to singular. We can avoid this by using ℓ_2 regularization, as we discuss in Section 10.2.7.

10.2.4 Stochastic gradient descent

Our goal is to solve the following optimization problem

$$\hat{\boldsymbol{w}} \triangleq \operatorname*{argmin}_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) \tag{10.26}$$

where $\mathcal{L}(\boldsymbol{w})$ is the loss function, in this case the negative log likelihood:

$$\text{NLL}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N}[y_n \log \mu_n + (1-y_n)\log(1-\mu_n)] \tag{10.27}$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, and $a_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n$ is the log odds.

There are many algorithms we could use to solve Equation (10.26), as we discuss in Chapter 8. Perhaps the simplest is to use stochastic gradient descent (Section 8.4). If we use a minibatch of size 1, then we get the following simple update equation:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta_t \nabla_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}_t) = \boldsymbol{w}_t - \eta_t(\mu_n - y_n)\boldsymbol{x}_n \tag{10.28}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where we replaced the average over all N examples in the gradient of Equation (10.21) with a single stochastically chosen sample n. (The index n changes with t.)

Since we know the objective is convex (see Section 10.2.3.4), then one can show that this procedure will converge to the global optimum, provided we decay the learning rate at the appropriate rate (see Section 8.4.3). We can improve the convergence speed using variance reduction techniques such as SAGA (Section 8.4.5.2).

10.2.5 Perceptron algorithm

A **perceptron**, first introduced in [Ros58], is a deterministic binary classifier of the following form:

$$f(\boldsymbol{x}_n; \boldsymbol{\theta}) = \mathbb{I}\left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + b > 0\right) \tag{10.29}$$

This can be seen to be a limiting case of a binary logistic regression classifier, in which the sigmoid function $\sigma(a)$ is replaced by the Heaviside step function $H(a) \triangleq \mathbb{I}(a > 0)$. See Figure 2.10 for a comparison of these two functions.

Since the Heaviside function is not differentiable, we cannot use gradient-based optimization methods to fit this model. However, Rosenblatt proposed the **perceptron learning algorithm** instead. The basic idea is to start with random weights, and then iteratively update them whenever the model makes a prediction mistake. More precisely, we update the weights using

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta_t(\hat{y}_n - y_n)\boldsymbol{x}_n \tag{10.30}$$

where $(\boldsymbol{x}_n, y_n)$ is the labeled example sampled at iteration t, and η_t is the learning rate or step size. (We can set the step size to 1, since the magnitude of the weights does not affect the decision boundary.) See perceptron_demo_2d.ipynb for a simple implementation of this algorithm.

The perceptron update rule in Equation (10.30) has an intuitive interpretation: if the prediction is correct, no change is made, otherwise we move the weights in a direction so as to make the correct answer more likely. More precisely, if $y_n = 1$ and $\hat{y}_n = 0$, we have $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \boldsymbol{x}_n$, and if $y_n = 0$ and $\hat{y}_n = 1$, we have $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \boldsymbol{x}_n$.

By comparing Equation (10.30) to Equation (10.28), we see that the perceptron update rule is equivalent to the SGD update rule for binary logistic regression using the approximation where we replace the soft probabilities $\mu_n = p(y_n = 1|\boldsymbol{x}_n)$ with hard labels $\hat{y}_n = f(\boldsymbol{x}_n)$. The advantage of the perceptron method is that we don't need to compute probabilities, which can be useful when the label space is very large. The disadvantage is that the method will only converge when the data is linearly separable [Nov62], whereas SGD for minimizing the NLL for logistic regression will always converge to the globally optimal MLE, even if the data is not linearly separable.

In Section 13.2, we will generalize perceptrons to nonlinear functions, thus significantly enhancing their usefulness.

10.2.6 Iteratively reweighted least squares

Gradient descent is a **first order** optimization method, which means it only uses first order gradients to navigate through the loss landscape. This can be slow, especially when some directions of space point steeply downhill, whereas other have a shallower gradient, as is the case in Figure 10.5a. In such problems, it can be much faster to use a **second order** optimization method, that takes the curvature of the space into account.

We discuss such methods in more detail in Section 8.3. Here we just consider a simple second order method that works well for logistic regression. We focus on the full batch setting (so we assume N is small), since it is harder to make second order methods work in the stochastic setting (see e.g., [Byr+16; Liu+18b] for some methods).

The classic second-order method is **Newton's method**. This consists of updates of the form

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta_t \mathbf{H}_t^{-1} \boldsymbol{g}_t \tag{10.31}$$

where

$$\mathbf{H}_t \triangleq \nabla^2 \mathcal{L}(\boldsymbol{w})|_{\boldsymbol{w}_t} = \nabla^2 \mathcal{L}(\boldsymbol{w}_t) = \mathbf{H}(\boldsymbol{w}_t) \tag{10.32}$$

is assumed to be positive-definite to ensure the update is well-defined. If the Hessian is exact, we can set the step size to $\eta_t = 1$.

We now apply this method to logistic regression. Recall from Section 10.2.3.3 that the gradient and Hessian are given by

$$\nabla_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} (\mu_n - y_n) \boldsymbol{x}_n \tag{10.33}$$

$$\mathbf{H} = \frac{1}{N} \mathbf{X}^\mathsf{T} \mathbf{S} \mathbf{X} \tag{10.34}$$

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1-\mu_1), \ldots, \mu_N(1-\mu_N)) \tag{10.35}$$

Hence the Newton update has the form

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \mathbf{H}^{-1} \boldsymbol{g}_t \tag{10.36}$$

$$= \boldsymbol{w}_t + (\mathbf{X}^\mathsf{T} \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\mathsf{T} (\boldsymbol{y} - \boldsymbol{\mu}_t) \tag{10.37}$$

$$= (\mathbf{X}^\mathsf{T} \mathbf{S}_t \mathbf{X})^{-1} \left[(\mathbf{X}^\mathsf{T} \mathbf{S}_t \mathbf{X}) \boldsymbol{w}_t + \mathbf{X}^\mathsf{T} (\boldsymbol{y} - \boldsymbol{\mu}_t) \right] \tag{10.38}$$

$$= (\mathbf{X}^\mathsf{T} \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\mathsf{T} \left[\mathbf{S}_t \mathbf{X} \boldsymbol{w}_t + \boldsymbol{y} - \boldsymbol{\mu}_t \right] \tag{10.39}$$

$$= (\mathbf{X}^\mathsf{T} \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\mathsf{T} \mathbf{S}_t \boldsymbol{z}_t \tag{10.40}$$

where we have defined the **working response** as

$$\boldsymbol{z}_t \triangleq \mathbf{X} \boldsymbol{w}_t + \mathbf{S}_t^{-1} (\boldsymbol{y} - \boldsymbol{\mu}_t) \tag{10.41}$$

and $\mathbf{S}_t = \text{diag}(\mu_{t,n}(1-\mu_{t,n}))$. Since $\mathbf{S}_t$ is a diagonal matrix, we can rewrite the targets in component form as follows:

$$z_{t,n} = \boldsymbol{w}_t^\mathsf{T} \boldsymbol{x}_n + \frac{y_n - \mu_{t,n}}{\mu_{t,n}(1-\mu_{t,n})} \tag{10.42}$$

Equation (10.40) is an example of a weighted least squares problem (Section 11.2.2.4), which is a minimizer of

$$\sum_{n=1}^{N} S_{t,n} (z_{t,n} - \boldsymbol{w}_t^\mathsf{T} \boldsymbol{x}_n)^2 \tag{10.43}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

Algorithm 10.1: Iteratively reweighted least squares (IRLS)

$\boldsymbol{w} = \mathbf{0}$
repeat
  for $n = 1 : N$ **do**
    $a_n = \boldsymbol{w}^\mathsf{T} \boldsymbol{x}_n$
    $\mu_n = \sigma(a_n)$
    $s_n = \mu_n(1 - \mu_n)$
    $z_n = a_n + \frac{y_n - \mu_n}{s_n}$
  $\mathbf{S} = \text{diag}(s_{1:N})$
  $\boldsymbol{w} = (\mathbf{X}^\mathsf{T}\mathbf{S}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{S}\boldsymbol{z}$
until *converged*

The overall method is therefore known as the **iteratively reweighted least squares** (**IRLS**) algorithm, since at each iteration we solve a weighted least squares problem, where the weight matrix $\mathbf{S}_t$ changes at each iteration. See Algorithm 10.1 for some pseudocode.

Note that **Fisher scoring** is the same as IRLS except we replace the Hessian of the actual log-likelihood with its expectation, i.e., we use the Fisher information matrix (Section 4.7.2) instead of $\mathbf{H}$. Since the Fisher information matrix is independent of the data, it can be precomputed, unlike the Hessian, which must be reevaluated at every iteration. This can be faster for problems with many parameters.

10.2.7 MAP estimation

In Figure 10.4, we saw how logistic regression can overfit when there are too many parameters compared to training examples. This is a consequence of the ability of maximum likelihood to find weights that force the decision boundary to "wiggle" in just the right way so as to curve around the examples. To get this behavior, the weights often need to be set to large values. For example, in Figure 10.4, when we use degree $K = 1$, we find that the MLE for the two input weights (ignoring the bias) is

$$\hat{\boldsymbol{w}} = [0.51291712, 0.11866937] \tag{10.44}$$

When we use degree $K = 2$, we get

$$\hat{\boldsymbol{w}} = [2.27510513, 0.05970325, 11.84198867, 15.40355969, 2.51242311] \tag{10.45}$$

And when $K = 4$, we get

$$\hat{\boldsymbol{w}} = [-3.07813766, \cdots, -59.03196044, 51.77152431, 10.25054164] \tag{10.46}$$

One way to reduce such overfitting is to prevent the weights from becoming so large. We can do this by using a zero-mean Gaussian prior, $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\mathbf{0}, C\mathbf{I})$, and then using MAP estimation, as we discussed in Section 4.5.3. The new training objective becomes

$$\mathcal{L}(\boldsymbol{w}) = \text{NLL}(\boldsymbol{w}) + \lambda||\boldsymbol{w}||_2^2 \tag{10.47}$$

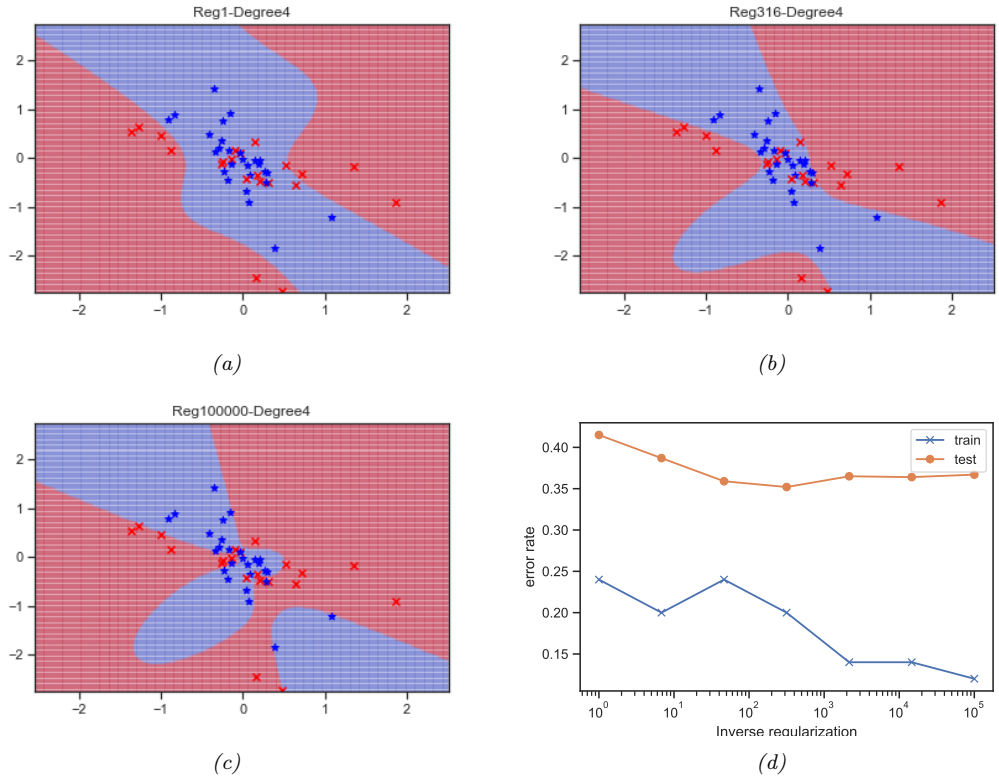


Figure 10.6: Weight decay with variance C applied to two-class, two-dimensional logistic regression problem with a degree 4 polynomial. (a) $C = 1$. (b) $C = 316$. (c) $C = 100,000$. (d) Train and test error vs C. Generated by logreg_poly_demo.ipynb.

where $||\boldsymbol{w}||_2^2 = \sum_{d=1}^{D} w_d^2$ and $\lambda = 1/C$. This is called ℓ_2 **regularization** or **weight decay**. The larger the value of λ, the more the parameters are penalized for being "large" (deviating from the zero-mean prior), and thus the less flexible the model. See Figure 10.6 for an illustration.

We can compute the MAP estimate by slightly modifying the input to the above gradient-based optimization algorithms. The gradient and Hessian of the penalized negative log likelihood have the following forms:

$$\text{PNLL}(\boldsymbol{w}) = \text{NLL}(\boldsymbol{w}) + \lambda \boldsymbol{w}^\mathsf{T} \boldsymbol{w} \tag{10.48}$$

$$\nabla_{\boldsymbol{w}} \text{PNLL}(\boldsymbol{w}) = \boldsymbol{g}(\boldsymbol{w}) + 2\lambda \boldsymbol{w} \tag{10.49}$$

$$\nabla^2_{\boldsymbol{w}} \text{PNLL}(\boldsymbol{w}) = \mathbf{H}(\boldsymbol{w}) + 2\lambda \mathbf{I} \tag{10.50}$$

where $\boldsymbol{g}(\boldsymbol{w})$ is the gradient and $\mathbf{H}(\boldsymbol{w})$ is the Hessian of the unpenalized NLL.

For an interesting exercise related to ℓ_2 regularized logistic regression, see Exercise 10.2.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

10.2.8 Standardization

In Section 10.2.7, we use an isotropic prior $\mathcal{N}(\boldsymbol{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$ to prevent overfitting. This implicitly encodes the assumption that we expect all weights to be similar in magnitude, which in turn encodes the assumption we expect all input features to be similar in magnitude. However, in many datasets, input features are on different scales. In such cases, it is common to **standardize** the data, to ensure each feature has mean 0 and variance 1. We can do this by subtracting the mean and dividing by the standard deviation of each feature, as follows:

$$\text{standardize}(x_{nd}) = \frac{x_{nd} - \hat{\mu}_d}{\hat{\sigma}_d} \tag{10.51}$$

$$\hat{\mu}_d = \frac{1}{N}\sum_{n=1}^{N} x_{nd} \tag{10.52}$$

$$\hat{\sigma}_d^2 = \frac{1}{N}\sum_{n=1}^{N}(x_{nd} - \hat{\mu}_d)^2 \tag{10.53}$$

An alternative is to use **min-max scaling**, in which we rescale the inputs so they lie in the interval $[0, 1]$. Both methods ensure the features are comparable in magnitude, which can help with model fitting and inference, even if we don't use MAP estimation. (See Section 11.7.5 for a discussion of this point.)

10.3 Multinomial logistic regression

Multinomial logistic regression is a discriminative classification model of the following form:

$$p(y|\boldsymbol{x}, \boldsymbol{\theta}) = \text{Cat}(y|\text{softmax}(\mathbf{W}\boldsymbol{x} + \boldsymbol{b})) \tag{10.54}$$

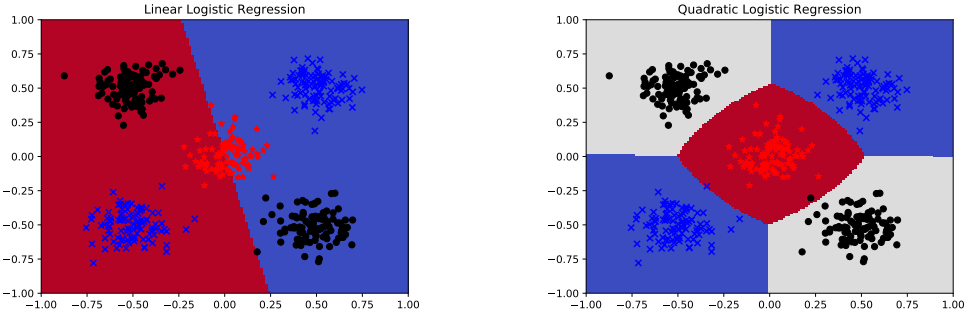
where $\boldsymbol{x} \in \mathbb{R}^D$ is the input vector, $y \in \{1, \ldots, C\}$ is the class label, softmax() is the softmax function (Section 2.5.2), $\mathbf{W}$ is a $C \times D$ weight matrix, $\boldsymbol{b}$ is C-dimensional bias vector, $\boldsymbol{\theta} = (\mathbf{W}, \boldsymbol{b})$ are all the parameters. (We will henceforth assume we have prepended each $\boldsymbol{x}$ with a 1, and added $\boldsymbol{b}$ to the first column of $\mathbf{W}$, so this simplifies to $\boldsymbol{\theta} = \mathbf{W}$.)

If we let $\boldsymbol{a} = \mathbf{W}\boldsymbol{x}$ be the C-dimensional vector of **logits**, then we can rewrite the above as follows:

$$p(y = c|\boldsymbol{x}, \boldsymbol{\theta}) = \frac{e^{a_c}}{\sum_{c'=1}^{C} e^{a_{c'}}} \tag{10.55}$$

Because of the normalization condition $\sum_{c=1}^{C} p(y_n = c|\boldsymbol{x}_n, \boldsymbol{\theta}) = 1$, we can set $\boldsymbol{w}_C = \mathbf{0}$. (For example, in binary logistic regression, where $C = 2$, we only learn a single weight vector.) Therefore the parameters $\boldsymbol{\theta}$ correspond to a weight matrix $\mathbf{W}$ of size $(C - 1) \times D$, where $\boldsymbol{x}_n \in \mathbb{R}^D$.

Note that this model assumes the labels are mutually exclusive, i.e., there is only one true label. For some applications (e.g., **image tagging**), we want to predict one or more labels for an input; in this case, the output space is the set of subsets of $\{1, \ldots, C\}$. This is called **multi-label classification**, as opposed to **multi-class classification**. This can be viewed as a bit vector, $\mathcal{Y} = \{0, 1\}^C$, where the c'th output is set to 1 if the c'th tag is present. We can tackle this using a modified version of



(a) (b)

Figure 10.7: Example of 3-class logistic regression with 2d inputs. (a) Original features. (b) Quadratic features. Generated by logreg_ multiclass_ demo.ipynb.

binary logistic regression with multiple outputs:

$$p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}) = \prod_{c=1}^{C} \text{Ber}(y_c|\sigma(\boldsymbol{w}_c^{\mathsf{T}}\boldsymbol{x})) \tag{10.56}$$

10.3.1 Linear and nonlinear classifiers

Logistic regression computes linear decision boundaries in the input space, as shown in Figure 10.7(a) for the case where $\boldsymbol{x} \in \mathbb{R}^2$ and we have $C = 3$ classes. However, we can always transform the inputs in some way to create nonlinear boundaries. For example, suppose we replace $\boldsymbol{x} = (x_1, x_2)$ by

$$\boldsymbol{\phi}(\boldsymbol{x}) = [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2] \tag{10.57}$$

This lets us create quadratic decision boundaries, as illustrated in Figure 10.7(b).

10.3.2 Maximum likelihood estimation

In this section, we discuss how to compute the maximum likelihood estimate (MLE) by minimizing the negative log likelihood (NLL).

10.3.2.1 Objective

The NLL is given by

$$\text{NLL}(\boldsymbol{\theta}) = -\frac{1}{N}\log \prod_{n=1}^{N}\prod_{c=1}^{C} \mu_{nc}^{y_{nc}} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C} y_{nc}\log \mu_{nc} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{H}_{ce}(\boldsymbol{y}_n, \boldsymbol{\mu}_n) \tag{10.58}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\mu_{nc} = p(y_{nc} = 1|\boldsymbol{x}_n, \boldsymbol{\theta}) = \text{softmax}(f(\boldsymbol{x}_n, \boldsymbol{\theta}))_c$, $\boldsymbol{y}_n$ is the one-hot encoding of the label (so $y_{nc} = \mathbb{I}(y_n = c)$), and $\mathbb{H}_{ce}(\boldsymbol{y}_n, \boldsymbol{\mu}_n)$ is the cross-entropy:

$$\mathbb{H}_{ce}(\boldsymbol{p}, \boldsymbol{q}) = -\sum_{c=1}^{C} p_c \log q_c \tag{10.59}$$

10.3.2.2 Optimizing the objective

To find the optimum, we need to solve $\nabla_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) = \mathbf{0}$, where $\boldsymbol{w}$ is a vectorized version of the weight matrix $\mathbf{W}$, and where we are ignoring the bias term for notational simplicity. We can find such a stationary point using any gradient-based optimizer; we give some examples below. But first we derive the gradient and Hessian, and then prove that the objective is convex.

10.3.2.3 Deriving the gradient

To derive the gradient of the NLL, we need to use the Jacobian of the softmax function, which is as follows (see Exercise 10.1 for the proof):

$$\frac{\partial \mu_c}{\partial a_j} = \mu_c(\delta_{cj} - \mu_j) \tag{10.60}$$

where $\delta_{cj} = \mathbb{I}(c = j)$. For example, if we have 3 classes, the Jacobian matrix is given by

$$\left[\frac{\partial \mu_c}{\partial a_j}\right]_{cj} = \begin{pmatrix} \mu_1(1-\mu_1) & -\mu_1\mu_2 & -\mu_1\mu_3 \\ -\mu_2\mu_1 & \mu_2(1-\mu_2) & -\mu_2\mu_3 \\ -\mu_3\mu_1 & -\mu_3\mu_2 & \mu_3(1-\mu_3) \end{pmatrix} \tag{10.61}$$

In matrix form, this can be written as

$$\frac{\partial \boldsymbol{\mu}}{\partial \boldsymbol{a}} = (\boldsymbol{\mu}\mathbf{1}^\mathsf{T}) \odot (\mathbf{I} - \mathbf{1}\boldsymbol{\mu}^\mathsf{T}) \tag{10.62}$$

where $\odot$ is elementwise product, $\boldsymbol{\mu}\mathbf{1}^\mathsf{T}$ copies $\boldsymbol{\mu}$ across each column, and $\mathbf{1}\boldsymbol{\mu}^\mathsf{T}$ copies $\boldsymbol{\mu}$ across each row.

We now derive the gradient of the NLL for a single example, indexed by n. To do this, we flatten the $D \times C$ weight matrix into a vector $\boldsymbol{w}$ of size CD (or $(C-1)D$ if we freeze one of the classes to have zero weight) by concatenating the rows, and then transposing into a column vector. We use $\boldsymbol{w}_j$ to denote the vector of weights associated with class j. The gradient wrt this vector is giving by the

following (where we use the Kronecker delta notation, δ_{jc}, which equals 1 if $j = c$ and 0 otherwise):

$$\nabla_{\boldsymbol{w}_j} \text{NLL}_n = \sum_c \frac{\partial \text{NLL}_n}{\partial \mu_{nc}} \frac{\partial \mu_{nc}}{\partial a_{nj}} \frac{\partial a_{nj}}{\partial \boldsymbol{w}_j} \tag{10.63}$$

$$= -\sum_c \frac{y_{nc}}{\mu_{nc}} \mu_{nc}(\delta_{jc} - \mu_{nj})\boldsymbol{x}_n \tag{10.64}$$

$$= \sum_c y_{nc}(\mu_{nj} - \delta_{jc})\boldsymbol{x}_n \tag{10.65}$$

$$= (\sum_c y_{nc})\mu_{nj}\boldsymbol{x}_n - \sum_c \delta_{jc} y_{nj}\boldsymbol{x}_n \tag{10.66}$$

$$= (\mu_{nj} - y_{nj})\boldsymbol{x}_n \tag{10.67}$$

We can repeat this computation for each class, to get the full gradient vector. The gradient of the overall NLL is obtained by summing over examples, to give the $D \times C$ matrix

$$\boldsymbol{g}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}_n (\boldsymbol{\mu}_n - \boldsymbol{y}_n)^\mathsf{T} \tag{10.68}$$

This has the same form as in the binary logistic regression case, namely an error term times the input.

10.3.2.4 Deriving the Hessian

Exercise 10.1 asks you to show that the Hessian of the NLL for multinomial logistic regression is given by

$$\mathbf{H}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} (\text{diag}(\boldsymbol{\mu}_n) - \boldsymbol{\mu}_n \boldsymbol{\mu}_n^\mathsf{T}) \otimes (\boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T}) \tag{10.69}$$

where $\mathbf{A} \otimes \mathbf{B}$ is the Kronecker product (Section 7.2.5). In other words, the block c, c' submatrix is given by

$$\mathbf{H}_{c,c'}(\boldsymbol{w}) = \frac{1}{N} \sum_n \mu_{nc}(\delta_{c,c'} - \mu_{n,c'}) \boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T} \tag{10.70}$$

For example, if we have 3 features and 2 classes, this becomes

$$\mathbf{H}(\boldsymbol{w}) = \frac{1}{N} \sum_n \begin{pmatrix} \mu_{n1} - \mu_{n1}^2 & -\mu_{n1}\mu_{n2} \\ -\mu_{n1}\mu_{n2} & \mu_{n2} - \mu_{n2}^2 \end{pmatrix} \otimes \begin{pmatrix} x_{n1}x_{n1} & x_{n1}x_{n2} & x_{n1}x_{n3} \\ x_{n2}x_{n1} & x_{n2}x_{n2} & x_{n2}x_{n3} \\ x_{n3}x_{n1} & x_{n3}x_{n2} & x_{n3}x_{n3} \end{pmatrix} \tag{10.71}$$

$$= \frac{1}{N} \sum_n \begin{pmatrix} (\mu_{n1} - \mu_{n1}^2)\mathbf{X}_n & -\mu_{n1}\mu_{n2}\mathbf{X}_n \\ -\mu_{n1}\mu_{n2}\mathbf{X}_n & (\mu_{n2} - \mu_{n2}^2)\mathbf{X}_n \end{pmatrix} \tag{10.72}$$

where $\mathbf{X}_n = \boldsymbol{x}_n \boldsymbol{x}_n^\mathsf{T}$. Exercise 10.1 also asks you to show that this is a positive definite matrix, so the objective is convex.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

10.3.3 Gradient-based optimization

It is straightforward to use the gradient in Section 10.3.2.3 to derive the SGD algorithm. Similarly, we can use the Hessian in Section 10.3.2.4 to derive a second-order optimization method. However, computing the Hessian can be expensive, so it is common to approximate it using quasi-Newton methods, such as limited memory BFGS. (BFGS stands for Broyden, Fletcher, Goldfarb and Shanno.) See Section 8.3.2 for details. Another approach, which is similar to IRLS, is described in Section 10.3.4.

All of these methods rely on computing the gradient of the log-likelihood, which in turn requires computing normalized probabilities, which can be computed from the logits vector $\boldsymbol{a} = \mathbf{W}\boldsymbol{x}$ using

$$p(y = c|\boldsymbol{x}) = \exp(a_c - \text{lse}(\boldsymbol{a})) \tag{10.73}$$

where lse is the log-sum-exp function defined in Section 2.5.4. For this reason, many software libraries define a version of the cross-entropy loss that takes unnormalized logits as input.

10.3.4 Bound optimization

In this section, we consider an approach for fitting logistic regression using a class of algorithms known as bound optimization, which we describe in Section 8.7. The basic idea is to iteratively construct a lower bound on the function you want to maximize, and then to update the bound, so it "pushes up" on the true function. Optimizing the bound is often easier than updating the function directly.

If $\ell(\boldsymbol{\theta})$ is a concave function we want to maximize, then one way to obtain a valid lower bound is to use a bound on its Hessian, i.e., to find a negative definite matrix $\mathbf{B}$ such that $\mathbf{H}(\boldsymbol{\theta}) \succ \mathbf{B}$. In this case, one can show that

$$\ell(\boldsymbol{\theta}) \geq \ell(\boldsymbol{\theta}^t) + (\boldsymbol{\theta} - \boldsymbol{\theta}^t)^\mathsf{T}\boldsymbol{g}(\boldsymbol{\theta}^t) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^t)^\mathsf{T}\mathbf{B}(\boldsymbol{\theta} - \boldsymbol{\theta}^t) \tag{10.74}$$

where $\boldsymbol{g}(\boldsymbol{\theta}^t) = \nabla\ell(\boldsymbol{\theta}^t)$. Defining $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$ as the right-hand-side of Equation (10.74), the update becomes

$$\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \mathbf{B}^{-1}\boldsymbol{g}(\boldsymbol{\theta}^t) \tag{10.75}$$

This is similar to a Newton update, except we use $\mathbf{B}$, which is a fixed matrix, rather than $\mathbf{H}(\boldsymbol{\theta}^t)$, which changes at each iteration. This can give us some of the advantages of second order methods at lower computational cost.

Let us now apply this to logistic regression, following [Kri+05], Let $\boldsymbol{\mu}_n(\boldsymbol{w}) = [p(y_n = 1|\boldsymbol{x}_n, \boldsymbol{w}), \ldots, p(y_n = C|\boldsymbol{x}_n, \boldsymbol{w})]$ and $\boldsymbol{y}_n = [\mathbb{I}(y_n = 1), \ldots, \mathbb{I}(y_n = C)]$. We want to *maximize* the log-likelihood, which is as follows:

$$\ell(\boldsymbol{w}) = \sum_{n=1}^{N}\left[\sum_{c=1}^{C} y_{nc}\boldsymbol{w}_c^\mathsf{T}\boldsymbol{x}_n - \log\sum_{c=1}^{C}\exp(\boldsymbol{w}_c^\mathsf{T}\boldsymbol{x}_n)\right] \tag{10.76}$$

The gradient is given by the following (see Section 10.3.2.3 for details of the derivation):

$$\boldsymbol{g}(\boldsymbol{w}) = \sum_{n=1}^{N}(\boldsymbol{y}_n - \boldsymbol{\mu}_n(\boldsymbol{w})) \otimes \boldsymbol{x}_n \tag{10.77}$$

where $\otimes$ denotes Kronecker product (which, in this case, is just outer product of the two vectors). The Hessian is given by the following (see Section 10.3.2.4 for details of the derivation):

$$\mathbf{H}(\boldsymbol{w}) = -\sum_{n=1}^{N}(\text{diag}(\boldsymbol{\mu}_n(\boldsymbol{w})) - \boldsymbol{\mu}_n(\boldsymbol{w})\boldsymbol{\mu}_n(\boldsymbol{w})^\mathsf{T}) \otimes (\boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T}) \tag{10.78}$$

We can construct a lower bound on the Hessian, as shown in [Boh92]:

$$\mathbf{H}(\boldsymbol{w}) \succ -\frac{1}{2}[\mathbf{I} - \mathbf{1}\mathbf{1}^\mathsf{T}/C] \otimes \left(\sum_{n=1}^{N}\boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T}\right) \triangleq \mathbf{B} \tag{10.79}$$

where $\mathbf{I}$ is a C-dimensional identity matrix, and $\mathbf{1}$ is a C-dimensional vector of all 1s.[1] In the binary case, this becomes

$$\mathbf{H}(\boldsymbol{w}) \succ -\frac{1}{2}\left(1 - \frac{1}{2}\right)\left(\sum_{n=1}^{N}\boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T}\right) = -\frac{1}{4}\mathbf{X}^\mathsf{T}\mathbf{X} \tag{10.80}$$

This follows since $\mu_n \leq 0.5$ so $-(\mu_n - \mu_n^2) \geq -0.25$.

We can use this lower bound to construct an MM algorithm to find the MLE. The update becomes

$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^t - \mathbf{B}^{-1}\boldsymbol{g}(\boldsymbol{w}^t) \tag{10.81}$$

This iteration can be faster than IRLS (Section 10.2.6) since we can precompute $\mathbf{B}^{-1}$ in time independent of N, rather than having to invert the Hessian at each iteration. For example, let us consider the binary case, so $\boldsymbol{g}^t = \nabla\ell(\boldsymbol{w}^t) = \mathbf{X}^\mathsf{T}(\boldsymbol{y} - \boldsymbol{\mu}^t)$, where $\boldsymbol{\mu}^t = [p_n(\boldsymbol{w}^t), (1 - p_n(\boldsymbol{w}^t))]_{n=1}^N$. The update becomes

$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^t - 4(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\boldsymbol{g}^t \tag{10.82}$$

Compare this to Equation (10.37), which has the following form:

$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^t - \mathbf{H}^{-1}\boldsymbol{g}(\boldsymbol{w}^t) = \boldsymbol{w}^t - (\mathbf{X}^\mathsf{T}\mathbf{S}^t\mathbf{X})^{-1}\boldsymbol{g}^t \tag{10.83}$$

where $\mathbf{S}^t = \text{diag}(\boldsymbol{\mu}^t \odot (1 - \boldsymbol{\mu}^t))$. We see that Equation (10.82) is faster to compute, since we can precompute the constant matrix $(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}$.

10.3.5 MAP estimation

In Section 10.2.7 we discussed the benefits of ℓ_2 regularization for binary logistic regression. These benefits hold also in the multi-class case. However, there is also an additional, and surprising, benefit to do with **identifiability** of the parameters, as pointed out in [HTF09, Ex.18.3]. (We say that the parameters are identifiable if there is a unique value that maximizes the likelihood; equivalently, we require that the NLL be *strictly* convex.)

1. If we enforce that $\boldsymbol{w}_C = \mathbf{0}$, we can use $C - 1$ dimensions for these vectors / matrices.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

To see why identifiability is an issue, recall that multiclass logistic regression has the form

$$p(y=c|\boldsymbol{x},\mathbf{W}) = \frac{\exp(\boldsymbol{w}_c^T\boldsymbol{x})}{\sum_{k=1}^{C}\exp(\boldsymbol{w}_k^T\boldsymbol{x})} \tag{10.84}$$

where $\mathbf{W}$ is a $C \times D$ weight matrix. We can arbitrarily define $\boldsymbol{w}_c = \mathbf{0}$ for one of the classes, say $c = C$, since $p(y = C|\boldsymbol{x},\mathbf{W}) = 1 - \sum_{c=1}^{C-1} p(y = c|\boldsymbol{x},\boldsymbol{w})$. In this case, the model has the form

$$p(y=c|\boldsymbol{x},\mathbf{W}) = \frac{\exp(\boldsymbol{w}_c^T\boldsymbol{x})}{1+\sum_{k=1}^{C-1}\exp(\boldsymbol{w}_k^T\boldsymbol{x})} \tag{10.85}$$

If we don't "clamp" one of the vectors to some constant value, the parameters will be unidentifiable.

However, suppose we don't clamp $\boldsymbol{w}_c = \mathbf{0}$, so we are using Equation 10.84, but we add ℓ_2 regularization by optimizing

$$\text{PNLL}(\mathbf{W}) = -\sum_{n=1}^{N}\log p(y_n|\boldsymbol{x}_n,\mathbf{W}) + \lambda\sum_{c=1}^{C}||\boldsymbol{w}_c||_2^2 \tag{10.86}$$

where we have absorbed the $1/N$ term into λ. At the optimum we have $\sum_{c=1}^{C}\hat{w}_{cj} = 0$ for $j = 1 : D$, so the weights automatically satisfy a sum-to-zero constraint, thus making them uniquely identifiable.

To see why, note that at the optimum we have

$$\nabla\text{NLL}(\boldsymbol{w}) + 2\lambda\boldsymbol{w} = \mathbf{0} \tag{10.87}$$

$$\sum_n(\boldsymbol{y}_n - \boldsymbol{\mu}_n)\otimes\boldsymbol{x}_n = \lambda\boldsymbol{w} \tag{10.88}$$

Hence for any feature dimension j we have

$$\lambda\sum_c w_{cj} = \sum_n\sum_c(y_{nc}-\mu_{nc})x_{nj} = \sum_n(\sum_c y_{nc} - \sum_c\mu_{nc})x_{nj} = \sum_n(1-1)x_{nj} = 0 \tag{10.89}$$

Thus if $\lambda > 0$ we have $\sum_c\hat{w}_{cj} = 0$, so the weights will sum to zero across classes for each feature dimension.

10.3.6 Maximum entropy classifiers

Recall that the multinomial logistic regression model can be written as

$$p(y=c|\boldsymbol{x},\mathbf{W}) = \frac{\exp(\boldsymbol{w}_c^\intercal\boldsymbol{x})}{Z(\boldsymbol{w},\boldsymbol{x})} = \frac{\exp(\boldsymbol{w}_c^\intercal\boldsymbol{x})}{\sum_{c'=1}^{C}\exp(\boldsymbol{w}_{c'}^\intercal\boldsymbol{x})} \tag{10.90}$$

where $Z(\boldsymbol{w},\boldsymbol{x}) = \sum_c\exp(\boldsymbol{w}_c^\intercal\boldsymbol{x})$ is the partition function (normalization constant). This uses the same features, but a different weight vector, for every class. There is a slight extension of this model that allows us to use features that are class-dependent. This model can be written as

$$p(y=c|\boldsymbol{x},\boldsymbol{w}) = \frac{1}{Z(\boldsymbol{w},\boldsymbol{x})}\exp(\boldsymbol{w}^\intercal\boldsymbol{\phi}(\boldsymbol{x},c)) \tag{10.91}$$

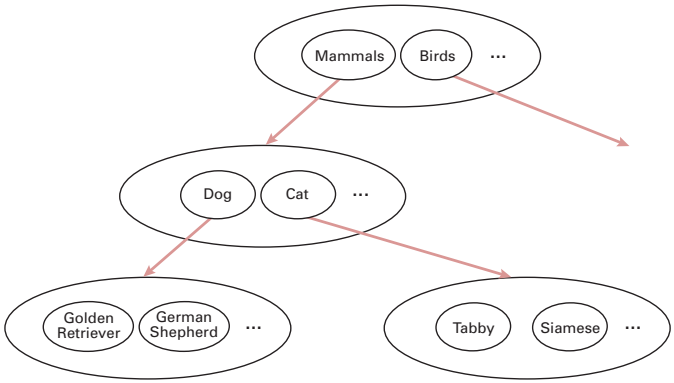


Figure 10.8: A simple example of a label hierarchy. Nodes within the same ellipse have a mutual exclusion relationship between them.

where $\boldsymbol{\phi}(\boldsymbol{x}, c)$ is the feature vector for class c. This is called a **maximum entropy classifer**, or **maxent classifier** for short. (The origin of this term is explained in Section 3.4.4.)

Maxent classifiers include multinomial logistic regression as a special case. To see this let $\boldsymbol{w} = [\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C]$, and define the feature vector as follows:

$$\boldsymbol{\phi}(\boldsymbol{x}, c) = [\mathbf{0}, \ldots, \boldsymbol{x}, \ldots, \mathbf{0}] \tag{10.92}$$

where $\boldsymbol{x}$ is embedded in the c'th block, and the remaining blocks are zero. In this case, $\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x}, c) = \boldsymbol{w}_c^\mathsf{T}\boldsymbol{x}$, so we recover multinomial logistic regression.

Maxent classifiers are very widely used in the field of natural language processing. For example, consider the problem of **semantic role labeling**, where we classify a word $\boldsymbol{x}$ into a semantic role y, such as person, place or thing. We might define (binary) features such as the following:

$$\phi_1(\boldsymbol{x}, y) = \mathbb{I}\left(y = \text{person} \wedge \boldsymbol{x} \text{ occurs after "Mr." or "Mrs"}\right) \tag{10.93}$$
$$\phi_2(\boldsymbol{x}, y) = \mathbb{I}\left(y = \text{person} \wedge \boldsymbol{x} \text{ is in whitelist of common names}\right) \tag{10.94}$$
$$\phi_3(\boldsymbol{x}, y) = \mathbb{I}\left(y = \text{place} \wedge \boldsymbol{x} \text{ is in Google maps}\right) \tag{10.95}$$
$$\vdots$$

We see that the features we use depend on the label.

There are two main ways of creating these features. The first is to manually specify many possibly useful features using various templates, and then use a feature selection algorithm, such as the group lasso method of Section 11.4.7. The second is to incrementally add features to the model, using a heuristic feature generation method.

10.3.7 Hierarchical classification

Sometimes the set of possible labels can be structured into a **hierarchy** or **taxonomy**. For example, we might want to predict what kind of an animal is in an image: it could be a dog or a cat; if it is a

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

dog, it could be a golden retriever or a German shepherd, etc. Intuitively, it makes sense to try to predict the most precise label for which we are confident [Den+12], that is, the system should "hedge its bets".

One simple way to achieve this, proposed in [RF17], is as follows. First, create a model with a binary output label for every possible node in the tree. Before training the model, we will use **label smearing**, so that a label is propagated to all of its parents (**hypernyms**). For example, if an image is labeled "golden retriever", we will also label it "dog". If we train a multi-label classifier (which produces a vector $p(\boldsymbol{y}|\boldsymbol{x})$ of binary labels) on such smeared data, it will perform hierarchical classification, predicting a set of labels at different levels of abstraction.

However, this method could predict "golden retriever", "cat" and "bird" all with probability 1.0, since the model does not capture the fact that some labels are mutually exclusive. To prevent this, we can add a mutual exclusion constraint between all label nodes which are siblings, as shown in Figure 10.8. For example, this model enforces that $p(\text{mammal}|\boldsymbol{x}) + p(\text{bird}|\boldsymbol{x}) = 1$, since these two labels are children of the root node. We can further partition the mammal probability into dogs and cats, so we have $p(\text{dog}|\boldsymbol{x}) + p(\text{cat}|\boldsymbol{x}) = p(\text{mammal}|\boldsymbol{x})$.

[Den+14; Din+15] generalize the above method by using a conditional graphical model where the graph structure can be more complex than a tree. In addition, they allow for soft constraints between labels, in addition to hard constraints.

10.3.8 Handling large numbers of classes

In this section, we discuss some issues that arise when there are a large number of potential labels, e.g., if the labels correspond to words from a language.

10.3.8.1 Hierarchical softmax

In regular softmax classifiers, computing the normalization constant, which is needed to compute the gradient of the log likelihood, takes $O(C)$ time, which can become the bottleneck if C is large. However, if we structure the labels as a tree, we can compute the probability of any label in $O(\log C)$ time, by multiplying the probabilities of each edge on the path from the root to the leaf. For example, consider the tree in Figure 10.9. We have

$$p(y = \text{I'm}|C) = 0.57 \times 0.68 \times 0.72 = 0.28 \tag{10.96}$$

Thus we replace the "flat" output softmax with a tree-structured sequence of binary classifiers. This is called **hierarchical softmax** [Goo01; MB05].

A good way to structure such a tree is to use Huffman encoding, where the most frequent labels are placed near the top of the tree, as suggested in [Mik+13a]. (For a different appproach, based on clustering the most common labels together, see [Gra+17]. And for yet another approach, based on sampling labels, see [Tit16].)

10.3.8.2 Class imbalance and the long tail

Another issue that often arises when there are a large number of classes is that for most classes, we may have very few examples. More precisely, if N_c is the number of examples of class c, then the empirical distribution $p(N_1, \ldots, N_C)$ may have a **long tail**. The result is an extreme form of **class**

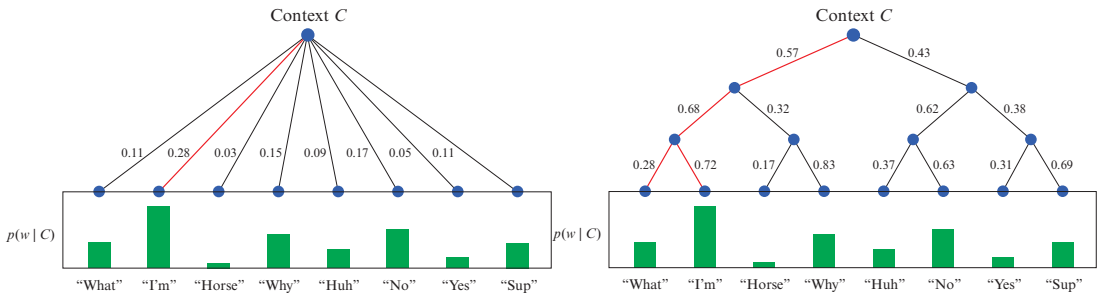


Figure 10.9: A flat and hierarchical softmax model $p(w|C)$, where C are the input features (context) and w is the output label (word). Adapted from https://www.quora.com/What-is-hierarchical-softmax.

imbalance (see e.g., [ASR15]). Since the rare classes will have a smaller effect on the overall loss than the common classes, the model may "focus its attention" on the common classes.

One method that can help is to set the bias terms $\boldsymbol{b}$ such that $\text{softmax}(\boldsymbol{b})_c = N_c/N$; such a model will match the empirical label prior even when using weights of $\boldsymbol{w} = \boldsymbol{0}$. We can then "subtract off" the prior term by using **logit adjustment** [Men+21], which ensures good performance across all groups.

Another common approach is to resample the data to make it more balanced, before (or during) training. In particular, suppose we sample a datapoint from class c with probability

$$p_c = \frac{N_c^q}{\sum_i^C N_i^q} \tag{10.97}$$

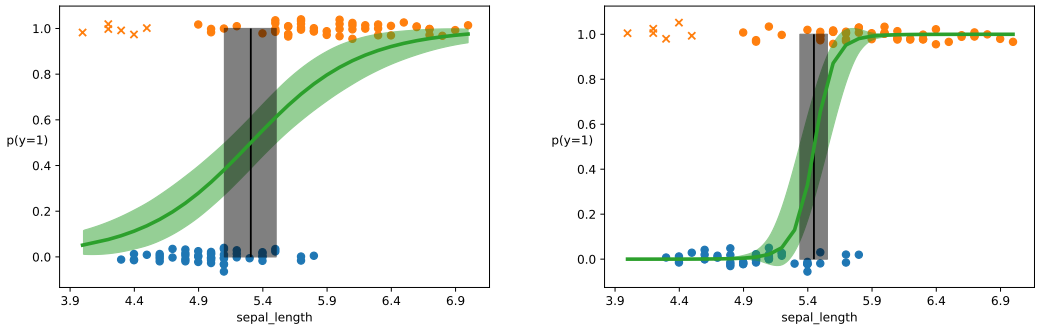
If we set $q = 1$, we recover standard **instance-balanced sampling**, where $p_c \propto N_c$; the common classes will be sampled more than rare classes. If we set $q = 0$, we recover **class-balanced sampling**, where $p_c = 1/C$; this can be thought of as first sampling a class uniformly at random, and then sampling an instance of this class. Finally, we can consider other options, such as $q = 0.5$, which is known as **square-root sampling** [Mah+18].

Yet another method that is simple and can easily handle the long tail is to use the **nearest class mean classifier**. This has the form

$$f(\boldsymbol{x}) = \underset{c}{\text{argmin}} \, ||\boldsymbol{x} - \boldsymbol{\mu}_c||_2^2 \tag{10.98}$$

where $\boldsymbol{\mu}_c = \frac{1}{N_c} \sum_{n:y_n=c} \boldsymbol{x}_n$ is the mean of the features belonging to class c. This induces a softmax posterior, as we discussed in Section 9.2.5. We can get much better results if we first use a neural network (see Part III) to learn good features, by training a DNN classifier with cross-entropy loss on the original unbalanced data. We then replace $\boldsymbol{x}$ with $\boldsymbol{\phi}(\boldsymbol{x})$ in Equation (10.98). This simple approach can give very good performance on long-tailed distributions [Kan+20].

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 10.10: (a) Logistic regression on some data with outliers (denoted by x). Training points have been (vertically) jittered to avoid overlapping too much. Vertical line is the decision boundary, and its posterior credible interval. (b) Same as (a) but using robust model, with a mixture likelihood. Adapted from Figure 4.13 of [Mar18]. Generated by logreg_iris_bayes_robust_1d_pymc3.ipynb.

10.4 Robust logistic regression *

Sometimes we have **outliers** in our data, which are often due to labeling errors, also called **label noise**. To prevent the model from being adversely affected by such contamination, we will use **robust logistic regression**. In this section, we discuss some approaches to this problem. (Note that the methods can also be applied to DNNs. For a more thorough survey of label noise, and how it impacts deep learning, see [Han+20].)

10.4.1 Mixture model for the likelihood

One of the simplest ways to define a robust logistic regression model is to modify the likelihood so that it predicts that each output label y is generated uniformly at random with probability π, and otherwise is generated using the usual conditional model. In the binary case, this becomes

$$p(y|\boldsymbol{x}) = \pi \text{Ber}(y|0.5) + (1-\pi)\text{Ber}(y|\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x})) \tag{10.99}$$

This approach, of using a mixture model for the observation model to make it robust, can be applied to many different models (e.g., DNNs).

We can fit this model using standard methods, such as SGD or Bayesian inference methods such as MCMC. For example, let us create a "contaminated" version of the 1d, two-class Iris dataset that we discussed in Section 4.6.7.2. We will add 6 examples of class 1 (Versicolor) with abnormally low sepal length. In Figure 10.10a, we show the results of fitting a standard (Bayesian) logistic regression model to this dataset. In Figure 10.10b, we show the results of fitting the above robust model. In the latter case, we see that the decision boundary is similar to the one we inferred from non-contaminated data, as shown in Figure 4.20b. We also see that the posterior uncertainty about the decision boundary's location is smaller than when using a non-robust model.

10.4.2 Bi-tempered loss

In this section, we present an approach to robust logistic regression proposed in [Ami+19].

The first observation is that examples that are far from the decision boundary, but mislabeled, will have undue adverse affect on the model if the loss function is convex [LS10]. This can be overcome by replacing the usual cross entropy loss with a "tempered" version, that uses a temperature parameter $0 \leq t_1 < 1$ to ensure the loss from outliers is bounded. In particular, consider the standard relative entropy loss function:

$$\mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \mathbb{H}_{ce}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \sum_c y_c \log \hat{y}_c \tag{10.100}$$

where $\boldsymbol{y}$ is the true label distribution (often one-hot) and $\hat{\boldsymbol{y}}$ is the predicted distribution. We define the **tempered cross entropy** loss as follows:

$$\mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \sum_c \left[y_c (\log_{t_1} y_c - \log_{t_1} \hat{y}_c) - \frac{1}{2-t_1}(y_c^{2-t_1} - \hat{y}_c^{2-t_1}) \right] \tag{10.101}$$

which simplifes to the following when the true distribution $\boldsymbol{y}$ is one-hot, with all its mass on class c:

$$\mathcal{L}(c, \hat{\boldsymbol{y}}) = -\log_{t_1} \hat{y}_c - \frac{1}{2-t_1}\left(1 - \sum_{c'=1}^{C} \hat{y}_{c'}^{2-t_1}\right) \tag{10.102}$$

Here $\log_t$ is tempered version of the log function:

$$\log_t(x) \triangleq \frac{1}{1-t}(x^{1-t} - 1) \tag{10.103}$$

This is mononotically increasing and concave, and reduces to the standard (natural) logarithm when $t = 1$. (Similarly, tempered cross entropy reduces to standard cross entropy when $t = 1$.) However, the tempered log function is bounded from below by $-1/(1-t)$ for $0 \leq t < 1$, and hence the cross entropy loss is bounded from above (see Figure 10.11).

The second observation is that examples that are near the decision boundary, but mislabeled, need to use a transfer function (that maps from activations $\mathbb{R}^C$ to probabilities $[0,1]^C$) that has heavier tails than the softmax, which is based on the exponential, so it can "look past" the neighborhood of the immediate examples. In particular, the standard softmax is defined by

$$\hat{y}_c = \frac{a_c}{\sum_{c'=1}^{C} \exp(a_{c'})} = \exp\left[a_c - \log \sum_{c'=1}^{C} \exp(a_{c'}) \right] \tag{10.104}$$

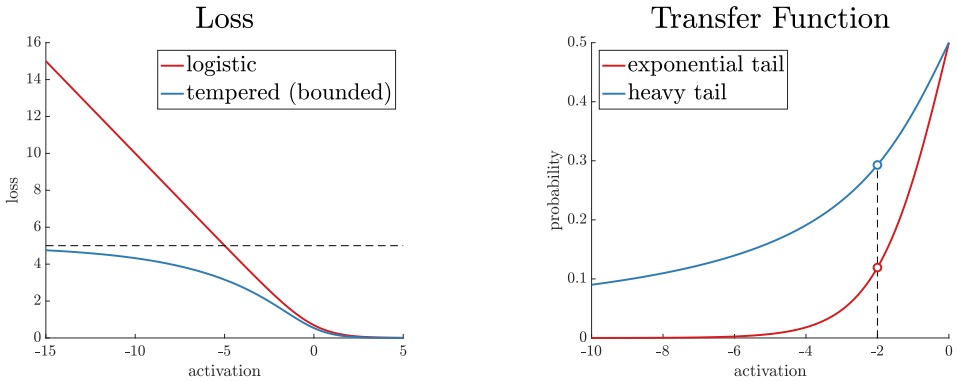
where $\boldsymbol{a}$ is the logits vector. We can make a heavy tailed version by using the **tempered softmax**, which uses a temperature parameter $t_2 > 1 > t_1$ as follows:

$$\hat{y}_c = \exp_{t_2}(a_c - \lambda_{t_2}(\boldsymbol{a})) \tag{10.105}$$

where

$$\exp_t(x) \triangleq [1 + (1-t)x]_+^{1/(1-t)} \tag{10.106}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) *(b)*

Figure 10.11: (a) Illustration of logistic and tempered logistic loss with $t_1 = 0.8$. (b) Illustration of sigmoid and tempered sigmoid transfer function with $t_2 = 2.0$. From https://ai.googleblog.com/2019/08/bi-tempered-logistic-loss-for-training.html. Used with kind permission of Ehsan Amid.

is a tempered version of the exponential function. (This reduces to the standard exponental function as $t \to 1$.) In Figure 10.11(right), we show that the tempered softmax (in the two-class case) has heavier tails, as desired.

All that remains is a way to compute $\lambda_{t_2}(\boldsymbol{a})$. This must satisfy the following fixed point equation:

$$\sum_{c=1}^{C} \exp_{t_2}(a_c - \lambda(\boldsymbol{a})) = 1 \tag{10.107}$$

We can solve for λ using binary search, or by using the iterative procedure in Algorithm 10.2.

Algorithm 10.2: Iterative algorithm for computing $\lambda(\boldsymbol{a})$ in Equation (10.107). From [AWS19].

Input: logits $\boldsymbol{a}$, temperature $t > 1$
$\mu := \max(\boldsymbol{a})$
$\boldsymbol{a} := \boldsymbol{a} - \mu$
while $\boldsymbol{a}$ *not converged* **do**
  $Z(\boldsymbol{a}) := \sum_{c=1}^{C} \exp_t(a_c)$
  $\boldsymbol{a} := Z(\boldsymbol{a})^{1-t}(\boldsymbol{a} - \mu\mathbf{1})$
Return $-\log_t \frac{1}{Z(\boldsymbol{a})} + \mu$

Combining the tempered softmax with the tempered cross entropy results in a method called **bi-tempered logistic regression**. In Figure 10.12, we show an example of this in 2d. The top row is standard logistic regression, the bottom row is bi-tempered. The first column is clean data. The second column has label noise near the boundary. The robust version uses $t_1 = 1$ (standard cross entropy) but $t_2 = 4$ (tempered softmax with heavy tails). The third column has label noise far

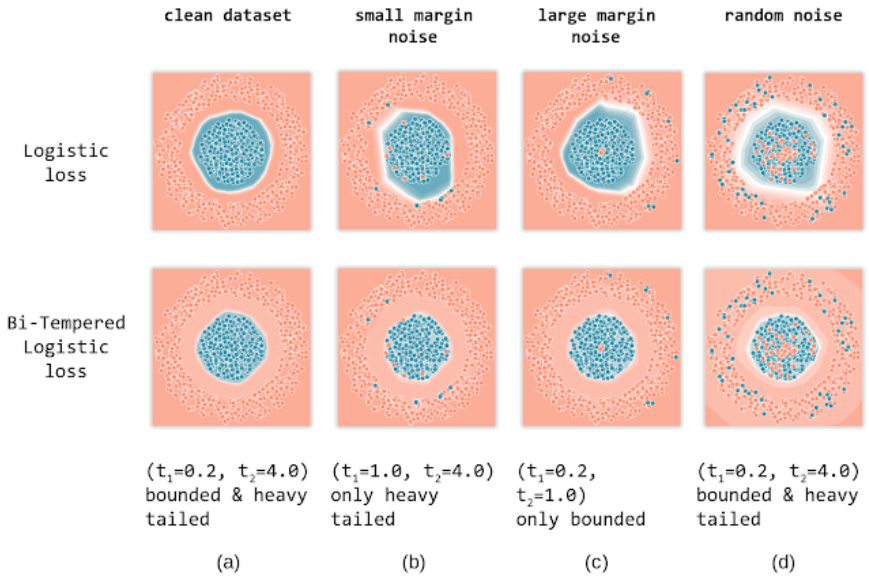


Figure 10.12: Illustration of standard and bi-tempered logistic regression on data with label noise. From https://ai.googleblog.com/2019/08/bi-tempered-logistic-loss-for-training.html . Used with kind permission of Ehsan Amid.

from the boundary. The robust version uses $t_1 = 0.2$ (tempered cross entropy with bounded loss) but $t_2 = 1$ (standard softmax). The fourth column has both kinds of noise; in this case, the robust version uses $t_1 = 0.2$ and $t_2 = 4$.

10.5 Bayesian logistic regression *

So far we have focused on point estimates of the parameters, either the MLE or the MAP estimate. However, in some cases we want to compute the posterior, $p(\boldsymbol{w}|\mathcal{D})$, in order to capture our uncertainty. This can be particularly useful in settings where we have little data, and where choosing the wrong decision may be costly.

Unlike with linear regression, it is not possible to compute the posterior exactly for a logistic regression model. A wide range of approximate algorithms can be used,. In this section, we use one of the simplest, known as the Laplace approximation (Section 4.6.8.2). See the sequel to this book, [Mur23] for more advanced approximations.

10.5.1 Laplace approximation

As we discuss in Section 4.6.8.2, the Laplace approximation approximates the posterior using a Gaussian. The mean of the Gaussian is equal to the MAP estimate $\hat{\boldsymbol{w}}$, and the covariance is equal to the inverse Hessian $\mathbf{H}$ computed at the MAP estimate, i.e., $p(\boldsymbol{w}|\mathcal{D}) \approx \mathcal{N}(\boldsymbol{w}|\hat{\boldsymbol{w}}, \mathbf{H}^{-1})$, We can find the mode using a standard optimization method (see Section 10.2.7), and then we can use the results

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

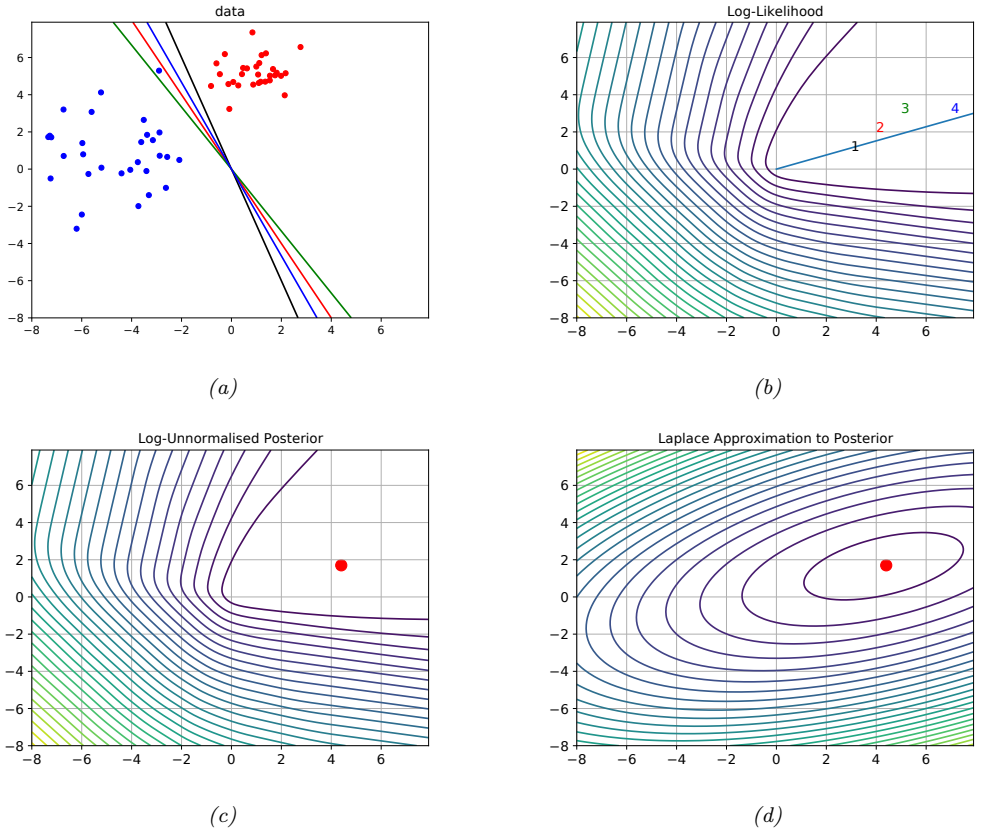


Figure 10.13: (a) Illustration of the data. (b) Log-likelihood for a logistic regression model. The line is drawn from the origin in the direction of the MLE (which is at infinity). The numbers correspond to 4 points in parameter space, corresponding to the lines in (a). (c) Unnormalized log posterior (assuming vague spherical prior). (d) Laplace approximation to posterior. Adapted from a figure by Mark Girolami. Generated by logreg_laplace_demo.ipynb.

from Section 10.2.3.4 to compute the Hessian at the mode.

As an example, consider the data illustrated in Figure 10.13(a). There are many parameter settings that correspond to lines that perfectly separate the training data; we show 4 example lines. The likelihood surface is shown in Figure 10.13(b). The diagonal line connects the origin to the point in the grid with maximum likelihood, $\hat{\boldsymbol{w}}_{\text{mle}} = (8.0, 3.4)$. (The unconstrained MLE has $||\boldsymbol{w}|| = \infty$, as we discussed in Section 10.2.7; this point can be obtained by following the diagonal line infinitely far to the right.)

For each decision boundary in Figure 10.13(a), we plot the corresponding parameter vector in Figure 10.13(b). These parameters values are $\boldsymbol{w}_1 = (3, 1)$, $\boldsymbol{w}_2 = (4, 2)$, $\boldsymbol{w}_3 = (5, 3)$, and $\boldsymbol{w}_4 = (7, 3)$. These points all approximately satisfy $\boldsymbol{w}_i(1)/\boldsymbol{w}_i(2) \approx \hat{\boldsymbol{w}}_{\text{mle}}(1)/\hat{\boldsymbol{w}}_{\text{mle}}(2)$, and hence are close to the

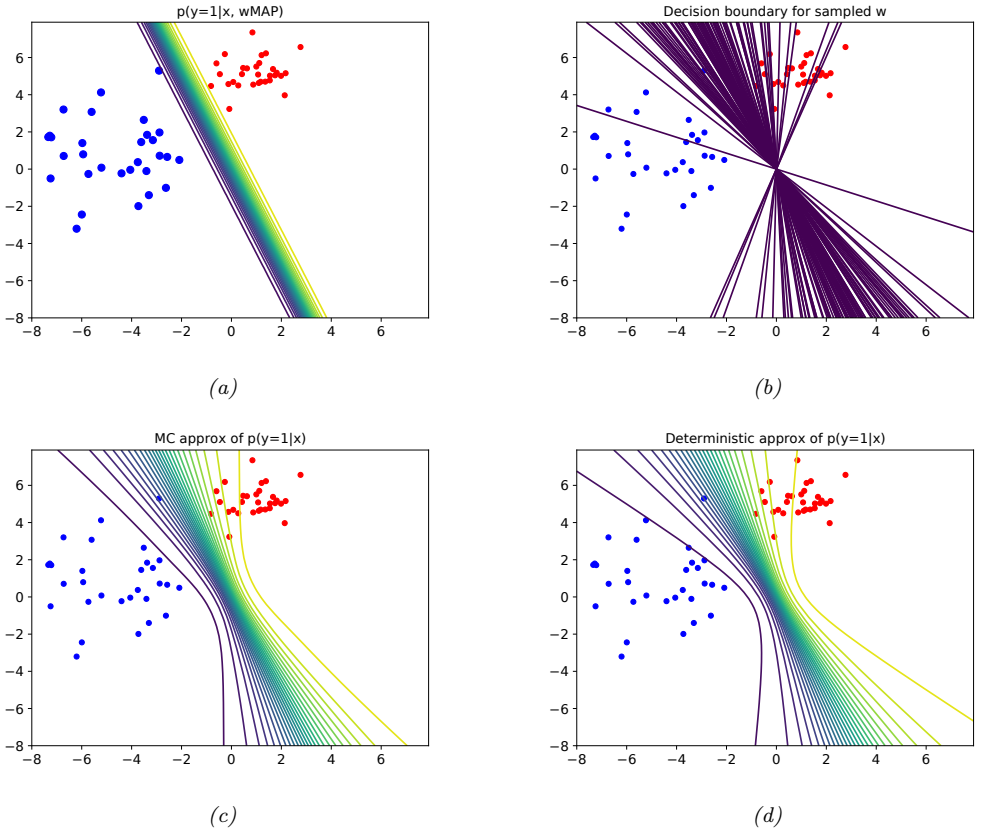


Figure 10.14: Posterior predictive distribution for a logistic regression model in 2d. (a): contours of $p(y = 1|\boldsymbol{x}, \hat{\boldsymbol{w}}_{map})$. (b): samples from the posterior predictive distribution. (c): Averaging over these samples. (d): moderated output (probit approximation). Adapted from a figure by Mark Girolami. Generated by logreg_ laplace_ demo.ipynb.

orientation of the maximum likelihood decision boundary. The points are ordered by increasing weight norm (3.16, 4.47, 5.83, and 7.62).

To ensure a unique solution, we use a (spherical) Gaussian prior centered at the origin, $\mathcal{N}(\boldsymbol{w}|\mathbf{0}, \sigma^2\mathbf{I})$. The value of σ^2 controls the strength of the prior. If we set $\sigma^2 = 0$, we force the MAP estimate to be $\boldsymbol{w} = \mathbf{0}$; this will result in maximally uncertain predictions, since all points $\boldsymbol{x}$ will produce a predictive distribution of the form $p(y = 1|\boldsymbol{x}) = 0.5$. If we set $\sigma^2 = \infty$, the prior becomes uninformative, and MAP estimate becomes the MLE, resulting in minimally uncertain predictions. (In particular, all positively labeled points will have $p(y = 1|\boldsymbol{x}) = 1.0$, and all negatively labeled points will have $p(y = 1|\boldsymbol{x}) = 0.0$, since the data is separable.) As a compromise (to make a nice illustration), we pick the value $\sigma^2 = 100$.

Multiplying this prior by the likelihood results in the unnormalized posterior shown in Fig-

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

ure 10.13(c). The MAP estimate is shown by the red dot. The Laplace approximation to this posterior is shown in Figure 10.13(d). We see that it gets the mode correct (by construction), but the shape of the posterior is somewhat distorted. (The southwest-northeast orientation captures uncertainty about the magnitude of $\boldsymbol{w}$, and the southeast-northwest orientation captures uncertainty about the orientation of the decision boundary.)

In Figure 10.14, we show contours of the posterior predictive distribution. Figure 10.14(a) shows the plugin approximation using the MAP estimate. We see that there is no uncertainty about the decision boundary, even though we are generating probabilistic predictions over the labels. Figure 10.14(b) shows what happens when we plug in samples from the Gaussian posterior. Now we see that there is considerable uncertainty about the orientation of the "best" decision boundary. Figure 10.14(c) shows the average of these samples. By averaging over multiple predictions, we see that the uncertainty in the decision boundary "splays out" as we move further from the training data. Figure 10.14(d) shows that the probit approximation gives very similar results to the Monte Carlo approximation.

10.5.2 Approximating the posterior predictive

The posterior $p(\boldsymbol{w}|\mathcal{D})$ tells us everything we know about the parameters of the model given the data. However, in machine learning applications, the main task of interest is usually to predict an output y given an input $\boldsymbol{x}$, rather than to try to understand the parameters of our model. Thus we need to compute the **posterior predictive distribution**

$$p(y|\boldsymbol{x}, \mathcal{D}) = \int p(y|\boldsymbol{x}, \boldsymbol{w}) p(\boldsymbol{w}|\mathcal{D}) d\boldsymbol{w} \tag{10.108}$$

As we discussed in Section 4.6.7.1, a simple approach to this is to first compute a point estimate $\hat{\boldsymbol{w}}$ of the parameters, such as the MLE or MAP estimate, and then to ignore all posterior uncertainty, by assuming $p(\boldsymbol{w}|\mathcal{D}) = \delta(\boldsymbol{w} - \hat{\boldsymbol{w}})$. In this case, the above integral reduces to the following plugin approximation:

$$p(\boldsymbol{y}|\boldsymbol{x}, \mathcal{D}) \approx \int p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{w}) \delta(\boldsymbol{w} - \hat{\boldsymbol{w}}) d\boldsymbol{w} = p(\boldsymbol{y}|\boldsymbol{x}, \hat{\boldsymbol{w}}) \tag{10.109}$$

However, if we want to compute uncertainty in our predictions, we should use a non-degenerate posterior. It is common to use a Gaussian posterior, as we will see. But we still need to approximate the integral in Equation (10.108). We discuss some approaches to this below.

10.5.2.1 Monte Carlo approximation

The simplest approach is to use a **Monte Carlo approximation** to the integral. This means we draw S samples from the posterior, $\boldsymbol{w}_s \sim p(\boldsymbol{w}|\mathcal{D})$. and then compute

$$p(y = 1|\boldsymbol{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^{S} \sigma(\boldsymbol{w}_s^{\mathsf{T}} \boldsymbol{x}) \tag{10.110}$$

10.5.2.2 Probit approximation

Although the Monte Carlo approximation is simple, it can be slow, since we need to draw S samples *at test time* for each input $\boldsymbol{x}$. Fortunately, if $p(\boldsymbol{w}|\mathcal{D}) = \mathcal{N}(\boldsymbol{w}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, there is a simple yet accurate

deterministic approximation, first suggested in [SL90]. To explain this approximation, we follow the presentation of [Bis06, p219]. The key observation is that the sigmoid function $\sigma(a)$ is similar in shape to the Gaussian cdf (see Section 2.6.1) $\Phi(a)$. In particular we have $\sigma(a) \approx \Phi(\lambda a)$, where $\lambda^2 = \pi/8$ ensures the two functions have the same slope at the origin. This is useful since we can integrate a Gaussian cdf wrt a Gaussian pdf exactly:

$$\int \Phi(\lambda a)\mathcal{N}(a|m,v)da = \Phi\left(\frac{m}{(\lambda^{-2}+v)^{\frac{1}{2}}}\right) = \Phi\left(\frac{\lambda m}{(1+\lambda^2 v)^{\frac{1}{2}}}\right) \approx \sigma(\kappa(v)m) \tag{10.111}$$

where we have defined

$$\kappa(v) \triangleq (1+\pi v/8)^{-\frac{1}{2}} \tag{10.112}$$

Thus if we define $a = \boldsymbol{x}^\mathsf{T}\boldsymbol{w}$, we have

$$p(y=1|\boldsymbol{x},\mathcal{D}) \approx \sigma(\kappa(v)m) \tag{10.113}$$

$$m = \mathbb{E}[a] = \boldsymbol{x}^\mathsf{T}\boldsymbol{\mu} \tag{10.114}$$

$$v = \mathbb{V}[a] = \mathbb{V}\left[\boldsymbol{x}^\mathsf{T}\boldsymbol{w}\right] = \boldsymbol{x}^\mathsf{T}\boldsymbol{\Sigma}\boldsymbol{x} \tag{10.115}$$

where we used Equation (2.165) in the last line. Since Φ is the inverse of the probit function, we will call this the **probit approximation**.

Using Equation (10.113) results in predictions that are less extreme (in terms of their confidence) than the plug-in estimate. To see this, note that $0 < \kappa(v) < 1$ and hence $\kappa(v)m < m$, so $\sigma(\kappa(v)m)$ is closer to 0.5 than $\sigma(m)$ is. However, the decision boundary itself will not be affected. To see this, note that the decision boundary is the set of points $\boldsymbol{x}$ for which $p(y=1|\boldsymbol{x},\mathcal{D}) = 0.5$. This implies $\kappa(v)m = 0$, which implies $m = \overline{\boldsymbol{w}}^\mathsf{T}\boldsymbol{x} = 0$; but this is the same as the decision boundary from the plugin estimate. Thus "being Bayesian" doesn't change the misclassification rate (in this case), but it does change the confidence estimates of the model, which can be important, as we illustrate in Section 10.5.1.

In the multiclass case we can use the **generalized probit approximation** [Gib97]:

$$p(y=c|\boldsymbol{x},\mathcal{D}) \approx \frac{\exp(\kappa(v_c)m_c)}{\sum_{c'}\exp(\kappa(v_{c'})m_{c'})} \tag{10.116}$$

$$m_c = \overline{\boldsymbol{m}}_c^\mathsf{T}\boldsymbol{x} \tag{10.117}$$

$$v_c = \boldsymbol{x}^\mathsf{T}\mathbf{V}_{c,c}\boldsymbol{x} \tag{10.118}$$

where κ is defined in Equation (10.112). Unlike the binary case, taking into account posterior covariance gives different predictions than the plug-in approach (see Exercise 3.10.3 of [RW06]).

For further approximations of Gaussian integrals combined with sigmoid and softmax functions, see [Dau17].

10.6 Exercises

Exercise 10.1 [Gradient and Hessian of log-likelihood for multinomial logistic regression]

a. Let $\mu_{ik} = \text{softmax}(\boldsymbol{\eta}_i)_k$, where $\boldsymbol{\eta}_i = \boldsymbol{w}^T\boldsymbol{x}_i$. Show that the Jacobian of the softmax is

$$\frac{\partial \mu_{ik}}{\partial \eta_{ij}} = \mu_{ik}(\delta_{kj} - \mu_{ij}) \tag{10.119}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

where $\delta_{kj} = I(k = j)$.

b. Hence show that the gradient of the NLL is given by

$$\nabla_{\boldsymbol{w}_c} \ell = \sum_i (y_{ic} - \mu_{ic}) \boldsymbol{x}_i \tag{10.120}$$

Hint: use the chain rule and the fact that $\sum_c y_{ic} = 1$.

c. Show that the block submatrix of the Hessian for classes c and c' is given by

$$\mathbf{H}_{c,c'} = -\sum_i \mu_{ic}(\delta_{c,c'} - \mu_{i,c'}) \boldsymbol{x}_i \boldsymbol{x}_i^T \tag{10.121}$$

Hence show that the Hessian of the NLL is positive definite.

Exercise 10.2 [Regularizing separate terms in 2d logistic regression †]

(Source: Jaakkola.)

a. Consider the data in Figure 10.15a, where we fit the model $p(y = 1|\boldsymbol{x}, \boldsymbol{w}) = \sigma(w_0 + w_1 x_1 + w_2 x_2)$. Suppose we fit the model by maximum likelihood, i.e., we minimize

$$J(\boldsymbol{w}) = -\ell(\boldsymbol{w}, \mathcal{D}_{\text{train}}) \tag{10.122}$$

where $\ell(\boldsymbol{w}, \mathcal{D}_{\text{train}})$ is the log likelihood on the training set. Sketch a possible decision boundary corresponding to $\hat{\boldsymbol{w}}$. (Copy the figure first (a rough sketch is enough), and then superimpose your answer on your copy, since you will need multiple versions of this figure). Is your answer (decision boundary) unique? How many classification errors does your method make on the training set?

b. Now suppose we regularize only the w_0 parameter, i.e., we minimize

$$J_0(\boldsymbol{w}) = -\ell(\boldsymbol{w}, \mathcal{D}_{\text{train}}) + \lambda w_0^2 \tag{10.123}$$

Suppose λ is a very large number, so we regularize w_0 all the way to 0, but all other parameters are unregularized. Sketch a possible decision boundary. How many classification errors does your method make on the training set? Hint: consider the behavior of simple linear regression, $w_0 + w_1 x_1 + w_2 x_2$ when $x_1 = x_2 = 0$.

c. Now suppose we heavily regularize only the w_1 parameter, i.e., we minimize

$$J_1(\boldsymbol{w}) = -\ell(\boldsymbol{w}, \mathcal{D}_{\text{train}}) + \lambda w_1^2 \tag{10.124}$$

Sketch a possible decision boundary. How many classification errors does your method make on the training set?

d. Now suppose we heavily regularize only the w_2 parameter. Sketch a possible decision boundary. How many classification errors does your method make on the training set?

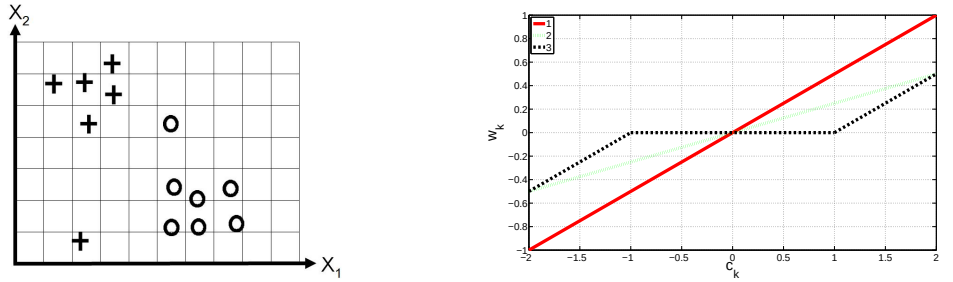
Exercise 10.3 [Logistic regression vs LDA/QDA †]

(Source: Jaakkola.) Suppose we train the following binary classifiers via maximum likelihood.

a. GaussI: A generative classifier, where the class-conditional densities are Gaussian, with both covariance matrices set to $\mathbf{I}$ (identity matrix), i.e., $p(\boldsymbol{x}|y = c) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_c, \mathbf{I})$. We assume $p(y)$ is uniform.

b. GaussX: as for GaussI, but the covariance matrices are unconstrained, i.e., $p(\boldsymbol{x}|y = c) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.

c. LinLog: A logistic regression model with linear features.



(a) (b)

Figure 10.15: (a) Data for logistic regression question. (b) Plot of $\hat{w}_k$ vs amount of correlation c_k for three different estimators.

d. QuadLog: A logistic regression model, using linear and quadratic features (i.e., polynomial basis function expansion of degree 2).

After training we compute the performance of each model M on the training set as follows:

$$L(M) = \frac{1}{n}\sum_{i=1}^{n} \log p(y_i|\boldsymbol{x}_i, \hat{\boldsymbol{\theta}}, M) \tag{10.125}$$

(Note that this is the *conditional* log-likelihood $p(y|\boldsymbol{x}, \hat{\boldsymbol{\theta}})$ and not the joint log-likelihood $p(y, \boldsymbol{x}|\hat{\boldsymbol{\theta}})$.) We now want to compare the performance of each model. We will write $L(M) \leq L(M')$ if model M *must* have lower (or equal) log likelihood (on the training set) than M', for any training set (in other words, M is worse than M', at least as far as training set logprob is concerned). For each of the following model pairs, state whether $L(M) \leq L(M')$, $L(M) \geq L(M')$, or whether no such statement can be made (i.e., M might sometimes be better than M' and sometimes worse); also, for each question, briefly (1-2 sentences) explain why.

a. GaussI, LinLog.
b. GaussX, QuadLog.
c. LinLog, QuadLog.
d. GaussI, QuadLog.
e. Now suppose we measure performance in terms of the average misclassification rate on the training set:

$$R(M) = \frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}(\boldsymbol{x}_i)) \tag{10.126}$$

Is it true in general that $L(M) > L(M')$ implies that $R(M) < R(M')$? Explain why or why not.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

11 Linear Regression

11.1 Introduction

In this chapter, we discuss **linear regression**, which is a very widely used method for predicting a real-valued output (also called the **dependent variable** or **target**) $y \in \mathbb{R}$, given a vector of real-valued inputs (also called **independent variables**, **explanatory variables**, or **covariates**) $\boldsymbol{x} \in \mathbb{R}^D$. The key property of the model is that the expected value of the output is assumed to be a linear function of the input, $\mathbb{E}[y|\boldsymbol{x}] = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}$, which makes the model easy to interpret, and easy to fit to data. We discuss nonlinear extensions later in this book.

11.2 Least squares linear regression

In this section, we discuss the most common form of linear regression model.

11.2.1 Terminology

The term "linear regression" usually refers to a model of the following form:

$$p(y|\boldsymbol{x}, \boldsymbol{\theta}) = \mathcal{N}(y|w_0 + \boldsymbol{w}^\mathsf{T}\boldsymbol{x}, \sigma^2) \tag{11.1}$$

where $\boldsymbol{\theta} = (w_0, \boldsymbol{w}, \sigma^2)$ are all the parameters of the model. (In statistics, the parameters w_0 and $\boldsymbol{w}$ are usually denoted by β_0 and $\boldsymbol{\beta}$.)

The vector of parameters $\boldsymbol{w}_{1:D}$ are known as the **weights** or **regression coefficients**. Each coefficient w_d specifies the change in the output we expect if we change the corresponding input feature x_d by one unit. For example, suppose x_1 is the age of a person, x_2 is their education level (represented as a continuous number), and y is their income. Thus w_1 corresponds to the increase in income we expect as someone becomes one year older (and hence get more experience), and w_2 corresponds to the increase in income we expect as someone's education level increases by one level. The term w_0 is the **offset** or **bias** term, and specifies the output value if all the inputs are 0. This captures the unconditional mean of the response, $w_0 = \mathbb{E}[y]$, and acts as a baseline. We will usually assume that $\boldsymbol{x}$ is written as $[1, x_1, \ldots, x_D]$, so we can absorb the offset term w_0 into the weight vector $\boldsymbol{w}$.

If the input is one-dimensional (so $D = 1$), the model has the form $f(\boldsymbol{x}; \boldsymbol{w}) = ax + b$, where $b = w_0$ is the intercept, and $a = w_1$ is the slope. This is called **simple linear regression**. If the input is multi-dimensional, $\boldsymbol{x} \in \mathbb{R}^D$ where $D > 1$, the method is called **multiple linear regression**. If the

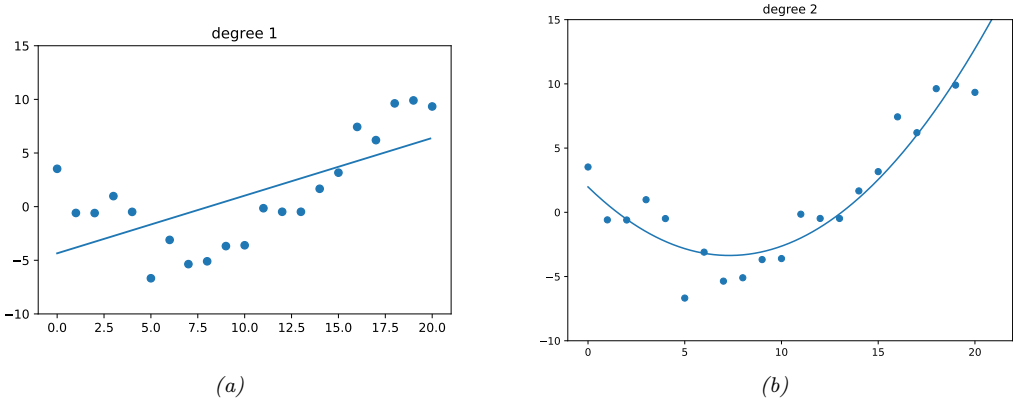


Figure 11.1: Polynomial of degrees 1 and 2 fit to 21 datapoints. Generated by linreg_poly_vs_degree.ipynb.

output is also multi-dimensional, $\boldsymbol{y} \in \mathbb{R}^J$, where $J > 1$, it is called **multivariate linear regression**,

$$p(\boldsymbol{y}|\boldsymbol{x}, \mathbf{W}) = \prod_{j=1}^{J} \mathcal{N}(y_j|\boldsymbol{w}_j^\mathsf{T}\boldsymbol{x}, \sigma_j^2) \tag{11.2}$$

See Exercise 11.1 for a simple numerical example.

In general, a straight line will not provide a good fit to most data sets. However, we can always apply a nonlinear transformation to the input features, by replacing $\boldsymbol{x}$ with $\boldsymbol{\phi}(\boldsymbol{x})$ to get

$$p(y|\boldsymbol{x}, \boldsymbol{\theta}) = \mathcal{N}(y|\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x}), \sigma^2) \tag{11.3}$$

As long as the parameters of the **feature extractor** $\boldsymbol{\phi}$ are fixed, the model remains *linear in the parameters*, even if it is not linear in the inputs. (We discuss ways to learn the feature extractor, and the final linear mapping, in Part III.)

As a simple example of a nonlinear transformation, consider the case of **polynomial regression**, which we introduced in Section 1.2.2.2. If the input is 1d, and we use a polynomial expansion of degree d, we get $\boldsymbol{\phi}(x) = [1, x, x^2, \ldots, x^d]$. See Figure 11.1 for an example. (See also Section 11.5 where we discuss splines.)

11.2.2 Least squares estimation

To fit a linear regression model to data, we will minimize the negative log likelihood on the training set. The objective function is given by

$$\text{NLL}(\boldsymbol{w}, \sigma^2) = -\sum_{n=1}^{N} \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2}(y_n - \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n)^2 \right) \right] \tag{11.4}$$

$$= \frac{1}{2\sigma^2} \sum_{n=1}^{N} (y_n - \hat{y}_n)^2 + \frac{N}{2} \log(2\pi\sigma^2) \tag{11.5}$$

where we have defined the predicted response $\hat{y}_n \triangleq \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n$. The MLE is the point where $\nabla_{\boldsymbol{w},\sigma}\text{NLL}(\boldsymbol{w},\sigma^2) = \mathbf{0}$. We can first optimize wrt $\boldsymbol{w}$, and then solve for the optimal σ.

In this section, we just focus on estimating the weights $\boldsymbol{w}$. In this case, the NLL is equal (up to irrelevant constants) to the **residual sum of squares**, which is given by

$$\text{RSS}(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n)^2 = \frac{1}{2}||\mathbf{X}\boldsymbol{w} - \boldsymbol{y}||_2^2 = \frac{1}{2}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y}) \tag{11.6}$$

We discuss how to optimize this below.

11.2.2.1 Ordinary least squares

From Equation (7.264) we can show that the gradient is given by

$$\nabla_{\boldsymbol{w}}\text{RSS}(\boldsymbol{w}) = \mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w} - \mathbf{X}^\mathsf{T}\boldsymbol{y} \tag{11.7}$$

Setting the gradient to zero and solving gives

$$\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w} = \mathbf{X}^\mathsf{T}\boldsymbol{y} \tag{11.8}$$

These are known as the **normal equations**, since, at the optimal solution, $\boldsymbol{y} - \mathbf{X}\boldsymbol{w}$ is normal (orthogonal) to the range of $\mathbf{X}$, as we explain in Section 11.2.2.2. The corresponding solution $\hat{\boldsymbol{w}}$ is the **ordinary least squares** (**OLS**) solution, which is given by

$$\hat{\boldsymbol{w}} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} \tag{11.9}$$

The quantity $\mathbf{X}^\dagger = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}$ is the (left) pseudo inverse of the (non-square) matrix $\mathbf{X}$ (see Section 7.5.3 for more details).

We can check that the solution is unique by showing that the Hessian is positive definite. In this case, the Hessian is given by

$$\mathbf{H}(\boldsymbol{w}) = \frac{\partial^2}{\partial \boldsymbol{w}^2}\text{RSS}(\boldsymbol{w}) = \mathbf{X}^\mathsf{T}\mathbf{X} \tag{11.10}$$

If $\mathbf{X}$ is full rank (so the columns of $\mathbf{X}$ are linearly independent), then $\mathbf{H}$ is positive definite, since for any $\boldsymbol{v} > \mathbf{0}$, we have

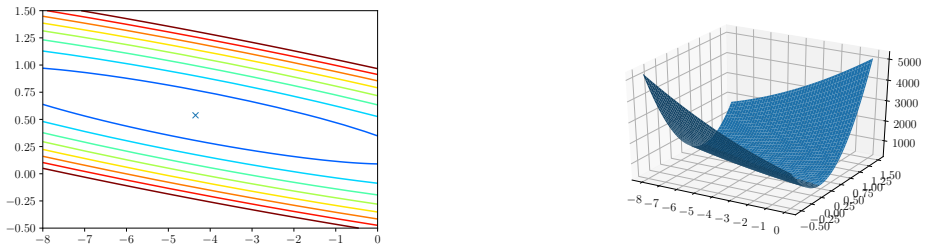
$$\boldsymbol{v}^\mathsf{T}(\mathbf{X}^\mathsf{T}\mathbf{X})\boldsymbol{v} = (\mathbf{X}\boldsymbol{v})^\mathsf{T}(\mathbf{X}\boldsymbol{v}) = ||\mathbf{X}\boldsymbol{v}||^2 > 0 \tag{11.11}$$

Hence in the full rank case, the least squares objective has a unique global minimum. See Figure 11.2 for an illustration.

11.2.2.2 Geometric interpretation of least squares

The normal equations have an elegant geometrical interpretation, deriving from Section 7.7, as we now explain. We will assume $N > D$, so there are more observations than unknowns. (This is known

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 11.2: (a) Contours of the RSS error surface for the example in Figure 11.1a. The blue cross represents the MLE. (b) Corresponding surface plot. Generated by linreg_contours_sse_plot.ipynb.

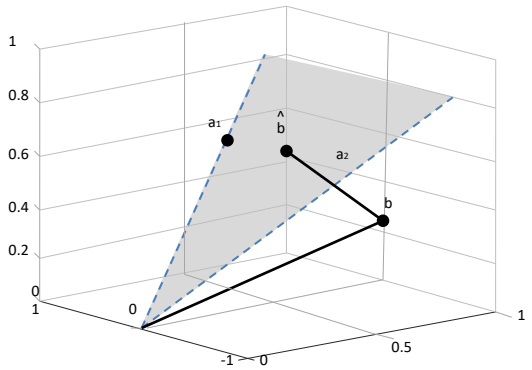


Figure 11.3: Graphical interpretation of least squares for $m = 3$ equations and $n = 2$ unknowns when solving the system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. $\boldsymbol{a}_1$ and $\boldsymbol{a}_2$ are the columns of $\mathbf{A}$, which define a 2d linear subspace embedded in $\mathbb{R}^3$. The target vector $\boldsymbol{b}$ is a vector in $\mathbb{R}^3$; its orthogonal projection onto the linear subspace is denoted $\hat{\boldsymbol{b}}$. The line from $\boldsymbol{b}$ to $\hat{\boldsymbol{b}}$ is the vector of residual errors, whose norm we want to minimize.

as an **overdetermined system**.) We seek a vector $\hat{\boldsymbol{y}} \in \mathbb{R}^N$ that lies in the linear subspace spanned by $\mathbf{X}$ and is as close as possible to $\boldsymbol{y}$, i.e., we want to find

$$\underset{\hat{\boldsymbol{y}} \in \text{span}(\{\boldsymbol{x}_{:,1},\ldots,\boldsymbol{x}_{:,d}\})}{\text{argmin}} \|\boldsymbol{y} - \hat{\boldsymbol{y}}\|_2. \tag{11.12}$$

where $\boldsymbol{x}_{:,d}$ is the d'th column of $\mathbf{X}$. Since $\hat{\boldsymbol{y}} \in \text{span}(\mathbf{X})$, there exists some weight vector $\boldsymbol{w}$ such that

$$\hat{\boldsymbol{y}} = w_1\boldsymbol{x}_{:,1} + \cdots + w_D\boldsymbol{x}_{:,D} = \mathbf{X}\boldsymbol{w} \tag{11.13}$$

To minimize the norm of the residual, $\boldsymbol{y} - \hat{\boldsymbol{y}}$, we want the residual vector to be orthogonal to every column of $\mathbf{X}$. Hence

$$\boldsymbol{x}_{:,d}^{\mathsf{T}}(\boldsymbol{y} - \hat{\boldsymbol{y}}) = 0 \Rightarrow \mathbf{X}^{\mathsf{T}}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w}) = \mathbf{0} \Rightarrow \boldsymbol{w} = (\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathsf{T}}\boldsymbol{y} \tag{11.14}$$

Hence our projected value of $\boldsymbol{y}$ is given by

$$\hat{\boldsymbol{y}} = \mathbf{X}\boldsymbol{w} = \mathbf{X}(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} \tag{11.15}$$

This corresponds to an **orthogonal projection** of $\boldsymbol{y}$ onto the column space of $\mathbf{X}$. For example, consider the case where we have $N = 3$ training examples, each of dimensionality $D = 2$. The training data defines a 2d linear subspace, defined by the 2 columns of $\mathbf{X}$, each of which is a point in 3d. We project $\boldsymbol{y}$, which is also a point in 3d, onto this 2d subspace, as shown in Figure 11.3.

The **projection matrix**

$$\text{Proj}(\mathbf{X}) \triangleq \mathbf{X}(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T} \tag{11.16}$$

is sometimes called the **hat matrix**, since $\hat{\boldsymbol{y}} = \text{Proj}(\mathbf{X})\boldsymbol{y}$. In the special case that $\mathbf{X} = \boldsymbol{x}$ is a column vector, the orthogonal projection of $\boldsymbol{y}$ onto the line $\boldsymbol{x}$ becomes

$$\text{Proj}(\boldsymbol{x})\boldsymbol{y} = \boldsymbol{x}\,\frac{\boldsymbol{x}^\mathsf{T}\boldsymbol{y}}{\boldsymbol{x}^\mathsf{T}\boldsymbol{x}} \tag{11.17}$$

11.2.2.3 Algorithmic issues

Recall that the OLS solution is

$$\hat{\boldsymbol{w}} = \mathbf{X}^\dagger\boldsymbol{y} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} \tag{11.18}$$

However, even if it is theoretically possible to compute the pseudo-inverse by inverting $\mathbf{X}^\mathsf{T}\mathbf{X}$, we should not do so for numerical reasons, since $\mathbf{X}^\mathsf{T}\mathbf{X}$ may be ill conditioned or singular.

A better (and more general) approach is to compute the pseudo-inverse using the SVD. Indeed, if you look at the source code for the function sklearn.linear_model.fit, you will see that it uses the scipy.linalg.lstsq function, which in turns calls DGELSD, which is an SVD-based solver implemented by the LAPACK library, written in Fortran.[1]

However, if $\mathbf{X}$ is tall and skinny (i.e., $N \gg D$), it can be quicker to use QR decomposition (Section 7.6.2). To do this, let $\mathbf{X} = \mathbf{QR}$, where $\mathbf{Q}^\mathsf{T}\mathbf{Q} = \mathbf{I}$. In Section 7.7, we show that OLS is equivalent to solving the system of linear equations $\mathbf{X}\boldsymbol{w} = \boldsymbol{y}$ in a way that minimizes $||\mathbf{X}\boldsymbol{w} - \boldsymbol{y}||_2^2$. (If $N = D$ and $\mathbf{X}$ is full rank, the equations have a unique solution, and the error will be 0.) Using QR decomposition, we can rewrite this system of equations as follows:

$$(\mathbf{QR})\boldsymbol{w} = \boldsymbol{y} \tag{11.19}$$

$$\mathbf{Q}^\mathsf{T}\mathbf{QR}\boldsymbol{w} = \mathbf{Q}^\mathsf{T}\boldsymbol{y} \tag{11.20}$$

$$\boldsymbol{w} = \mathbf{R}^{-1}(\mathbf{Q}^\mathsf{T}\boldsymbol{y}) \tag{11.21}$$

Since $\mathbf{R}$ is upper triangular, we can solve this last set of equations using backsubstitution, thus avoiding matrix inversion. See linsys_solve_demo.ipynb for a demo.

An alternative to the use of direct methods based on matrix decomposition (such as SVD and QR) is to use iterative solvers, such as the **conjugate gradient** method (which assumes $\mathbf{X}$ is symmetric

1. Note that a lot of the "Python" scientific computing stack sits on top of source code that is written in Fortran or C++, for reasons of speed. This makes it hard to change the underlying algorithms. By contrast, the scientific computing libraries in the Julia language are written in Julia itself, aiding clarity without sacrificing speed.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

positive definite), and the **GMRES** (generalized minimal residual method), that works for general $\mathbf{X}$. (In SciPy, this is implemented by `sparse.linalg.gmres`.) These methods just require the ability to perform matrix-vector multiplications (i.e., an implementation of a **linear operator**), and thus are well-suited to problems where $\mathbf{X}$ is sparse or structured. For details, see e.g., [TB97].

A final important issue is that it is usually essential to **standardize** the input features before fitting the model, to ensure that they are zero mean and unit variance. We can do this using Equation (10.51).

11.2.2.4 Weighted least squares

In some cases, we want to associate a weight with each example. For example, in **heteroskedastic regression**, the variance depends on the input, so the model has the form

$$p(y|\boldsymbol{x};\boldsymbol{\theta}) = \mathcal{N}(y|\boldsymbol{w}^\mathsf{T}\boldsymbol{x}, \sigma^2(\boldsymbol{x})) = \frac{1}{\sqrt{2\pi\sigma^2(\boldsymbol{x})}} \exp\left(-\frac{1}{2\sigma^2(\boldsymbol{x})}(y - \boldsymbol{w}^\mathsf{T}\boldsymbol{x})^2\right) \tag{11.22}$$

Thus

$$p(\boldsymbol{y}|\boldsymbol{x};\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{y}|\mathbf{X}\boldsymbol{w}, \boldsymbol{\Lambda}^{-1}) \tag{11.23}$$

where $\boldsymbol{\Lambda} = \text{diag}(1/\sigma^2(\boldsymbol{x}_n))$. This is known as **weighted linear regression**. One can show that the MLE is given by

$$\hat{\boldsymbol{w}} = (\mathbf{X}^\mathsf{T}\boldsymbol{\Lambda}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{\Lambda}\boldsymbol{y} \tag{11.24}$$

This is known as the **weighted least squares** estimate.

11.2.3 Other approaches to computing the MLE

In this section, we discuss other approaches for computing the MLE.

11.2.3.1 Solving for offset and slope separately

Typically we use a model of the form $p(y|\boldsymbol{x},\boldsymbol{\theta}) = \mathcal{N}(y|w_0 + \boldsymbol{w}^\mathsf{T}\boldsymbol{x}, \sigma^2)$, where w_0 is an offset or "bias" term. We can compute $(w_0, \boldsymbol{w})$ at the same time by adding a column of 1s to $\mathbf{X}$, and the computing the MLE as above. Alternatively, we can solve for $\boldsymbol{w}$ and w_0 separately. (This will be useful later.) In particular, one can show that

$$\hat{\boldsymbol{w}} = (\mathbf{X}_c^\mathsf{T}\mathbf{X}_c)^{-1}\mathbf{X}_c^\mathsf{T}\boldsymbol{y}_c = \left[\sum_{i=1}^N (\boldsymbol{x}_n - \overline{\boldsymbol{x}})(\boldsymbol{x}_n - \overline{\boldsymbol{x}})^\mathsf{T}\right]^{-1}\left[\sum_{i=1}^N (y_n - \overline{y})(\boldsymbol{x}_n - \overline{\boldsymbol{x}})\right] \tag{11.25}$$

$$\hat{w}_0 = \frac{1}{N}\sum_n y_n - \frac{1}{N}\sum_n \boldsymbol{x}_n^\mathsf{T}\hat{\boldsymbol{w}} = \overline{y} - \overline{\boldsymbol{x}}^\mathsf{T}\hat{\boldsymbol{w}} \tag{11.26}$$

where $\mathbf{X}_c$ is the centered input matrix containing $\boldsymbol{x}_n^c = \boldsymbol{x}_n - \overline{\boldsymbol{x}}$ along its rows, and $\boldsymbol{y}_c = \boldsymbol{y} - \overline{\boldsymbol{y}}$ is the centered output vector. Thus we can first compute $\hat{\boldsymbol{w}}$ on centered data, and then estimate w_0 using $\overline{y} - \overline{\boldsymbol{x}}^\mathsf{T}\hat{\boldsymbol{w}}$.

Note that, if we write the model in the form $\hat{y} = \beta_0 + \boldsymbol{\beta}^\mathsf{T}(\boldsymbol{x} - \overline{\boldsymbol{x}})$, then we have $\hat{\boldsymbol{\beta}} = \hat{\boldsymbol{w}}$ and $\hat{w}_0 = \hat{\beta}_0 - \hat{\boldsymbol{\beta}}^\mathsf{T}\overline{\boldsymbol{x}}$, so $\hat{\beta}_0 = \overline{y}$.

11.2.3.2 Simple linear regression (1d inputs)

In the case of 1d (scalar) inputs, the results from Section 11.2.3.1 reduce to the following simple form, which may be familiar from basic statistics classes:

$$\hat{w}_1 = \frac{\sum_n (x_n - \overline{x})(y_n - \bar{y})}{\sum_n (x_n - \bar{x})^2} = \frac{C_{xy}}{C_{xx}} = \rho_{xy} \frac{\sigma_y}{\sigma_x} \tag{11.27}$$

$$\hat{w}_0 = \mathbb{E}[y] - w_1 \mathbb{E}[x] \approx \bar{y} - \hat{w}_1 \bar{x} \tag{11.28}$$

where $C_{xx} = \mathrm{Cov}[X, X] = \mathbb{V}[X] = \sigma_x^2$, $C_{yy} = \mathrm{Cov}[Y, Y] = \mathbb{V}[Y] = \sigma_y^2$, $C_{xy} = \mathrm{Cov}[X, Y]$, and $\rho_{xy} = \frac{C_{xy}}{\sigma_x \sigma_y}$. Hence the prediction becomes $\hat{y} = \bar{y} - \hat{w}_1 \bar{x} + \hat{w}_1 x = \bar{y} = \rho \sigma_y \frac{(x - \bar{x})}{\sigma_y}$. We can interpret this equation as follows: we estimate how many standard deviations x is away from the mean of X, and then we predict the outcome to be the mean of Y plus ρ times that many standard deviations above or below.

11.2.3.3 Partial regression

From Equation (11.27), we can compute the **regression coefficient** of Y on X as follows:

$$R_{YX} \triangleq \frac{\partial}{\partial x} \mathbb{E}[Y|X = x] = w_1 = \frac{C_{xy}}{C_{xx}} \tag{11.29}$$

This is the slope of the linear prediction for Y given X.

Now consider the case where we have 2 inputs, so $Y = w_0 + w_1 X_1 + w_2 X_2 + \epsilon$, where $\mathbb{E}[\epsilon] = 0$. One can show that the optimal regression coefficient for w_1 is given by $R_{YX_1 \cdot X_2}$, which is the **partial regression coefficient** of Y on X_1, keeping X_2 constant:

$$w_1 = R_{YX_1 \cdot X_2} = \frac{\partial}{\partial x} \mathbb{E}[Y|X_1 = x, X_2] \tag{11.30}$$

Note that this quantity is invariant to the specific value of X_2 we condition on.

We can derive w_2 in a similar manner. Indeed, we can extend this to multiple input variables. In each case, we find the optimal coefficients are equal to the partial regression coefficients. This means that we can interpret the j'th coefficient $\hat{w}_j$ as the change in output y we expect per unit change in input x_j, keeping all the other inputs constant.

11.2.3.4 Recursively computing the MLE

OLS is a batch method for computing the MLE. In some applications, the data arrives in a continual stream, so we want to compute the estimate online, or **recursively**, as we discussed in Section 4.4.2. In this section, we show how to do this for the case of simple (1d) linear regession.

Recall from Section 11.2.3.2 that the batch MLE for simple linear regression is given by

$$\hat{w}_1 = \frac{\sum_n (x_n - \overline{x})(y_n - \bar{y})}{\sum_n (x_n - \bar{x})^2} = \frac{C_{xy}}{C_{xx}} \tag{11.31}$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x} \tag{11.32}$$

where $C_{xy} = \mathrm{Cov}[X, Y]$ and $C_{xx} = \mathrm{Cov}[X, X] = \mathbb{V}[X]$.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

We now discuss how to compute these results in a recursive fashion. To do this, let us define the following sufficient statistics:

$$\overline{x}^{(n)} = \frac{1}{n}\sum_{i=1}^{n} x_i, \;\; \overline{y}^{(n)} = \frac{1}{n}\sum_{i=1}^{n} y_i \tag{11.33}$$

$$C_{xx}^{(n)} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2, \;\; C_{xy}^{(n)} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y}), \;\; C_{yy}^{(n)} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \overline{y})^2 \tag{11.34}$$

We can update the means online using

$$\overline{x}^{(n+1)} = \overline{x}^{(n)} + \frac{1}{n+1}(x_{n+1} - \overline{x}^{(n)}), \;\; \overline{y}^{(n+1)} = \overline{y}^{(n)} + \frac{1}{n+1}(y_{n+1} - \overline{y}^{(n)}) \tag{11.35}$$

To update the covariance terms, let us first rewrite $C_{xy}^{(n)}$ as follows:

$$C_{xy}^{(n)} = \frac{1}{n}\left[(\sum_{i=1}^{n} x_i y_i) + (\sum_{i=1}^{n} \overline{x}^{(n)}\overline{y}^{(n)}) - \overline{x}^{(n)}(\sum_{i=1}^{n} y_i) - \overline{y}^{(n)}(\sum_{i=1}^{n} x_i)\right] \tag{11.36}$$

$$= \frac{1}{n}\left[(\sum_{i=1}^{n} x_i y_i) + n\overline{x}^{(n)}\overline{y}^{(n)} - \overline{x}^{(n)} n\overline{y}^{(n)} - \overline{y}^{(n)} n\overline{x}^{(n)}\right] \tag{11.37}$$

$$= \frac{1}{n}\left[(\sum_{i=1}^{n} x_i y_i) - n\overline{x}^{(n)}\overline{y}^{(n)}\right] \tag{11.38}$$

Hence

$$\sum_{i=1}^{n} x_i y_i = nC_{xy}^{(n)} + n\overline{x}^{(n)}\overline{y}^{(n)} \tag{11.39}$$

and so

$$C_{xy}^{(n+1)} = \frac{1}{n+1}\left[x_{n+1}y_{n+1} + nC_{xy}^{(n)} + n\overline{x}^{(n)}\overline{y}^{(n)} - (n+1)\overline{x}^{(n+1)}\overline{y}^{(n+1)}\right] \tag{11.40}$$

We can derive the update for $C_{xx}^{(n+1)}$ in a similar manner.

See Figure 11.4 for a simple illustration of these equations in action for a 1d regression model.

To extend the above analysis to D-dimensional inputs, the easiest approach is to use SGD. The resulting algorithm is called the **least mean squares** algorithm; see Section 8.4.2 for details.

11.2.3.5 Deriving the MLE from a generative perspective

Linear regression is a discriminative model of the form $p(y|\boldsymbol{x})$. However, we can also use generative models for regression, by analogy to how we use generative models for classification in Chapter 9, The goal is to compute the conditional expectation

$$f(\boldsymbol{x}) = \mathbb{E}[y|\boldsymbol{x}] = \int y \; p(y|\boldsymbol{x})dy = \frac{\int y \; p(\boldsymbol{x}, y)dy}{\int p(\boldsymbol{x}, y)dy} \tag{11.41}$$

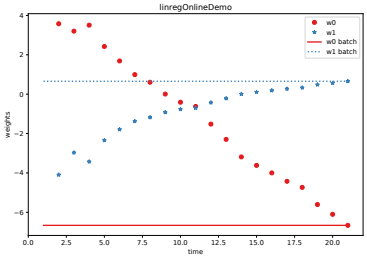


Figure 11.4: Regression coefficients over time for the 1d model in Figure 1.7a(a). Generated by linregOnlineDemo.ipynb.

Suppose we fit $p(\boldsymbol{x}, y)$ using an MVN. The MLEs for the parameters of the joint distribution are the empiricial means and covariances (see Section 4.2.6 for a proof of this result):

$$\boldsymbol{\mu}_x = \frac{1}{N}\sum_n \boldsymbol{x}_n \tag{11.42}$$

$$\mu_y = \frac{1}{N}\sum_n y_n \tag{11.43}$$

$$\boldsymbol{\Sigma}_{xx} = \frac{1}{N}\sum_n (\boldsymbol{x}_n - \overline{\boldsymbol{x}})(\boldsymbol{x}_n - \overline{\boldsymbol{x}})^\mathsf{T} = \frac{1}{N}\mathbf{X}_c^\mathsf{T}\mathbf{X}_c \tag{11.44}$$

$$\boldsymbol{\Sigma}_{xy} = \frac{1}{N}\sum_n (\boldsymbol{x}_n - \overline{\boldsymbol{x}})(y_n - \overline{y}) = \frac{1}{N}\mathbf{X}_c^\mathsf{T}\boldsymbol{y}_c \tag{11.45}$$

Hence from Equation (3.28), we have

$$\mathbb{E}\left[y|\boldsymbol{x}\right] = \mu_y + \boldsymbol{\Sigma}_{xy}^\mathsf{T}\boldsymbol{\Sigma}_{xx}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_x) \tag{11.46}$$

We can rewrite this as $\mathbb{E}\left[y|\boldsymbol{x}\right] = w_0 + \boldsymbol{w}^\mathsf{T}\boldsymbol{x}$ by defining

$$w_0 = \mu_y - \boldsymbol{w}^\mathsf{T}\boldsymbol{\mu}_x = \overline{y} - \boldsymbol{w}^\mathsf{T}\overline{\boldsymbol{x}} \tag{11.47}$$

$$\boldsymbol{w} = \boldsymbol{\Sigma}_{xx}^{-1}\boldsymbol{\Sigma}_{xy} = \left(\mathbf{X}_c^\mathsf{T}\mathbf{X}_c\right)^{-1}\mathbf{X}_c^\mathsf{T}\boldsymbol{y}_c \tag{11.48}$$

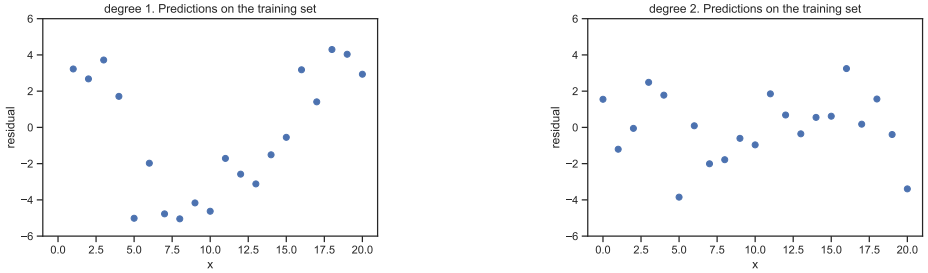
This matches the MLEs for the discriminative model as we showed in Section 11.2.3.1. Thus we see that fitting the joint model, and then conditioning it, yields the same result as fitting the conditional model. However, this is only true for Gaussian models (see Section 9.4 for further discussion of this point).

11.2.3.6 Deriving the MLE for σ^2

After estimating $\hat{\boldsymbol{w}}_{\text{mle}}$ using one of the above methods, we can estimate the noise variance. It is easy to show that the MLE is given by

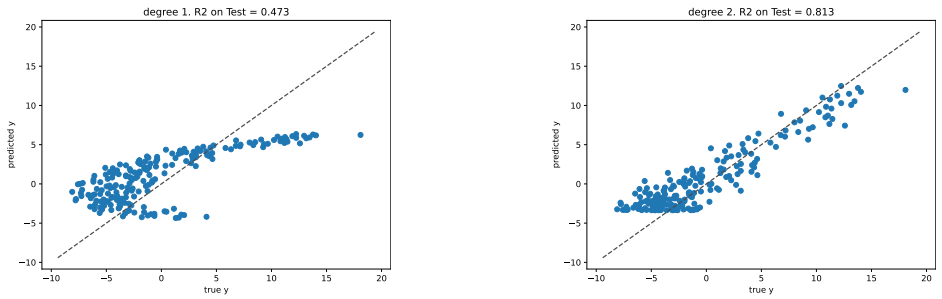
$$\hat{\sigma}^2_{\text{mle}} = \operatorname*{argmin}_{\sigma^2} \text{NLL}(\hat{\boldsymbol{w}}, \sigma^2) = \frac{1}{N}\sum_{n=1}^N (y_n - \boldsymbol{x}_n^\mathsf{T}\hat{\boldsymbol{w}})^2 \tag{11.49}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 11.5: Residual plot for polynomial regression of degree 1 and 2 for the functions in Figure 1.7a(a-b). Generated by linreg_poly_vs_degree.ipynb.



(a) (b)

Figure 11.6: Fit vs actual plots for polynomial regression of degree 1 and 2 for the functions in Figure 1.7a(a-b). Generated by linreg_poly_vs_degree.ipynb.

This is just the MSE of the residuals, which is an intuitive result.

11.2.4 Measuring goodness of fit

In this section, we discuss some simple ways to assess how well a regression model fits the data (which is known as **goodness of fit**).

11.2.4.1 Residual plots

For 1d inputs, we can check the reasonableness of the model by plotting the residuals, $r_n = y_n - \hat{y}_n$, vs the input x_n. This is called a **residual plot**. The model assumes that the residuals have a $\mathcal{N}(0, \sigma^2)$ distribution, so the residual plot should be a cloud of points more or less equally above and below the horizontal line at 0, without any obvious trends.

As an example, in Figure 11.5(a), we plot the residuals for the linear model in Figure 1.7a(a). We see that there is some curved structure to the residuals, indicating a lack of fit. In Figure 11.5(b), we plot the residuals for the quadratic model in Figure 1.7a(b). We see a much better fit.

To extend this approach to multi-dimensional inputs, we can plot predictions $\hat{y}_n$ vs the true output y_n, rather than plotting vs x_n. A good model will have points that lie on a diagonal line. See Figure 11.6 for some examples.

11.2.4.2 Prediction accuracy and R^2

We can assess the fit quantitatively by computing the RSS (residual sum of squares) on the dataset: $\text{RSS}(\boldsymbol{w}) = \sum_{n=1}^{N}(y_n - \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n)^2$. A model with lower RSS fits the data better. Another measure that is used is **root mean squared error** or **RMSE**:

$$\text{RMSE}(\boldsymbol{w}) \triangleq \sqrt{\frac{1}{N}\text{RSS}(\boldsymbol{w})} \tag{11.50}$$

A more interpretable measure can be computed using the **coefficient of determination**, denoted by R^2:

$$R^2 \triangleq 1 - \frac{\sum_{n=1}^{N}(\hat{y}_n - y_n)^2}{\sum_{n=1}^{N}(\overline{y} - y_n)^2} = 1 - \frac{\text{RSS}}{\text{TSS}} \tag{11.51}$$

where $\overline{y} = \frac{1}{N}\sum_{n=1}^{N} y_n$ is the empirical mean of the response, $\text{RSS} = \sum_{n=1}^{N}(y_n - \hat{y}_n)^2$ is the residual sum of squares, and $\text{TSS} = \sum_{n=1}^{N}(y_n - \overline{y})^2$ is the total sum of squares. Thus we see that R^2 measures the variance in the predictions relative to a simple constant prediction of $\hat{y}_n = \overline{y}$. If a model does no better at predicting than using the mean of the output, it we have $R^2 = 0$. If the model perfectly fits the data, then the RSS will be 0, so $R^2 = 1$. In general, larger values imply greater reduction in variance (better fit). This is illustrated in Figure 11.6.

11.3 Ridge regression

Maximum likelihood estimation can result in overfitting, as we discussed in Section 1.2.2.2. A simple solution to this is to use MAP estimation with a zero-mean Gaussian prior on the weights, $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\boldsymbol{0}, \lambda^{-1}\mathbf{I})$, as we discused in Section 4.5.3. This is called **ridge regression**.

In more detail, we compute the MAP estimate as follows:

$$\hat{\boldsymbol{w}}_{\text{map}} = \text{argmin}\, \frac{1}{2\sigma^2}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w})^\mathsf{T}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w}) + \frac{1}{2\tau^2}\boldsymbol{w}^\mathsf{T}\boldsymbol{w} \tag{11.52}$$

$$= \text{argmin}\, \text{RSS}(\boldsymbol{w}) + \lambda||\boldsymbol{w}||_2^2 \tag{11.53}$$

where $\lambda \triangleq \frac{\sigma^2}{\tau^2}$ is proportional to the strength of the prior, and

$$||\boldsymbol{w}||_2 \triangleq \sqrt{\sum_{d=1}^{D}|w_d|^2} = \sqrt{\boldsymbol{w}^\mathsf{T}\boldsymbol{w}} \tag{11.54}$$

is the ℓ_2 norm of the vector $\boldsymbol{w}$. Thus we are penalizing weights that become too large in magnitude. In general, this technique is called ℓ_2 **regularization** or **weight decay**, and is very widely used. See Figure 4.5 for an illustration.

Note that we do not penalize the offset term w_0, since that only affects the global mean of the output, and does not contribute to overfitting. See Exercise 11.2.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

11.3.1 Computing the MAP estimate

In this section, we discuss algorithms for computing the MAP estimate.

The MAP estimate corresponds to minimizing the following penalized objective:

$$J(\boldsymbol{w}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{w})^\mathsf{T}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w}) + \lambda||\boldsymbol{w}||_2^2 \tag{11.55}$$

where $\lambda = \sigma^2/\tau^2$ is the strength of the regularizer. The derivative is given by

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = 2\left(\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w} - \mathbf{X}^\mathsf{T}\boldsymbol{y} + \lambda\boldsymbol{w}\right) \tag{11.56}$$

and hence

$$\hat{\boldsymbol{w}}_{\text{map}} = (\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda\mathbf{I}_D)^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} = (\sum_n \boldsymbol{x}_n\boldsymbol{x}_n^\mathsf{T} + \lambda\mathbf{I}_D)^{-1}(\sum_n y_n\boldsymbol{x}_n) \tag{11.57}$$

11.3.1.1 Solving using QR

Naively computing the primal estimate $\boldsymbol{w} = (\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$ using matrix inversion is a bad idea, since it can be slow and numerically unstable. In this section, we describe a way to convert the problem to a standard least squares problem, to which we can apply QR decomposition, as discussed in Section 11.2.2.3.

We assume the prior has the form $p(\boldsymbol{w}) = \mathcal{N}(\mathbf{0}, \boldsymbol{\Lambda}^{-1})$, where $\boldsymbol{\Lambda}$ is the precision matrix. In the case of ridge regression, $\boldsymbol{\Lambda} = (1/\tau^2)\mathbf{I}$. We can emulate this prior by adding "virtual data" to the training set to get

$$\tilde{\mathbf{X}} = \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\boldsymbol{\Lambda}} \end{pmatrix}, \quad \tilde{\boldsymbol{y}} = \begin{pmatrix} \boldsymbol{y}/\sigma \\ \mathbf{0}_{D\times 1} \end{pmatrix} \tag{11.58}$$

where $\boldsymbol{\Lambda} = \sqrt{\boldsymbol{\Lambda}}\sqrt{\boldsymbol{\Lambda}}^\mathsf{T}$ is a Cholesky decomposition of $\boldsymbol{\Lambda}$. We see that $\tilde{\mathbf{X}}$ is $(N + D) \times D$, where the extra rows represent pseudo-data from the prior.

We now show that the RSS on this expanded data is equivalent to penalized RSS on the original data:

$$f(\boldsymbol{w}) = (\tilde{\boldsymbol{y}} - \tilde{\mathbf{X}}\boldsymbol{w})^\mathsf{T}(\tilde{\boldsymbol{y}} - \tilde{\mathbf{X}}\boldsymbol{w}) \tag{11.59}$$

$$= \left(\begin{pmatrix} \boldsymbol{y}/\sigma \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\boldsymbol{\Lambda}} \end{pmatrix}\boldsymbol{w}\right)^\mathsf{T}\left(\begin{pmatrix} \boldsymbol{y}/\sigma \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\boldsymbol{\Lambda}} \end{pmatrix}\boldsymbol{w}\right) \tag{11.60}$$

$$= \begin{pmatrix} \frac{1}{\sigma}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w}) \\ -\sqrt{\boldsymbol{\Lambda}}\boldsymbol{w} \end{pmatrix}^\mathsf{T}\begin{pmatrix} \frac{1}{\sigma}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w}) \\ -\sqrt{\boldsymbol{\Lambda}}\boldsymbol{w} \end{pmatrix} \tag{11.61}$$

$$= \frac{1}{\sigma^2}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w})^\mathsf{T}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w}) + (\sqrt{\boldsymbol{\Lambda}}\boldsymbol{w})^\mathsf{T}(\sqrt{\boldsymbol{\Lambda}}\boldsymbol{w}) \tag{11.62}$$

$$= \frac{1}{\sigma^2}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w})^\mathsf{T}(\boldsymbol{y} - \mathbf{X}\boldsymbol{w}) + \boldsymbol{w}^\mathsf{T}\boldsymbol{\Lambda}\boldsymbol{w} \tag{11.63}$$

Hence the MAP estimate is given by

$$\hat{\boldsymbol{w}}_{\text{map}} = (\tilde{\mathbf{X}}^\mathsf{T}\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^\mathsf{T}\tilde{\boldsymbol{y}} \tag{11.64}$$

which can be solved using standard OLS methods. In particular, we can compute the QR decomposition of $\tilde{\mathbf{X}}$, and then proceed as in Section 11.2.2.3. This takes $O((N + D)D^2)$ time.

11.3.1.2 Solving using SVD

In this section, we assume $D > N$, which is the usual case when using ridge regression. In this case, it is faster to use SVD than QR. To see how this works, let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{I}_N$, $\mathbf{U}\mathbf{U}^\mathsf{T} = \mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{I}_N$, and $\mathbf{S}$ is a diagonal $N \times N$ matrix. Now let $\mathbf{R} = \mathbf{U}\mathbf{S}$ be an $N \times N$ matrix. One can show (see Exercise 18.4 of [HTF09]) that

$$\hat{\boldsymbol{w}}_{\text{map}} = \mathbf{V}(\mathbf{R}^\mathsf{T}\mathbf{R} + \lambda\mathbf{I}_N)^{-1}\mathbf{R}^\mathsf{T}\boldsymbol{y} \tag{11.65}$$

In other words, we can replace the D-dimensional vectors $\boldsymbol{x}_i$ with the N-dimensional vectors $\boldsymbol{r}_i$ and perform our penalized fit as before. The overall time is now $O(DN^2)$ operations, which is less than $O(D^3)$ if $D > N$.

11.3.2 Connection between ridge regression and PCA

In this section, we discuss an interesting connection between ridge regression and PCA (which we describe in Section 20.1), in order to gain further insight into why ridge regression works well. Our discussion is based on [HTF09, p66].

Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$ be the SVD of $\mathbf{X}$, where $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{I}_N$, $\mathbf{U}\mathbf{U}^\mathsf{T} = \mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{I}_N$, and $\mathbf{S}$ is a diagonal $N \times N$ matrix. Using Equation (11.65) we can see that the ridge predictions on the training set are given by

$$\hat{\boldsymbol{y}} = \mathbf{X}\hat{\boldsymbol{w}}_{\text{map}} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}\mathbf{V}(\mathbf{S}^2 + \lambda\mathbf{I})^{-1}\mathbf{S}\mathbf{U}^\mathsf{T}\boldsymbol{y} \tag{11.66}$$

$$= \mathbf{U}\tilde{\mathbf{S}}\mathbf{U}^\mathsf{T}\boldsymbol{y} = \sum_{j=1}^{D} \boldsymbol{u}_j \tilde{S}_{jj} \boldsymbol{u}_j^\mathsf{T}\boldsymbol{y} \tag{11.67}$$

where

$$\tilde{S}_{jj} \triangleq [\mathbf{S}(\mathbf{S}^2 + \lambda I)^{-1}\mathbf{S}]_{jj} = \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \tag{11.68}$$

and σ_j are the singular values of $\mathbf{X}$. Hence

$$\hat{\boldsymbol{y}} = \mathbf{X}\hat{\boldsymbol{w}}_{\text{map}} = \sum_{j=1}^{D} \boldsymbol{u}_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \boldsymbol{u}_j^\mathsf{T}\boldsymbol{y} \tag{11.69}$$

In contrast, the least squares prediction is

$$\hat{\boldsymbol{y}} = \mathbf{X}\hat{\boldsymbol{w}}_{\text{mle}} = (\mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T})(\mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\mathsf{T}\boldsymbol{y}) = \mathbf{U}\mathbf{U}^\mathsf{T}\boldsymbol{y} = \sum_{j=1}^{D} \boldsymbol{u}_j \boldsymbol{u}_j^\mathsf{T}\boldsymbol{y} \tag{11.70}$$

If σ_j^2 is large compared to λ, then $\sigma_j^2/(\sigma_j^2 + \lambda) \approx \sigma_j^2/\sigma_j^2 = 1$, so direction $\boldsymbol{u}_j$ is not affected, but if σ_j^2 is small compared to λ, and if λ is large, then $\sigma_j^2/(\sigma_j^2 + \lambda) \approx 1/\lambda \approx 0$, so direction $\boldsymbol{u}_j$ will be downweighted. In view of this, we define the effective number of **degrees of freedom** of the model as follows:

$$\text{dof}(\lambda) = \sum_{j=1}^{D} \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \tag{11.71}$$

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

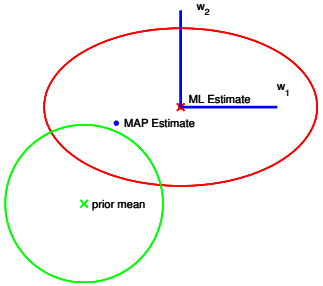


Figure 11.7: Geometry of ridge regression. The likelihood is shown as an ellipse, and the prior is shown as a circle centered on the origin. Adapted from Figure 3.15 of [Bis06]. Generated by geom_ridge.ipynb.

When $\lambda = 0$, $\text{dof}(\lambda) = D$, and as $\lambda \to \infty$, $\text{dof}(\lambda) \to 0$.

Let us try to understand why this behavior is desirable. In Section 11.7, we show that $\text{Cov}[\boldsymbol{w}|\mathcal{D}] \propto (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}$, if we use a uniform prior for $\boldsymbol{w}$. Thus the directions in which we are most uncertain about $\boldsymbol{w}$ are determined by the eigenvectors of $(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}$ with the largest eigenvalues, as shown in Figure 7.6. These drections correspond to the eigenvectors of $\mathbf{X}^\mathsf{T}\mathbf{X}$ with the smallest eigenvalues, and hence (from Section 7.5.2) the smallest singular values. So if σ_j^2 is small relative to λ, ridge regression will downweight direction $\boldsymbol{u}_j$.

This process is illustrated in Figure 11.7. The horizontal w_1 parameter is not-well determined by the data (has high posterior variance), but the vertical w_2 parameter is well-determined. Hence $w_{\text{map}}(2)$ is close to $w_{\text{mle}}(2)$, but $w_{\text{map}}(1)$ is shifted strongly towards the prior mean, which is 0. In this way, ill-determined parameters are reduced in size towards 0. This is called **shrinkage**.

There is a related, but different, technique called **principal components regression**, which is a supervised version of PCA, which we explain in Section 20.1. The idea is this: first use PCA to reduce the dimensionality to K dimensions, and then use these low dimensional features as input to regression. However, this technique does not work as well as ridge regression in terms of predictive accuracy [HTF01, p70]. The reason is that in PC regression, only the first K (derived) dimensions are retained, and the remaining $D - K$ dimensions are entirely ignored. By contrast, ridge regression uses a "soft" weighting of all the dimensions.

11.3.3 Choosing the strength of the regularizer

To find the optimal value of λ, we can try a finite number of distinct values, and use cross validation to estimate their expected loss, as discussed in Section 4.5.5.2. See Figure 4.5d for an example.

This approach can be quite expensive if we have many values to choose from. Fortunately, we can often **warm start** the optimization procedure, using the value of $\hat{\boldsymbol{w}}(\lambda_k)$ as an initializer for $\hat{\boldsymbol{w}}(\lambda_{k+1})$, where $\lambda_{k+1} < \lambda_k$; in other words, we start with a highly constrained model (strong regularizer), and then gradually relax the constraints (decrease the amount of regularization). The set of parameters $\hat{\boldsymbol{w}}_k$ that we sweep out in this way is known as the **regularization path**. See Figure 11.10(a) for an example.

We can also use an empirical Bayes approach to choose λ. In particular, we choose the hyperparameter by computing $\hat{\lambda} = \text{argmax}_\lambda \log p(\mathcal{D}|\lambda)$, where $p(\mathcal{D}|\lambda)$ is the marginal likelihood or evidence.

Figure 4.7b shows that this gives essentially the same result as the CV estimate. However, the Bayesian approach has several advantages: computing $p(\mathcal{D}|\lambda)$ can be done by fitting a single model, whereas CV has to fit the same model K times; and $p(\mathcal{D}|\lambda)$ is a smooth function of λ, so we can use gradient-based optimization instead of discrete search.

11.4 Lasso regression

In Section 11.3, we assumed a Gaussian prior for the regression coefficients when fitting linear regression models. This is often a good choice, since it encourages the parameters to be small, and hence prevents overfitting. However, sometimes we want the parameters to not just be small, but to be exactly zero, i.e., we want $\hat{\boldsymbol{w}}$ to be **sparse**, so that we minimize the **L0-norm**:

$$||\boldsymbol{w}||_0 = \sum_{d=1}^{D} \mathbb{I}\left(|w_d| > 0\right) \tag{11.72}$$

This is useful because it can be used to perform **feature selection**. To see this, note that the prediction has the form $f(\boldsymbol{x}; \boldsymbol{w}) = \sum_{d=1}^{D} w_d x_d$, so if any $w_d = 0$, we ignore the corresponding feature x_d. (The same idea can be applied to nonlinear models, such as DNNs, by encouraging the first layer weights to be sparse.)

11.4.1 MAP estimation with a Laplace prior (ℓ_1 regularization)

There are many ways to compute such sparse estimates (see e.g., [Bha+19]). In this section we focus on MAP estimation using the Laplace distribution (which we discussed in Section 11.6.1) as the prior:

$$p(\boldsymbol{w}|\lambda) = \prod_{d=1}^{D} \text{Laplace}(w_d|0, 1/\lambda) \propto \prod_{d=1}^{D} e^{-\lambda|w_d|} \tag{11.73}$$

where λ is the sparsity parameter, and

$$\text{Laplace}(w|\mu, b) \triangleq \frac{1}{2b} \exp\left(-\frac{|w-\mu|}{b}\right) \tag{11.74}$$

Here μ is a location parameter and $b > 0$ is a scale parameter. Figure 2.15 shows that $\text{Laplace}(w|0, b)$ puts more density on 0 than $\mathcal{N}(w|0, \sigma^2)$, even when we fix the variance to be the same.

To perform MAP estimation of a linear regression model with this prior, we just have to minimize the following objective:

$$\text{PNLL}(\boldsymbol{w}) = -\log p(\mathcal{D}|\boldsymbol{w}) - \log p(\boldsymbol{w}|\lambda) = ||\mathbf{X}\boldsymbol{w} - \boldsymbol{y}||_2^2 + \lambda||\boldsymbol{w}||_1 \tag{11.75}$$

where $||\boldsymbol{w}||_1 \triangleq \sum_{d=1}^{D} |w_d|$ is the ℓ_1 norm of $\boldsymbol{w}$. This method is called **lasso**, which stands for "least absolute shrinkage and selection operator" [Tib96]. (We explain the reason for this name below.) More generally, MAP estimation with a Laplace prior is called ℓ_1**-regularization**.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

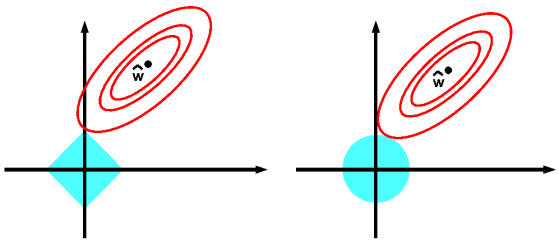


Figure 11.8: Illustration of ℓ_1 (left) vs ℓ_2 (right) regularization of a least squares problem. Adapted from Figure 3.12 of [HTF01].

Note also that we could use other norms for the weight vector. In general, the q-norm is defined as follows:

$$\|\boldsymbol{w}\|_q = \left(\sum_{d=1}^{D} |w_d|^q\right)^{1/q} \tag{11.76}$$

For $q < 1$, we can get even sparser solutions. In the limit where $q = 0$, we get the ℓ_0**-norm**:

$$\|\boldsymbol{w}\|_0 = \sum_{d=1}^{D} \mathbb{I}\left(|w_d| > 0\right) \tag{11.77}$$

However, one can show that for any $q < 1$, the problem becomes non-convex (see e.g., [HTW15]). Thus ℓ_1-norm is the tightest **convex relaxation** of the ℓ_0-norm.

11.4.2 Why does ℓ_1 regularization yield sparse solutions?

We now explain why ℓ_1 regularization results in sparse solutions, whereas ℓ_2 regularization does not. We focus on the case of linear regression, although similar arguments hold for other models.

The lasso objective is the following non-smooth objective (see Section 8.1.4 for a discussion of smoothness):

$$\min_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) + \lambda||\boldsymbol{w}||_1 \tag{11.78}$$

This is the Lagrangian for the following quadratic program (see Section 8.5.4):

$$\min_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) \quad \text{s.t.} \quad ||\boldsymbol{w}||_1 \leq B \tag{11.79}$$

where B is an upper bound on the ℓ_1-norm of the weights: a small (tight) bound B corresponds to a large penalty λ, and vice versa.

Similarly, we can write the ridge regression objective $\min_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) + \lambda||\boldsymbol{w}||_2^2$ in bound constrained form:

$$\min_{\boldsymbol{w}} \text{NLL}(\boldsymbol{w}) \quad \text{s.t.} \quad ||\boldsymbol{w}||_2^2 \leq B \tag{11.80}$$

In Figure 11.8, we plot the contours of the NLL objective function, as well as the contours of the ℓ_2 and ℓ_1 constraint surfaces. From the theory of constrained optimization (Section 8.5) we know that the optimal solution occurs at the point where the lowest level set of the objective function intersects the constraint surface (assuming the constraint is active). It should be geometrically clear that as we relax the constraint B, we "grow" the ℓ_1 "ball" until it meets the objective; the corners of the ball are more likely to intersect the ellipse than one of the sides, especially in high dimensions, because the corners "stick out" more. The corners correspond to sparse solutions, which lie on the coordinate axes. By contrast, when we grow the ℓ_2 ball, it can intersect the objective at any point; there are no "corners", so there is no preference for sparsity.

11.4.3 Hard vs soft thresholding

The lasso objective has the form $\mathcal{L}(\boldsymbol{w}) = \text{NLL}(\boldsymbol{w}) + \lambda||\boldsymbol{w}||_1$. One can show (Exercise 11.3) that the gradient for the smooth NLL part is given by

$$\frac{\partial}{\partial w_d}\text{NLL}(\boldsymbol{w}) = a_d w_d - c_d \tag{11.81}$$

$$a_d = \sum_{n=1}^{N} x_{nd}^2 \tag{11.82}$$

$$c_d = \sum_{n=1}^{N} x_{nd}(y_n - \boldsymbol{w}_{-d}^{\mathsf{T}}\boldsymbol{x}_{n,-d}) \tag{11.83}$$

where $\boldsymbol{w}_{-d}$ is $\boldsymbol{w}$ without component d, and similarly $\boldsymbol{x}_{n,-d}$ is feature vector $\boldsymbol{x}_n$ without component d. We see that c_d is proportional to the correlation between d'th column of features, $\boldsymbol{x}_{:,d}$, and the residual error obtained by predicting using all the other features, $\boldsymbol{r}_{-d} = \boldsymbol{y} - \mathbf{X}_{:,-d}\boldsymbol{w}_{-d}$. Hence the magnitude of c_d is an indication of how relevant feature d is for predicting $\boldsymbol{y}$, relative to the other features and the current parameters. Setting the gradient to 0 gives the optimal update for w_d, keeping all other weights fixed:

$$w_d = c_d/a_d = \frac{\boldsymbol{x}_{:,d}^{\mathsf{T}}\boldsymbol{r}_{-d}}{||\boldsymbol{x}_{:,d}||_2^2} \tag{11.84}$$

The corresponding new prediction for $\boldsymbol{r}_{-d}$ becomes $\hat{\boldsymbol{r}}_{-d} = w_d\boldsymbol{x}_{:,d}$, which is the orthogonal projection of the residual onto the column vector $\boldsymbol{x}_{:,d}$, consistent with Equation (11.15).

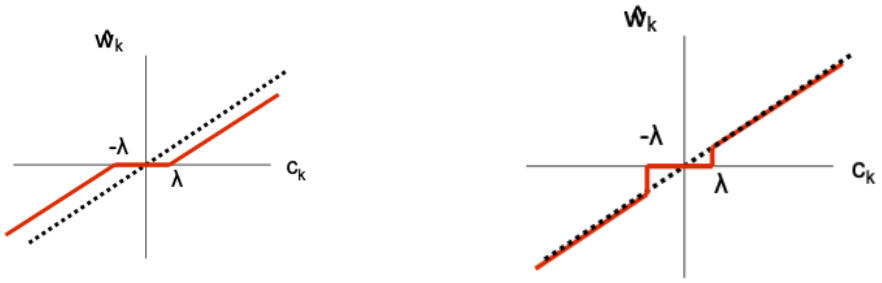
Now we add in the ℓ_1 term. Unfortunately, the $||\boldsymbol{w}||_1$ term is not differentiable whenever $w_d = 0$. Fortunately, we can still compute a subgradient at this point. Using Equation (8.14) we find that

$$\partial_{w_d}\mathcal{L}(\boldsymbol{w}) = (a_d w_d - c_d) + \lambda\partial_{w_d}||\boldsymbol{w}||_1 \tag{11.85}$$

$$= \begin{cases} \{a_d w_d - c_d - \lambda\} & \text{if } w_d < 0 \\ [-c_d - \lambda, -c_d + \lambda] & \text{if } w_d = 0 \\ \{a_d w_d - c_d + \lambda\} & \text{if } w_d > 0 \end{cases} \tag{11.86}$$

Depending on the value of c_d, the solution to $\partial_{w_d}\mathcal{L}(\boldsymbol{w}) = 0$ can occur at 3 different values of w_d, as follows:

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license



(a) (b)

Figure 11.9: Left: soft thresholding. Right: hard thresholding. In both cases, the horizontal axis is the residual error incurred by making predictions using all the coefficients except for w_k, and the vertical axis is the estimated coefficient $\hat{w}_k$ that minimizes this penalized residual. The flat region in the middle is the interval $[-\lambda, +\lambda]$.

1. If $c_d < -\lambda$, so the feature is strongly negatively correlated with the residual, then the subgradient is zero at $\hat{w}_d = \frac{c_d+\lambda}{a_d} < 0$.
2. If $c_d \in [-\lambda, \lambda]$, so the feature is only weakly correlated with the residual, then the subgradient is zero at $\hat{w}_d = 0$.
3. If $c_d > \lambda$, so the feature is strongly positively correlated with the residual, then the subgradient is zero at $\hat{w}_d = \frac{c_d-\lambda}{a_d} > 0$.

In summary, we have

$$\hat{w}_d(c_d) = \begin{cases} (c_d + \lambda)/a_d & \text{if } c_d < -\lambda \\ 0 & \text{if } c_d \in [-\lambda, \lambda] \\ (c_d - \lambda)/a_d & \text{if } c_d > \lambda \end{cases} \tag{11.87}$$

We can write this as follows:

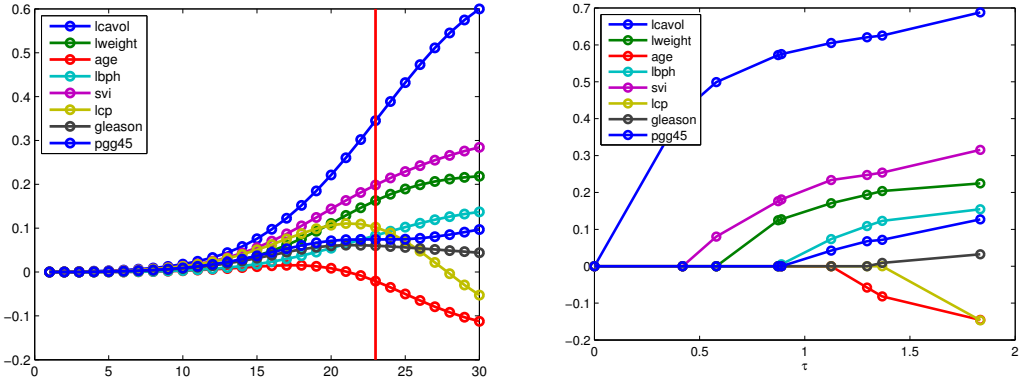
$$\hat{w}_d = \text{SoftThreshold}(\frac{c_d}{a_d}, \lambda/a_d) \tag{11.88}$$

where

$$\text{SoftThreshold}(x, \delta) \triangleq \text{sign}(x)\,(|x| - \delta)_+ \tag{11.89}$$

and $x_+ = \max(x, 0)$ is the positive part of x. This is called **soft thresholding** (see also Section 8.6.2). This is illustrated in Figure 11.9(a), where we plot $\hat{w}_d$ vs c_d. The dotted black line is the line $w_d = c_d/a_d$ corresponding to the least squares fit. The solid red line, which represents the regularized estimate $\hat{w}_d$, shifts the dotted line down (or up) by λ, except when $-\lambda \leq c_d \leq \lambda$, in which case it sets $w_d = 0$.

By contrast, in Figure 11.9(b), we illustrate **hard thresholding**. This sets values of w_d to 0 if $-\lambda \leq c_d \leq \lambda$, but it does not shrink the values of w_d outside of this interval. The slope of the soft



(a) (b)

Figure 11.10: (a) Profiles of ridge coefficients for the prostate cancer example vs bound B on ℓ_2 norm of $\boldsymbol{w}$, so small B (large λ) is on the left. The vertical line is the value chosen by 5-fold CV using the 1 standard error rule. Adapted from Figure 3.8 of [HTF09]. Generated by ridgePathProstate.ipynb. (b) Same as (a) but using ℓ_1 norm of $\boldsymbol{w}$. The x-axis shows the critical values of $\lambda = 1/B$, where the regularization path is discontinuous. Adapted from Figure 3.10 of [HTF09]. Generated by lassoPathProstate.ipynb.

thresholding line does not coincide with the diagonal, which means that even large coefficients are shrunk towards zero. This is why lasso stands for "least absolute selection *and shrinkage* operator". Consequently, lasso is a biased estimator (see Section 4.7.6.1).

A simple solution to the biased estimate problem, known as **debiasing**, is to use a two-stage estimation process: we first estimate the support of the weight vector (i.e., identify which elements are non-zero) using lasso; we then re-estimate the chosen coefficients using least squares. For an example of this in action, see Figure 11.13.

11.4.4 Regularization path

If $\lambda = 0$, we get the OLS solution. which will be dense. As we increase λ, the solution vector $\hat{\boldsymbol{w}}(\lambda)$ will tend to get sparser. If λ is bigger than some critical value, we get $\hat{\boldsymbol{w}} = \mathbf{0}$. This critical value is obtained when the gradient of the NLL cancels out with the gradient of the penalty:

$$\lambda_{\max} = \max_d |\nabla_{w_d} \text{NLL}(\mathbf{0})| = \max_d c_d(\boldsymbol{w} = 0) = \max_d |\boldsymbol{y}^\mathsf{T} \boldsymbol{x}_{:,d}| = ||\mathbf{X}^\mathsf{T} \boldsymbol{y}||_\infty \tag{11.90}$$

Alternatively, we can work with the bound B on the ℓ_1 norm. When $B = 0$, we get $\hat{\boldsymbol{w}} = \mathbf{0}$. As we increase B, the solution becomes denser. The largest value of B for which any component is zero is given by $B_{\max} = ||\hat{\boldsymbol{w}}_{\text{mle}}||_1$.

As we increase λ, the solution vector $\hat{\boldsymbol{w}}$ gets sparser, although not necessarily monotonically. We can plot the values $\hat{w}_d$ vs λ (or vs the bound B) for each feature d; this is known as the **regularization path**. This is illustrated in Figure 11.10(b), where we apply lasso to the prostate cancer regression dataset from [HTF09]. (We treat features `gleason` and `svi` as numeric, not categorical.) On the left,

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

0	0	0	0	0	0	0	0
0.4279	0	0	0	0	0	0	0
0.5015	0.0735	0	0	0	0	0	0
0.5610	0.1878	0	0	0.0930	0	0	0
0.5622	0.1890	0	0.0036	0.0963	0	0	0
0.5797	0.2456	0	0.1435	0.2003	0	0	0.0901
0.5864	0.2572	-0.0321	0.1639	0.2082	0	0	0.1066
0.6994	0.2910	-0.1337	0.2062	0.3003	-0.2565	0	0.2452
0.7164	0.2926	-0.1425	0.2120	0.3096	-0.2890	-0.0209	0.2773

Table 11.1: Values of the coefficients for linear regression model fit to prostate cancer dataset as we vary the strength of the ℓ_1 regularizer. These numbers are plotted in Figure 11.10(b).

when $B = 0$, all the coefficients are zero. As we increase B, the coefficients gradually "turn on".[2] The analogous result for ridge regression is shown in Figure 11.10(a). For ridge, we see all coefficients are non-zero (assuming $\lambda > 0$), so the solution is not sparse.

Remarkably, it can be shown that the lasso solution path is a piecewise linear function of λ [Efr+04; GL15]. That is, there are a set of critical values of λ where the active set of non-zero coefficients changes. For values of λ between these critical values, each non-zero coefficient increases or decreases in a linear fashion. This is illustrated in Figure 11.10(b). Furthermore, one can solve for these critical values analytically [Efr+04]. In Table 11.1. we display the actual coefficient values at each of these critical steps along the regularization path (the last line is the least squares solution).

By changing λ from $\lambda_{\max}$ to 0, we can go from a solution in which all the weights are zero to a solution in which all weights are non-zero. Unfortunately, not all subset sizes are achievable using lasso. In particular, one can show that, if $D > N$, the optimal solution can have at most N variables in it, before reaching the complete set corresponding to the OLS solution of minimal ℓ_1 norm. In Section 11.4.8, we will see that by using an ℓ_2 regularizer as well as an ℓ_1 regularizer (a method known as the elastic net), we can achieve sparse solutions which contain more variables than training cases. This lets us explore model sizes between N and D.

11.4.5 Comparison of least squares, lasso, ridge and subset selection

In this section, we compare least squares, lasso, ridge and subset selection. For simplicity, we assume all the features of $\mathbf{X}$ are orthonormal, so $\mathbf{X}^\mathsf{T}\mathbf{X} = \mathbf{I}$. In this case, the NLL is given by

$$\text{NLL}(\boldsymbol{w}) = ||\boldsymbol{y} - \mathbf{X}\boldsymbol{w}||^2 = \boldsymbol{y}^\mathsf{T}\boldsymbol{y} + \boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w} - 2\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} \tag{11.91}$$

$$= \text{const} + \sum_d w_d^2 - 2\sum_d \sum_n w_d x_{nd} y_n \tag{11.92}$$

so we see this factorizes into a sum of terms, one per dimension. Hence we can write down the MAP and ML estimates analytically for each w_d separately, as given below.

- **MLE** From Equation (11.85), the OLS solution is given by

$$\hat{w}_d^{\text{mle}} = c_d / a_d = \boldsymbol{x}_{:d}^\mathsf{T}\boldsymbol{y} \tag{11.93}$$

where $\boldsymbol{x}_{:d}$ is the d'th column of $\mathbf{X}$.

2. It is common to plot the solution versus the **shrinkage factor**, defined as $s(B) = B/B_{\max}$, rather than against B. This merely affects the scale of the horizontal axis, not the shape of the curves.

Term	OLS	Best Subset	Ridge	Lasso
intercept	2.465	2.477	2.467	2.465
lcalvol	0.676	0.736	0.522	0.548
lweight	0.262	0.315	0.255	0.224
age	-0.141	0.000	-0.089	0.000
lbph	0.209	0.000	0.186	0.129
svi	0.304	0.000	0.259	0.186
lcp	-0.287	0.000	-0.095	0.000
gleason	-0.021	0.000	0.025	0.000
pgg45	0.266	0.000	0.169	0.083
Test error	0.521	0.492	0.487	0.457
Std error	0.176	0.141	0.157	0.146

Figure 11.11: Results of different methods on the prostate cancer data, which has 8 features and 67 training cases. Methods are: OLS = ordinary least squares, Subset = best subset regression, Ridge, Lasso. Rows represent the coefficients; we see that subset regression and lasso give sparse solutions. Bottom row is the mean squared error on the test set (30 cases). Adapted from Table 3.3. of [HTF09]. Generated by prostate_comparison.ipynb.

- **Ridge** One can show that the ridge estimate is given by

$$\hat{w}_d^{\text{ridge}} = \frac{\hat{w}_d^{\text{mle}}}{1+\lambda} \tag{11.94}$$

- **Lasso** From Equation (11.88), and using the fact that $\hat{w}_d^{\text{mle}} = c_d/a_d$, we have

$$\hat{w}_d^{\text{lasso}} = \text{sign}(\hat{w}_d^{\text{mle}}) \left(|\hat{w}_d^{\text{mle}}| - \lambda\right)_+ \tag{11.95}$$

  This corresponds to soft thresholding, shown in Figure 11.9(a).

- **Subset selection** If we pick the best K features using subset selection, the parameter estimate is as follows

$$\hat{w}_d^{\text{ss}} = \begin{cases} \hat{w}_d^{\text{mle}} & \text{if rank}(|\hat{w}_d^{\text{mle}}|) \leq K \\ 0 & \text{otherwise} \end{cases} \tag{11.96}$$

  where rank refers to the location in the sorted list of weight magnitudes. This corresponds to hard thresholding, shown in Figure 11.9(b).

We now experimentally compare the prediction performance of these methods on the prostate cancer regression dataset from [HTF09]. (We treat features `gleason` and `svi` as numeric, not categorical.) Figure 11.11 shows the estimated coefficients at the value of λ (or K) chosen by cross-validation; we see that the subset method is the sparsest, then lasso. In terms of predictive performance, all methods are very similar, as can be seen from Figure 11.12.

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

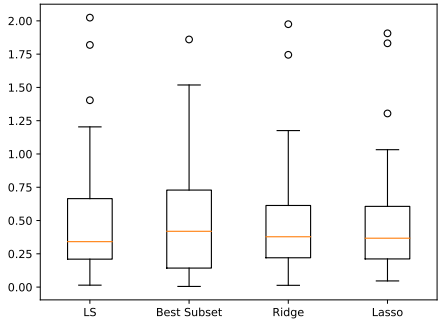


Figure 11.12: Boxplot displaying (absolute value of) prediction errors on the prostate cancer test set for different regression methods. Generated by prostate_ comparison.ipynb.

11.4.6 Variable selection consistency

It is common to use ℓ_1 regularization to estimate the set of relevant variables, a process known as **variable selection**. A method that can recover the true set of relevant variables (i.e., the support of $\boldsymbol{w}^*$) in the $N \to \infty$ limit is called **model selection consistent**. (This is a theoretical notion that assumes the data comes from the model.)

Let us give an example. We first generate a sparse signal $\boldsymbol{w}^*$ of size $D = 4096$, consisting of 160 randomly placed ± 1 spikes. Next we generate a random design matrix $\mathbf{X}$ of size $N \times D$, where $N = 1024$. Finally we generate a noisy observation $\boldsymbol{y} = \mathbf{X}\boldsymbol{w}^* + \boldsymbol{\epsilon}$, where $\epsilon_n \sim \mathcal{N}(0, 0.01^2)$. We then estimate $\boldsymbol{w}$ from $\boldsymbol{y}$ and $\mathbf{X}$. The original $\boldsymbol{w}^*$ is shown in the first row of Figure 11.13. The second row is the ℓ_1 estimate $\hat{\boldsymbol{w}}_{L1}$ using $\lambda = 0.1\lambda_{\max}$. We see that this has "spikes" in the right places, so it has correctly identified the relevant variables. However, although we see that $\hat{\boldsymbol{w}}_{L1}$ has correctly identified the non-zero components, but they are too small, due to shrinkage. In the third row, we show the results of using the debiasing technique discussed in Section 11.4.3. This shows that we can recover the original weight vector. By contrast, the final row shows the OLS estimate, which is dense. Furthermore, it is visually clear that there is no single threshold value we can apply to $\hat{\boldsymbol{w}}_{\text{mle}}$ to recover the correct sparse weight vector.

To use lasso to perform variable selection, we have to pick λ. It is common to use cross validation to pick the optimal value on the regularization path. However, it is important to note that cross validation is picking a value of λ that results in good predictive accuracy. This is not usually the same value as the one that is likely to recover the "true" model. To see why, recall that ℓ_1 regularization performs selection *and* shrinkage, that is, the chosen coefficients are brought closer to 0. In order to prevent relevant coefficients from being shrunk in this way, cross validation will tend to pick a value of λ that is not too large. Of course, this will result in a less sparse model which contains irrelevant variables (false positives). Indeed, it was proved in [MB06] that the prediction-optimal value of λ does not result in model selection consistency. However, various extensions to the basic method have been devised that are model selection consistent (see e.g., [BG11; HTW15]).

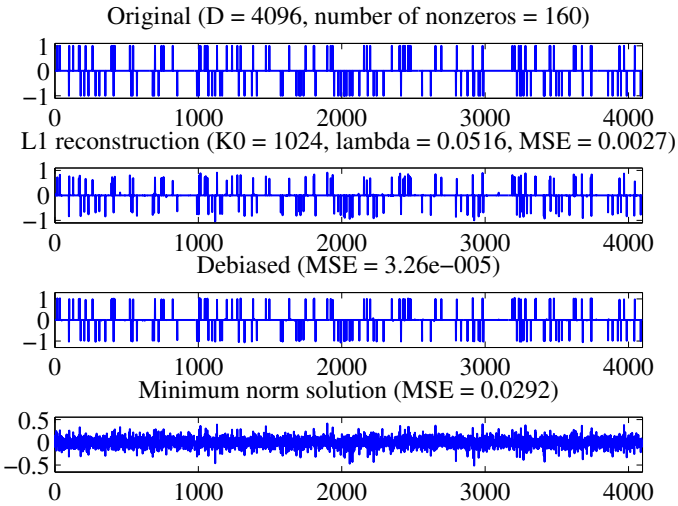


Figure 11.13: Example of recovering a sparse signal using lasso. See text for details. Adapted from Figure 1 of [FNW07]. Generated by sparse_sensing_demo.ipynb.

11.4.7 Group lasso

In standard ℓ_1 regularization, we assume that there is a 1:1 correspondence between parameters and variables, so that if $\hat{w}_d = 0$, we interpret this to mean that variable d is excluded. But in more complex models, there may be many parameters associated with a given variable. In particular, each variable d may have a vector of weights $\boldsymbol{w}_d$ associated with it, so the overall weight vector has block structure, $\boldsymbol{w} = [\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_D]$. If we want to exclude variable d, we have to force the whole subvector $\boldsymbol{w}_d$ to go to zero. This is called **group sparsity**.

11.4.7.1 Applications

Here are some examples where group sparsity is useful:

- Linear regression with categorical inputs: If the d'th variable is categorical with K possible levels, then it will be represented as a one-hot vector of length K (Section 1.5.3.1), so to exclude variable d, we have to set the whole vector of incoming weights to 0.
- Multinomial logistic regression: The d'th variable will be associated with C different weights, one per class (Section 10.3), so to exclude variable d, we have to set the whole vector of outgoing weights to 0.
- Neural networks: the k'th neuron will have multiple inputs, so if we want to "turn the neuron off", we have to set all the incoming weights to zero. This allows us to use group sparsity to learn neural network structure (for details, see e.g., [GEH19]).

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

- Multi-task learning: each input feature is associated with C different weights, one per output task. If we want to use a feature for all of the tasks or none of the tasks, we should select weights at the group level [OTJ07].

11.4.7.2 Penalizing the two-norm

To encourage group sparsity, we partition the parameter vector into G groups, $\boldsymbol{w} = [\boldsymbol{w}_1, \ldots, \boldsymbol{w}_G]$. Then we minimize the following objective

$$\text{PNLL}(\boldsymbol{w}) = \text{NLL}(\boldsymbol{w}) + \lambda \sum_{g=1}^{G} ||\boldsymbol{w}_g||_2 \tag{11.97}$$

where $||\boldsymbol{w}_g||_2 = \sqrt{\sum_{d \in g} w_d^2}$ is the 2-norm of the group weight vector. If the NLL is least squares, this method is called **group lasso** [YL06; Kyu+10].

Note that if we had used the sum of the squared 2-norms in Equation (11.97), then the model would become equivalent to ridge regression, since

$$\sum_{g=1}^{G} ||\boldsymbol{w}_g||_2^2 = \sum_{g} \sum_{d \in g} w_d^2 = ||\boldsymbol{w}||_2^2 \tag{11.98}$$

By using the square root, we are penalizing the radius of a ball containing the group's weight vector: the only way for the radius to be small is if all elements are small.

Another way to see why the square root version enforces sparsity at the group level is to consider the gradient of the objective. Suppose there is only one group of two variables, so the penalty has the form $\sqrt{w_1^2 + w_2^2}$. The derivative wrt w_1 is

$$\frac{\partial}{\partial w_1}(w_1^2 + w_2^2)^{\frac{1}{2}} = \frac{w_1}{\sqrt{w_1^2 + w_2^2}} \tag{11.99}$$

If w_2 is close to zero, then the derivative approaches 1, and w_1 is driven to zero as well, with force proportional to λ. If, however, w_2 is large, the derivative approaches 0, and w_1 is free to stay large as well. So all the coefficients in the group will have similar size.

11.4.7.3 Penalizing the infinity norm

A variant of this technique replaces the 2-norm with the infinity-norm [TVW05; ZRY05]:

$$||\boldsymbol{w}_g||_\infty = \max_{d \in g} |w_d| \tag{11.100}$$

It is clear that this will also result in group sparsity, since if the largest element in the group is forced to zero, all the smaller ones will be as well.

11.4.7.4 Example

An illustration of these techniques is shown in Figure 11.14 and Figure 11.15. We have a true signal $\boldsymbol{w}$ of size $D = 2^{12} = 4096$, divided into 64 groups each of size 64. We randomly choose 8 groups

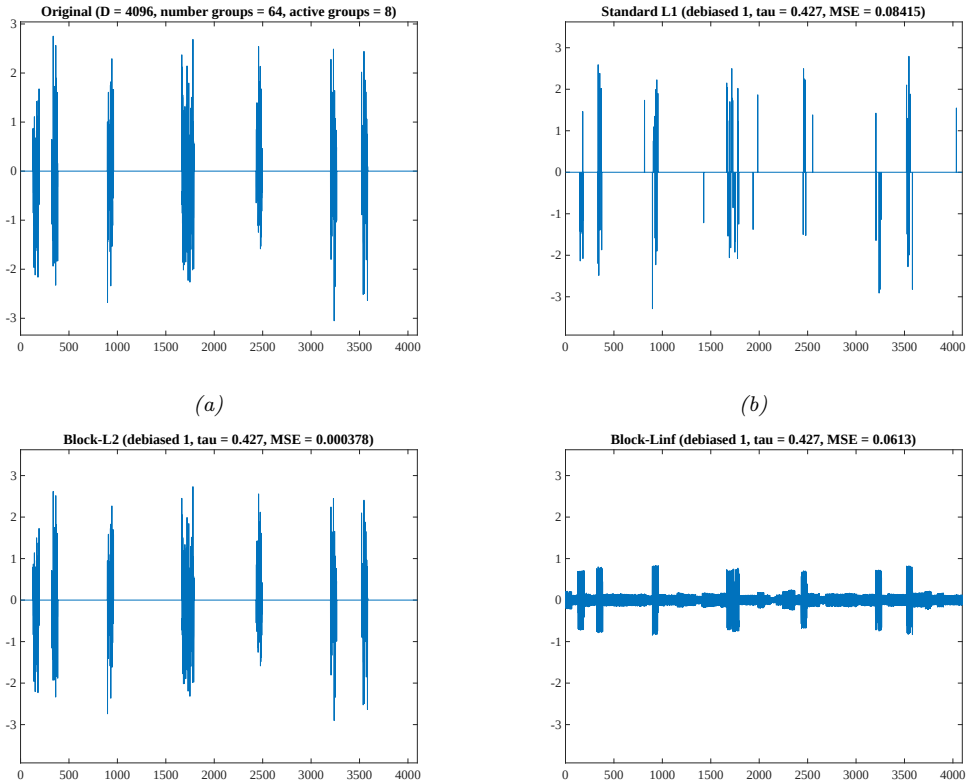


Figure 11.14: Illustration of group lasso where the original signal is piecewise Gaussian. (a) Original signal. (b) Vanilla lasso estimate. (c) Group lasso estimate using an ℓ_2 norm on the blocks. (d) Group lasso estimate using an ℓ_∞ norm on the blocks. Adapted from Figures 3-4 of [WNF09]. Generated by groupLassoDemo.ipynb.

of $\boldsymbol{w}$ and assign them non-zero values. In Figure 11.14 the values are drawn from a $\mathcal{N}(0,1)$; in Figure 11.15, the values are all set to 1. We then sample a random design matrix $\mathbf{X}$ of size $N \times D$, where $N = 2^{10} = 1024$. Finally, we generate $\boldsymbol{y} = \mathbf{X}\boldsymbol{w} + \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, 10^{-4}\mathbf{I}_N)$. Given this data, we estimate the support of $\boldsymbol{w}$ using ℓ_1 or group ℓ_1, and then estimate the non-zero values using least squares (debiased estimate).

We see from the figures that group lasso does a much better job than vanilla lasso, since it respects the known group structure. We also see that the ℓ_∞ norm has a tendency to make all the elements within a block to have similar magnitude. This is appropriate in the second example, but not the first. (The value of λ was the same in all examples, and was chosen by hand.)

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

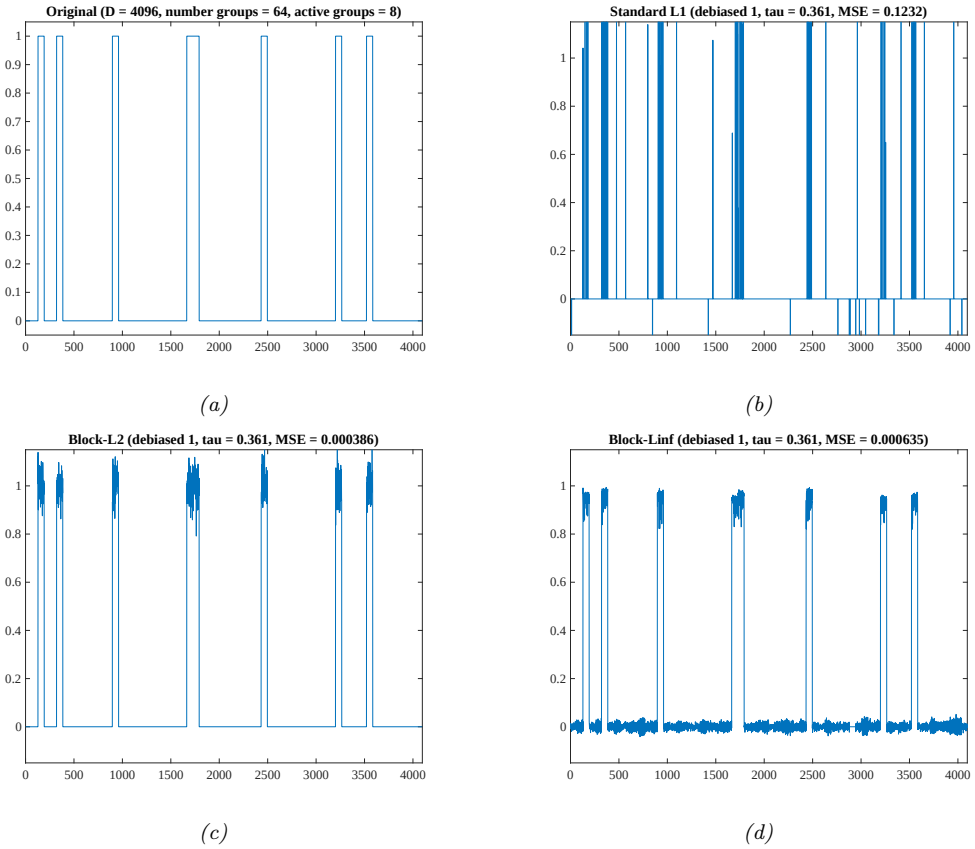


Figure 11.15: Same as Figure 11.14, except the original signal is piecewise constant. Generated by groupLassoDemo.ipynb.

11.4.8 Elastic net (ridge and lasso combined)

In group lasso, we need to specify the group structure ahead of time. For some problems, we don't know the group structure, and yet we would still like highly correlated coefficients to be treated as an implicit group. One way to achieve this effect, proposed in [ZH05], is to use the **elastic net**, which is a hybrid between lasso and ridge regression.[3] This corresponds to minimizing the following objective:

$$\mathcal{L}(\boldsymbol{w}, \lambda_1, \lambda_2) = ||\boldsymbol{y} - \mathbf{X}\boldsymbol{w}||^2 + \lambda_2 ||\boldsymbol{w}||_2^2 + \lambda_1 ||\boldsymbol{w}||_1 \tag{11.101}$$

This penalty function is *strictly convex* (assuming $\lambda_2 > 0$) so there is a unique global minimum, even if $\mathbf{X}$ is not full rank. It can be shown [ZH05] that any strictly convex penalty on $\boldsymbol{w}$ will exhibit a **grouping effect**, which means that the regression coefficients of highly correlated variables tend to

3. It is apparently called the "elastic net" because it is "like a stretchable fishing net that retains all the big fish" [ZH05].

be equal. In particular, if two features are identically equal, so $\mathbf{X}_{:j} = \mathbf{X}_{:k}$, one can show that their estimates are also equal, $\hat{w}_j = \hat{w}_k$. By contrast, with lasso, we may have that $\hat{w}_j = 0$ and $\hat{w}_k \neq 0$ or vice versa, resulting in less stable estimates.

In addition to its soft grouping behavior, elastic net has other advantages. In particular, if $D > N$, the maximum number of non-zero elements that can be selected (excluding the MLE, which has D non-zero elements) is N. By contrast, elastic net can select more than N non-zero variables on its path to the dense estimate, thus exploring more possible subsets of variables.

11.4.9 Optimization algorithms

A large variety of algorithms have been proposed to solve the lasso problem, and other ℓ_1-regularized convex objectives. In this section, we briefly mention some of the most popular methods.

11.4.9.1 Coordinate descent

Sometimes it is hard to optimize all the variables simultaneously, but it easy to optimize them one by one. In particular, we can solve for the j'th coefficient with all the others held fixed as follows:

$$w_j^* = \underset{\eta}{\operatorname{argmin}}\, \mathcal{L}(\boldsymbol{w} + \eta \boldsymbol{e}_j) \tag{11.102}$$

where $\boldsymbol{e}_j$ is the j'th unit vector. This is called **coordinate descent**. We can either cycle through the coordinates in a deterministic fashion, or we can sample them at random, or we can choose to update the coordinate for which the gradient is steepest.

This method is particularly appealing if each one-dimensional optimization problem can be solved analytically, as is the case for lasso (see Equation (11.87)). This is known as the **shooting** algorithm [Fu98; WL08]. (The term "shooting" is a reference to cowboy theme inspired by the term "lasso".) See Algorithm 11.1 for details.

This coordinate descent method has been generalized to the GLM case in [FHT10], and is the basis of the popular **glmnet** software library.

Algorithm 11.1: Coordinate descent for lasso (aka shooting algorithm)

Initialize $\boldsymbol{w} = (\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda \mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$
repeat
  for $d = 1, \ldots, D$ **do**
    $a_d = \sum_{n=1}^N x_{nd}^2$
    $c_d = \sum_{n=1}^N x_{nd}(y_n - \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_n + w_d x_{nd})$
    $w_d = \text{SoftThreshold}(\frac{c_d}{a_d}, \lambda/a_d)$
until *converged*

11.4.9.2 Projected gradient descent

In this section, we convert the non-differentiable ℓ_1 penalty into a smooth regularizer. To do this, we first use the **split variable trick** to define $\boldsymbol{w} = \boldsymbol{w}^+ - \boldsymbol{w}^-$, where $\boldsymbol{w}^+ = \max\{\boldsymbol{w}, 0\}$ and

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

$\boldsymbol{w}^- = -\min\{\boldsymbol{w}, 0\}$. Now we can replace $||\boldsymbol{w}||_1$ with $\sum_d (w_d^+ + w_d^-)$. We also have to replace $\text{NLL}(\boldsymbol{w})$ with $\text{NLL}(\boldsymbol{w}^+ + \boldsymbol{w}^-)$. Thus we get the following smooth, but constrained, optimization problem:

$$\min_{\boldsymbol{w}^+ \geq 0, \boldsymbol{w}^- \geq 0} \text{NLL}(\boldsymbol{w}^+ - \boldsymbol{w}^-) + \lambda \sum_{d=1}^{D} (w_d^+ + w_d^-) \tag{11.103}$$

In this case of a Gaussian likelihood, the NLL becomes a least squares loss, and the objective becomes a quadratic program (Section 8.5.4). One way to solve such problems is to use projected gradient descent (Section 8.6.1). Specifically, we can enforce the constraint by projecting onto the positive orthant, which we can do using $w_d := \max(w_d, 0)$; this operation is denoted by P_+. Thus the projected gradient update takes the following form:

$$\begin{pmatrix} \boldsymbol{w}_{t+1}^+ \\ \boldsymbol{w}_{t+1}^- \end{pmatrix} = P_+ \left(\begin{bmatrix} \boldsymbol{w}_t^+ - \eta_t \nabla \text{NLL}(\boldsymbol{w}_t^+ - \boldsymbol{w}_t^-) - \eta_t \lambda \boldsymbol{e} \\ \boldsymbol{w}_t^- + \eta_t \nabla \text{NLL}(\boldsymbol{w}_t^+ - \boldsymbol{w}_t^-) - \eta_t \lambda \boldsymbol{e} \end{bmatrix} \right) \tag{11.104}$$

where $\boldsymbol{e}$ is the unit vector of all ones.

11.4.9.3 Proximal gradient descent

In Section 8.6, we introduced proximal gradient descent, which can be used to optimize smooth functions with non-smooth penalties, such as ℓ_1. In Section 8.6.2, we showed that the proximal operator for the ℓ_1 penalty corresponds to soft thresholding. Thus the proximal gradient descent update can be written as

$$\boldsymbol{w}_{t+1} = \text{SoftThreshold}(\boldsymbol{w}_t - \eta_t \nabla \text{NLL}(\boldsymbol{w}_t), \eta_t \lambda) \tag{11.105}$$

where the soft thresholding operator (Equation (8.134)) is applied elementwise. This is called the **iterative soft thresholding algorithm** or **ISTA** [DDDM04; Don95]. If we combine this with Nesterov acceleration, we get the method known as "fast ISTA" or **FISTA** [BT09], which is widely used to fit sparse linear models.

11.4.9.4 LARS

In this section, we discuss methods that can generate a set of solutions for different values of λ, starting with the empty set, i.e., they compute the full regularization path (Section 11.4.4). These algorithms exploit the fact that one can quickly compute $\hat{\boldsymbol{w}}(\lambda_k)$ from $\hat{\boldsymbol{w}}(\lambda_{k-1})$ if $\lambda_k \approx \lambda_{k-1}$; this is known as **warm starting**. In fact, even if we only want the solution for a single value of λ, call it λ_*, it can sometimes be computationally more efficient to compute a set of solutions, from $\lambda_{\max}$ down to λ_*, using warm-starting; this is called a **continuation method** or **homotopy** method. This is often much faster than directly "cold-starting" at λ_*; this is particularly true if λ_* is small.

The **LARS** algorithm [Efr+04], which stands for "least angle regression and shrinkage", is an example of a homotopy method for the lasso problem. This can compute $\hat{\boldsymbol{w}}(\lambda)$ for all possible values of λ in an efficient manner. (A similar algorithm was independently invented in [OPT00b; OPT00a]).

LARS works as follows. It starts with a large value of λ, such that only the variable that is most correlated with the response vector $\boldsymbol{y}$ is chosen. Then λ is decreased until a second variable is found which has the same correlation (in terms of magnitude) with the current residual as the first variable,

where the residual at step k on the path is defined as $\boldsymbol{r}_k = \boldsymbol{y} - \mathbf{X}_{:,F_k}\boldsymbol{w}_k$, where F_k is the current **active set** (cf., Equation (11.83)). Remarkably, one can solve for this new value of λ analytically, by using a geometric argument (hence the term "least angle"). This allows the algorithm to quickly "jump" to the next point on the regularization path where the active set changes. This repeats until all the variables are added.

It is necessary to allow variables to be removed from the current active set, even as we increase λ, if we want the sequence of solutions to correspond to the regularization path of lasso. If we disallow variable removal, we get a slightly different algorithm called **least angle regression** or **LAR**. LAR is very similar to **greedy forward selection**, and a method known as **least squares boosting** (see e.g., [HTW15]).

11.5 Regression splines *

We have seen how we can use polynomial basis functions to create nonlinear mappings from input to output, even though the model remains linear in the parameters. One problem with polynomials is that they are a global approximation to the function. We can achieve more flexibility by using a series of local approximations. To do this, we just need to define a set of basis functions that have local support. The notion of "locality" is hard to define in high-dimensional input spaces, so in this section, we restrict ourselves to 1d inputs. We can then approximate the function using

$$f(x;\boldsymbol{\theta}) = \sum_{i=1}^{m} w_i B_i(x) \tag{11.106}$$

where B_i is the i'th basis function.

A common way to define such basis functions is to use **B-splines**. ("B" stands for "basis", and the term "spline" refers to a flexible piece of material used by artists to draw curves.) We discuss this in more detail in Section 11.5.1.

11.5.1 B-spline basis functions

A spline is a piecewise polynomial of degree D, where the locations of the pieces are defined by a set of **knots**, $t_1 < \cdots < t_m$. More precisely, the polynomial is defined on each of the intervals $(-\infty, t_1)$, $[t_1, t_2]$, $\cdots$, $[t_m, \infty)$. The function is continuous and has continuous derivatives of orders $1, \ldots, D-1$ at its knot points. It is common to use **cubic splines**, in which $D = 3$. This ensures the function is continuous, and has continuous first and second derivatives at each knot.

We will skip the details on how B-splines are computed, since it is not relevant to our purposes. Suffice it to say that we can call the patsy.bs function to convert the $N \times 1$ data matrix $\mathbf{X}$ into an $N \times (K + D + 1)$ design matrix $\mathbf{B}$, where K is the number of knots and D is the degree. (Alternatively, you can specify the desired number of basis functions, and let patsy work out the number and locations of the knots.)

Figure 11.16 illustrates this approach, where we use B-splines of degree 0, 1 and 3, with 3 knots. By taking a weighted combination of these basis functions, we can get increasingly smooth functions, as shown in the bottom row.

We see from Figure 11.16 that each individual basis function has local support. At any given input point x, only $D + 1$ basis functions will be "active". This is more obvious if we plot the design matrix

Author: Kevin P. Murphy. (C) MIT Press. CC-BY-NC-ND license

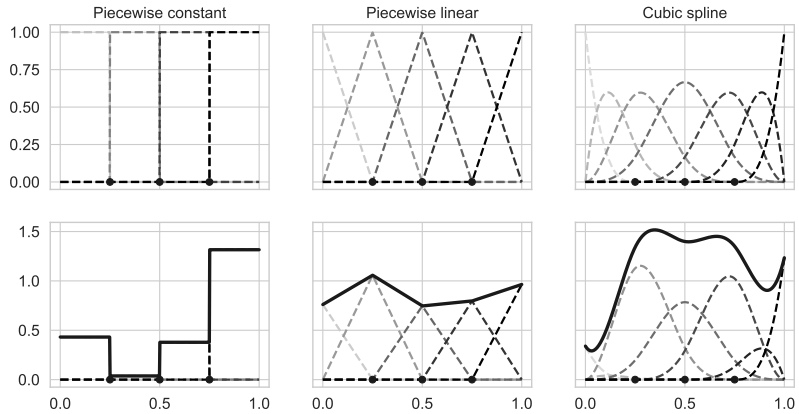


Figure 11.16: Illustration of B-splines of degree 0, 1 and 3. Top row: unweighted basis functions. Dots mark the locations of the 3 internal knots at $[0.25, 0.5, 0.75]$. *Bottom row: weighted combination of basis functions using random weights. Generated by splines_basis_weighted.ipynb. Adapted from Figure 5.4 of [MKL11]. Used with kind permission of Osvaldo Martin.*

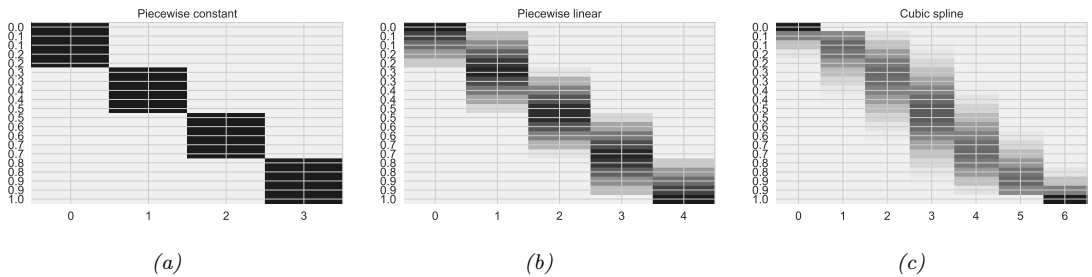


Figure 11.17: Design matrix for B-splines of degree (a) 0, (b) 1 and (c) 3. We evaluate the splines on 20 inputs ranging from 0 to 1. Generated by splines_basis_heatmap.ipynb. Adapted from Figure 5.6 of [MKL11]. Used with kind permission of Osvaldo Martin.

B itself. Let us first consider the piecewise constant spline, shown in Figure 11.17(a). The first B-spline (column 1) is 1 for the first 5 observations, and otherwise 0. The second B-spline (column 0) is 0 for the first 5 observations, 1 for the second 5, and then 0 again. And so on. Now consider the linear spline, shown in Figure 11.17(b). The first B-spline (column 0) goes from 1 to 0, the next three splines go from 0 to 1 and back to 0; and the last spline (column 4) goes from 0 to 1; this reflects the triangular shapes shown in the top middle panel of Figure 11.16. Finally consider the cubic spline, shown in Figure 11.17(c). Here the pattern of activations is smoother, and the resulting model fits will be smoother too.